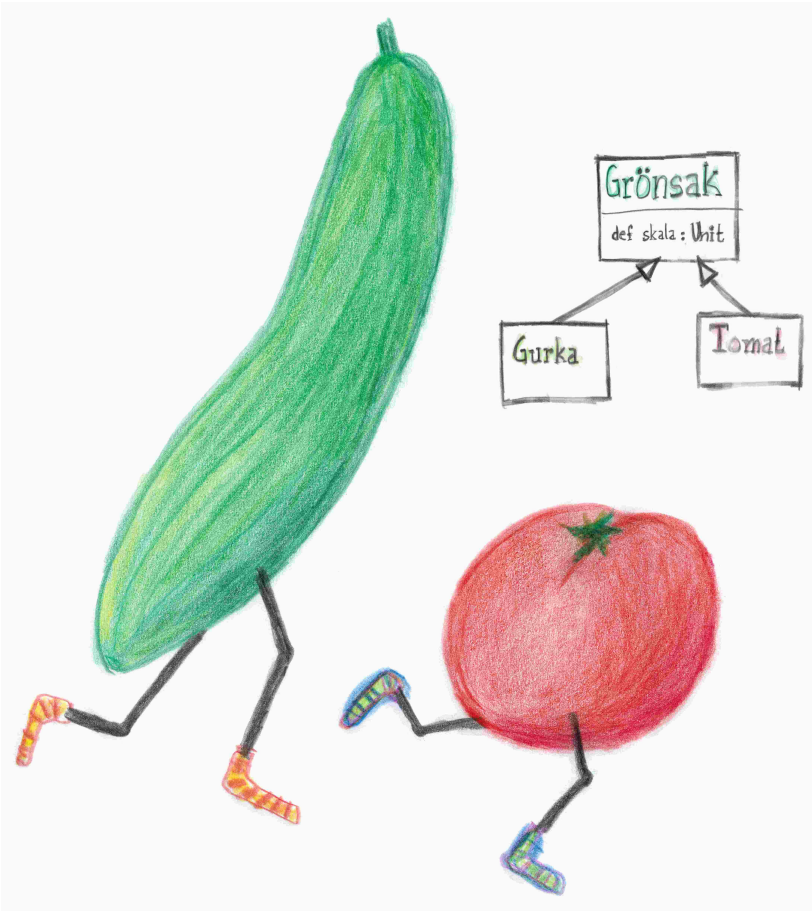
Introduction to programming with Scala

Exercises



H-ICT-SE-2-25 2026
Object Oriented Software Development
The Hague University of Applied Science

Compilation date: April 6, 2026
https://lunduniversity.github.io/pgk

Editor: Björn Regnell

Contributors in alphabetical order: Anders Buhl, André Philipsson Eriksson, Anna Axelsson, Anna Palmqvist Sjövall, Annie Predel, Anton Andersson, Benjamin Lindberg, Björn Regnell, Casper Schreiter, Cecilia Lindskog, Dag Hemberg, Elias Åradsson, Elliot Bräck, Elsa Cervetti Ogestad, Emelie Engström, Emil Wihlander, Erik Bjäreholt, Erik Grampp, Evelyn Beck, Felix Ohrgren, Fredrik Danebjer, Fritjof Bengtsson, Gustav Cedersjö, Henrik Olsson, Hjalmar Rutberg, Hussein Taher, Jakob Hök, Jakob Sinclair, Johan Ravnborg, Johannes Nydahl, Jonas Danebjer, Jos Rosenqvist, Lovisa Löfgren, Maj Stenmark, Maria Kulesh, Melker Widén, Måns Magnusson, Nicholas Boyd Isacsson, Niklas Sandén, Oliver Levay, Oliver Persson, Oscar Sigurdsson, Oskar Berg, Oskar Widmark, Patrik Persson, Per Holm, Philip Sadrian, Sandra Nilsson, Sebastian Hegardt, Simon Persson, Stefan Jonsson, Theodor Lundqvist, Tim Borglund, Tom Postema, Valthor Halldorsson, Viktor Claesson, Wilhelm Wanecek, William Karlsson.

Home: https://lunduniversity.github.io/pgk

Repo: https://github.com/lunduniversity/introprog

This compendium is on-going work.
Contributions are welcome!
Contact: bjorn.regnell@cs.lth.se

Versions:
Scala 3.7.2
JDK 21
introprog-scalalib 1.4.0

You can use this work if you respect this **LICENSE**: CC BY-SA 4.0
http://creativecommons.org/licenses/by-sa/4.0/
Do *not* distribute solutions to lab assignments and projects.

Copyright © 2015-2026.
Björn Regnell, Dept. of Computer Science, LTH, Lund University.

Contents

I Exercises **1**

1. Exercise `Excercise01` 3
2. Exercise `Excercise02` 17
3. Exercise `Excercise03` 27
4. Exercise `Excercise04` 37
5. Exercise `Excercise05` 48
6. Exercise `Excercise06` 63
7. Exercise `Excercise07` 74
8. Exercise `Excercise08` 90
9. Exercise `Excercise09` 98
10. Exercise `Excercise10` 105
11. Exercise `Excercise11` 116
12. Exercise `Excercise12` 123
13. Exercise `Excercise13` 139

II Solutions **141**

Solutions to the exercises **143**

1. Solution `Excercise01` 144
2. Solution `Excercise02` 159
3. Solution `Excercise03` 168
4. Solution `Excercise04` 176
5. Solution `Excercise05` 187
6. Solution `Excercise06` 199
7. Solution `Excercise07` 208
8. Solution `Excercise08` 222
9. Solution `Excercise09` 230
10. Solution `Excercise10` 235
11. Solution `Excercise11` 244
12. Solution `Excercise12` 253
13. Solution `Excercise13` 263
14. Solution `java` 264

Part I

Exercises

1. Exercise Excercise01

Goals

- ☐ Understand what happens when statements are executed and expressions are evaluated.
- ☐ Be familiar with the meaning of the concepts sequence, alternative, repetition, and abstraction.
- ☐ Be familiar with literals for simple values, their types, and scope.
- ☐ Be able to declare and use variables and assignment, as well as be able to draw diagrams of the memory situation when variable values change.
- ☐ Understand the difference between different numeric types, be able to convert between them, and be aware of precision issues that may arise.
- ☐ Understand boolean expressions and the values **`true`** and **`false`**, and be able to simplify boolean expressions.
- ☐ Understand the difference between integer division and floating-point division, as well as the use of remainder in integer division.
- ☐ Understand precedence rules and the use of parentheses in expressions.
- ☐ Be able to use **`if`** statements and **`if`** expressions.
- ☐ Be able to use **`for`** loops and **`while`** loops.
- ☐ Be able to use `math.random()` to generate random numbers within different intervals.

Preparations

- ☐ Study the concepts in chapter 1
- ☐ You need a computer with Scala and Kojo, see appendix C and A.

1.1 Basic Tasks; preparation for the lab

Task 1. *Match concepts with description.*

Connect each concept with the (simplified) description that fits best:

literal	1	A	introduce new concepts that simplify coding
string	2	B	either true or false
statement	3	C	translate code into executable form
expression	4	D	represents a specific data value
function	5	E	decimal number with limited precision
procedure	6	F	a line of code that does something; can be separated with a semicolon
runtime error	7	G	a sequence of characters
compile error	8	H	combines values and functions into a new value
abstract	9	I	describes what the data can be used for
compile	10	J	performs a side effect when called; return value is empty
type	11	K	computes a return value when called
for-loop	12	L	to change the value of a variable
while-loop	13	M	may occur before execution starts
assignment	14	N	may occur while the program runs
floating-point	15	O	useful when the number of repetitions is known beforehand
boolean	16	P	useful when the number of repetitions is unknown beforehand

Task 2. *Output in Scala REPL.*

Start Scala REPL (eng. *Read-Evaluate-Print-Loop*).

```
> scala
Welcome to Scala 3.1.2 (17.0.2, Java OpenJDK 64-Bit Server VM).
Type in expressions for evaluation. Or try :help.
scala -version.
scala>
```

a) Write after the prompt `scala>` a statement that prints any (common/crazy) greeting phrase, by calling the procedure `println` with some string argument. Press *Enter* so that the statement compiles and executes.

b) Write the same statement again (or press the up arrow) but ”forget” to write the right parenthesis after the argument before pressing *Enter*. What happens?

> *Tip for continuation:* There are many useful shortcuts and other tricks to work fast in REPL. Ask someone who knows these tricks to show you how to work faster. Read appendix C.4.2 and then try copying and pasting text. Use arrow keys to scroll through history, Ctrl+A to go to the start of a line, Ctrl+K to delete from cursor to end of line, etc.

Task 3. *String concatenation.*

a) Write an expression that concatenates two strings, e.g. `"gurk"` and `"burk"`, using the operator + and observe the result. What is the value and type of the expression? What digit follows the word `res` in the variable that stores the result?

b) Use the result from the concatenation, e.g. `res0` (replace the `0` with the digit after `res` in the output from the previous evaluation if necessary), and write an expression using the operator $*$ that repeats the result from the previous subtask 42 times.

Task 4. *When is the error detected?*

a) What is the type and value of the expression `"hej" * 3` ? Test in REPL.

b) Replace the 3 above with a sufficiently large integer so that memory runs out, but not so large that the number is outside the range of the base type `Int`. How does the error message start? Is this a runtime error or a compile-time error?

c) Choose a value for the argument after the operator $*$ such that a type error is generated. How does the error message start? Is this a runtime error or a compile-time error?

> *Tip for continuation:* Feel free to make mistakes when coding to learn more! Practice interpreting different error messages and ask someone for help if you don't understand. The compiler's messages can be very helpful, though sometimes cryptic. If you get stuck and cannot proceed yourself, ask for help, *but ask for hints rather than complete solutions* so you keep the initiative and control over the next step in your learning.

Task 5. *Literals and types.*

a) Use the REPL command `:type` (can be abbreviated `:t`) if needed to match the literals below with the correct type.

`1`	1	A	`String`
`1L`	2	B	`Boolean`
`1.0`	3	C	`Boolean`
`1D`	4	D	`Unit`
`1F`	5	E	`Int`
`'1'`	6	F	`Double`
`"1"`	7	G	`Long`
`true`	8	H	`Float`
`false`	9	I	`Char`
`()`	10	J	`Double`

b) What happens if you add 1 to the largest possible value of the `Int` type?
Tip: see the quick reference [1] under the heading "The Scala type system" section "Methods on numbers".

c) What is the difference between the types `Long` and `Int`?

d) What is the difference between the types `Double` and `Float`?

Task 6. *Mathematical functions. Using documentation.*

[1] https://fileadmin.cs.lth.se/pgk/quickref.pdf

a) Suppose you have a chessboard with 64 squares. Imagine you start by placing a single grain of rice on the first square and then pile twice as many grains in a new pile for each subsequent square: 1, 2, 4, 8, ... etc. When you have done this for all squares, how many grains of rice have you placed on the chessboard in total?[2]
Tip: You should calculate $2^{64} - 1$. If you type `math.` in the REPL and press TAB you will see built-in mathematical functions in Scala's standard library:

```
scala> math.    // Press TAB right after the dot and observe the list
```

Use the function `math.pow` and appropriate arguments. If you enter `math.pow` or `math.pow()` without arguments, you will see the function header with the parameter list.
If you browse to http://www.scala-lang.org/api/current/ and write `math` into the search box and then, after clicking **`scala.math`**, write **`pow`** in the box further down, the page is filtered and you find the documentation of `def pow` which you can click on and read more about.

b) Define the function `circumference` below in REPL. Can you omit the return type annotation? Why? Is there reason to keep it?

```
def circumference(radius: Double): Double = 2 * math.Pi * radius
```

c) Earth's (average) diameter (at the equator) is about 12750 km. Write an expression that calls the function `circumference` above to calculate approximately how many kilometers per day you must travel if you want to go around the Earth in 80 days.

Task 7. *Variables and assignment. Mutable and immutable variables.*

a) Draw a *new* picture of the computer's memory after *each* executed line 1–6 below. Each picture should show all variables existing in memory along with their variable name, type, and value.

```
1 scala> var a = 13
2 scala> val b = a + 1
3 scala> var c = (a + b) * 2.0
4 scala> b = 0
5 scala> a = 0
6 scala> c = c + 1
```

After the first line the memory situation looks like this:

a:	Int	13

b) Why does line 4 cause an error? Is it a compile-time error or a runtime error? What does the error message say?

Task 8. *Random numbers with `math.random()`.*

a) What does the function `math.random()` return? What type? What are the maximum and minimum possible values?
Tip: Search here: http://www.scala-lang.org/api/current/and try in REPL.

[2] https://en.wikipedia.org/wiki/Wheat_and_chessboard_problem

b) Declare the parameterless function `def roll: Int = ???` that should represent a dice roll and return a random integer between 1 and 6. Test the function by calling it many times.
Tip: Use `math.random()` and multiply and add with appropriate integers. Surround the calculation with parentheses and end with `.toInt` to truncate decimals and convert the type from `Double` to `Int`.

Task 9. *Repetition with `for`, `foreach` and `while`.*

a) Here is an example of a `for` statement:

```
for i <- 1 to 10 do print(s"$i, ")
```

Use a `for` statement to print out the result of 100 dice rolls with the function `roll` from exercise 8.

b) Here is an example of a `foreach` statement:

```
(1 to 10).foreach(i => print(s"$i, "))
```

Use a `foreach` statement to print out the result of 100 dice rolls with the function `roll` from exercise 8.

c) Here is an example of a `while` statement:

```
var i = 1
while i <= 10 do { print(s"$i, "); i = i + 1 }
```

Use a `while` statement to print the result of 100 dice rolls with the function `roll` from exercise 8. What happens if you forget `i = i + 1`?

Task 10. *Alternatives with `if` statement and `if` expression.*

a) Here is an example of an `if` statement (note the double equal signs):

```
if roll == 3 then println("THREE") else println("NOT THREE")
```

Try the above in REPL. Then write a `for` statement that rolls 100 dice and prints the string `"CONGRATULATIONS! "` if it is a six, else a sad smiley: `":("`

b) Here is an example of an `if` expression:

```
if roll < 6 then 0 else 1
```

Try this in REPL. Then write a `while` statement that rolls 100 dice and counts the number of sixes. Print the count after the `while` statement.

c) Simplify the implementation of this function:

```
def isAdult(age: Int) = if age >= 18 then true else false
```

Task 11. *Sequence, statement and block.*

a) What do these statements do?

```
scala> def p = { print("true"); print("!"); println("hej")}
scala> p;p;p;p
```

b) Use the up arrow to get back the line where you defined the procedure `p`. Swap the strings in the print calls in the procedure `p` so that the output becomes:

```
hejtrue!
hejtrue!
hejtrue!
hejtrue!
```

c) How does the compiler interpret curly braces and semicolons? What is a block?

Task 12. *Integer division.* Which value and which type correspond to which expression? If you are unsure, test in REPL.

`4 / 42`	1	A	`4: Int`
`42.0 / 2`	2	B	`10: Int`
`42 / 4`	3	C	`21.0: Double`
`42 % 4`	4	D	`true : Boolean`
`4 % 42`	5	E	`false: Boolean`
`40 % 4 == 0`	6	F	`0: Int`
`42 % 4 == 0`	7	G	`2: Int`

Task 13. *Boolean values.* What is the value of these expressions?

a) `true && true`
b) `false && true`
c) `true || true`
d) `false || true`
e) `false || false`
f) `true == true`
g) `true != false`
h) `true > false`
i) `true && (1 / 0 > 1)`
j) `false && (1 / 0 > 1)`

Task 14. *Boolean variables.* What is printed on line 2 and 4 below?

```
1 scala> var monster = false
2 scala> if monster then println("watch out!!!")
3 scala> monster = true
4 scala> if monster then println("watch out!!!")
```

Task 15. *Turtle graphics with Kojo.* In this week's lab you will use Kojo to verify that you can use sequence, alternative, repetition, and abstraction. With Kojo you create Scala programs that draw colorful shapes using an easy-to-use Scala library for *turtle graphics*[3].
If you use Kojo as a graphics library (recommended) and run with `scala run` (see Appendix A) you can use Scala 3. But if you run Kojo Desktop, Scala 2 applies and

[3] https://en.wikipedia.org/wiki/Turtle_graphics

even though most of this week's lab works similarly in Scala 2 and Scala 3, Scala 2 requires the old syntax for control structures with necessary parentheses around condition expressions, without either **do** or **then**, and neither optional curly braces nor indentation syntax.
Enter and run the program below with any method following Appendix A. Note the connection between the order of statements and what happens in the drawing window.

```
front; right
front; left
color(green)
front
```

If you use Kojo Desktop it is good to start the program with `erase` (why?[4]).

a) Write code that draws a square as in the image below.

Feel free to write your code in different ways *without* changing the result: write statements in sequence on multiple lines or statements in sequence on the same line separated by semicolons; use whitespace and blank lines in the code. How do you want to group your statements so they are easy for a person to read?

b) Draw stairs as shown in the image below.

c) Draw any picture on any background using some of the procedures in the table below. For example, you can draw a pink triangle with purple outlines on a black background. Try to ease code readability using appropriate line breaks and grouping.

`front(100)`	'The turtle moves forward 100 steps (25 if no argument).'
`color(pink)`	'Sets the pen's color to pink.'
`fill(purple)`	'Sets the filling color to purple.'
`fill(transparant)`	'Makes the turtle *not* fill anything when drawing.'
`width(20)`	'Makes the pen width 20.'

[4]When you press the play button in Kojo Desktop the canvas in the drawing window nor the turtle's state is reset. By starting your Kojo Desktop programs with `erase` you start execution in exactly the same initial condition: a blank canvas where the turtle points upward, the pen is down, and the pen color is red.

`background(black)`	'The background color becomes black.'
`penDown`	'Puts the turtle's pen down so it draws when moving.'
`penUp`	'Lifts the turtle's pen so it *does not* draw when moving.'
`right(45)`	'The turtle turns 45 degrees to the right.'
`left(45)`	'The turtle turns 45 degrees to the left.'
`jump`	'The turtle jumps 25 steps without drawing.'
`jump(100)`	'The turtle jumps 100 steps without drawing.'
`jumpTo(100, 200)`	'The turtle jumps to position (100, 200) without drawing.'
`goTo(100, 200)`	'The turtle turns and moves to position (100, 200).'
`east`	'The turtle turns so its nose points right.'
`west`	'The turtle turns so its nose points left.'
`north`	'The turtle turns so its nose points upward.'
`south`	'The turtle turns so its nose points downward.'
`setAngle(90)`	'The turtle turns its nose to angle 90 degrees.'

Tip for continuation: Keep both REPL and an editor open simultaneously. Then you can investigate how different code fragments work in REPL, while you *stepwise* create increasingly larger programs in the editor. This way of working will be very useful throughout the rest of the course. Regardless of which other tools you run, it is useful to keep REPL running in a separate window to help in your creative process, while chasing bugs, and while learning new concepts. Whenever you wonder how something works in Scala: **bring up REPL and try it!**

1.2 Extra Tasks; additional practice

Task 16. *Value and type.* Which value and type correspond to which expression? If you are unsure, test in REPL.

`1.0 + 18`	1	A	`1.042E42: Double`
`(41 + 1).toDouble`	2	B	`65: Int`
`1.042e42 + 1`	3	C	`113: Int`
`12E6.toLong`	4	D	`48: Int`
`32.toChar.toString`	5	E	`" ": String`
`'A'.toInt`	6	F	`0: Int`
`0.toInt`	7	G	`'*': Char`
`'0'.toInt`	8	H	`19.0: Double`
`'9'.toInt`	9	I	`12000000: Long`
`'A' + '0'`	10	J	`'q': Char`
`('A' + '0').toChar`	11	K	`42.0: Double`
`"*!%#".charAt(0)`	12	L	`57: Int`

Task 17. *Statements and expressions.*

a) What is the difference between a statement and an expression?

b) Give examples of statements that are not expressions?

c) Explain what happens for each evaluated line:

```
scala> def valueMissing = ()
scala> valueMissing
scala> valueMissing.toString
scala> println(valueMissing)
scala> println(println("hej"))
```

d) What is the type of the literal `()`?

e) What is the return type of `println`?

Task 18. *Procedure with parameter.* A procedure is a function that causes a side effect, for example printing or a variable assignment, but that does not return any interesting result value.[5]

a) Declare a mutable variable `highscore` initialized to 0.

b) Declare a procedure `updateHighscore` that takes a parameter `points` and assigns `highscore` a new value if `points` is greater than `highscore`, and prints the string `"REKORD!"`. If `points` is not greater than `highscore`, print the string `"DON'T GIVE UP!"`. Test the procedure in REPL.

c) Make a new variant of `updateHighscore` that is *not* a procedure but instead a function returning a string for later printing. Test the function in REPL.

[5] In Scala, procedures are functions returning the *empty value*, written `()` and typed `Unit`. In Java and several other languages there is no empty value and a special syntax for procedures using the keyword `void`.

Task 19. *Floating point arithmetic.*

a) What is the smallest positive value of type `Double`?

b) What is the value of this expression? Why does it become so?

```
scala> Double.MaxValue + Double.MinPositiveValue == Double.MaxValue
```

Task 20. ***`if`*** *statements.* For each line below, describe what is printed.

```
scala> if !true then println("true") else println("false")
scala> if !false then println("true") else println("false")
scala> def flipCoin = if math.random() < 0.5 then "kop" else "munt"
scala> for i <- 1 to 5 do print(s"$i:$flipCoin")
```

Task 21. Declare the following variables with these initial values:

```
scala> var vegetable = "cucumber"
scala> var fruit = "banana"
```

For each line below, give the value and type of the expression:

```
scala> if vegetable == "tomato" then "good" else "not good"
scala> if fruit == "banana" then "good" else "not good"
scala> if true then vegetable else 42
scala> if false then vegetable else 42
```

Task 22. *Modulo operator `%` and Boolean values.*

a) Declare a function `def isEven(n: Int): Boolean = ???` that returns `true` if the number `n` is even, else `false`.

b) Declare a function `def isOdd(n: Int): Boolean = ???` that returns `false` if the number `n` is even, else `true`.

Task 23. *Differences between* ***`var`***, ***`val`***, ***`def`***.

a) Evaluate each line one at a time in order in Scala REPL. For each line below: explain what happens and note value and possible errors.

```
scala> var x = 30
scala> x + 1
scala> x = x + 1
scala> x == x + 1
scala> val y = 20
scala> y = y + 1
scala> var z = { println("hej z!"); math.random() }
scala> def w = { println("hej w!"); math.random() }
scala> z
scala> z
scala> z = z + 1
scala> w
scala> w
scala> w = w + 1
```

b) What is the difference between `var`, `val` and `def`?

Task 24. *Difference between* ***if*** *and* ***while***. What is the result of line 3 and 4?

```
1 scala> def lotto1 = if math.random() > 0.5 then print("win :) ")
2 scala> def lotto2 = while math.random() > 0.5 do print("win :) ")
3 scala> lotto1
4 scala> lotto2
```

1.3 Advanced Tasks; challenges

Task 25. *Logic and De Morgan's Laws.* Simplify the following expressions. Assume `point` and `highscore` are integer variables and `clear` is of Boolean type.

a) `point > 100 && point > 1000`

b) `point > 100 || point > 1000`

c) `!(point > highscore)`

d) `!(point > 0 && point < highscore)`

e) `!(point < 0 || point > highscore)`

f) `clear == true`

g) `clear == false`

Task 26. *The string interpolator s.* Putting an `s` in front of a string literal allows the compiler to safely insert variable values into the string. Variable names must be preceded by a dollar sign, e.g. `s"Hej $namn"`. If you want to evaluate an expression, place it in braces immediately after the dollar sign, e.g. `s"Doubled length: ${namn.size * 2}"`

a) What is printed below?

```
scala> val f = "Kim"
scala> val e = "Finkodare"
scala> println(s"The name '$f $e' has ${f.size + e.size} letters.")
```

b) Create the following output using the `s` string interpolator and the variables `f` and `e` from the previous subtask.

```
Kim has 3 letters.
Finkodare has 9 letters.
```

Task 27. *Assignment operators.* You can shorten an assignment that changes a variable, e.g. `x = x + 1`, by using so-called assignment operators and write `x += 1` which means the same. Draw a new picture of computer memory after each line below. The pictures should show variables' names, type, and value.

```
scala> var a = 40
scala> var b = a + 40
scala> a += 10
scala> b -= 10
scala> a *= 2
scala> b /= 2
```

Task 28. *Big numbers.* If we want to compute $2^{64} - 1$ as an exact integer[6] it becomes larger than `Int.MaxValue`, so unfortunately we cannot use fast `Int`. Luckily, `BigInt` to the rescue.

a) Read about `BigInt` and `BigDecimal` in http://www.scala-lang.org/api/current/ Note what they can be used for.

b) You create a `BigInt` integer using `BigInt(2)` and can call the method `pow` on a `BigInt` with dot notation. Compute $2^{64} - 1$ as an exact integer.

[6] https://en.wikipedia.org/wiki/Wheat_and_chessboard_problem

c) What disadvantages do `BigInt` and `BigDecimal` have?

Task 29. *Precedence rules* Evaluation order can be controlled with parentheses. What value and type do the following expressions have?

a) `23 + 2 * 2 + (23 + 2) * 2`

b) `(-(2 - 42)) / (1 + 1 + 1)`

c) `(-(2 - 42)) / (-1)/(1 + 1 + 1)`

Task 30. *Documentation of packages in Java and Scala.*

a) By pressing the tab key you can see what is in various packages. What is the constant π called in `java.lang.Math` (note capital M) and `scala.math.`?

```
scala> java.lang.Math.    //press TAB after the dot
scala> scala.math.        //press TAB after the dot
```

b) Compare the documentation for class `java.lang.Math` here:
https://docs.oracle.com/javase/8/docs/api/
with the documentation for the package `scala.math` here:
http://www.scala-lang.org/api
Give examples of what you can do on the Scala documentation website that you *cannot* do on the corresponding Java documentation website.

c) What does the method `hypot` do? Why is it better to use `hypot` instead of implementing the calculation yourself with square root, multiplication, and addition?

Task 31. *Accuracy and exceptions in arithmetic expressions.* What is the result of the expressions below? Note exceptions and accuracy issues.

a) `Int.MaxValue + 1`

b) `1 / 0`

c) `1E8 + 1E-8`

d) `1E9 + 1E-9`

e) `math.pow(math.hypot(3,6), 2)`

★ f) `1.0 / 0`

★ g) `(1.0 / 0).toInt`

★ h) `math.sqrt(-1)`

★ i) `math.sqrt(Double.NaN)`

j) `throw new Exception("PANG!!!")`

★ **Task 32.** *Modulo arithmetic with negative numbers.* Read about modulo arithmetic here:
en.wikipedia.org/wiki/Modulo_operation
and investigate how it works with different signs (positive vs negative) for modulo calculation with $dividend\%divisor$ in Scala.

★ **Task 33.** *Literal identifiers.* Read about identifiers in Scala and especially *literal identifiers* here: http://www.artima.com/pins1ed/functional-objects.html#6.10.

a) Explain what happens below:

```
scala> val `literal val` = 42
scala> println(`literal val`)
```

b) Scala and Java have different sets of reserved words. In what way can 'backticks' be useful considering this?

Task 34. *`java.lang.Integer`, hexadecimal literals, BigDecimal.* ★

a) Find the documentation for `java.lang.Integer`.
Use the methods `toBinaryString` and `toHexString` to fill in the table below.

decimal integer	binary value	hexadecimal value
33		
42		
64		

b) How do you write the hexadecimal integer 10c (which equals 268 decimal) as a literal in Scala?

c) What is `0x10` raised to the power c = speed of light in m/s? *Tip:* Use `BigDecimal`.

Task 35. *String formatting.* Read about the `f`-interpolator here: ★
http://docs.scala-lang.org/overviews/core/string-interpolation.html
How can you use `f`-interpolator to produce the following output in REPL? Change line 2 at `???` so that the floating point number `g` is rounded to three decimals before printing.

```
1 scala> val g = 2 / 3.0
2 scala> val str = f"The giant cucumber is $g??? meter lang"
3 scala> println(str)
4 The giant cucumber is 0.667 meter lang
```

Task 36. *Multiplication warning.* Look up the documentation for `java.lang.Math.multiplyExact` and read what that method does.

a) What happens here?

```
scala> Math.multiplyExact(1, 2)
scala> Int.MaxValue * 2
scala> Math.multiplyExact(Int.MaxValue, 2)
```

b) Why might one want to use `java.lang.Math.multiplyExact` instead of 'regular' multiplication?

Task 37. *Extra operators for exact multiplication.* Kim CodeWizard thinks `Math.multiplyExact` is too cumbersome to write and extends the type `Int` with an extra operator: ★

```
extension (i: Int) def *!(j: Int) = Math.multiplyExact(i,j)
```

a) Paste the above code into REPL and try the extra operator.

b) Help Kim CodeWizard add more operators for values of type `Int` that allow using `Math.subtractExact` and `Math.addExact` conveniently.

c) Test a compound expression that uses all extra methods on `Int`. Do you find it more readable or more cryptic with the new operators?

2. Exercise Excercise02

Goals

- ☐ Be able to create, compile, and run a simple application in the terminal.
- ☐ Be able to create the collections Range, Array, and Vector with integers and strings.
- ☐ Be able to index an indexable collection, e.g., Array and Vector.
- ☐ Be able to call the operations size, mkString, sum, min, max on collections containing integers.
- ☐ Be familiar with the differences and similarities between the collections Range, Array, and Vector.
- ☐ Understand the difference between a while statement and a for expression.
- ☐ Be able to create collections with integer values as a result of simple for expressions.
- ☐ Understand the difference between an algorithm in pseudo-code and its implementation.
- ☐ Be able to implement the algorithms SUM, MIN, MAX with an indexable collection and a while loop.

Preparations

- ☐ Study the concepts in chapter 2
- ☐ Familiarize yourself with basic terminal commands, see appendix B.
- ☐ Familiarize yourself with VS Code, see appendix C.
- ☐ Retrieve given code via the course's GitHub repository.

2.1 Basic Tasks

Task 1. *Match concepts with descriptions.*

Match each concept with the (simplified) description that fits best:

compile	1	A	used in for-expressions to create a new collection
script	2	B	an immutable, indexable sequence collection
object	3	C	machine code is generated from one or more source files
@main	4	D	a mutable, indexable sequence collection
program argument	5	E	where execution of the compiled program starts
data structure	6	F	data structure with elements in a certain order
collection	7	G	single code file, no main program needed
sequence collection	8	H	step-by-step description of a solution to a problem
Array	9	I	a specific realization of an algorithm
Vector	10	J	many elements in a whole; element-wise access
Range	11	K	data structure with elements of the same type
yield	12	L	can be passed via the args parameter to main
algorithm	13	M	groups variables and functions
implementation	14	N	a collection representing a range of integers

Task 2. *Using the terminal.* Read about the terminal in appendix B.

a) What three commands should you run to 1) create a directory named `hello` and 2) navigate into the directory and 3) display the current directory name? Open a terminal window and run these three commands.

b) What two commands should you run to 1) navigate back "up" one step in the directory tree and 2) list all files and directories at that location? Run these two commands in the terminal.

Task 3. *Create and run a Scala script.*

a) Create a file named `sum.sc` in the `hello` directory you created in the previous task using an editor, e.g. VS `code`.

```
> cd hello
> code sum.sc
```

The file should contain the following lines:

```
val n = 1000
val summa = (1 to n).sum
println(s"The sum of the first $n numbers is: $summa")
```

Save the file and run the command `scala run sum.sc` in the terminal:

```
> scala run sum.sc
```

What is the sum of the first 1000 numbers?

b) Modify the file `sum.sc` so that the right parenthesis on the last line is missing. Save the file (Ctrl+S) and run the script file again in the terminal (up arrow key). What is the error message? Is it a runtime error or a compilation error?

c) Modify `sum.sc` so that instead of `1000` it says `args(0).toInt` after `val n =` and save and rerun your program with the argument 5001 as follows:

```
> scala run sum.sc -- 5001
```

What is the sum of the first 5001 numbers?

d) What error message do you get if you forget to give the script an argument? Is it a runtime error or a compilation error?

Task 4. *Scala application with `@main`.* Create, using an editor, a file named `hello.scala`.

```
> mkdir w02
> cd w02
> code hello.scala
```

Write the following code in the file:

```
@main def run(): Unit = {
  val message = "Hello world!"
  println(message)
}
```

a) Compile with `scala compile hello.scala`. What are the names of the files the compiler creates? Look for files ending with .class in the directories under the folder starting with w02, the name of the folder containing the file hello.scala

```
> scala compile hello.scala
> ls .scala-build/w02*/classes/main/
```

b) How should you modify your code so that the compiler gives the following error message:
`Syntax Error: '}' expected, but eof found?`

c) In Scala, curly braces are optional ((eng. *optional braces*)) and code is instead structured in contiguous blocks using indentation[7]. It is fine to switch styles in the same file if you think this makes the code more readable.
The above code can be written as:

```
@main def run(): Unit =
  val message = "Hello world!"
  println(message)
```

What happens if you remove the indentation of the last line?

d) What does the `@main` annotation mean?

Task 5. *Create and use collections.* In Scala's standard library there are many different collections that can be used uniformly (with some exceptions for `Array`). Match the expressions that create or use collections with the explanations, so that all connections are correct (at least one explanation fits more than one expression, but there is only one solution where all connections are pairwise correct):

`val xs = Vector(2)`	1	A	new collection with a zero appended at the end
`val ys = Array.fill(9)(0)`	2	B	new collection, elements converted to integers
`Vector.fill(9)(' ')`	3	C	new reference to a mutable sequence
`xs(0)`	4	D	new collection, elements converted to strings
`xs.apply(0)`	5	E	shortcut for `apply(0)`
`xs :+ 0`	6	F	indexing, returns the first element
`0 +: xs`	7	G	new string with commas between elements
`ys.mkString`	8	H	new collection with a zero prepended at the beginnin
`ys.mkString(",")`	9	I	new reference to a sequence of length 1
`xs.map(_.toString)`	10	J	new immutable sequence with blank characters
`xs.map(_.toInt)`	11	K	new string with all elements concatenated

Practice with your own variants in the REPL until you have learned to use expressions like the above by heart. Then you will find it easier to get started with coming labs.

Task 6. *Compare `Array` and `Vector`.* Match each collection type with the description that fits best:

a) What applies regarding mutability?

[7] Optional curly braces and significant indentation came with the new Scala 3. In old Scala 2, curly braces were necessary if multiple statements should be combined and indentation did not affect meaning.

Vector	1	A	mutable
Array	2	B	immutable

b) What applies when adding elements at the beginning ((eng. *prepend*)) and end ((eng. *append*)), or changing a subsequence at an arbitrary position (also called *to patch*)?

Vector	1	A	slow when changing size (requires copying everything)
Array	2	B	creates variants with more/other elements quickly from existing ones

c) What applies regarding equality testing?

Vector	1	A	`xs == ys` is **`true`** if all elements are equal
Array	2	B	unlike other Scala collections, == does not check content equality

Task 7. *Calculate sum, min and max in `args`.* Write a program that prints the sum, min, and max for a sequence of integers in `args`. You can assume the program is only run with integer program parameters. *Tip:* Using expressions `args.sum`, `args.min`, and `args.max` gives the sum, smallest and largest values respectively.
Example of running in terminal:

```
> code sum-min-max.scala
> scala run sum-min-max.scala -- 1 2 42 3 4
52 1 42
```

What error message appears if you give the argument `hej` when an integer is expected?

Task 8. *Algorithm: SWAP.* It is common when working with mutable data structures to need to swap places between elements, and then the SWAP algorithm is needed, as illustrated here by swapping values:
Problem: Swap values of two variables.
Solution idea: Use a temporary variable for storage.

a) Write the SWAP algorithm in *pseudo-code* (step by step in common language) below.
Input: Two integer variables x and y
???
Output: Variables x and y with their values swapped.

b) Implement the SWAP algorithm. Replace `???` below with code that swaps the values in the variables `x` and `y`:

```
scala> var x = 42; var y = 43
scala> ???
scala> println(s"x is $x, y is $y")
x is 43, y is 42
```

Task 9. *Indexing and assignment in Array with SWAP.* Write a program that swaps the first and last elements in the parameter `args`. The swap should only happen if there are at least two elements in `args`. Regardless of whether the change happened or not, `args` should then be printed with spaces between the arguments. *Tip:* You can access the last element with `args(args.length - 1)`
Example run in terminal:

```
> code swap-args.scala
> scala run swap-args.scala -- hej alla barn
barn alla hej
```

Task 10. ***for*** *expressions and* `map` *expressions.* The variable `xs` below refers to the collection
`Vector(1, 2, 3)`. Match the expressions on the left with the correct value on the right.

`for x <- xs yield x * 2`	1	A	`Vector(2, 4, 6)`
`for i <- xs.indices yield i`	2	B	`Vector(1, 2)`
`xs.map(x => x + 1)`	3	C	`Vector(1, 2, 3)`
`for i <- 0 to 1 yield xs(i)`	4	D	`Vector(2, 3, 4)`
`(1 to 3).map(i => i)`	5	E	`Vector(0, 1, 2)`
`(1 until 3).map(i => xs(i))`	6	F	`Vector(2, 3)`

Practice with your own variants in REPL until you have learned to use expressions like above by heart. Then it will be easier to get started with upcoming labs.

Task 11. *Algorithm: SUMBUG* Below is the pseudo-code for SUMBUG.

Input : integer n
Output: sum of the positive integers 1 through n

$sum \leftarrow 0$
$i \leftarrow 1$
while $i \leq n$ **do**
  $sum \leftarrow sum + 1$
end
sum

a) Step through the algorithm with pen and paper, writing down how the values for each variable change. There are two bugs in the algorithm. What are they? Correct the bugs and test again by "running" the algorithm on paper and check it works for $n = 0$, $n = 1$, and $n = 5$. What happens if $n = -1$?

b) Using an editor, create the file `sumn.scala`. Implement the SUM algorithm according to the corrected pseudocode and place the implementation in a `@main` annotated method named `sumn`. You can create input `n` to the algorithm with this declaration at the beginning of your method:
`val n = args(0).toInt`
or directly have `n` as a parameter to the method.
What does the application print if you run it with the argument 8888?

```
scala sumn.scala -- 8888
```

Check that your implementation calculates correctly by comparing the answer with the value of this expression evaluated in Scala REPL:

```
scala> (1 to 8888).sum
```

2.2 Extra Tasks; additional practice

Task 12. *Algorithm: MAXBUG* Below is the pseudo-code for MAXBUG.

```
Input  : Array args of strings all containing integers
Output : the largest integer
max ← the smallest integer value possible
n ← number of integers
i ← 0
while i < n do
    x ← args(i).toInt
    if ( x > max) then
        max ← x
    end
end
max
```

a) Run with pen and paper. There is a bug in the above algorithm. What is it? Fix the bug.

b) Implement the MAX algorithm (without bug) as a Scala application. Tips:

- The smallest `Int` value that can ever occur: `Int.MinValue`
- Number of elements in *args* given by: `args.length` or `args.size`

```
> code maxn.scala
> scala maxn.scala -- 7 42 1 -5 9
42
```

c) Rewrite the algorithm so that variable *max* is initialized to the first number in the sequence.

d) Implement the new algorithm variant from task c and test the program. Make sure the program works even if *args* is empty.

Task 13. *Algorithm MIN-INDEX.* Implement the MIN-INDEX algorithm that searches for the index of the smallest integer in a sequence. Pseudocode for the MIN-INDEX algorithm:

```
Input  : Sequence xs with n integers.
Output : Index of the smallest number or −1 if xs is empty.
minPos ← 0
i ← 1
while i < n do
    if xs(i) < xs(minPos) then
        minPos ← i
    end
    i ← i + 1
end
if n > 0 then
    minPos
else
    −1
end
```

a) Try the algorithm with pen and paper on the sequence $(1, 2, -1, 4)$ and draw the memory state after each round in the loop. What difference occurs in the execution flow if the loop variable i is initialized to 0 instead of 1?

b) Implement the MIN-INDEX algorithm in a Scala program with the following function:

```
def indexOfMin(xs: Array[Int]): Int = ???
```

- Let the program have a `main` function that creates a new array of integers from `args` which is sent to `indexOfMin`, and then print the result.
- Test for several cases:
  - empty sequences
  - sequence with only one number
  - long sequence with the minimal number first, somewhere in the middle, and last.

Task 14. *The data structure `Range`.* Evaluate the following expressions in Scala REPL. What are the value and type of each expression?

a) `Range(1, 10)`
b) `Range(1, 10).inclusive`
c) `Range(0, 50, 5)`
d) `Range(0, 50, 5).size`
e) `Range(0, 50, 5).inclusive`
f) `Range(0, 50, 5).inclusive.size`
g) `0.until(10)`
h) `0 until (10)`
i) `0 until 10`
j) `0.to(10)`
k) `0 to 10`
l) `0.until(50).by(5)`
m) `0 to 50 by 5`
n) `(0 to 50 by 5).size`
o) `(1 to 1000).sum`

2.3 Advanced Tasks; challenges

Task 15. *Rock-Paper-Scissors game.* Build on the code below to make a Rock-Paper-Scissors game[8]. The code works as it should, except the `winner` function cheats in favor of the computer. Add also a main function so the program can be compiled and run in the terminal. The game becomes more fun if you count the number of wins and losses. You can also make the computer choose not with uniform distribution.

```
object Game:
  val choices = Vector("Rock", "Paper", "Scissors")

  def printChoices(): Unit =
    for i <- 1 to choices.size do println(s"$i: ${choices(i - 1)}")

  def userChoice(): Int =
    printChoices()
    scala.io.StdIn.readLine("What do you choose? [1|2|3]<ENTER>:").toInt - 1

  def computerChoice(): Int = (math.random() * 3).toInt

  /** Should return "You", "Computer", or "None" */
  def winner(user: Int, computer: Int): String = "Computer"

  def play(): Unit =
    val u = userChoice()
    val c = computerChoice()
    println(s"You chose ${choices(u)}")
    println(s"Computer chose ${choices(c)}")
    val w = winner(u, c)
    println(s"$w is the winner!")
    if w == "None" then play()
```

★ **Task 16.** *Compare execution time for resizing between `Array` and `Vector`.* Paste the code below into REPL:

```
def time(block: => Unit): Double =
  val t = System.nanoTime
  block
  (System.nanoTime-t)/1e6  // gives milliseconds
```

a) Write code that performs these steps in order:
   1. declare a `val` as which is an `Array` filled with one million integer zeros,
   2. declare a `val` vs which is a `Vector` filled with one million integer zeros,
   3. run `time(as :+ 0)` 10 times and calculate the average of the timings,
   4. run `time(vs :+ 0)` 10 times and calculate the average of the timings.

b) Which of `Array` and `Vector` is fastest for adding elements? Why is that?

[8] https://en.wikipedia.org/wiki/Rock_paper_scissors

Task 17. *Memory usage for Range.* The data structure `Range` keeps track of start and end values and step size for an enumeration, but all the numbers in the enumeration are not generated until requested. An `Int` takes 4 bytes in memory. Approximately how much space in memory do the objects the variables (a) `interval` and (b) `sequence` refer to below take? ★

```
scala> val interval = (1 to Int.MaxValue by 2)
scala> val sequence = interval.toArray
```

Tip: Use expression `BigInt(Int.MaxValue) * 2` in your calculations.

Task 18. *Examine the generated byte-code.* The compiler generates bytecode, pronounced "byte code", which the virtual machine interprets and translates to machine code while the program runs. ★
Create a file plusxy.scala with:

```
@main def plusxy(x: Int, y: Int) = x + y
```

Compile the program with

```
scala compile plusxy.scala
```

Navigate with `cd .scala-build/` and further down with `ls` and `cd` as deep as you can until you reach directory `main`. Note what files the compiler has created with `ls`. With the command `javap -v 'plusxy$package$.class'` you can inspect the bytecode directly in the terminal.

```
javap -v 'plusxy$package$.class'
```

a) Find the line `public int plusxy(int, int);` and study the code after `Code:` and try to guess which instruction performs the actual addition.

b) What happens if we add a parameter?
Create a new file `plusxyz.scala`:

```
@main def plusxyz(x: Int, y: Int, z: Int) = x + y + z
```

Compile and then study the bytecode with `javap -v 'plusxyz$package$.class'`. What differs in bytecode between `plusxy` and `plusxyz`?

c) Read about bytecode here: en.wikipedia.org/wiki/Java_bytecode. What does the initial letter in the addition instruction mean?

3. Exercise Excercise03

Goals

- ☐ Be able to create and use functions with one or more parameters, default arguments, and named arguments.
- ☐ Be able to explain nested function calls with activation records on the stack.
- ☐ Be able to explain the difference between genuine and "pseudo" functions.
- ☐ Be able to apply a function to all elements in a collection.
- ☐ Be able to use functions as first-class values.
- ☐ Be able to create and use anonymous functions (also known as lambda functions).
- ☐ Be aware that functions can have a split parameter list.
- ☐ Be aware that it is possible to partially apply arguments to functions with split parameter lists to create so-called curried functions.
- ☐ Be aware of recursion and be able to describe the characteristics of a recursive function.
- ☐ Be aware that it is possible to create custom control structures using name calls.
- ☐ Be aware of the difference between call-by-value and call-by-name.
- ☐ Be able to interpret a stack trace.

Preparations

- ☐ Study the concepts in chapter 3

3.1 Basic Tasks; preparation for the lab

Task 1. *Match concepts with description.* Match each concept with the (simplified) description that fits best:

function header	1	A	describes names and types of parameters
function body	2	B	argument evaluated before call
parameter list	3	C	has parameter list and optionally a return type
block	4	D	delayed evaluation of argument
named arguments	5	E	function without a name, also called lambda
default argument	6	F	applies a function to each element of a collection
value call	7	G	a function returning a boolean value
name call	8	H	arguments can be passed in any order
map	9	I	always returns same result for same arguments
pure function	10	J	list of calls at runtime error
predicate	11	K	a function that calls itself
random seed	12	L	arguments can be omitted
anonymous function	13	M	produces a repeatable pseudo-random sequence
recursive function	14	N	code executed on function call
stack trace	15	O	may have local names; last line gives value

Task 2. *Define and call functions.* A function with one parameter is defined with the following syntax in Scala:

```
def name(parameter: Type = defaultArgument): ReturnType = returnValue
```

a) Define the function `increase` that has an integer parameter `x` and whose return value is the argument plus 1. The default argument should be 1. Specify the return type explicitly.

b) What is the value of the expression `increase(increase(increase(increase())))`?

c) Define the function `reduce` that has an integer parameter `x` and whose return value is the argument minus 1. The default argument should be 1. Specify the return type explicitly.

d) What is the value of the expression `increase(reduce(increase(increase(reduce(reduce()))))))`

e) What is the difference between parameter and argument?

Task 3. *Implement function in different ways.* Create a function that can sum the first `n` positive integers.

a) First write the function header with `???` as the function body. Give the function a good name. Specify the return type. Check that your function compiles without errors before proceeding.

b) Implement the function using a range and the method `sum`. Test that the function works. What happens if you give a negative argument?

c) Implement the function using **`while-do`**. What happens if you give a negative argument?

Task 4. *The text game AliensOnEarth.* Download the game below [9] and study the code.

```
object AliensOnEarth:
  def readChoice(msg: String, options: Vector[String]): String =
    options.indices.foreach(i => println(s"$i: ${options(i)}"))
    val selected = scala.io.StdIn.readLine(msg).toInt
    options(selected)

  def isAnswerYes(msg: String): Boolean =
    scala.io.StdIn.readLine(s"$msg (Y/n)").toLowerCase.startsWith("y")

  def randomChoice(options: Vector[String]): String =
    val selected = scala.util.Random.nextInt(options.size)
    options(selected)

  def playGame(alien: String, maxPoints: Int = 1000): Int =
    val xs = Vector("penguin", "window", "apple")
    val correct = if math.random() < 0.5 then xs(0) else randomChoice(xs)
    val cheatCode = (xs.indexOf(correct) + 1) * math.Pi
    println(s"""|Hello $alien!
                |You are an alien on Earth.
                |Your encrypted password is $cheatCode.
                |You see three strange Earth objects.""".stripMargin)
    val choice = readChoice(s"$alien wants? ", xs)
    if choice == correct then maxPoints else 0

```

[9] https://raw.githubusercontent.com/lunduniversity/introprog/master/compendium/examples/AliensOnEarth.scala

```
  def main(args: Array[String]): Unit =
    try
      val name = if args.size > 0 then args(0) else "Captain Zoom"
      val points = playGame(alien = name)
      if points > 0 then println(s"Congratulations $name! :)")
      println(s"You got $points points.")
    catch case e: Exception =>
      println(s"Game over. The Earth was hit by an asteroid. :(")
      if isAnswerYes("Do you want to trace the asteroid?") then
        e.printStackTrace()
```

a) While reading the code, try to figure out what is the best strategy to get as many points as possible. Compile and run the game in the terminal with your favorite name as argument. Which of the three objects on planet Earth has the highest probability to be the best option?

b) Match the code snippets below with the best description.[10]

`options.indices`	1	A	convert string to lowercase
`"1X2".toLowercase`	2	B	integer sequence with all indices in a sequence
`Random.nextInt(n)`	3	C	random number in range `0 until n`
`try { } catch { }`	4	D	removes margin up to vertical bar
`""" ... """`	5	E	catch exceptions to prevent crash
`s.stripMargin`	6	F	string spanning multiple lines
`e.printStackTrace`	7	G	prints exception information

Tip: Using the REPL you can find out how different parts work, for example:

```
scala> val xs = Vector("p", "w", "a")
scala> xs.indices
scala> xs.indices.foreach(i => println(i))
scala> xs.indexOf("w")
scala> xs.indexOf("gurka")
scala> Vector("hej", "hejsan", "hej").indexOf("hej")
scala> try 1 / 0 catch case e: Exception => println(e)
```

Tips for continuing:

- When I created `AliensOnEarth` I started with a very small program with a simple `main` function that just printed something fun. Then I built the program step-by-step and compiled and tested after every small change.
- When I code, I have REPL running in one terminal window and my code editor in another. In a third window I have a terminal running compilation in *watch mode*, see appendix F.2.1. Ask a supervisor how you can work efficiently with stepwise experimentation in REPL to build larger programs little by little.
- This way of working takes time to get used to, but is a good way to invent larger and better programs. Large programs are easiest built stepwise and debugging is much easier if you just add small parts at a time.
- It also becomes much easier for you to understand your program if you divide the code into many short functions with good names. You can then more easily

[10] Do your best even if not everything is clear. Some things will be covered in detail only in later course modules.

invent more advanced functions by reusing existing ones.

- During this week's lab you will develop your own text game. You will benefit from reusing the input and random choice functions from the example program `AliensOnEarth`.

Task 5. *Pure functions.* A pure function[11] (eng. *pure function*) always gives the same result with the same arguments (as we are used to in mathematics) and has no externally observable side effects (for example printing).
Which of the functions below are pure functions?

```
var x = 0
val y = x

def inc(i: Int) = i + 1

def nöff(i: Int) =
  x = x + i
  "nöff " * x
end nöff

def addX(i: Int) = x + i

def addY(i: Int) = y + i

def isPalindrome(s: String) = s == s.reverse

def rnd(min: Int, max: Int) = math.random() * max + min
```

Tip: Copy and test the functions in REPL one by one so that you understand exactly what happens.

Task 6. *Apply function to each element in a collection. Function as argument.*
Declare the function `increase` and the variable `xs` below in REPL:

```
scala> def increase(x: Int) = x + 1
scala> val xs = Vector(3, 4, 5)
```

Match the expressions below on the left with the expressions on the right that have the same value. If you wonder about something, test the expressions and various variants in REPL.

`for i <- 1 to 3 yield increment(i)`	1	A	`xs`
`Vector(2, 3, 4).map(i => increment(i))`	2	B	`Vector(4, 5, 6)`
`xs.map(increment)`	3	C	`()`
`xs.map(increment).map(increment)`	4	D	`Vector(5, 6, 7)`
`xs.foreach(increment)`	5	E	`Vector(2, 3, 4)`

[11] Pure functions by definition satisfy *referential transparency* (eng. *referential transparency*) which you can read more about here: simple.wikipedia.org/wiki/Referential_transparency

Task 7. *Anonymous functions.* We have several times seen the syntax `i => i + 1`, for example in a loop `(1 to 10).map(i => i + 1)` where the function `i => i + 1` is applied to all integers from 1 through 10 and the result is a new sequence collection. The syntax `(i: Int) => i + 1` is a literal to create a *function value* (also called an *anonymous function* or *lambda expression*). The syntax resembles that for function declarations, but the keyword `def` is missing in the function header and instead of equal sign `=>` separates the parameter list from the function body. If the compiler can infer the type from the context, the short form `i => i + 1` can be used.
There is an *even* shorter way to write an anonymous function *if* the type can be inferred *and* it only uses its parameter *once*; then function literals can be written with so-called *placeholder syntax* using underscores, such as `_ + 1` which the compiler automatically expands to `ngtnamn => ngtnamn + 1` (the name of the parameter does not matter; the compiler chooses some internal one).
Match expressions on the left with expressions on the right that have the same value:

`(0 to 2).map(i => i + 1)`	1	A	`Vector(9.0, 16.0, 25.0)`
`(1 to 3).map(_ + 1)`	2	B	`(2 to 4).map(i => i - 1)`
`(2 to 4).map(math.pow(2, _))`	3	C	`Vector(2.0, 2.5, 3.0)`
`(3 to 5).map(math.pow(_, 2))`	4	D	`Vector(2, 3, 4)`
`(4 to 6).map(_.toDouble).map(_ / 2)`	5	E	`Vector(4.0, 8.0, 16.0)`

Function literals are called *anonymous functions* since they have no name, unlike e.g. `def increase(i: Int): Int = i + 1`, which is named `increase`. Another common name is *lambda expression* after the computer science mathematical tool lambda calculus.

Task 8. *Create your own control structure using call-by-name parameter.* Call-by-name is written with a colon-rocket after the colon before the parameter type and means that the argument is evaluated on the spot each time.

a) Use call-by-name combined with a split parameter list to create your own control structure as below.[12]

```
def repeat(n: Int)(block: => Unit): Unit =
  var i = 0
  while i < n do
    ???
  end while
```

b) Test your control structure in REPL. Let it repeat 100 times rolling a random number between 1 and 6 and then print it out. Also try using fewer braces with help of colon.

c) Why is call-by-name needed here?

Task 9. *Learn to read a stack trace.* Write a program in the file `fel.scala` that causes a *runtime error* and then run it in terminal with `scala run fel.scala`. Study the stack trace printed. What does a `stack trace` contain? Discuss with the supervisor how you can benefit from a stack trace when debugging.

[12] This is how the loop `repeat` in Kojo is defined.

3.2 Extra Tasks; additional practice

Task 10. *Function with multiple parameters.*

a) Define in REPL two functions `sum` and `diff` with two integer parameters that return the sum respectively the difference of the arguments:

```
def sum(x: Int, y: Int): Int = ???

def diff(x: Int, y: Int): Int = ???
```

What are the values of the expressions below? Explain what happens.

b) `diff(0, 100)`

c) `diff(100, sum(42, 43))`

d) `sum(sum(42, 43), diff(100, sum(0, 0)))`

e) `sum(diff(Byte.MaxValue, Byte.MinValue), 1)`

Task 11. *Average value.* Write and test a function `avg` that computes the average between two integers and returns a `Double`.

Task 12. *Function calls with named arguments.*

```
1 scala> def skrivNamn(efternamn: String, förnamn: String) =
2          println(s"Namn: $efternamn, $förnamn")
3 scala> skrivNamn(förnamn = "Stina", efternamn = "Triangelsson")
4 scala> skrivNamn(efternamn = "Oval", "Viktor")
```

a) What is printed after line 3 and line 4 above?

b) Name three advantages with named arguments.

Task 13. *Function as a genuine value.* Functions are *genuine values* in Scala. This means variables can have functions as values and function values can be arguments to functions that have function parameters. Functions that take functions as arguments are called *higher-order functions*.

A function that has an integer parameter and returns an integer has the function type `Int => Int` (pronounced *int-to-int*) and the value of the function forms an object which has a method called `apply` with the corresponding function type.

a) Declare the functions and variables below in REPL. Then match the expressions on the left with the expressions on the right that create the same output. If you wonder about something, test the expressions and various variants in REPL.

```
scala> def health(): Unit = println("Hej!")
scala> def multipleCalls(aantal: Int, f: () => Unit): Unit =
         for _ <- 1 to aantal do f()
scala> val f1 = () => health()
scala> var f2 = (s: String) => println(s)
scala> val f3 = () => f2("Thunk")
```

`multipleCalls(1, greet)`	1	A	`f2("Hello!\nHello!")`
`multipleCalls(3, greet)`	2	B	`multipleCalls(3, f1)`
`multipleCalls(2, f1)`	3	C	`f3()`
`multipleCalls(1, f3)`	4	D	`f2("Hello!")`

b) What types do the variables `f1`, `f2` and `f3` have?

c) Does this work? Why? `f2 = f1`

d) Does this work? Why? `val f4 = multipleCalls`

e) Does this work? Why? `val f4 = health`

f) Does this work? Why? `val f4: () => Unit = health`

Task 14. *Discarded result values and return type `Unit`.* Investigate the code below in REPL and explain what happens.

a)

```
scala> def tom = println("")
scala> println(tom)
```

b)

```
scala> def wasted: Unit = 1 + 1
scala> println(wasted)
```

c)

```
scala> def wasted2 = { val x = 1 + 1 }
scala> println(wasted2)
```

d) Why is it good to explicitly specify `Unit` as the return type for procedures?

Task 15. *Call-by-name.*
Declare this procedure in REPL:

```
def doThisTwice(b: => Unit): Unit = { b; b }
```

Call `doThisTwice` with a block as parameter. The block should contain a print statement. Explain what happens.

3.3 Advanced Tasks; challenges

Task 16. *Mutability of parameters.* What do you think about this: Is a parameter mutable in the function body ...

a) ... in Scala? (Yes/No)

b) ... in Java? (Yes/No)

c) ... in Python? (Yes/No)

Task 17. *Call-by-value and call-by-name.* Normally in Scala (and in Java) *call-by-value* is used when calling functions, which means the argument expression is evaluated *before* the binding to the parameter name happens.
One can also in Scala (but not in Java) with syntax `=>` before the parameter type declare that *call-by-name* shall happen, which means the evaluation of the argument expression is *delayed* and happens *each time* the name is used in the method body.
Declare below functions in REPL.

```
def snark: Int = { print("snark "); Thread.sleep(1000); 42 }
def callByValue(x: Int):   Int = x + x
def callByName(x: => Int): Int = x + x
lazy val zzz = snark
```

Explain what happens when the following expressions are evaluated.

a) `snark + snark`

b) `callByValue(snark)`

c) `callByName(snark)`

d) `callByName(zzz)`

Task 18. *Create your own control structure for iteration with loop variable.*

a) The advantage of `repeat` in task 7 is that it is concise and easy to use. But it is not as easy to use if you need access to a loop variable. Implement the control structure below.

```
def repeat(n: Int)(p: Int => Unit): Unit =
  var i = 0
  while i < n do
    ???
```

b) Use `repeat` to write the loop variable and a random decimal number between 0 and 1 one hundred times.

Task 19. *Split parameter list and curried functions.* One can split the parameters to a function into several parameter lists. The function `add1` below has a parameter list with two parameters while `add2` has two parameter lists with one parameter each:

```
  def add1(a: Int, b: Int) = a + b
  def add2(a: Int)(b: Int) = a + b
```

a) When calling the function `add2` the arguments must be written in two separate parentheses. How can you use `add2` to compute `1 + 1`?

b) One advantage with split parameter lists is that one can create so-called *curried functions*[13] where arguments are partially applied. Try the curried function value `singLa` below. What is printed after lines 3 and 5?

```
1 scala> def repeat(s: String)(n: Int): String = s * n
2 scala> val song = repeat("doremi ")(3)
3 scala> println(song)
4 scala> val singLa = repeat("la")
5 scala> println(singLa(7))
```

★ **Task 20.** *Recursion.* A recursive function calls itself.

a) Explain what happens below.

```
scala> def countdown(x: Int): Unit =
         if x > 0 then {println(x); countdown(x - 1)}
scala> countdown(10)
scala> countdown(-1)
scala> def finalCountdown(x: Byte): Unit =
         {println(x); Thread.sleep(100); finalCountdown((x-1).toByte); 1 / x}
scala> finalCountdown(Byte.MaxValue)
```

b) What happens if you put the statement that risks division by zero *before* the recursive call in the function `finalCountdown` above?

c) Explain what happens below. Why does the last line take longer than the penultimate line?

```
scala> def signum(a: Int): Int = if a >= 0 then 1 else -1
scala> def add(x: Int, y: Int): Int =
         if y == 0 then x else add(x + 1, y - signum(y))
scala> add(100, 100)
scala> add(Int.MaxValue, 0)
scala> add(0, Int.MaxValue)
```

★ **Task 21.** *Investigate tail recursion by throwing exceptions.* Explain what happens. Can you find evidence that the compiler can optimize recursion into an ordinary loop?

```
scala> def explode = throw Exception("BANG!!!")
scala> explode
scala> def countdown(n: Int): Unit =
         if n == 0 then explode else countdown(n-1)
scala> countdown(10)
scala> countdown(10000)
scala> def countdown2(n: Int): Unit =
         if n == 0 then explode else {countdown2(n-1); print("no tailrec")}
scala> countdown2(10)
scala> countdown2(10000)
```

★ **Task 22.** *`@tailrec` annotation.* You can ask the compiler to give an error message if it cannot optimize the code to an equivalent while-loop. This can be used in cases where you want to be absolutely sure that the compiler can optimize the code and there is no risk of a *stack overflow* due to too deep call nesting.
Try the lines below in REPL and explain what happens.

[13] Also called Curry functions after the mathematician and logician Haskell Brooks Curry.

```
scala> def countNoTailrec(n: Long): Unit =
         if n <= 0L then println("Klar! " + n) else {countNoTailrec(n-1L); ()}
scala> countNoTailrec(1000L)
scala> countNoTailrec(100000L)
scala> import scala.annotation.tailrec
scala> @tailrec def countNoTailrec(n: Long): Unit =
         if n <= 0L then println("Klar! " + n) else {countNoTailrec(n-1L); ()}
scala> @tailrec def countTailrec(n: Long): Unit =
         if n <= 0L then println("Klar! " + n) else countTailrec(n-1L)
scala> countTailrec(1000L)
scala> countTailrec(100000L)
scala> countTailrec(Int.MaxValue.toLong * 2L)
```

4. Exercise Excercise04

Goals

- ☐ Be able to create and use objects as modules.
- ☐ Be able to explain how nested blocks affect name visibility and name shadowing.
- ☐ Be able to explain the concepts of visibility, private member, namespace, and name shadowing.
- ☐ Be able to create and use tuples.
- ☐ Be able to create functions that have multiple return values.
- ☐ Be able to explain the semantic relationship between functions and objects in Scala.
- ☐ Be able to explain the connection between package structure and code file structure.
- ☐ Be able to use ready-made code libraries in jar files.
- ☐ Be able to use import of members in objects and packages.
- ☐ Be able to rename on import.
- ☐ Be able to explain the difference between import and export.
- ☐ Be able to create and use variables with delayed initialization.

Preparations

- ☐ Study the concepts in chapter 4
- ☐ Read about how to fix bugs in appendix D.

4.1 Basic Tasks; preparation for the lab

Task 1. *Match concepts with descriptions.*

Connect each concept with the (simplified) description that fits best:

module	1	A	function that is a member of an object
singleton object	2	B	module that can have state; exists in a single instance
package	3	C	code unit with abstractions that can be reused
import	4	D	module creating namespace; machine code gets own folder
export	5	E	belongs to an object; accessed via dot notation if visible
lazy initialization	6	F	makes name locally available without full path
member	7	G	allocation occurs only when name is referenced
attribute	8	H	variable representing (part of) an object's state
method	9	I	environment where all names are unique
private	10	J	methods with same name but different parameter types
overloading	11	K	modifies visibility of an object member
name shadowing	12	L	local name hides same name in enclosing block
namespace	13	M	changing between def and val does not affect usage
uniform access	14	N	alternative name for type, often improves readability
dot notation	15	O	used to access non-private parts
type alias	16	P	makes name visible outside as a member of this object

Task 2. *Nested singleton objects, import, visibility and dot notation.* In the two-dimensional Underworld live the Mole and the Worm. The Worm has hidden from the Mole and is located far away. The Worm has also made parts of its position invisible to the outside world:

```
object Underworld:
  var x = 0
  var y = 1

  object Mole:
    var x = Underworld.x + 10
    var y = Underworld.y + 9

  object Worm:
    private var x = Mole.x
    var y = Mole.y + 190
    def isMoleFood: Boolean = ???
```

a) Create the above code in the file `Underworld.scala` with an editor and implement the predicate `isMoleFood` so that it returns true if the mole's coordinates are the same as the worm's.

b) Test the life in Underworld by pasting your module in REPL. Import Underworld's members with an asterisk so that you can see Mole and Worm. Using assignments, move Worm's y-coordinate so that it ends up at the same location as the Mole. Check that the predicate `isMoleFood` works as intended.

c) Then import everything from Mole and then everything from Worm and assign `x` a new value according to lines 1–3 below. What values do the expressions in lines 4–6 below give? Explain what happens in terms of name shadowing and visibility.

```
1 scala> import Mole.* // Mullvaden.*
2 scala> import Worm.* // Masken.*
3 scala> x = -1
4 scala> Mole.x
5 scala> Worm.x
6 scala> Underworld.x
```

Task 3. *Export.*

a) Compare **`import`** and **`export`** by describing one similarity and one difference.

b) Create an example in REPL demonstrating the usefulness of **`export`**.

Task 4. *Tuples.* Tuples combine multiple different values into an immutable object. Below, tuples are used to represent a 3D point in the Underworld with coordinates `(x, y, z)` of type `(Int, Int, Double)`, where the z-coordinate indicates how deep down in the underworld the point is. At a secret location is the entrance to the uberworld.

```
object Underworld3D:
  private val secret = ("the ascent to the uberworld", (0, 0, 0.0))

  object Mole:
    var pos = (5, 3, math.random() * 10 + 1)
```

```
    def depth  = ???

  object Worm:
    private var pos = (0, 0, 10.0)
    def isMolefood : Boolean = ???
    def isRightUnderTheRise: Boolean = ???
```

a) The function `depth` should give the z-coordinate for Mole. What is the type of `depth`?

b) What is the type of `secret`?

c) Write the code for `Underworld3D` in an editor and implement the missing parts. The predicate `isMoleFood` should be true if Worm is at the same place as Mole. The predicate `isRightUnderTheRise` should be true if the x and y coordinates match the secret entrance to the uberworld. Test your implementation in REPL.

d) A tuple with n values is called an n-tuple. If you consider the empty value `()` as a tuple, what can this value be called?

Task 5. *Lazy initialization.* With `lazy val` you can delay initialization.

a) What do lines 2 and 3 below produce?

```
1 scala> lazy val z = { println("nu!"); Array.fill(1e1.toInt)(0)}
2 scala> z
3 scala> z
```

b) Try again with a so large array that the memory gets full. When does the allocation happen?

c) Singleton objects are lazy. Initialization order can be wrong.

```
object test:
  object zzz    { val a = { println("nu!"); 42} }
  object buggig { val a = b ; val b = 42        }
  object funkar { lazy val a = b; val b = 42    }
```

Paste the module `test` into REPL. When is `"nu!"` printed?

d) What happens in REPL if you refer to the three different `a` variables?

e) What is the difference between `lazy val a = expression` and `def b = expression` ?

Task 6. *Extension methods.* Extension methods allow dot notation on values of existing types.

a) Create an extension method on integers that allows incrementing.

```
scala> 42.inc
val res0: Int = 43
```

b) Create an extension method on integers that allows decrementing.

```
scala> 42.dec
val res1: Int = 41
```

c) Combine the extension methods so that they become *collective*, i.e. under the same `extension`. Also use `math.incrementExact` and `math.decrementExact` after

consulting the documentation here: https://docs.oracle.com/en/java/javase/17/docs/api/

d) What is the advantage of `math.incrementExact` and `math.decrementExact`?

Task 7. *Jar file. Classpath. Package.* A jar file is used to collect compiled programs, code, documentation, resource files, etc, into a single file. A jar file is compressed similarly to a zip file. In the course we use a package named `introprog` which is in a jar file named something like `introprog_3-1.4.0.jar` (or later version) where the first number indicates the Scala version the library is compiled for, and the second number indicates the library version which changes with each new release.

a) In this week's lab we will use the class `PixelWindow` that exists in the package `introprog`. What parameters does the class `PixelWindow` have and which default arguments are present? How do you write to create an instance of `PixelWindow`?
Tip: See the code for `PixelWindow` here (search for the class's parameters):
https://github.com/lunduniversity/introprog-scalalib/blob/master/src/main/scala/introprog/PixelWindow.scala

b) Download the latest release of the jar file with the `introprog` package here:
https://github.com/lunduniversity/introprog-scalalib/releases
Save the file named `introprog_3-1.4.0.jar` (or later version) in a suitable location.

c) Test `PixelWindow` in REPL as below. Use the option `-jar` with the jar file name as argument. Write code that draws a square with side 100 and upper left corner at point (100, 100), continuing the partially started code below (adjust the jar file name according to the version you downloaded):

```
> scala repl --jar introprog_3-1.4.0.jar
scala> val w = introprog.PixelWindow(400,300,"HEJ")
scala> w.line(100, 100, 200, 100)
scala> w.line(200, 100, 200, 200)
scala> // continue so that a full square is drawn
```

d) Write the program below with an editor in the file `hello-window.scala` and fill in the missing parts so that a red square is drawn, guided by the documentation:
https://fileadmin.cs.lth.se/pgk/api

```scala
package hello

object Main:
  val w = new introprog.PixelWindow(400, 300, "HEJ")

  var color = java.awt.Color.red

  /** Square with upper corner at point p and size side pixels. */
  def square(p: (Int, Int))(side: Int): Unit =
    if side > 0 then
      // side == 1 gives a square that is a single pixel
      val d = side - 1

      w.line(p._1,     p._2,     p._1 + d, p._2,     color)
      w.line(p._1 + d, p._2,     p._1 + d, p._2 + d, color)
      w.line(p._1 + d, p._2 + d, p._1,     p._2 + d, color)
      ???
```

```
  def main(args: Array[String]): Unit =
    println("Draw square:")
    square(300,100)(50)
```

Run the program with

```
> scala run hello-window.scala --jar introprog_3-1.4.0.jar
Found several main classes. Which would you like to run?
[0] hello.Main
[1] introprog.examples.TestBlockGame
[2] introprog.examples.TestIO
[3] introprog.examples.TestPixelWindow
```

Besides your own main program named `hello.Main`, there are several example main programs in package `introprog.examples`. When several main programs are detected, you are asked which one you want to run. Choose your own main program.

e) You can avoid the question by explicitly specifying the main program with the option `--main-class`. Try that!

f) You can avoid downloading `introprog` by yourself using the option `--dep` when running from terminal, as described in the library's `README.md` on github here:
https://github.com/lunduniversity/introprog-scalalib
Try it!

g) You can also add the dependency inside your code file using a magic comment, also described in the above `README.md`. Try that!

Task 8. *Color.* There are many ways to describe colors. In natural language we have different names for colors, e.g. *white*, *pink* and *magenta*. In computer image memories it is common to describe colors as a mixture of *red*, *green* and *blue* in the so-called RGB system.
In this week's lab we will use `PixelWindow`, which describes RGB colors with the class `java.awt.Color`. There are some predefined colors in `java.awt.Color`, e.g. `java.awt.Color.black` for black and `java.awt.Color.green` for green, see further documentation of `java.awt.Color` in JDK[14]. Other colors can be created by specifying the particular amount of red, green and blue needed to blend a certain color. The three parameters to `new java.awt.Color(r, g, b)` specify how much *red*, *green* and *blue* the color should contain, and the amounts should be in the range 0–255. The color $(153, 102, 51)$ means quite a lot of red, somewhat less green and even less blue and it is perceived as brown.

a) In the lab you need these three brownish colors and it is convenient to collect them in their own namespace via a singleton object called `Color` as below.

```
object Color:
  val mole   = new java.awt.Color( 51,  51,   0)
  val soil   = new java.awt.Color(153, 102,  51)
  val tunnel = new java.awt.Color(204, 153, 102)
```

But we would prefer importing `java.awt.Color` so that we can use the class name without repeating the entire path, even though the name collides with the name of our singleton object. Rewrite the above code using import aliasing so that colors can

[14] https://docs.oracle.com/en/java/javase/21/docs/api/

be created with `new JColor(...)`. Make the import local inside the singleton object `Color`.

b) Inspired by the REPL experiments below, modify your program in `hello-window.scala` so that *three* overlapping color-filled squares are drawn as in the top picture to the right. Instead of drawing with the existing method `fill` from `PixelWindow`, you practice iteration by implementing the drawing procedures `straight` and `fill` yourself as below. The procedure `straight` should draw a horizontal line with left point `p` and length `d` pixels. The procedure `fill` should, with many horizontal lines, draw a filled square with upper left corner at point `p` and side `s` pixels. The drawing should look like the top picture to the right. If you for example include one pixel too many in your coordinate calculations, you may get the incorrect picture below.

```
> scala repl --dep se.lth.cs::introprog:1.4.0
scala> val w = new introprog.PixelWindow(400,300,"Three shades of brown")
scala> type Pt = (Int, Int)
scala> var color = java.awt.Color.red
scala> def straight(p: Pt)(d: Int) = w.line(p._1, p._2, ???, ???, color)
scala> def fill(p: Pt)(s: Int) = for i <- ??? do straight((p._1, ???))(s)

scala> object Color:
     |   ???

scala> color = Color.soil
scala> fill(100,100)(75)

scala> color = Color.tunnel
scala> fill(100,100)(50)

scala> color = Color.mole
scala> fill(150,150)(25)
```

c) On which calls above is the omission of tuple parentheses utilized?

Task 9. *Events.* In this week's lab, you will implement a simple game where the user can control a block mole with the keyboard. With `introprog.PixelWindow` you can handle events generated when the user presses or releases a keyboard or mouse button.

a) Study the documentation for the singleton object `introprog.PixelWindow.Event`. What is the name of the immutable integer variable that represents a key press event? What is its value?

b) From the documentation for singleton object `introprog.examples.TestPixelWindow` you can access the code that implements the object by clicking the `Source` link above the search box. Which line in the main program in the `main` method takes care of the case that a key press event has occurred?

c) Run with `scala run .` (where the dot stands for current directory) the main program in `TestPixelWindow` with options
`--main-class introprog.examples.TestPixelWindow` and
`--dep se.lth.cs::introprog:1.3.1`
A test window opens when `main` runs. Click in the window at different places and press different keys and observe what is printed. What is printed when the up arrow key is pressed and released?

d) Inspired by the implementation of `TestPixelWindow`, write a program that draws green lines between the positions for each mouse button press and mouse button release the user makes.
Tip: When the mouse button is pressed, save the position in a variable named `start`. When the mouse is released, draw the line from the saved position to `w.lastMousePos`.

4.2 Extra Tasks; additional practice

Task 10. *Functions are objects with an `apply` method.*
The method `apply` is special.

```
scala> object plus { def apply(x: Int, y: Int) = x + y }
scala> plus.apply(42, 43)
```

Is it possible to omit `.apply` and call `plus` as a function?

Task 11. *Create modules using singleton objects.*

a) Investigate in REPL what the expression `"pearIceScream".split('i')` evaluates to.

b) What is printed if you call `Test()` which calls the `apply` method below?

```
object stringUtils:
  object split:
    def sentences(s: String): Array[String] = s.split('.')
    def words(s: String): Array[String] = s.split(' ').filter(_.nonEmpty)

  object count:
    def letters(s: String):   Int = s.count(_.isLetter)
    def words(s: String):     Int = split.words(s).size
    def sentences(s: String): Int = split.sentences(s).size

  object statistics:
    var history = ""
    def printFreq(s: String = history): Unit =
      println(s"\n--- FREQUENCY ANALYSIS OF:\n$s")
      println(s"# letters : ${count.letters(s)}")
      println(s"# words   : ${count.words(s)}")
      println(s"# sentences: ${count.sentences(s)}")
      history = (s"$history $s").trim

object Test:
  import stringUtils.*
  def apply(): Unit =
    val s1 = "Five ants are more than four elephants. Eat cucumber."
    val s2 = "Galaxies in my breams. Tomato is good. Pear split."
    statistics.printFreq(s1)
    statistics.printFreq(s2)
    statistics.printFreq()
```

c) Which of the objects in the module `stringUtils` has state? Is it mutable?

d) Change the methods in the singleton object `count` to become extension methods so they can be called like this:

```
scala> import stringUtils.count

scala>  val s = "Hejsan hoppsan. Gurka är gott."
val s: String = Hejsan hoppsan. Gurka är gott.

scala>  (s.nbrOfLetters, s.nbrOfWords, s.nbrOfSentences)
val res0: (Int, Int, Int) = (24,5,2)
```

Task 12. *Tuples as parameters.* Implement below different variants of calculating the distance between two points. *Tip:* Use `math.hypot`.

```
def distxy(x1: Int, y1: Int, x2: Int, y2: Int): Double = ???
def distpt(p1: (Int, Int), p2: (Int, Int)):     Double = ???
def distp(p1: (Int, Int))(p2: (Int, Int)):      Double = ???
```

Task 13. *Tuples as function results.* Tuples enable a function to return several different values simultaneously. Implement the function statistics below. It should return a 3-tuple containing the number of elements in `xs`, the average of the elements, and a 2-tuple with the range (min,max). Specify the return type explicitly in your implementation. Test that it works in REPL. *Tip:* You can use methods `size`, `sum`, `min` and `max` which work on numeric sequences.

```
/** Returns the size, the mean, and the range of xs */
def statistics(xs: Vector[Double]) = ???
```

Task 14. *Create modules using packages.*

a) The code below is in the file `paket.scala`. Draw a picture of the directory structure that is created in the current library under subdirectory `main` in `.scala-build` when the code below is compiled with: `scala compile paket.scala`

```
package gurka.tomat.banan

package p1:
  package p11:
    object hello:
      def hello = println("Hej paket p1.p11!")
  package p12:
    object hello:
      def hello = println("Hej paket p1.p12!")

package p2:
  package p21:
    object hello:
      def hello = println("Hej paket p2.p21!")

object Main:
  def main(args: Array[String]): Unit =
    import p1.*
    p11.hello.hello
    p12.hello.hello
    import p2.{p21 as orange}
    orange.hello.hello
```

b) What is printed when the program is run?

c) Are packages allowed to have state variables without placing them inside a singleton object or a class?

4.3 Advanced Tasks; challenges

Task 15. *How to do without* **do while** *in Scala 3?* In many languages there is a construct with the following syntax: **do** `<statements>` **while** `<condition>` where the <statements> run at least once before the condition is tested. This "backwards while" is not used often but can be convenient if you want to run a repetition at least once.
This construct exists in Scala 2 but not Scala 3 because the keyword **do** in Scala 3 is used for optional braces and indentation syntax in "regular while". One reason it is considered okay to remove **do** `<statements>` **while** `<condition>` is that in Scala 3 the "backwards while" can be rewritten as a regular while by enclosing the statements to be done at least once in a block at the condition place and having the loop body be the empty value, i.e.:

```
while
  <statements>
  <condition>
do ()
```

a) The below works in Scala 2, but what happens if you try to do this in Scala 3:

```
> scala repl --scala 2
Welcome to Scala 2.13.8 (OpenJDK 64-Bit Server VM, Java 17.0.3).
Type in expressions for evaluation. Or try :help.

scala> var i = 0
var i: Int = 0

scala> do i += 1 while (i < 10)

scala> i
val res20: Int = 10
```

b) Rewrite the "backwards" **do while** to an equivalent "regular" **while do** that works in Scala 3.

Task 16. *Postfix operators for increment and decrement.* In many languages, e.g. Java, C++, C, you can write `i++` and `i--` to increment or decrement an integer variable `i`. Use Scala's extension methods to make it possible to use the operators ++ and `--` on integers as below:

```
scala> 42.++
val res0: Int = 43

scala> 42.--
val res1: Int = 41

scala> import language.postfixOps    // allows postfix operator notation

scala> 43 ++
val res2: Int = 44

scala> 43 --
val res3: Int = 42
```

```
scala> val i = 42
val i: Int = 42

scala> i++
val res4: Int = 43

scala> i--
val res5: Int = 41
```

Task 17. *Use ready package: Color chooser.* In the lab it is useful to be able to mix your own colors so you can draw clear blue sky and lush grass. You can create a color chooser with the `introprog` package as below.

```
> scala repl --dep se.lth.cs::introprog:1.3.1
scala> introprog.Dialog.selectColor()
```

a) What happens if you press **Ok** after selecting a green color?

b) What happens if you press **Cancel** ?

c) What happens if you press **Reset** ?

d) Read the documentation for the method `selectColor` in the singleton object `Dialog` in the package `introprog`. Call `selectColor` with the default color `java.awt.Color.green`.

Task 18. *Use ready package: user dialogs.*

a) Read the documentation for the singleton object `Dialog` in the package `introprog`.

b) Use the procedure `introprog.Dialog.show` and give a message to the user that it is `"Game over!"`.

c) Use the function `introprog.Dialog.input` to ask the question `"What is your name?"` and find out the user's name. What happens if the user clicks *Cancel*?

d) Use the function `introprog.Dialog.select` to ask the user to choose among rock, scissors and paper. What is the return type?

★ **Task 19.** *Create your own `jar` file.*

a) Write the command `jar` in terminal and check with `jar --help` what options exist. Which options should you use to create a jar in a named file with verbose output about what happens?

b) Create with an editor in the file `hello.scala` a simple program that prints `"Hello package!"` or similar. The code should be in package `hello` and contain an object `Main` with a `main` method. Compile your file with the option `--destination .` so that your code ends up in the current directory instead of in `.scala-build`.

c) Write a jar command in terminal that packages the code into a jar file named `my.jar` and start REPL with the jar on the classpath. Call your `main` function in REPL by specifying the path *`packagename.objectname.methodname`* with an empty array as argument.

d) With which command can you run the compiled and jar-packaged program directly in terminal (i.e. without starting REPL)?

★ **Task 20.** *How large is JDK8?* Find out using `http://stackoverflow.com/` how many classes and packages the Java platform JDK8 includes.

5. Exercise `Excercise05`

Goals

- ☐ Be able to declare classes with class parameters.
- ☐ Be able to create instances with and without **`new`**.
- ☐ Be able to provide arguments during instantiation.
- ☐ Understand the meaning of reference variables and the value **`null`**.
- ☐ Be able to use the keyword **`private`** to control the visibility of attributes and constructor parameters.
- ☐ Understand the purpose of getters and setters.
- ☐ Be able to explain access rules for companion objects.
- ☐ Be able to create a factory method in a companion object.
- ☐ Be aware of the usefulness of a private constructor.
- ☐ Understand the difference between reference equality and structural equality.
- ☐ Be aware of the difference between `==` and `eq`, as well as `!=` and `ne`.
- ☐ Be able to explain how case classes handle instantiation.
- ☐ Be aware of how case classes handle equality.

Preparations

- ☐ Study the concepts in chapter 5

5.1 Basic Tasks; preparation for the lab

Task 1. *Match concepts with description.*

Connect each concept with the (simplified) description that fits best:

class	1	A	indirect assignment to attribute value
instance	2	B	instances are considered different even if states are equal
constructor	3	C	keyword for direct instance creation of a class
class parameter	4	D	sees private members of class with same name
reference equality	5	E	helper function for indirect construction
structural equality	6	F	no need to write new; automatic structural equality
case class	7	G	a value that does not refer to any instance
getter	8	H	template for creating multiple instances of the same type
setter	9	I	copy of an object with its own state memory
companion object	10	J	instances considered equal if they have the same state
factory method	11	K	binds to arguments given at construction
`null`	12	L	indirect access to attribute value
`new`	13	M	creates instance, allocates space for state memory

Task 2. *Class and instance.* You have in exercise `Excercise04` seen how singleton objects in their own namespace can collect functions (methods) and have state (attributes). But singleton objects exist only in one copy. If you want to create many objects of the same type you need a *class*. An object instance created from a class is

called an *instance* of the class. Each instance has its own state. Declare the singleton object and the class below and paste into REPL.

```
object Singelpunkt { var x = 1; var y = 2 }
class  Punkt       { var x = 3; var y = 2 }
```

a) Suppose the expressions on the left are evaluated from top to bottom. Which result on the right corresponds to each expression? Try in REPL if you are unsure.[15]

`Singelpunkt.x`	1	A	`java.lang.NullPointerException`
`Punkt.x`	2	B	`1`
`val p  = new Singelpunkt`	3	C	`Not found: type`
`val p1 = new Punkt`	4	D	`p1: Punkt = Punkt@27a1a53c`
`val p2 = Punkt()`	5	E	`3`
`{ p1.x = 1; p2.x }`	6	F	`p2: Punkt = Punkt@51ab04bd`
`(new Punkt).y`	7	G	`value is not a member of object`
`{ val p: Punkt = null; p.x }`	8	H	`2`

b) In three cases there is an error. Why? Is it a compile-time error or runtime error?

Task 3. *Class parameters.* The class Punkt in the previous task is not so handy to use because you *after* instantiation can only give the attributes `x` and `y` the coordinate values you want, and this must be done with explicit assignment statements.
You can solve this problem with *class parameters* that allow you to initialize attributes with constructor arguments and thereby specify an initial state directly at instantiation.
Declare the class below in the REPL.

```
class Point(var x: Int, var y: Int)
```

a) Suppose the expressions on the left are evaluated from top to bottom. Which result on the right corresponds to each expression? Try in REPL if you are unsure.

`val p1 = Point(1, 2)`	1	A	`missing argument for parameter`
`val p2 = Point()`	2	B	`2`
`val p2 = Point(3, 4)`	3	C	`p1: Point = Point@30ef773e`
`p2.x - p1.x`	4	D	`too many arguments for constructor`
`Point(0, 1).y`	5	E	`p2: Point = Point@218cf600`
`Point(0, 1, 2)`	6	F	`1`

b) In two cases there is an error. Why? Is it a compile-time error or runtime error?

Task 4. *Immutable class with default arguments.* What applies to parameters and arguments to functions also applies to class parameters, e.g. default arguments and

[15] The string after the `@`-sign is a hexadecimal representation of the integer assigned to each object so that the system can distinguish different instances. https://stackoverflow.com/questions/4712139

named arguments. One can *additionally* put the keywords `var` and `val` before class parameters and then the parameter becomes a visible attribute. If one wants private attributes one can specify e.g. `private val` before the class parameter name. If nothing is specified before a class parameter, the most restrictive visibility `private[this] val` applies, meaning that the name is only visible within the current instance[16].
Declare below the class in REPL.

```
class Point3D(val x: Int = 0, val y: Int = 0, z: Int = 0)
```

a) Suppose the expressions on the left are evaluated from top to bottom. Which result on the right corresponds to each expression? Try in REPL if you are unsure.

`val p1 = Point3D()`	1	A	`false`
`val p2 = Point3D(y = 1)`	2	B	`Reassignment to val`
`Point3D(z = 2).z`	3	C	`p1: Point3D = Point3D@2eb37eee`
`p2.y = 0`	4	D	`true`
`p2.y == 0`	5	E	`value cannot be accessed`
`p1.x == Point3D().x`	6	F	`p2: Point3D = Point3D@65a9e8d7`

b) What is the problem with the above class if you want to use it to represent points in 3 dimensions?

Task 5. *Case class, `this`, equality, `toString` and companion object.*
Paste the classes below into REPL.

```
case class Pt(x: Int = 0, y: Int = 0):
  def moved(dx: Int = 0, dy: Int = 0): Pt = Pt(x + dx, y + dy)

class MutablePt(private var p: (Int, Int) = (0, 0)):
  def x: Int = p._1
  def y: Int = p._2
  def move(dx: Int = 0, dy: Int = 0) = { p = (x + dx, y + dy); this }
  override def toString = s"MPt($x,$y)"
```

a) Suppose the expressions on the left are evaluated from top to bottom. Which REPL responses on the right correspond to each expression? Try in REPL if you are unsure.

`val p1 = Pt(1, 2)`	1	A	`MPt(5,6)`
`val p2 = Pt(y = 3)`	2	B	`false`
`val p3 = MutablePt(5, 6)`	3	C	`Pt(0,3)`
`val p4 = Mutable()`	4	D	`Not found`
`p2.moved(dx = 1) == Pt(1, 3)`	5	E	`Pt(1,2)`
`p3.move(dy = 1) == MutablePt(5, 7)`	6	F	`true`

[16] For case classes, as we shall soon see, `val` applies meaning visibility and immutability (and not `private[this] val`).

b) What return type will the compiler infer here for the function `MutablePt.move`?

c) What is the difference between instantiation with universal apply methods and instantiation with **new**? Is there any case where **new** must be used?

d) What are methods like **def** `x` and **def** `y` above called?

Task 6. *Implement parts of the classes `Pos`, `KeyControl`, `Mole` and `BlockWindow` needed during the lab `blockbattle1`.* In the next lab you will build upon the `blockmole` lab and make a game for two players where each player controls their *own* instance of a `blockmole`. We must then change `Mole` so that it becomes a class instead of a singleton object. Complete the classes below and test carefully so that they work.
All classes should belong to **package** `blockbattle` and be in separate files with the same name as the class, e.g. `Pos.scala`.
Tip: Keep a separate terminal window open and run Scala CLI with change watching according to the command below. Then your changed code is automatically compiled each time you save a scala file in the current directory.

```
scala compile . --watch
```

The option `--watch` can be shortened to `-w` instead.

a) During the lab it is convenient to be able to represent movable positions in a pixel window. Implement the case class `Pos` in a new terminal window as below so that it works according to the following REPL tests.

```
package blockbattle

case class Pos(x: Int, y: Int):
  def moved(delta: (Int, Int)): Pos = ???
```

Test that `Pos` works with REPL as below:

```
> scala repl .
Welcome to Scala 3.3.0 (17.0.6, Java OpenJDK 64-Bit Server VM).
Type in expressions for evaluation. Or try :help.

scala> blockbattle.Pos(1,2)
val res0: blockbattle.Pos = Pos(1,2)

scala> import blockbattle.*

scala> val p = Pos(1,2)
val p: blockbattle.Pos = Pos(1,2)

scala> p.moved(0,1)
val res1: blockbattle.Pos = Pos(1,3)
```

Also test calling `moved` on the class name, e.g. `Pos.moved(0,1)`. Does this work? Why/why not? How do method calls differ on methods in singleton objects vs. class instances?

b) During the lab it is convenient to be able to represent which keys correspond to the different directions a user can control their mole in. Complete the case class `KeyControl` as below so that it works according to the following REPL tests. The method `direction` should give a delta step in the correct `(x, y)` direction for a given key name. The method `has` should return **true** if the key name exists in any of the four directional keys in this `KeyControl` instance, otherwise **false**.

```scala
package blockbattle

case class KeyControl(left: String, right: String, up: String, down: String):
  def direction(key: String): (Int, Int) = ???

  def has(key: String): Boolean = ???
```

```
scala> import blockbattle.*

scala> val kc1 = KeyControl(right="d",left="a",up="w",down="s")
val kc1: blockbattle.KeyControl = KeyControl(a,d,w,s)

scala> val kc2 = KeyControl("Left","Right","Up","Down")
val kc2: blockbattle.KeyControl = KeyControl(Left,Right,Up,Down)

scala> kc2.left
val res0: String = Left

scala> kc2.has("a")
val res1: Boolean = false

scala> kc2.has("Up")
val res2: Boolean = true

scala> kc1.direction("a")
val res3: (Int, Int) = (-1,0)

scala> kc1.direction("s")
val res4: (Int, Int) = (0,1)

scala> kc1.direction("d")
val res5: (Int, Int) = (1,0)

scala> kc1.direction("w")
val res6: (Int, Int) = (0,-1)

scala> Pos(1,2).moved(kc1.direction("a"))
val res7: blockbattle.Pos = Pos(0,2)
```

c) Complete the class `Mole` as below. Mole is a class representing a block mole with mutable attributes for position, direction and score. Each instance also has immutable attributes that keep track of its name, color and which keys can be used to control the mole. Implement the class members one by one and test carefully with suitable test cases after each addition/bugfix. Create a main program e.g. in the file `Main.scala` with your tests creating instances and printing out attributes etc.

```scala
package blockbattle

class Mole(
  val name: String,
  var pos: Pos,
  var dir: (Int, Int),
  val color: java.awt.Color,
  val keyControl: KeyControl
):
```

```
  var points = 0

  override def toString =
    s"Mole[name=$name, pos=$pos, dir=$dir, points=$points]"

  /** If keyControl.has(key) then the direction dir is updated according to keyControl */
  def setDir(key: String): Unit = ???

  /** Updates dir to the opposite direction. */
  def reverseDir(): Unit = ???

  /** Updates pos to become nextPos */
  def move(): Unit = ???

  /** Returns the next position according to the direction dir without updating pos */
  def nextPos: Pos = ???
```

d) During the lab you need a class `blockbattle.BlockWindow` which with the help of `introprog.PixelWindow` offers block graphics. Each instance of `BlockWindow` should have an attribute that refers to a `PixelWindow` instance. This is called **aggregation** (eng. *aggregation*).[17]
To be able to compile and test your `BlockWindow` class you need to have the `introprog` package on the classpath. Download the file https://fileadmin.cs.lth.se/introprog.jar via your browser or with the command `curl` below (note the capital letter `O` and not a zero in option `-sLO`):

```
curl -o introprog.jar -sLO  https://fileadmin.cs.lth.se/introprog.jar
scala run . --jar introprog.jar
```

Then `introprog.jar` is automatically on the classpath.
Complete the class `BlockWindow` below. The method `setBlock` should using the method `pixelWindow.fill` fill a square area with side `blockSize` pixels at a certain position `pos` in block coordinates and with a certain color `color`. The method `getBlock` should using the method `pixelWindow.getPixel` return the color for the upper-left corner in the block at position `pos` in block coordinates.

```
package blockbattle

class BlockWindow(
  val nbrOfBlocks: (Int, Int),
  val title: String = "BLOCK WINDOW",
  val blockSize: Int = 14
):
  import introprog.PixelWindow

  val pixelWindow = new PixelWindow(
    nbrOfBlocks._1 * blockSize, nbrOfBlocks._2 * blockSize, title)

  def setBlock(pos: Pos, color: java.awt.Color): Unit = ???

  def getBlock(pos: Pos): java.awt.Color = ???

  def write(
    text: String,
```

[17] https://en.wikipedia.org/wiki/Object_composition#Aggregation

```
    pos: Pos,
    color: java.awt.Color,
    textSize: Int = blockSize
  ): Unit = pixelWindow.drawText(
      text, pos.x * blockSize, pos.y * blockSize, color, textSize)

  def nextEvent(maxWaitMillis: Int = 10): BlockWindow.Event.EventType  =
    import BlockWindow.Event._
    pixelWindow.awaitEvent(maxWaitMillis)
    pixelWindow.lastEventType match
      case PixelWindow.Event.KeyPressed   => KeyPressed(pixelWindow.lastKey)
      case PixelWindow.Event.WindowClosed => WindowClosed
      case _                              => Undefined

object BlockWindow:
  def delay(millis: Int): Unit = Thread.sleep(millis)

  object Event:
    trait EventType
    case class  KeyPressed(key: String) extends EventType
    case object WindowClosed            extends EventType
    case object Undefined               extends EventType
```

In the instructions to lab `blockbattle1` there are tips on how you can handle events in a `BlockWindow` using the method `nextEvent` above.

e) Make the main program in `Main.scala` draw some arbitrary blocks in an instance of the class `BlockWindow`. Also create a **`while`** `(!quit)` loop which using `nextEvent()` prints events to the terminal that are not of type `Undefined`.
The method `nextEvent()` is in the class `BlockWindow`. Each loop round should also contain a `200` milliseconds delay by calling the `delay` method defined in the companion object `BlockWindow` above. If the event `WindowClosed` occurs the loop should terminate. Run the main program and check that the result is as expected.

5.2 Extra Tasks; additional practice

Task 7. *Instantiation with* ***`new`*** *and the value* ***`null`***. Instances of classes are created with **`new`**. Then the constructor is called and space is reserved in the computer's memory for the object. Variables of reference type that do not refer to any object have the value **`null`**.

a) What happens below? Which lines give error messages and if so, what does the error say?

```
1 scala> class Cucumber(val weight: Int)
2 scala> var g: Cucumber = null
3 scala> g.weight
4 scala> g = new Cucumber(42)
5 scala> g.weight
6 scala> g = null
7 scala> g.weight
```

b) Draw the memory situation after lines 2, 4, 6.

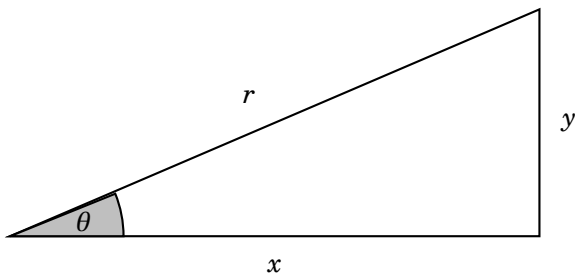
Task 8. *Create a point class that can handle polar coordinates and a class that represents a polygon using these points.* You shall create an immutable case class `Point` which can represent a coordinate both with "normal" Cartesian coordinates[18] and with polar coordinates [19]. Then you shall use this class to create regular polygons with an immutable case class `Polygon`.

a) Create code with the help of an editor, e.g. VS Code, in the file `Point.scala` following these guidelines:

1. `Point` shall belong to the package `graphics`.
2. `Point` shall have the following two public, immutable class parameters:
   - `x: Double` for the x-coordinate.
   - `y: Double` for the y-coordinate.
3. `Point` shall have the following public members (two immutable attributes and one method):
   - `val r: Double` shall give the corresponding polar coordinate's distance to origin.
   - `val theta: Double` shall give the polar coordinate's angle in radians.
   - `def +(p: Point): Point` shall give a new point whose coordinate is the sum of the x and y coordinates of this instance and point `p`.
4. `Point` shall have a companion object with a method that constructs a point from polar coordinates. The method shall have this header:
   `def polar(r: Double, theta: Double): Point`

Tips:

- You will find useful the methods $r =$ `math.hypot(x, y)` and $\theta =$ `math.atan2(y, x)` when converting to polar coordinates:



- You will find useful the methods `math.cos(theta)` and `math.sin(theta)` when converting from polar coordinates.

- Note that the class's attributes are of type `Double` and not `Int`, even though later you will use the point to describe a discrete pixel position in a `PixelWindow`. The reason for this is that rounding errors may occur during numerical computations. This is particularly noticeable during repeated calculations with small values, e.g. when drawing an approximated circle with many line segments.

b) The class `PolygonWindow` below contains a `PixelWindow` and provides the ability to draw polygons. Copy the code for `PolygonWindow` to a new code file `PolygonWindow.scala` in the same directory as your `Point` above.

[18] https://sv.wikipedia.org/wiki/Kartesiskt_koordinatsystem
[19] https://sv.wikipedia.org/wiki/Pol%C3%A4ra_koordinater

```
//> using dep "se.lth.cs::introprog:1.4.0"
package graphics

import introprog.PixelWindow
import java.awt.Color

extension (p: Point) def toPixels: Seq[Int] =
  Seq(p.x.round.toInt, p.y.round.toInt)

class PolygonWindow:
  val black     = Color(0, 0, 0)
  val coolGreen = Color(0, 255, 111)
  val width     = 500
  val height    = 500

  val window =
    PixelWindow(width, height, title = "Polygons",
                background = black, foreground = coolGreen)

  def draw(polygon: Polygon): Unit =
    for i <- 0 until polygon.nbrOfCorners do
      val from = polygon.points(i).toPixels.map(_ + width / 2)
      val next = polygon.points((i+1) % polygon.nbrOfCorners)
      val to = next.toPixels.map(_ + width / 2)
      window.line(from(0), from(1), to(0), to(1), lineWidth = 2)
```

Create a case class named `Polygon` with a parameter `points: Vector[Point]` and an attribute `val nbrOfCorners: Int`. The case class `Polygon` shall also be in the package `graphics`.
Like the class `Point` above, `Polygon` shall have a companion object. The companion object shall have a method `regular` which creates a regular polygon. The method shall have the following parameters: `nbrOfCorners: Int, radius: Double, midPoint: Point`
Consider how the case class `Polygon` and its companion object should look for the code above in `PolygonWindow` to work as intended. Test that everything works in REPL.

c) Can you use the method `regular` to draw circles? Can you use `Polygon` to represent irregular polygons? Test in REPL.

Task 9. *Classes, instances and garbage.* Long ago in a galaxy far far away...

```
case class Arm(isToLeft: Boolean)

case class Leg(isToLeft: Boolean)

case class Head(hasHair: Boolean = true)

case class SpaceCreature(
      arm1:   Arm   = Arm(true),
      arm2:   Arm   = Arm(false),
      leg1:   Leg   = Leg(true),
      leg2:   Leg   = Leg(false),
```

```
      head1: Head = Head(hasHair = false),
  var head2: Head = Head()
):
  def isBald = !head1.hasHair && !head2.hasHair
```

a) Paste the above space code in REPL and evaluate the lines below. Draw the memory situation after line 5 and describe what happens.

```
1 scala> val alien = SpaceCreature()
2 scala> alien.isBald
3 scala> val predator = SpaceCreature()
4 scala> predator.isBald
5 scala> predator.head1 = alien.head1
6 scala> predator.head2 eq alien.head1  // test of reference equality
7 scala> println(predator)
8 scala> predator.isBald
```

b) What happens eventually to the original head2 object in predator after the assignment on line 5? Is it possible to refer to this object in some way?

Task 10. *Case class. Immutable square.*

a) Implement the square below with an editor and paste it in REPL.

```
case class Square(val x: Int = 0, val y: Int = 0, val side: Int = 1):
  val area: Int = ???

  /** Creates a new Square moved to position (x + dx, y + dy) */
  def moved(dx: Int, dy: Int): Square = ???

  def isEqualSizeAs(that: Square): Boolean = ???

  /** Multiplies the side with factor and rounded to nearest integer */
  def scale(factor: Double): Square = ???

object Square:
  /** A Square at (0, 0) with side 1 */
  val unit: Square = ???
```

b) Test your square as below. Explain what happens.

```
scala> val (s1, s2) = (Square(), Square(1, 10, 1))
scala> val s3 = s1 moved (1,-5)
scala> s1 isEqualSizeAs s3
scala> s2 isEqualSizeAs s1
scala> s1 isEqualSizeAs Square.unit
scala> s2.scale(math.Pi) isEqualSizeAs s2
scala> s2.scale(math.Pi) isEqualSizeAs s2.scale(math.Pi)
scala> s2.scale(math.Pi) eq s2.scale(math.Pi)
scala> Square.unit eq Square.unit
```

5.3 Advanced Tasks; challenges

Task 11. *Content equality between different types.* Paste below classes into REPL and investigate what happens.

```
class Cucumber(val weight: Int)

class Car(val typ: String)
```

```
scala> class Cucumber(val weight: Int)
     |
     | class Car(val typ: String)
// defined class Cucumber
// defined class Car

scala> 42 == "Fourtytwo"

scala> Cucumber(50) == Car("Sedan")
```

Is there any result that is problematic, and if so, why?

Task 12. *Attribute representation. Private constructor. Factory method.* Kim Kodkunnig long ago created this class which is used in many places in existing code:

```
class Point private (val x: Int, val y: Int)
object Point:
  def apply(x: Int = 0, y: Int = 0): Point = new Point(x, y)
  val origo = apply()
```

a) What happens if you try to instantiate Kim Kodkunnig's class directly with the keyword `new`?

b) Why does Kim Kodkunnig use a companion object with a factory method? What access rules apply between a companion object and a class with the same name?

c) Help Kim Kodkunnig change the attribute representation so that the immutable state consists of a 2-tuple `val p: (Int, Int)` instead. Existing code should not have to change and the class `Point` should behave from the "outside" just like before.

Task 13. *Visibility of class parameters and constructor,* ***`private`****`[this]`*.

a) One of the cucumber classes below is broken. Why and what error do you get?

```
class Cucumber1(weight: Int)

class Cucumber2(val weight: Int)

class Cucumber3(private val weight: Int)

class Cucumber4(private val weight: Int, buddy: Cucumber4):
  def buddyWeight = buddy.weight

class Cucumber5(private[this] val weight: Int, buddy: Cucumber5):
  def buddyWeight = buddy.weight
```

```
class Cucumber6 private (weight: Int)

class Cucumber7 private (var weight: Int)
object Cucumber7:
  def apply(weight: Int) =
    require(weight >= 0, "negative weight: " + weight)
    new Cucumber7(weight)
```

b) Investigate below what effect the keywords **val** and **private** have. Explain what happens. Which lines give which error messages?

```
scala> new Cucumber1(42).weight
scala> new Cucumber2(42).weight
scala> new Cucumber3(42).weight
scala> val noneCucumber: Cucumber4 = null
scala> new Cucumber4(42, noneCucumber).buddyWeight
scala> new Cucumber4(42, new Cucumber4(84, null)).buddyWeight
scala> new Cucumber6(42)
scala> new Cucumber7(-42)
scala> Cucumber7(-42)
scala> val g = Cucumber7(42)
scala> g.weight
scala> g.weight = -1
scala> g.weight
```

Task 14. *User-defined setter combined with private constructor.* Paste this code in REPL:

```
class Cucumber8 private (private var _weight: Int):
  def weight = _weight
  def weight_=(v: Int): Unit =
    require(v >= 0, "negative weight: " +v)
    _weight = v

object Cucumber8:
  def apply(weight: Int) =
    require(weight >= 0, "negative weight: " + weight)
    new Cucumber8(weight)
```

a) Explain what happens below. Which lines produce which error messages?

```
scala> val g = Cucumber8(-42)
scala> val g = Cucumber8(42)
scala> g.weight
scala> g.weight = 0
scala> g.weight = -1
scala> g.weight += 42
scala> g.weight -= 1000
```

b) What is the advantage of being able to create user-defined setters?

Task 15. *Object with mutable state.* You shall implement a model of a jumping frog fulfilling the following requirements:

1. Each frog object shall keep track of where it is.
2. Each frog object shall keep track of how far the frog has jumped in total.
3. Each frog object shall be able to calculate how far it is between the frog's current position and the starting position.
4. All frogs start their jumping at the origin.
5. A frog can jump according to two methods:
   - relative move according to the parameters `dx` and `dy`,
   - random relative move $[1, 10]$ in x-direction change and $[1, 10]$ in y-direction change.

a) Implement the class `Frog` according to the code skeleton below and the above requirements.

```
class Frog private (initX: Int = 0, initY: Int = 0):
  def x: Int = ???
  def y: Int = ???

  def jump(dx: Int, dy: Int): Unit = ???
  def randomJump: Unit = ???

  def distanceToStart: Double = ???
  def distanceJumped: Double = ???
  def distanceTo(that: Frog): Double = ???

object Frog:
  def spawn(): Frog = ???
```

Tips:

- If the name you want to give a private mutable attribute "collides" with a method name, it is common to prefix the attribute name with underscore, e.g. `private var _x` to avoid name conflict.
- Introduce one method at a time and paste the new frog in REPL after each extension and test.

b) Create a method `def test(): Unit` in a singleton object `FrogTest` that contains code that does at least one check for each requirement. If no check fails, `"Test Ok!"` should be printed, otherwise execution should abort. *Tip:* Use `assert(b, msg)` which aborts execution and prints `msg` if `b` is false.

c) What is a method called that only returns the value of a private attribute?

d) Introduce setters for the attributes tracking the x and y position. Changes of position in x- or y-direction should count as a jump and thus be recorded in the attribute that holds the accumulated jump distance.

e) Simulate massive frog jumping with collision detection by creating 100 frogs initially placed on the x-axis at distance 8 units apart. For each round in a `while` loop, let a randomly selected frog perform a `randomJump` until some frog pair is closer than 0.5 length units, which is the definition of collision. Count how many loop rounds are needed before any frog pair collides and print the number. Also print the total number of

Tips: Start with pseudocode on paper. Use a frog vector.

Task 16. *Object with mutable state.* The webshop **UberSquare** sells movable squares. In the business model it includes charging per movement. You shall help UberSquare develop a simple prototype to impress venture capitalists.

a) Implement `Square` according to the documentation comments in the following code sketch and according to these requirements:

1. Each instance of `Square` shall count the number of moves made since the instance was constructed.
2. To monitor their customers UberSquare also want to count the total number of moves made by all squares ever created (so-called *big data*).
3. Each time a move happens a certain amount shall be added to the accumulated cost for the respective square, according to the cost calculation in requirement 4.
4. UberSquare worries that the squares get moved too far away and therefore decides that for each move the accumulated square cost is increased by the new position's distance to the origin at square construction multiplied by the current size of the square.
5. To appear generous UberSquare advertises that scaling squares is free. [20]

```
/** A mutable and expensive Square. */
class Square private (val initX: Int, val initY: Int, val initSide: Int):
  private var nMoves = 0
  private var sumCost = 0.0

  private var _x = initX
  private var _y = initY

  private var _side = initSide

  private def addCost(): Unit =
    sumCost += ???

  def x: Int = ???
  def y: Int = ???

  def side = ???

  /** Scales the side of this square and rounds it to nearest integer */
  def scale(factor: Double): Unit = ???

  /** Moves this square to position (x + xd, y + dy) */
  def move(dx: Int, dy: Int): Unit = ???

  /** Moves this square to position (x, y) */
  def moveTo(x: Int, y: Int): Unit = ???

  /** The accumulated cost of this Square */
  def cost: Double = ???

  /** Returns the accumulated cost. Sets the accumulated cost to zero. */
```

[20] That is, a call to the method `scale` does not cause any immediate cost.

```
  def pay: Double = ???

  override def toString: String =
    s"Square[($x, $y), side: $side, #moves: $nMoves times, cost: $sumCost]"


object Square:
  private var created = Vector[Square]()

  /** Constructs a new Square object at (x, y) with size side */
  def apply(x: Int, y: Int, side: Int): Square =
    require(side >= 0, s"side must be positive: $side")
    ???

  /** Constructs a new Square object at (0, 0) with side 1 */
  def apply(): Square = ???

  /** The total number of moves that have been made for all squares. */
  def totalNumberOfMoves: Int = ???

  /** The total cost of all squares. */
  def totalCost: Double = ???
```

b) Test your square prototype in REPL. Use e.g. the code below:

```
scala> val xs = Vector.fill(10)(Square())
scala> xs.foreach(_.move(2, 3))
scala> xs.foreach(_.scale(2.9))
scala> val (m, c) = (Square.totalNumberOfMoves, Square.totalCost)
val m: Int = 10
val c: Double = 36.055512754639885
```

Task 17. *Auxiliary constructor.* In previous tasks we have enabled alternative ways to create instances through default arguments and factory methods in companion objects. Another way to do this, which in Scala is unusual[21], is to define several constructors inside the class body. In Scala, such an extra constructor is called an **auxiliary constructor**. ★

You create an auxiliary constructor in Scala by defining a method with the special name `this`, i.e. a declaration `def this(...) = ...`. Auxiliary constructors must begin by calling another constructor, either the primary constructor (i.e. the one defined in the class header) or a previously defined auxiliary constructor.

a) Read more about auxiliary constructors here:
www.artima.com/pins1ed/functional-objects.html#6.7

b) Invent your own task with auxiliary constructors, based on one of the classes from previous exercises.

[21] But in Java this is very common since default arguments etc. are not part of the language.

6. Exercise Excercise06

Goals

- ☐ Be able to create and use **match** expressions with constant values, guards, and pattern matching with case classes.
- ☐ Be able to create and use case objects for matching on enumerated values.
- ☐ Be able to handle missing values using the `Option` type and pattern matching on `Some` and `None`.
- ☐ Be able to catch exceptions with `scala.util.Try`.
- ☐ Know about **try**, **catch**, and **throw**.
- ☐ Know the keyword **sealed** and understand the usefulness of sealed types.

Preparations

- ☐ Study the concepts in chapter 6

6.1 Basic Tasks; preparation for the lab

Task 1. *Match on constant values.*

a) Write the program below with a code editor and save it in the file `Match.scala`. Compile and run and give as argument your favorite vegetable. What happens? Explain how a **match** expression works.

```
object Match:
  def main(args: Array[String]): Unit =
    val favorite = if args.length > 0 then args(0) else "selleri"
    println("Your favourite vegetable: " + favorite)
    val firstChar = favorite.toLowerCase.charAt(0)
    val meThink = firstChar match
      case 'c' => "cucumber is good!"
      case 't' => "tomato is good!"
      case 'b' => "broccoli is good !"
      case  _  =>  s"$favorite is less good..."
    println(s"I think that $meThink")
```

b) What error message do you get if you remove the case branch for default values and input is chosen such that no case branch matches? Is it a runtime error or a compile-time error?

Task 2. *Guards in case branches.* With the help of a guard in a case branch, you can limit with a condition whether the branch should be chosen.
Start from the code in task 1a and replace the case branch for `'c'` matching with the variant below with a guard using the keyword **if** (note that parentheses around the condition are not needed):

```
    case 'c' if math.random() > 0.5 => "cucumber is sometimes good..."
```

Recompile and run the program repeatedly with different inputs until all branches in the **match** expression have executed. Explain what happens.

Task 3. *Pattern match on attributes in case classes.* Scala's `match` expression is extra powerful if used with `case` classes: then attributes can be automatically extracted and bound to local variables directly in the case branch as the example below shows (note that `w` and `rotten` do not need to be explicitly declared). This is called **pattern matching**. What is printed below? Why? Try changing the names `w` and `rotten`.

```
scala> case class Cucumber(weight: Int, isRotten: Boolean)
scala> val g = Cucumber(100, true)
scala> g match { case Cucumber(w,rotten) => println("G" + w + rotten) }
```

Task 4. *Match on case objects and the usefulness of* `sealed`. Write the following lines of code in a REPL one by one. Note the keyword `sealed`, which is used to seal a type. A **sealed type** must have all its subtypes in the same source file.

```
scala> sealed trait Color
scala> case object Spades extends Color
```

a) What is the error message and why does it occur? Is it a compile-time or a runtime error?

b) Now create the code below in an editor and paste it in the REPL.

```
object Deck:
  sealed trait Color
  object Color:
     val values = Vector(Spades, Hearts, Diamonds, Clubs)
  case object Spades extends Color
  case object Hearts extends Color
  case object Diamonds extends Color
  case object Clubs extends Color
```

c) Create a function `def paracolor(f: Color): Color` in an editor, which by means of a match expression returns the parallel color for a color. The parallel color to `Hearts` is `Diamonds` and vice versa, while the parallel color to `Clubs` is `Spades` and vice versa. Paste the function in the REPL. Also take the opportunity to write an `import` statement for the outer object **Deck**, so that the members of the object can be reached easily.

```
scala> paracolor(Spades)
scala> val xs = Vector.fill(5)(Color.values((math.random() * 4).toInt))
scala> xs.map(paracolor)
```

d) Now we will investigate what happens if you forget one of the case branches in the match in `paracolor`. Intentionally "forget" one of the case branches and paste the new `paracolor` with the incomplete match. What is the warning? Does the warning occur at runtime or compilation?

e) Call `paracolor` with the "forgotten" color. What is the error message? Is it a compile-time or runtime error?

f) Explain what the keyword `sealed` means and what benefit there is of **sealing** a supertype.

Task 5. *Pattern match enumeration.* Now we will investigate and compare the difference between the keywords **enum** and **sealed trait**. Write the code below in a REPL.

```
enum Color:
  case Spades, Hearts, Diamonds, Clubs
```

a) Create again with the help of an editor a function **def** `paracolor(f: Color): Color`, nearly identical to the one we created in subtask 4c. The function should again use a match expression to return the parallel color of the given argument. Note that this time `Color` is not a **sealed trait**, but instead an enumeration (**enum**). Paste the function in the REPL.

```
scala> paracolor(Color.Diamonds)
scala> val xs = Vector.fill(5)(Color.values((math.random() * 4).toInt))
scala> xs.map(paracolor)
```

b) Reflect on the differences and similarities between using **sealed trait** with **case** objects versus using **enum** when pattern matching.

Task 6. *Meaning of lowercase and uppercase initial letters when matching.* To enable names to be bound to variables during matching without needing prior declaration (as we saw in task 3), identifiers starting with a lowercase letter have a special meaning: the compiler interprets it as you wanting a variable to be bound to a value during matching. An identifier starting with an uppercase letter is interpreted as a constant value (e.g. a case object or a case class pattern).

a) *A case branch that captures everything*. A case branch with an identifier starting with a lowercase letter without a guard will match everything. Try the following in REPL, but try to anticipate what will happen. What happens?

```
scala> val x = "ucumber"
scala> x match
         case str if str.startsWith("c") => println("maybe cucumber")
         case whatever => println("not cucumber: " + whatever)
scala> val c = "cucumber"
scala> c match
         case str if str.startsWith("c") => println("maybe cucumber")
         case whatever => println("maybe cucumber: " + whatever)
```

b) *Pitfall with lowercase initial letters.* Before trying below in REPL, try to guess what will happen. What happens? What are the warnings and what do they mean?

```
scala> val any: Any = "neither tomato nor cucumber"
scala> case object Cucumber
scala> case object tomato
scala> any match
         case Cucumber => println("cucumber")
         case tomato => println("tomato")
         case _ => println("everything else")
```

c) *Use backticks to force a match on a constant value.* There is a workaround if you don't want the compiler to create a new local variable: use the special character *backtick*, written ` and requiring special keyboard strokes.[22] Rewrite the previous

[22] Ask someone if you don't know how to type backtick ` on your keyboard.

task but enclose the identifier `tomato` in the tomato case branch in backticks, like this:
`case` `tomato` `=>` `...`

Task 7. *Match on the contents of a Vector.* Run the code below in REPL. What is printed? Explain what happens.

```
scala> val xss = Vector(Vector("hej"),Vector("on", "dough"),Vector("4","x","2")
scala> xss.map( _ match
  case Vector() => "tom"
  case Vector(a) => a.reverse
  case Vector(_, b) => b.reverse
  case Seq(a, "x", b) => a + b
  case _ => "OTHERWISE THIS"
  ).foreach(println)
```

Task 8. *Use `Option` and match on potentially missing values.* One often needs to write code to handle values that may be missing, e.g., missing phone numbers in a person database. This situation is common enough that many languages have special support for missing values.
In Java[23] the `null` value is used to indicate that a reference lacks a value. One must remember to test if the value is missing each time such a value is handled, e.g. using `if (ref != null) { ...} else { ... }`. Another common trick is to use `-1` to indicate missing positive integers, e.g., missing indices, which must be handled with `if (i != -1) { ...} else { ... }`.
In Scala, there is a special type `Option` that allows smooth and type-safe handling of missing values. If a possibly missing value is wrapped in an `Option`, it is in a special kind of collection that can only contain *none* or *some* value, thus has size either `0` or `1`.

a) Explain what happens below.

```
scala> var maybe: Option[Int] = None
scala> maybe.size
scala> maybe = Some(42)
scala> maybe.size
scala> maybe.isEmpty
scala> maybe.isDefined
scala> def increaseIfExists(opt: Option[Int]): Option[Int] = opt match
         case Some(i) => Some(i + 1)
         case None    => None
scala> val anotherMaybe = increaseIfExists(maybe)
scala> def increase(i: Int) = i + 1
scala> val moreMaybe = maybe.map(increase)
```

b) The pattern matching above is at least as clumsy as an `if` statement, but since `Option` is a kind of (small) collection there are smoother methods. Explain what happens below.

```
val sentence = Some(42)
val notSentence = Option.empty[Int]
sentence.map(_ + 1)
notSentence.map(_ + 1)
notSentence.map(_ + 1).orElse(Some("missing")).foreach(println)
sentence.map(_ + 1).orElse(Some("missing")).foreach(println)
```

[23] Scala also has `null` but it is only needed when interoperating with Java code.

c) *Collection methods that return an `Option`.* Explain what happens for each line below. One line gives an error message; which line and what error?

```
val xs = (42 to 84 by 5).toVector
val e = Vector.empty[Int]
xs.headOption
xs.headOption.get
xs.headOption.getOrElse(0)
xs.headOption.orElse(Some(0))
e.headOption
e.headOption.get
e.headOption.getOrElse(0)
e.headOption.orElse(Some(0))
Vector(xs, e, e, e)
Vector(xs, e, e, e).map(_.lastOption)
Vector(xs, e, e, e).map(_.lastOption).flatten
xs.lift(0)
xs.lift(1000)
e.lift(1000).getOrElse(0)
xs.find(_ > 50)
xs.find(_ < 42)
e.find(_ > 42).foreach(_ => println("FOUND!"))
```

d) What are the advantages of `Option` compared to **`null`** or `-1` if one forgets to handle missing values in code?

Task 9. *Throw exceptions.* If you want to signal that an error or abnormal situation has occurred, you can **throw** an **exception**. Then the program aborts immediately with an error message unless you **catch** the exception. a) What happens below?

```
scala> throw new Exception("PANG!")
scala> java.lang.   // Press TAB after the dot
scala> throw new IllegalArgumentException("wrong wrong wrong")
scala> val carola =
         try
           throw new Exception("stormWind!")
           42
         catch
           case e: Throwable =>
             println("Handled by a " + e)
             -1
```

b) Name a couple of exceptions in package `java.lang` that you can guess their meaning and in what situations they are thrown.

c) What type does variable `carola` have above? What would its type be if the catch branch returned a string instead?

Task 10. *Catch exceptions with `scala.util.Try`.* In package `scala.util` there is type `Try` (capital T), a kind of collection that can contain either a ”successful” or ”failed” value. If the computation succeeds and no exceptions are thrown, the value is wrapped in a `Success`, otherwise the thrown exception is wrapped in a `Failure`. One can extract the value or exception via pattern matching but it is usually more convenient to use collection methods `map` and `foreach`, like with `Option`. There is also a convenient method `recover` on `Try` where one can provide code that is run if an exception occurs.

a) Explain what happens below.

```
scala> def pang = throw new Exception("PANG!")
scala> import scala.util.{Try, Success, Failure}
scala> Try{pang}
scala> Try{pang}.recover{case e: Throwable =>   "disarmed bomb: " + e}
scala> Try{"silent"}.recover{case e: Throwable => "disarmed bomb: " + e}
scala> def maybePang = if math.random() > 0.5 then "silent" else pang
scala> def maybeOk = Try{maybePang}
scala> val xs = Vector.fill(100)(maybeOk)
scala> xs(13) match
         case Success(x) => ":)"
         case Failure(e) => ":( " + e
scala> xs(13).isSuccess
scala> xs(13).isFailure
scala> xs.count(_.isFailure)
scala> xs.find(_.isFailure)
scala> val badOpt = xs.find(_.isFailure)
scala> val goodOpt = xs.find(_.isSuccess)
scala> badOpt
scala> badOpt.get
scala> badOpt.get.get
scala> badOpt.map(_.getOrElse("bomb disarmed!")).get
scala> goodOpt.map(_.getOrElse("bomb disarmed!")).get
scala> xs.map(_.getOrElse("bomb disarmed!")).foreach(println)
scala> xs.map(_.toOption)
scala> xs.map(_.toOption).flatten
scala> xs.map(_.toOption).flatten.size
```

b) What is the return type of the function `pang`?

c) Why does the function `maybePang` have the inferred return type `String`?

6.2 Advanced Tasks; challenges

Task 11. *Use matching or dynamic binding?* One can implement the distinction of edible vegetables in task 3 via dynamic binding instead of **`match`**.

a) Make a new variant of your program as per the guidelines below, save the modified code in `vegopoly.scala` and compile and run.

- Remove the predicate `isWorthEating` from object `Main` and instead add an abstract method **`def`** `isWorthEating: Boolean` in the trait `Vegetable`.
- Add concrete **`val`** members in each vegetable class that define edibility.
- Modify the main program accordingly so that `isWorthEating` is invoked as a method on the harvested vegetable objects when filtering edible ones.

b) Add a new vegetable **`case class`** `Broccoli` and define its edibility. Modify the randomization functions so broccoli becomes less common than cucumber.

c) Compare the solution with **`match`** in task 3 and the solution above with polymorphism. What are pros and cons of each? Discuss two different scenarios at a hypothetical company developing software for the agricultural sector: 1) the set of vegetables does not change often but edibility definitions change very frequently, and 2) the vegetable set changes very often but edibility definitions rarely change.

Task 12. *Method equals.* If you override the existing `equals` method, then == will behave differently. You can achieve content equality instead of reference equality. We start by studying the existing equals with reference equality.

a) What happens below? Inspect parameter type and return type for `equals`.

```
1 scala> class Cucumber(val weight: Int, val isEatable: Boolean)
2 scala> val g1 = new Cucumber(42, true)
3 scala> val g2 = g1
4 scala> val g3 = new Cucumber(42, true)
5 scala> g1 == g2
6 scala> g1 == g3
7 scala> g1.equals  // press ENTER to see function type
```

b) Draw the memory situation after line 4.

c) *Override methods `equals` and `hashCode`.*

Background: It turns out surprisingly complicated to implement content equality with `equals` so that it works correctly in all edge cases. For example, one must also override a method called `hashCode` if one overrides `equals`, because these are used jointly for efficiency to create internal storage of objects in some collections. Missing this can cause objects to become "invisible" in hash-code-based collections – more about this in later courses. When objects are in an open inheritance hierarchy it is also more complicated; it is easier if dealing with final classes. Also, special considerations are needed if the class has type parameters.

Define the class below in REPL with overridden `equals` and `hashCode`; it does not inherit anything and is final.

```
// works well if the class is final and does not inherit anything
final class Cucumber(val weight: Int, val isEatable: Boolean):
  override def equals(other: Any): Boolean = other match
    case that: Cucumber => weight == that.weight && isEatable == that.isEatable
    case _ => false
  override def hashCode: Int = (weight, isEatable).## //explained later
```

d) What happens now below, where `Cucumber` now has overridden `equals` with content equality?

```
scala> val g1 = new Cucumber(42, true)
scala> val g2 = g1
scala> val g3 = new Cucumber(42, true)
scala> g1 == g2
scala> g1 == g3
```

e) How do you notice above that the overridden `equals` causes == now to give content equality? Compare with subtask a.
In task 18 you get to try the full 8-step recipe to override `equals` according to all the rules. In the following course there will be more training in handling content equality and hash codes. In Scala one gets an object's hash code via the method ##.[24]

Task 13. *Polynomial.* With the code below, one can do the following:

```
scala> import polynomial.*

scala> Const(1) * x
```

[24] If interested in hash codes, read more here: en.wikipedia.org/wiki/Hash_function

```
res0: polynomial.Term = x

scala> (x*5)^2
res1: polynomial.Prod = 25x^2

scala> Poly(x*(-5), y^4, (z^2)*3)
res2: polynomial.Poly = -5x + y^4 + 3z^2
```

a) Explain what happens above by studying the code below[25].

```scala
object polynomial:

  sealed trait Term:
    def *(that: Term): Term

  case class Const(value: BigDecimal) extends Term:

    def toSilentString: String = this match
      case Const.One      => ""
      case Const.MinusOne => "-"
      case _              => value.toString

    override def toString = value.toString

    override def *(that: Term): Term = that match
      case Const(d)    => Const(d * value)
      case v: Var      => Prod(this, Set(v))
      case Prod(c, vs) => Prod(Const(c.value * value), vs)

    def *(d: BigDecimal): Const = Const(d * value)

    def ^(e: Int): Const = Const(value.pow(e))


  object Const:
    final val Zero     = Const(BigDecimal(0))
    final val One      = Const(BigDecimal(1))
    final val MinusOne = Const(BigDecimal(-1))

  case class Var(name: Char, exp: Int = 1) extends Term:

    private def silentExpString: String =
      if exp == 1 then "" else "^"+exp.toString

    override def toString = s"$name$silentExpString"

    def ^(e: Int): Var = Var(name, e * exp)

    def *(c: BigDecimal) = Prod(Const(c), Set(this))

    override def *(that: Term): Term = that match
```

[25] The code is also available here:
github.com/lunduniversity/introprog/tree/master/compendium/examples/polynomial

```
      case c: Const => Prod(c, Set(this))

      case v: Var =>
        if v.name == name then Var(name, v.exp + exp)
        else Prod(Const.One, Set(this, v))

      case p: Prod => p * this


  object Var:

    def apply(d: BigDecimal, name: Char): Prod =
      Prod(Const(d), Set(Var(name)))

    def apply(d: BigDecimal, name: Char, exp: Int): Prod =
      Prod(Const(d), Set(Var(name, exp)))

    def addExp(v1: Var, v2: Var): Var = Var(v1.name, v1.exp + v2.exp)

    def multiply(v1: Var, vs: Set[Var]): Set[Var] =
      if !vs.contains(v1) then vs + v1
      else vs.map(v2 => if v1.name == v2.name then addExp(v1, v2) else v2)

    def multiply(vs1: Set[Var], vs2: Set[Var]): Set[Var] =
      var result = vs2
      vs1.foreach{ v1 => result = multiply(v1, result) }
      result


  case class Prod(const: Const, vars: Set[Var]) extends Term :

    override def toString = s"${const.toSilentString}${vars.mkString}"

    override def *(that: Term): Term = that match
      case Const(d) => Prod(Const(d * const.value), vars)

      case v: Var => Prod(const, Var.multiply(v, vars))

      case Prod(Const(d), vs)  =>
        Prod(Const(const.value * d), Var.multiply(vs, vars))

    def ^(e: Int) = Prod(const ^ e, vars.map(_ ^ e))

  case class Poly(xs: Set[Term]):
    override def toString = xs.mkString(" + ")

  object Poly:
    def apply(ts: Term*) : Poly = Poly(ts.toSet)

  val (x, y, z, s, t) = (Var('x'), Var('y'), Var('z'), Var('s'), Var('t'))
```

b) Build further on **object** `polynomial` and implement addition between different terms.

Task 14. *`Option` as a collection.* Study the documentation for `Option` here and see if you recognize any methods that also exist on the collection `Vector`:
www.scala-lang.org/api/current/scala/Option.html
Explain how the method `contains` on an `Option` works using the documentation examples.

Task 15. *Catch exceptions with `catch` in Java and Scala.* Make the analogous program in Scala as shown in task **??**, utilizing that Scala's `try-catch` is an expression. Compile and run and test that from user's point of view it works exactly the same way. Note the main differences between the two programs.

Task 16. *Polynomial, continuation: reduction.* Build on `object polynomial` in task 13 on page 69 and implement the method `def reduce: Poly` in case class `Poly` which simplifies polynomials if multiple `Prod` terms can be added.

Task 17. *Type-safe content test with method ===.* Methods `equals` and == allow comparison with anything. Sometimes one wants a type-safe content comparison that only allows comparison of objects of a more specific type and gives a compile error otherwise. One usually defines a method === with a parameter `that` having a suitably specific desired type. Introduce the abstract method === in trait `polynomial.Term` in task 13 on page 69 and override it in all subclasses of Term. Test that you get a compilation error if you try to compare a `Term` with something completely different, e.g. a `String` or a `Vector`.

```
  def ===(that: Term): Boolean
```

Task 18. *Override `equals` with content equality also for non-final classes.* Below are parts of class `Complex` representing a complex number with real and imaginary parts. Instead of, as often done in Scala, using a case class and an `equals` method that automatically gives content equality, you should practice implementing your own `equals`.

```
class Complex(val re: Double, val im: Double):
  def abs: Double = math.hypot(re, im)
  override def toString = s"Complex($re, $im)"
  def canEqual(other: Any): Boolean = ???
  override def hashCode: Int  = ???
  override def equals(other: Any): Boolean = ???

case object Complex:
  def apply(re: Double, im: Double): Complex = new Complex(re, im)
```

Follow this **recipe**[26] in 8 steps to override `equals` with content equality that also works for non-final classes:

1. Introduce this method: `def canEqual(other: Any): Boolean`
   Note that the parameter type must be `Any`. If done in a subclass to a class that already has `canEqual`, you should also add `override`.

2. Method `canEqual` should return `true` if `other` is of the same type as `this`, e.g.:
   `def canEqual(other: Any): Boolean = other.isInstanceOf[Complex]`

[26] This recipe is based on `http://www.artima.com/pins1ed/object-equality.html`

3. Introduce method `equals` making sure parameter has type `Any`:
   `override def equals(other: Any): Boolean`

4. Implement `equals` with a match expression starting as:
   `other match`

5. The match should have two branches. The first branch should have a typed pattern for the class to compare:
   `case that: Complex =>`

6. If you implement `equals` in the class introducing `canEqual`, start the expression with:
   `(that canEqual this) &&`
   and continue with a comparison based on class content, e.g.: `this.re == that.re && this.im ==`
   If overriding a different equals than the standard one from `AnyRef`, you probably want to start the logical expression with calling super's equals method: `super.equals(that) &&` but you need to carefully consider what equality between subclasses should mean in your particular case.

7. The second branch in the match should be: `case _ => false`

8. Override `hashCode`, for example by making a tuple of the class contents and calling method ## on the tuple to get a good hash code:
   `override def hashCode: Int = (re, im).##`

Task 19. *Override equals with inheritance.* Build on the example below and override equals with inheritance by following the recipe in task 18.

```
trait Number:
  override def equals(other: Any): Boolean = ???

class Complex(re: Double, im: Double) extends Number:
  override def equals(other: Any): Boolean = ???

class Rational(numerator: Int, denominator: Int) extends Number:
  override def equals(other: Any): Boolean = ???
```

Task 20. *Special matchings.* Read about usage of special matchings here:
dotty.epfl.ch/docs/reference/changed-features/vararg-splices.html

a) Try variable binding with `@` in a match in REPL.

b) Try sequence patterns with `_` and `_*` in a match in REPL.

Task 21. *Extractors.* Read more about extractors here:
dotty.epfl.ch/docs/reference/changed-features/pattern-matching.html
Create your own extractor object for HTTP addresses like e.g.:
`http://my.host.domain/path/to/this`
that extracts `my.host.domain` and `path/to/this` with method `unapply` and test in a match.

Task 22. *Polynomial, continuation: polynomial division.* Implement polynomial division appropriately by building on `object polynomial` in task 13 on page 69.
Read more about polynomial division here: sv.wikipedia.org/wiki/Polynomdivision

7. Exercise `Excercise07`

Goals

- ☐ Be able to read and write pseudocode for sequence algorithms and implement sequence algorithms according to pseudocode.
- ☐ Be able to implement sequence algorithms, both by copying to a new sequence and by in-place modification of an existing sequence.
- ☐ Be able to use built-in methods for updating, linear searching in, and sorting sequence collections.
- ☐ Be able to describe the difference in the use of mutable and immutable sequences, especially during updates.
- ☐ Understand how the sorting order is defined for strings.
- ☐ Be able to sort sequence collections containing objects of basic types using built-in and custom sorting orders with the methods `sorted`, `sortBy` and `sortWith`.
- ☐ Be able to implement linear searching according to different search criteria.
- ☐ Be able to describe the properties of the sequence collections `Vector`, `List`, `Array`, `ArrayBuffer`, and `ListBuffer`.
- ☐ Understand side effects of updating shared references to mutable elements.
- ☐ Be able to use functions with repeated parameters.
- ☐ Know how to implement functions with repeated parameters.
- ☐ Be able to implement integer counting in an integer array.

Preparations

- ☐ Study the concepts in chapter 7

7.1 Basic Tasks; preparation for the lab

Task 1. *Match concepts with description.*

Match each concept with the (simplified) description that best fits:

element	1	A	defines how elements of a type are arranged
collection	2	B	data structure with elements of same type
collection library	3	C	algorithm arranging elements in a specific order
sequence(collection)	4	D	algorithm finding elements according to a search criterion
sequence algorithm	5	E	how execution time grows with problem size
order	6	F	searching a sequence until element found
sorting	7	G	object in a data structure
searching	8	H	algorithm counting elements with certain properties
linear search	9	I	solution to problem using a sequence collection
counting	10	J	many ready-made collections with different properties
time complexity	11	K	how memory usage grows with problem size
memory complexity	12	L	zero or more elements of same type in a specific order

Task 2. *Different sequence collections.* Match each sequence collection with the (simplified) description that best fits:

`Vector`	1	A	mutable, fast indexing, can change size
`List`	2	B	immutable, quickly returns any modified collection
`Array`	3	C	immutable, quickly returns new collection modified at start
`ArrayBuffer`	4	D	primitive, mutable, fast indexing, fixed size
`ListBuffer`	5	E	mutable, fast to modify at start

Task 3. *Using sequence collections.* Assume the following variables are visible in the current namespace:

```
val xs: Vector[Int] = Vector(1, 2, 3)
val x: Int = 0
```

a) Match each expression on the left with the corresponding result on the right. If you are unsure of the result, consult the quick reference and test in REPL.
Tip: "colon on the collection side".

`x +: xs`	1	A	`true`
`xs +: x`	2	B	`Vector(2, 2, 3)`
`xs :+ x`	3	C	`1`
`xs ++ xs`	4	D	`error: value tail is not a member of Int`
`xs.indices`	5	E	`(0 until 3)`
`xs apply 0`	6	F	`Vector(1, 2, 3)`
`xs(3)`	7	G	`Vector(0, 1, 2, 3)`
`xs.length`	8	H	`false`
`xs.take(4)`	9	I	`java.lang.IndexOutOfBoundsException`
`xs.drop(2)`	10	J	`Vector(1, 2, 3, 0)`
`xs.updated(0, 2)`	11	K	`Vector(3)`
`xs.tail.head`	12	L	`error: value +: is not a member of Int`
`xs.head.tail`	13	M	`Vector(1, 2, 3, 1, 2, 3)`
`xs.isEmpty`	14	N	`2`
`xs.nonEmpty`	15	O	`3`

b) There are three cases where errors occur. Why? Are these compile-time errors or runtime errors?

Tip for continuation: Scala's standard library has many useful collections with uniform method sets. Learning the most important collection methods gives you a powerful toolbox. Read more here:

- quick reference (only allowed exam aid): https://fileadmin.cs.lth.se/pgk/quickref.pdf

- overview (by Prof. Martin Odersky, inventor of Scala, and others): https://docs.scala-lang.org/overviews/collections-2.13/introduction.html
- API documentation: https://www.scala-lang.org/api/current/scala/collection/

Task 4. *Copying sequences.* The class `Mutant` below can be used to create mutable instances with integers.[27]

```
class Mutant(var int: Int = 0)
```

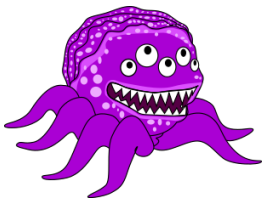


Figure 1: An instance of class Mutant where `int` might be 5.

Run the following in REPL after studying this: https://youtu.be/dpdOUEe9mm4

```
scala> val fem = new Mutant(5)
scala> val xs = Vector(fem, fem, fem)
scala> val ys = xs.toArray    // copies references to new Array
scala> val zs = xs.map(x => new Mutant(x.int)) // deep copies to new Vector
scala> xs(0).int = (new Mutant).int
```

a) Fill in the table below by writing the value of each expression to the right. Explain what happens. *Tip:* The method `eq` always compares references (not content).

`xs(0)`	
`ys(0).int`	
`zs(0).int`	
`xs(0) eq ys(0)`	
`xs(0) eq zs(0)`	
`(ys.toBuffer :+ new Mutant).apply(0).int`	

b) Implement the `deepCopy` function below using a **`while`** loop that performs *deep* copying, i.e., creates a new array with new, content-copied mutants.

```
def deepCopy(xs: Array[Mutant]): Array[Mutant] = ???
```

Use this algorithm:

[27] If the built-in base type Int, like `Mutant`, oddly could be used to create mutable instances, integer mathematics in Scala would dissolve into a frightening chaos.

Input : A mutant array xs
Output: A deep copy of xs

1 $result \leftarrow$ a new mutant array with space for as many elements as in xs
2 $i \leftarrow 0$
3 **while** *i less than the number of elements* **do**
4 create a copy of the element $xs(i)$ and place the copy in $result$ at position i
5 increment i by 1
6 **end**
7 $result$

c) Test your function and check that no tricky mutations via shared references are possible, as happened with `xs` and `ys` in the first subtask.

d) Is it common to do deep copying of all elements in immutable collections that only contain immutable elements "just to be safe"?

Tip for continuation: You can often solve basic subproblems using built-in collection methods from the standard library. For example, the copying in `deepCopy` in the previous task can easily be done using the collection method `map`.
But it's very good for your understanding if you can implement basic sequence algorithms yourself even though it is usually better to use ready-made, well-tested methods. In coming tasks you will implement your own versions of some sequence algorithms already existing in the standard library.

Task 5. *Updating sequences.* Declare these variables in REPL:

```
val xs = (1 to 4).toVector
val buf = xs.toBuffer
```

a) The expressions on the left are evaluated from top to bottom. Match with the correct result.

`{ buf(0) = -1; buf(0) }`	1	A	`error: value update is not a member`
`{ xs(0) = -1; xs(0) }`	2	B	`Vector(5, 2, 3, 4)`
`buf.update(1, 5)`	3	C	`ArrayBuffer(-1, 5, 3, 4, 5)`
`xs.updated(0, 5)`	4	D	`-1`
`{ buf += 5; buf }`	5	E	`Vector(1, -1, 5)`
`{ xs += 5; xs }`	6	F	`(): Unit`
`xs.patch(1,Vector(-1,5),3)`	7	G	`error: value += is not a member`
`xs`	8	H	`Vector(1, 2, 3, 4)`

Tip: Read about the methods in quick reference and test in REPL. Example:

```
scala> Vector(1,2,3,4).patch(from = 1, other = Vector(0,0), replaced = 3)
val res0: Vector[Int] = Vector(1, 0, 0)
```

b) Implement the `insert` function below using the sequence collection method `patch`. *Tip:* Give argument `0` to parameter `replaced`.

```
/** Creates a copy of xs but with elem inserted at position pos. */
def insert(xs: Array[Int], elem: Int, pos: Int): Array[Int] = ???
```

c) Write pseudocode for an algorithm that implements `insert` using **`while`**.

d) Implement `insert` according to your pseudocode. Test in REPL and observe what happens if `pos` is negative? What happens if `pos` is just one step beyond the last position in `xs`? What happens if `pos` is several steps beyond the last position?

> *Tip for continuation:* It's not easy to get all edge cases right even in small algorithms such as `insert` above. Therefore, it is important to think carefully about your sequence algorithm with regard to different special cases. Use this checklist:
>
> 1. What happens if the sequence is empty?
> 2. Does it work for exactly one element?
> 3. Can indices become negative?
> 4. Can indices exceed length minus one?
> 5. Can there be an infinite loop, e.g., due to missing loop variable increment?
>
> Sometimes you want exceptions to be thrown on invalid input or other error conditions and `require` or `assert` are good to use then. In other cases you want the result to, for example, be an empty sequence if input is invalid. Such behaviors must be documented so that other users of your algorithms (or you yourself after forgetting the details) understand what happens in various cases.

Task 6. *Comparing strings in Scala.* In Scala, strings can be compared with the operators ==, !=, <, <=, >, >=, where equality/inequality is based on whether all characters in the string are equal or not, while greater/less is based on order in accordance with each character's Unicode value.[28]

a) What values do the following comparisons return?

```
scala> 'a' < 'b'
scala> "aaa" < "aaaa"
scala> "aaa" < "bbb"
scala> "AAA" < "aaa"
scala> "ÄÄÄ" < "ÖÖÖ"
scala> "ÅÅÅ" < "ÄÄÄ"
```

Unfortunately, the order of ÄÅÖ does not follow Swedish rules, but we will ignore that for simplicity; if interested in how to fix this, do task 20.

b) Which of the strings $s1$ and $s2$ comes first (i.e., is "less") if $s1$ is a prefix of $s2$ and $s2$ contains more characters than $s1$?

[28] Advanced: All characters in a `java.lang.String` are represented according to the UTF-16 standard (https://en.wikipedia.org/wiki/UTF-16), meaning each Unicode code point is stored as either one or two 16-bit integers. String comparison in Scala and Java actually compares the 16-bit integers rather than characters. This difference does not matter when a string only contains characters that fit in one 16-bit integer, which practically includes almost all characters commonly used daily. The characters requiring two 16-bit integers are mostly rare Chinese characters, rare symbols, characters from extinct languages, and emoji. We will disregard such characters in this course.

Task 7. *Linear search according to different criteria.* Linear search means looking until you find what you are searching for in a sequence. This subproblem recurs often! Usually linear search starts from the beginning and continues until it finds an element that satisfies the criterion. Depending on the contents of the sequence and the criterion, it may happen that you go through all elements without finding what you are looking for.

a) Linear search with built-in sequence collection methods.

```
val xs = ((1 to 5).reverse ++ (0 to 5)).toVector
```

Declare the above variable in REPL and match the expressions below with the correct values. Explain what happens.

`xs.indexOf(0)`	1	A	`Vector(1, 1)`
`xs.indexOf(6)`	2	B	`-1`
`xs.indexWhere(_ < 2)`	3	C	`true`
`xs.indexWhere(_ != 5)`	4	D	`Some(1)`
`xs.find(_ == 1)`	5	E	`Vector(1, 0, 1)`
`xs.find(_ == 6)`	6	F	`5`
`xs.contains(0)`	7	G	`Vector(4, 6)`
`xs.filter(_ == 1)`	8	H	`4`
`xs.filterNot(_ > 1)`	9	I	`1`
`xs.zipWithIndex.filter(_._1 == 1).map(_._2)`	10	J	`None`

b) Implement linear search in a string vector with a string predicate.

```
/** Returns first index where p is true. Returns -1 if not found. */
def indexOf(xs: Vector[String], p: String => Boolean): Int = ???
```

A string predicate `p: String => Boolean` is a function that takes a string as input and returns a boolean. Implement `indexOf` using a **`while`** loop. You can, for example, use a local boolean variable `found` to keep track of whether you found what is searched for according to the predicate.
When elements satisfying the predicate are missing, you must decide what should happen. The requirement on your implementation in this case is given by the documentation comment above.
Your function should behave as follows:

```
scala> val xs = Vector("hej", "on", "dough")
val xs: Vector[String] = Vector(hej, on, dough)

scala> indexOf(xs, _.contains('p'))
val res0: Int = 1

scala> indexOf(xs, _.contains('q'))
val res1: Int = -1

scala> indexOf(Vector(), _.contains('q'))
val res2: Int = -1

```

```
scala> indexOf(Vector("q"), _.length == 1)
val res3: Int = 0
```

Task 8. *Lab preparation: Implement integer counting in Array.* Counting means counting the number of occurrences of different values. Each time a new value occurs, we need to increment a frequency counter. A counter is needed for each value to be counted. We keep counting until all values are counted.
In this week's lab you will count the occurrence of different poker card combinations. In this exercise, as preparation for the lab, solve a similar counting problem: frequency analysis of many dice rolls. Dice counting needs six different counters. You can advantageously use a sequence collection with space for six integers. For example, position `0` holds the count of ones, position `1` holds the count of twos, etc.

a) Implement the algorithm below according to the pseudocode:

```
def registerDiceRoll(xs: Seq[Int]): Vector[Int] =
  val result = ??? /* Array with six zeros */
  xs.foreach{ x =>
    require(x >= 1 && x <= 6, "dice roll must be between 1 & 6")
    ??? /* count the occurrence of x */
  }
  result.toVector
```

b) Use the function `toss` below when testing your counting algorithm with a sequence collection containing at least 1000 dice rolls.

```
def toss(n: Int) = Vector.fill(n)(util.Random.nextInt(6) + 1)
```

Task 9. *Built-in methods for sorting.* There are several ways to order sequences by various criteria. For primitive types `Int`, `Double`, `String`, etc., built-in orderings exist so that the sequence collection method `sorted` works without extra arguments (if you are satisfied with the built-in order definition). Methods `sortBy` and `sortWith` also exist if you want to order a sequence with elements of some primitive type using your own ordering definition, or if you have your own classes in your sequence.

```
val xs = Vector(1, 2, 1, 3, -1)
val ys = Vector("abra", "ka", "dabra").map(_.reverse)
val zs = Vector('a', 'A', 'b', 'c').sorted

case class Person(firstName: String, surName: String)

val ps = Vector(Person("Kim", "Ung"), Person("kamrat", "Clementin"))
```

Declare the above in REPL and match the expressions below with the correct results.
Tip: Uppercase letters sort before lowercase letters in the built-in ordering for the primitive types `String` and `Char`. Also, Swedish characters have a peculiar order.[29]
Read about the sorting methods in the quick reference and try in REPL.

[29] Order originates from outdated character encoding standards: https://sv.wikipedia.org/wiki/ASCII

`'a' < 'A'`	1	A	`"ka"`
`"AÄÖö" < "AÅÖö"`	2	B	`1`
`xs.sorted.head`	3	C	`-1`
`xs.sorted.reverse.head`	4	D	`error: ...`
`ys.sorted.head`	5	E	**`false`**
`zs.indexOf('a')`	6	F	`0`
`ps.sorted.head.firstName.take(2)`	7	G	`3`
`ps.sortBy(_.firstName).apply(1).firstName.take(2)`	8	H	**`true`**
`xs.sortWith((x1,x2) => x1 > x2).indexOf(3)`	9	I	`"ak"`

We will later implement our own sorting algorithms as practice, but normally built-in sorting is used because it is efficient and well-tested. However, it is not uncommon to define custom orderings for your own classes, which we will study later in the course.

Task 10. *Built-in shuffle method.* In this week's lab you will implement your own shuffle algorithm and use it to shuffle a deck of cards. There is already a built-in method `shuffle` in the singleton object `Random` in package `scala.util`.

a) Look up documentation for `Random.shuffle` and study the method header. It says complicated things about `CanBuildFrom` etc. This clever detail, which we will not delve into in this course, is so that the method can return the appropriate type of collection. When you see such a method header you can assume the method works fine with several types of suitable collections in Scala's standard library.
Click on the `shuffle` documentation so you can see all text. What does the documentation say about the result? Is it in-place shuffling or copying to a new collection?

b) Try repeated shuffles of different types of sequences with different element types in REPL.

Task 11. *Repeated parameters.* You can declare a function that takes an argument sequence of arbitrary length, so-called *varargs*. The syntax consists of an asterisk $*$ after the type. The function is then said to have repeated parameters. In the function body you get access to the arguments in a sequence collection. The arguments are given arbitrarily many, separated by commas. Example:

```
/** Gives a vector of string lengths for arbitrary number of strings. */
def stringSizes(xs: String*): Vector[Int] = xs.map(_.size).toVector
```

a) Declare and use `stringSizes` in REPL. What happens if you call `stringSizes` with an empty argument list?

b) Sometimes you already have a sequence collection, e.g., `xs`, and want to pass each element as an argument to a varargs function. The syntax is `xs: _*` which makes the compiler convert the sequence collection into an argument sequence of the right type. Try this syntax by giving a `xs` of type `Vector[String]` as argument to `stringSizes`. Does it work even if `xs` is a sequence of length 0?

7.2 Extra Tasks; additional practice

Task 12. *Counting boolean values. Flipping a coin.*

a) Implement a function that counts many coin flips according to the below function header. Input is a sequence of boolean values where heads is coded as `true` and tails as `false`. For counting, use a local `Array[Int]`. The result must be a tuple where the count of heads is in the first place and count of tails in the second place.

```
def registerCoinFlips(xs: Seq[Boolean]): (Int, Int) = ???
```

b) Create a function `flips(n)` that generates a boolean `Vector` with n coin flips and use it to test your coin flip counting algorithm.

Task 13. *Copying and appending at the end.* Create the function `copyAppend` that implements the following algorithm, *after* fixing the **two bugs** below:

Input : Integer array xs and integer x
Output : A new integer array which is a copy of xs but with x appended at the end as an extra element.

$ys \leftarrow$ a new array with one more slot than xs
$i \leftarrow 0$
while $i \leq xs.length$ **do**
  $ys(i) \leftarrow xs(i)$
end
put x at the last position in ys
ys

Inspect your code according to the checklist in the previous tip box. Test your function for different cases: empty sequence, sequence with exactly one element, sequence with many elements.

Task 14. *Copy and reverse sequence.* Implement `seqReverseCopy` according to:

Input : Integer array xs
Output : A new integer array with elements of xs in reverse order.

$n \leftarrow$ number of elements in xs
$ys \leftarrow$ new integer array with space for n elements
$i \leftarrow 0$
while $i < n$ **do**
  $ys(n-i-1) \leftarrow xs(i)$
  $i \leftarrow i+1$
end
ys

a) Use a `while` loop as in the algorithm. Test your implementation in REPL and check it works for different cases.

b) Create a new implementation that uses a `for` loop starting from the end. Run it in REPL and check it works for different cases.

Task 15. *Copy all except one.* Implement copying an array *except* for one element at a specified position. First write pseudocode before implementing:

```
def removeCopy(xs: Array[Int], pos: Int): Array[Int]
```

Task 16. *In-place removal in array.* Sometimes you want to remove an element at a given position in an array without copying all elements to a new collection. One way is to move all subsequent elements one step towards lower indexes and fill the last position with a padding value, e.g. 0. Write pseudocode for such an algorithm first. Then implement the algorithm in a procedure with this signature:

```
def removeAndPad(xs: Array[Int], pos: Int, pad: Int = 0): Unit
```

Task 17. *Copying and inserting.*

a) Implement a function with this header according to the algorithm below:

```
def insertCopy(xs: Array[Int], x: Int, pos: Int): Array[Int]
```

Input : A sequence xs of type `Array[Int]` and integers x and pos
Output: A new sequence of type `Array[Int]` which is a copy of xs but with x inserted at position pos

$n \leftarrow$ number of elements in xs
$ys \leftarrow$ a new `Array[Int]` with space for $n+1$ elements
for $i \leftarrow 0$ **to** $pos-1$ **do**
  $ys(i) \leftarrow xs(i)$
end
$ys(pos) \leftarrow x$
for $i \leftarrow pos$ **to** $n-1$ **do**
  $ys(i+1) \leftarrow xs(i)$
end
ys

b) What must `pos` be for this to work with an empty array as input?

c) What happens if your function is called with a negative argument for `pos`?

d) What happens if your function is called with `pos` equal to `xs.size`?

e) What happens if your function is called with `pos` greater than `xs.size`?

Task 18. *In-place insertion in array.* One way to implement insertion in an array, without copying all elements to a new array with an extra slot, is to move all elements after `pos` one step to higher index, making room for the new element. In this solution the last element may "disappear" due to overwriting, since arrays cannot grow.
Write such an algorithm in pseudocode first and then implement it in a procedure with this header:

```
def insertDropLast(xs: Array[Int], x: Int, pos: Int): Unit
```

Task 19. *More built-in methods for linear search.*

a) Read in quick reference about the methods `lastIndexOf`, `indexOfSlice`, `segmentLength` and `maxBy` and describe what each can be used for.

b) Test the methods in REPL.

7.3 Advanced Tasks; challenges

Task 20. *Fix Swedish sorting order of ÄÅÖ.* Swedish letters come in a strange order for Swedes if special actions are not taken. Using class `java.text.Collator` you can get a `Comparator` for strings that follows local language rules for many languages on earth.

a) Verify that the sorting order is correct in REPL as below.

```
scala> val wrong = Vector("ö","å","ä","z").sorted
scala> val svColl = java.text.Collator.getInstance(new java.util.Locale("sv"))
scala> val svOrd = Ordering.comparatorToOrdering(svColl)
scala> val right = Vector("ö","å","ä","z").sorted(svOrd)
```

b) Use the above method to write a program that prints lines in a text file in correct Swedish sorting order. The program should be runnable with:
`scala sorted -sv textfil.txt`

c) Read more here:
stackoverflow.com/questions/24860138/sort-list-of-string-with-localization-in-scala

Task 21. *Fibonacci sequence with ListBuffer.* The collection `ListBuffer` is a mutable sequence that is fast and memory efficient when prepending. Investigate what happens here:

```
1 scala> val xs = scala.collection.mutable.ListBuffer.empty[Int]
2 scala> xs.prependAll(Vector(1, 1))
3 scala> while xs.head < 100 do {xs.prepend(xs.take(2).sum); println(xs)}
4 scala> xs.reverse.toList
```

The numbers produced on line 4 above are called Fibonacci numbers [30] and grow quickly.

a) Define and test the following function. It should internally use mutable `ListBuffer` but return a sequence of immutable `List`.

```
/** Produces a list of numbers in the Fibonacci sequence 1, 1, 2, 3, 5, 8 ...
  * where the largest number is less than max. */
def fib(max: Long): List[Long] = ???
```

b) How long should a Fibonacci sequence be so that the last element is as close as possible to `Int.MaxValue`?

c) Implement `fibBig` that uses `BigInt` instead of `Long` and lets your computer use its large memory while the planet warms a bit.

Task 22. *In-place reverse sequence.* Implement the following algorithm in the function `reverseChars` and test that it works for various cases in REPL.

Task 23. *Palindrome predicate.* A palindrome[31] is a word that remains unchanged when read backwards. Examples: kajak, dallassallad.
One way to implement a palindrome predicate is shown below:

```
def isPalindrome(s: String): Boolean = s == s.reverse
```

[30] sv.wikipedia.org/wiki/Fibonaccital
[31] https://sv.wikipedia.org/wiki/Palindrom

Input : An array xs of characters
Output : xs updated in-place, with characters in reverse order

```
n ← number of elements in xs
for i ← 0 to n/2 − 1 do
    temp ← xs(i)
    xs(i) ← xs(n − i − 1)
    xs(n − i − 1) ← temp
end
```

a) The above implementation may require going through all characters twice and need memory for double the number of characters. Why?

b) Create a palindrome test that traverses the elements at most once and does not need extra memory for a copy of the string. *Idea:* Compare pairs first and last, second and second last, etc.

Task 24. *More useful sequence collection methods.* Search on the web and read about these methods and test them in REPL:

- `xs.tabulate(n)(f)`
- `xs.forall(p)`
- `xs.exists(p)`
- `xs.count(p)`
- `xs.zipWithIndex`

Task 25. *Arrays don't behave, but `ArraySeqs` do!* Although `Array` is primitive, Scala's collection library has "magic" under the hood so arrays behave almost like "real" collections. Thus, you don't have to do special handling, like boxing, wrapper classes, or type casting, common for Java programmers.
However, limitations and anomalies still exist, for example regarding equality tests. If you want an array to behave like other collections, you can easily "wrap" it using `toSeq`, which on arrays returns an `ArraySeq`. This behaves as a normal immutable sequence without sacrificing the speed of a primitive array.

```scala
val as = Array(1,2,3)
val xs = as.toSeq
```

a) How does equality testing between two `ArraySeqs` work? What is the type of `xs` above? Is it possible to update a wrapped array?

b) What type of argument sequence do you get access to inside a function with repeated parameters?

c) Read here: http://docs.scala-lang.org/overviews/collections/arrays.html ★ and give an example of what else you cannot do with an array, besides content equality testing.

Task 26. *List or Vector?* Compare the time complexity between List and Vector when handling at the start and at the end, based on the following REPL session on your own machine. The below was run on an AMD Ryzen 7 5800X (16) @ 3.800GHz under Arch Linux 5.12.8-arch1-1 with Scala 3.0.1 and openjdk 11.0.11, but you should use what you have on your own computer. ★

How fast does the below run on your computer? When is List fastest and when is Vector fastest? How large are the differences in performance? [32]

```
> head -5 /proc/cpuinfo
processor    : 0
vendor_id    : AuthenticAMD
cpu family    : 25
model        : 33
model name    : AMD Ryzen 7 5800X 8-Core Processor

scala> def time(n: Int)(block: => Unit): Double =
     |   def now = System.nanoTime
     |   var timestamp = now
     |   var sum = 0L
     |   var i = 0
     |   while i < n do
     |     block
     |     sum = sum + (now - timestamp)
     |     timestamp = now
     |     i = i + 1
     |   val average = sum.toDouble / n
     |   println("Average time: " + average + " ns")
     |   average


// Exiting paste mode, now interpreting.

time: (n: Int)(block: => Unit)Double


scala> val n = 100000
scala> val l = List.fill(n)(math.random())
scala> val v = Vector.fill(n)(math.random())

scala> (for i <- 1 to 20 yield time(n){l.take(10)}).min
average time: 97.66952 ns
average time: 91.90033 ns
average time: 79.88311 ns
average time: 69.5963 ns
average time: 69.69892 ns
average time: 69.8033 ns
average time: 69.7705 ns
average time: 69.68491 ns
average time: 69.54222 ns
average time: 69.66051 ns
average time: 69.73661 ns
average time: 69.54112 ns
average time: 69.69141 ns
average time: 69.46341 ns
average time: 69.4098 ns
average time: 61.34162 ns
average time: 41.1333 ns
average time: 40.97051 ns
average time: 40.9075 ns
average time: 41.12321 ns
```

[32] You will learn more about such measurements in the LTH course "Evaluation of Software Systems" given at the end of year 1 for Computer Engineering students.

```
val res0: Double = 40.9075

scala> (for i <- 1 to 20 yield time(n){v.take(10)}).min
average time: 84.56978 ns
average time: 75.20167 ns
average time: 57.16529 ns
average time: 34.84469 ns
average time: 34.38478 ns
average time: 34.77709 ns
average time: 34.77179 ns
average time: 35.0506 ns
average time: 34.7967 ns
average time: 35.04258 ns
average time: 34.82559 ns
average time: 36.3673 ns
average time: 34.91029 ns
average time: 34.87239 ns
average time: 34.51958 ns
average time: 34.83949 ns
average time: 34.56169 ns
average time: 34.80719 ns
average time: 34.84459 ns
average time: 34.89468 ns
val res1: Double = 34.38478

scala> (for i <- 1 to 20 yield time(1000){l.takeRight(10)}).min
average time: 131365.106 ns
average time: 118632.787 ns
average time: 118440.066 ns
average time: 118687.567 ns
average time: 118428.487 ns
average time: 118871.686 ns
average time: 118964.797 ns
average time: 119030.236 ns
average time: 119262.534 ns
average time: 119228.344 ns
average time: 119226.494 ns
average time: 119310.933 ns
average time: 119352.854 ns
average time: 119121.913 ns
average time: 119133.664 ns
average time: 119015.193 ns
average time: 119276.674 ns
average time: 119224.882 ns
average time: 119301.771 ns
average time: 119444.401 ns
val res2: Double = 118428.487

scala> (for i <- 1 to 20 yield time(1000){v.takeRight(10)}).min
average time: 805.989 ns
average time: 365.219 ns
average time: 225.49 ns
average time: 125.92 ns
average time: 124.98 ns
average time: 130.689 ns
average time: 139.86 ns
average time: 128.29 ns
```

```
average time: 132.59 ns
average time: 125.729 ns
average time: 125.46 ns
average time: 130.59 ns
average time: 122.03 ns
average time: 121.9 ns
average time: 119.69 ns
average time: 120.48 ns
average time: 125.239 ns
average time: 126.09 ns
average time: 125.92 ns
average time: 125.91 ns
val res3: Double = 119.69
```

Why do different rounds in the for loop run at different speeds even though each round does the same thing?

★ **Task 27.** *Time complexity of various collections in Scala's standard library.*
Study differences in time complexity between different collections here:
docs.scala-lang.org/overviews/collections/performance-characteristics.html
Also read the criticism of simplifications in the above description here:
www.lihaoyi.com/post/ScalaVectoroperationsarentEffectivelyConstanttime.html
Read this thorough empirical review of performance in Scala's collection libraries:
www.lihaoyi.com/post/BenchmarkingScalaCollections.html

8. Exercise `Excercise08`

Goals

- ☐ Be able to create and use matrices with nested structures of `Vector`.
- ☐ Be able to iterate over the elements in a matrix with nested **`for`** statements and **`for-yield`** expressions, as well as nested application of `map` and `foreach`.
- ☐ Be able to create and use functions that take matrices as parameters.
- ☐ Be able to create a simple generic class and simple generic functions using a type parameter.

Preparations

- ☐ Study the concepts in chapter 8

8.1 Basic Tasks; preparation for the lab

Task 1. *Match concepts with descriptions.*

Connect each concept with the (simplified) description that best fits:

matrix	1	A	indexable 2D data structure
row vector	2	B	has abstract type parameter, type is general
column vector	3	C	another word for column
column	4	D	concrete type, binds to type parameter at compile time
generic	5	E	compiler deduces type from context
type argument	6	F	matrix of dimension $1 \times m$ with m horizontal values
type inference	7	G	matrix of dimension $m \times 1$ with m vertical values

Task 2. *Create matrices using nested collections.* In a computer program, it is possible, using collections that contain collections, to create nested structures that can be indexed in two dimensions and thereby represent a **matrix**.[33]

a) Draw the memory situation after the assignment on line 1 below. What type and value does `m` have? What are the dimensions of `m`? How does indexing occur in a computer program compared to mathematics?

```
1 scala> val m = Vector((1 to 5).toVector, (3 to 7).toVector)
2 scala> m.apply(0).apply(1)
3 scala> m(1)
4 scala> m(1)(4)
```

b) What values and types are produced by the expressions on lines 2, 3, and 4 above?

c) It is also possible to create two-dimensional, nested structures where the rows *do not* contain the same number of elements. This is therefore not a true matrix in the strict mathematical sense, but such a structure is often still called a ”matrix”. What type do the variables `m2`, `m3`, `m4`, and `m5` below have?

[33] sv.wikipedia.org/wiki/Matrix

```
scala> val m2 = Vector(Vector(1,2,3),Vector(4,5),Vector(42))
scala> val m3 = Vector(Vector(1,2), Vector(1.0, 2.0, 3.0))
scala> val m4 = m3(1) +: Vector("a") +: m3
scala> val m5 = Vector.fill(42){ m2(1).map(e => (e * math.random()).toInt) }
```

d) Which of the variables `m2`, `m3`, `m4`, and `m5` above represents a true matrix in the mathematical sense? What are its dimensions?

Task 3. *Create and iterate over matrices.* You will create matrices where each row represents 5 throws of a die in the Yatzy game.[34]

a) Define in REPL a function `def throwDie: Int = ???` that returns a random integer between 1 and 6.

b) Create the integer matrix below in REPL. What dimension does this matrix have?

```
scala> val ds1 = for (i <- 1 to 1000) yield
            for (j <- 1 to 5) yield throwDie
```

c) You can also use the variants below to create an integer matrix. Which of the variants `ds1` ... `ds6` do you find easiest to read and understand? Try each variant in REPL and specify the type for `ds1` ... `ds6` as inferred by the compiler.

```
val ds2 = (1 to 1000).map(i => (1 to 5).map(j => throwDie))
val ds3 = (1 to 1000).map(i => Vector.fill(5)(throwDie))
val ds4 = for (i <- 1 to 1000) yield Vector.fill(5)(throwDie)
val ds5 = Vector.fill(1000)(Vector.fill(5)(throwDie))
val ds6 = Vector.fill(1000, 5)(throwDie)
```

d) Define a function
`def roll(n: Int): Vector[Int] = ???`
that produces an integer vector with n random die throws. The throws should be sorted in ascending order; use the collection method `sorted`.

e) Define in REPL a function `isYatzy(xs: Vector[Int]): Boolean = ???` which tests if all elements in an integer vector are the same. Use the collection method `forall`.

f) Create a function
`def diceMatrix(m: Int, n: Int): Vector[Vector[Int]] = ???`
which uses the `roll` function to create a matrix with `m` vectors each having `n` random die throws.

g) Create a function returning a print-friendly string
`def diceMatrixToString(xss: Vector[Vector[Int]]): String = ???`
using `map` and `mkString` which works as below.

```
scala> val dm2s = diceMatrixToString(diceMatrix(4, 5))
val dm2s: String = 1 4 4 6 6
1 1 2 6 6
2 4 4 5 6
1 1 5 6 6

scala> println(dm2s)
1 4 4 6 6
```

[34] sv.wikipedia.org/wiki/Yatzy

```
1 1 2 6 6
2 4 4 5 6
1 1 5 6 6
```

h) Implement the function
`def filterYatzy(xss: Vector[Vector[Int]]): Vector[Vector[Int]]`
which filters out all rows from the matrix `xss` that are yatzy-rows. Use your function `isYatzy` and the collection method `filter`.

```
scala> println(diceMatrixToString(filterYatzy(diceMatrix(10000, 5))))
4 4 4 4 4
6 6 6 6 6
4 4 4 4 4
6 6 6 6 6
4 4 4 4 4
4 4 4 4 4
2 2 2 2 2
```

i) Implement the function
`def yatzyPips(xss: Vector[Vector[Int]]): Vector[Int] = ???`
which should give a vector of the die values that resulted in yatzy for the rolls in the matrix `xss` as in the example below. Use your `filterYatzy`.

```
scala> val dm = Vector(Vector(1,2,3,4,5),Vector(4,4,4,4,4),Vector(3,3,3,3,3))
scala> yatzyPips(dm)
val res42: Vector[Int] = Vector(4, 3)
```

Task 4. *An immutable, generic matrix class for this week's lab* `life`. During this week's lab, you will simulate a simple form of "life" consisting of cells in a grid. For this purpose, we need a matrix class that you will implement step by step in this exercise. Create the case class below using an editor in the file `Matrix.scala`. Test your solution using any IDE such as `scalaide` or `idea`.

```
case class Matrix(data: Vector[Vector[String]]){
  def apply(row: Int, col: Int): String = data(row)(col)
}
object Matrix {
  def fill(dim: (Int, Int))(value: String): Matrix =
    Matrix(Vector.fill(dim._1, dim._2)(value))
}
```

```
scala> val m = Matrix.fill(3,4)("hej")
scala> val e = m(2, 2)
```

a) What type does `m` get above?

b) What type does `e` get above?

c) In how many places must you change the `Matrix` above for it to instead represent a matrix of integers?

d) You should now, with the help of a **type parameter**, make `Matrix` **generic**, so that it becomes a more useful matrix class which can contain elements of any type. Perform the following changes in `Matrix.scala`:

- Add a type parameter `T` inside square brackets after the name `Matrix` everywhere it occurs *except* after the name of the companion object[35].
- Replace `String` with `T` everywhere `String` appears.
- Add a type parameter `T` inside square brackets after `def fill`.

Test your generic class in REPL by creating a boolean matrix:

```
scala> val bm = Matrix.fill(3,4)(false)
scala> val be = bm(0, 0)
```

e) What type does `bm` have above?

f) What type does `be` have above?

g) Add a code line at the beginning of the class body that, using `require`, guarantees that all rows in the matrix are equally long.

h) Add a member `val dim: (Int, Int)` in the class body after the `require` statement which gives a pair (a 2-tuple) with the number of rows and columns in the matrix.

i) Add a method `def updated(row: Int, col: Int)(value: T): Matrix[T]` that returns a new matrix where the element at position `(row, col)` has been updated to `value`.

j) Add a method `def foreachIndex(f: (Int, Int) => Unit): Unit` which applies the function `f` to every index in `data`.

k) Add a method `override def toString` so that an instance of `Matrix` appears as follows:

```
scala> val dm = Matrix.fill(3,4)(42.0)
val dm: Matrix[Double] =
Matrix of dim (3,4):
42.0 42.0 42.0 42.0
42.0 42.0 42.0 42.0
42.0 42.0 42.0 42.0
```

[35] Singleton objects cannot have type parameters, but their members can.

8.2 Extra Tasks; additional practice

Task 5. *Imperative matrix algorithms.* Imperative approaches are necessary to master when working with collections and/or languages lacking functional methods and/or functional programming features. By studying imperative solutions to often more concise functional ones, you gain practice creating algorithms using change through assignment during iteration.

a) Implement `isYatzy` from problem 3e again, but now using an imperative approach with a **`while`** loop instead of the functional `forall`. Use a variable `i` to keep track of the index and a variable `foundDiff` to track whether a differing value is found. The function requires about 9 lines, so it's suitable to open an editor to write while figuring out the solution. Start with writing pseudocode, preferably with pencil and paper. Test by pasting it into REPL.

b) An imperative implementation of `diceMatrixToString` from problem 3g using mutable `StringBuilder`[36] is shown below. Explain how the code works. What happens if `xss` is empty? What happens if `xss` contains only empty vectors? Mention one advantage and one disadvantage of using **`val`** `sb: StringBuilder` and `append`, compared to ordinary immutable **`var`** `s: String` and + for appending at the end.

```
def diceMatrixToString(xss: Vector[Vector[Int]]): String =
  val sb = new StringBuilder()
  for(m <- xss.indices) do
    for(n <- xss(m).indices) do
      sb.append(xss(m)(n).toString)
      if n < xss(m).size - 1 then sb.append(" ")
      else if m < xss.size - 1 then sb.append("\n")
    end for
  end for
  sb.toString
```

c) As practice, make an imperative implementation of `filterYatzy` with a **`for`**-**`do`** loop (i.e., without using `filter` and without using **`yield`**).

d) Explain how the functional implementation of `filterYatzy` below works, using a **`for`**-**`yield`**-expression. Do you find your imperative solution easier or harder to read and understand compared to the functional one below?

```
def filterYatzy(xss: Vector[Vector[Int]]): Vector[Vector[Int]] =
  (for i <- xss.indices if isYatzy(xss(i)) yield xss(i)).toVector
```

Task 6. *String table with column headers.*

a) Implement the case class `Table` according to the specification below. You can assume all rows have as many columns as the number of elements in `headings`, and that all headings in `headings` are unique. The parameter `sep` indicates the character used to separate columns. This is also assumed to hold for input files read with `fromFile`.

Tips:

- The value `indexOfHeading` can be created using the method `zipWithIndex`, which works on all sequence collections, and the method `toMap`, which works on

[36] https://www.scala-lang.org/api/2.12.9/scala/collection/mutable/StringBuilder.html

sequences of 2-tuples. First investigate how these methods work in REPL and read their documentation.

- Create an input file to test that `Table` works correctly.

```
case class Table(
  data: Vector[Vector[String]],
  headings: Vector[String],
  sep: Char
):
  /** A 2-tuple with (number of rows, number of columns) in data */
  val dim: (Int, Int) = ???

  /** The element in row r and column c of data, counting from 0 */
  def apply(r: Int, c: Int): String = ???

  /** The row-vector r in data, counting from 0 */
  def row(r: Int): Vector[String]= ???

  /** The column-vector c in data, counting from 0 */
  def col(c: Int): Vector[String] = ???

  /** A map from heading to index counting from 0 */
  lazy val indexOfHeading: Map[String, Int] = ???

  /** The column-vector with heading h in data */
  def col(h: String): Vector[String] = ???

  /** A vector with the distinct, sorted values of col with heading h */
  def values(h: String): Vector[String] = ???

  /** Headings and data with columns separated by sep */
  override lazy val toString: String = ???

object Table:
  /** Creates a new Table from fileName with columns split by sep */
  def fromFile(fileName: String, sep: Char = ';'): Table = ???
```

b) Create a program using `Table` that can be run from the terminal with `scala run infile.csv ';'` which outputs the count of occurrences of different values in each column (i.e., a form of recording).

Task 7. *Create a yatzy game for terminal use.* Build a simplified yatzy game in the terminal where the user can decide which dice to re-throw. Start with something really simple and then build your game by gradually adding more and more features.

8.3 Advanced Tasks; challenges

Task 8. *Generic functions.* A generic function has (at least) one type parameter within square brackets after the name, for example `[T]`. This type then appears as the type of (some of) the parameters in the parameter list. The compiler deduces a concrete type at compile time and replaces the type parameter with this concrete type. This way, a function can work for many types.

a) Explain what happens on each line below.

```
scala> def tnirp[T](x: T): Unit = println(x.toString.reverse)
scala> tnirp(42)
scala> tnirp("hej")
scala> case class Cucumber(weight: Int)
scala> tnirp(Cucumber(42))
scala> tnirp[String](42)
scala> tnirp[Double](42)
```

b) You can combine generic functions with functions that take functions as parameters. This is how `map` and `foreach` are implemented. Explain for each line below what happens.

```
scala> def compose[A, B, C](f: A => B, g: B => C)(x: A): C = g(f(x))
scala> def inc(x: Int): Int = x + 1
scala> def half(x: Int): Double = x / 2.0
scala> compose(inc, half)(42)
scala> compose(half, inc)(42)
```

c) What is the error message on the last line above? Change `inc` and/or `half` so that the types fit.

Task 9. *Generic classes.* Classes can also be generic. A generic class has (at least) one type parameter within square brackets after the class name.

a) Test the generic class `Cell[T]` below in REPL. Create instances of `Cell[T]` where the type parameter `T` is bound to different concrete types, and explain what happens.

```
scala> class Cell[T](var value: T):
         override def toString = "Cell(" + value + ")"

scala> new Cell(42)
scala> new Cell("hej")
scala> new Cell(new Cell(math.Pi))
scala> new Cell[String](42)
scala> new Cell[Double](42)
```

b) Add the method `def concat[U](that: Cell[U]):Cell[String]` to the class `Cell` which concatenates the string representations of the two cell values.

```
scala> val a = new Cell("hej")
scala> val b = new Cell(42)
scala> a concat b
```

c) What kinds of cells can you concatenate if you remove the type parameter name `U` in `concat` while using type `Cell[T]` as the parameter type of `that`? What consequences does this have for cells of types other than `Cell[String]`?

Task 10. *Implement more generic methods in* `Matrix[T]`. Build further on problem 4 and implement the specification below. Create your own tests that verify all methods work as expected.

Specification **Matrix[T]**

```
/** An immutable, generic matrix class. */
case class Matrix[T](data: Vector[Vector[T]]):
  require(???)  // guarantee all rows have equal columns

  /** Returns a pair with number of rows and columns. */
  val dim: (Int, Int) = ???

  /** Returns the element at position (row, col). */
  def apply(row: Int, col: Int): T = ???

  /** Returns a new matrix where the element at (row, col) has the value value. */
  def updated(row: Int, col: Int)(value: T): Matrix[T] =  ???

  /** Applies function f to all elements. */
  def foreach(f: T => Unit): Unit = ???

  /** Applies function f to all indices. */
  def foreachIndex(f: (Int, Int) => Unit): Unit = ???

  /** Returns a new matrix with results of element-wise application of f. */
  def map[U](f: T => U): Matrix[U] = ???

  /** Returns a new matrix with results of applying f to each index. */
  def mapIndex[U](f: (Int, Int) => U): Matrix[U] = ???

  /** Returns a print-friendly string representation of the matrix. */
  override def toString = ???

object Matrix:
  /** Returns a matrix with dimension dim where all elements have value value. */
  def fill[T](dim: (Int, Int))(value: T): Matrix[T] = ???
```

9. Exercise `Excercise09`

Goals

- ☐ Be able to create and use tuples as parameters and return values.
- ☐ Know and be able to use basic methods on collections.
- ☐ Be able to create and use both immutable and mutable sets.
- ☐ Understand the differences and similarities between a set and a sequence.
- ☐ Be able to describe how the linear search algorithm works.
- ☐ Be able to create and use both immutable and mutable key-value tables.
- ☐ Be able to use key-value tables to implement registration.
- ☐ Understand similarities and differences between a key-value table and a sequence.
- ☐ Be able to save and read data to/from text files on disk.

Preparations

- ☐ Study the concepts in chapter 9

9.1 Basic Tasks; preparation for the lab

Task 1. *Match concepts with description.*

Match each concept with the (simplified) description that fits best:

set	1	A	search sequence until criterion met
key-value table	2	B	decode symbol sequence and recreate object in memory
mapping	3	C	a unique identifier
key	4	D	property of remaining after program ends
persistence	5	E	encode object into decodable sequence of symbols
serialize	6	F	unordered collection of mappings with unique keys
deserialize	7	G	`key -> value`
linear search	8	H	unordered collection with unique elements

Task 2. *What is a set?* Explain what happens below. Why do the elements end up in a ”strange” order? Why does one element ”disappear”?

```
scala> val xs = Vector(1,2,3,1,2,3,4,5,7).toSet
xs: scala.collection.immutable.Set[Int] = Set(5, 1, 2, 7, 3, 4)
scala> xs.foreach(print)
512734
```

Task 3. *Using sets.*

Match each expression on the left with an expression on the right that has the same value:

`Set(1, 2) ++ Set(1, 2)`	1	A	`3`
`(1 to 3).toSet`	2	B	`Set(3)`
`Vector.fill(3)(1).toSet`	3	C	`6`
`Set(1, 2, 3) diff Set(1, 2)`	4	D	`error: ...`
`(1 to 7).toSet.apply(8)`	5	E	`true`
`Set(1, 2, 3).sorted`	6	F	`Set(1, 2) - 2`
`Set(2,4) subsetOf (1 to 7).toSet`	7	G	`Set(1) + 2 + 3`
`Set(1, -1, 2, -2).map(_.abs).sum`	8	H	`false`
`Set(1, 1, 1, 1, 1, 5).sum`	9	I	`Set(1, 2)`

Task 4. *Count unique words using a set.* In this week's lab, we will perform automatic language processing of long texts which we split into words. With the method `s.split(' ').toVector` you can split a string `s` into a sequence of words, where `s` is split into many strings at each space character and all spaces are removed.

a) Use the methods `split` and `toSet` to create an expression that calculates how many unique words there are in the string `hej` below:

```
scala> val hej = "hej hej hemskt mycket hej"
```

b) Sets are fast at checking if an element is in the set, but you cannot expect the elements to be in any particular order. There is a sequence collection method that produces a sequence with unique elements from a sequence and preserves the original order. What is the method called?
Tip: Look in the quick reference or search online. The method works on all collections that are of type `Seq` and has a name that starts with the letters `di`.

Task 5. *Creating 2-tuples with the method `->` which can be read as "maps to".* We have previously seen how two different values can be collected in a 2-tuple, for example `(0, true)`. Pairs can also be created using the method `->` as shown below. Try this in REPL:

```
scala> ("Skåne", "Lund")          // a string pair with a regular 2-tuple
scala> "Skåne" -> "Lund"           // operator notation with ->
scala> "Skåne".->("Lund")         // dot notation with -> (not very common)
```

The method `->` works with all types and is a factory method for pairs. The method name resembles a right arrow and illustrates a mapping from the first to the second value.

a) Does it work on pairs created with `->` to use the methods `_1` and `_2`?

b) Declare a variable `val huvudstad: Vector[(String, String)]` that contains mappings between geographical areas and their capitals according to the table below.

Sweden	Stockholm
Denmark	Copenhagen
Greenland	Nuuk
Skåne	Lund

c) Write an expression that extracts `"Lund"` from `huvudstad`.

Task 6. *Linear search for key in sequence of mappings.*

a) Implement the function `lookupIndex` below using the collection method `indexWhere` so that a linear search is performed for the index of a pair in the sequence where `key` is in the first position of the pair.

```
def lookupIndex(xs: Vector[(String, String)])(key: String): Int = ???
```

b) Test your function in REPL by looking up the index for Skåne's capital in the sequence `huvudstad` from the previous task.

Task 7. *Key-value table.* A key-value table is a smart data structure that allows you to look up the value that a key maps to *without* the need for linear search. The value is retrieved directly in constant time, i.e., the time to look up a value does *not* depend on the number of elements in the collection, but happens with very little delay.
In Scala, key-value tables are called `Map` with a capital M, and are practical to use in many different contexts. `Map` exists in both an immutable and a mutable variant. There are easy methods of the form `toXXX` to convert between a `Map` and a sequence of pairs of type `XXX[(KeyType, ValueType)]`.

a) Declare the map `telnr` below in REPL and use `apply` to find the phone number to Fröken Ur.

b) What is the type of `telnr`?

c) What is the type of `telnr.toVector`?

```
val telnr = Map(
  "Anna"     -> 46462229812L,
  "Björn"     -> 46462229009L,
  "Sandra"    -> 46462220368L,
  "Fröken Ur" -> 4690510L,
)
```

A set of `Map` instances, nested if necessary, can advantageously be used to build an in-memory database where built-in collection methods, e.g. `map`, `filter`, and **`for`**-**`yield`** expressions, provide flexible and efficient search capabilities. You will do this in this week's lab.
The collection `Map` is a generalization of a sequence, where you can "index", not only by an integer, but with any type of value, e.g., a string. The `Map` data structure is also called an *associative array*[37] and is implemented as a so-called *hash table*[38], but you will have to wait until the advanced course to go through how such a data structure is implemented.

Task 8. *Using key-value table.*

a) Create the variables below in REPL.

```
val follow = for i <- 2 to 16 by 2 yield (i, i + 1)
val xs = follow.toMap
val ys = xs.toVector
```

[37] https://en.wikipedia.org/wiki/Associative_array
[38] https://en.wikipedia.org/wiki/Hash_table

Do the mappings in `ys` appear in the same order as `follow`? Why?

b) With `xs` and `ys` declared in REPL as above, match expressions on the left with the correct result on the right. If you are unsure about complex expressions, try simpler expressions in REPL and examine the value and type of partial results.

`xs(2) + xs(4)`	1	A	`8`
`ys(0)`	2	B	`7`
`xs(0)`	3	C	`(10, 11)`
`(xs + (0 -> 1)).apply(0)`	4	D	`1`
`xs.keySet.apply(2)`	5	E	`(16, 17)`
`xs isDefinedAt 0`	6	F	`-9`
`xs.getOrElse(0, 7)`	7	G	`true`
`xs.maxBy(_._2)`	8	H	`false`
`xs.map(p => p._1 -> -p._2)(8)`	9	I	`NoSuchElementException`

Task 9. *Registration in mutable key-value table.* In this task, you will implement a helper class for registration in a frequency table which you will then use in this week's lab. The class should be called `FreqMapBuilder` that, after repeated calls to the method `add(s: String): Unit`, can create frequency tables of type `Map[String, Int]`, where the key-value pairs in the table indicate the number of occurrences of a certain string. You should start from the code below.
The class uses a mutable table internally. After having added many strings, you can obtain an immutable table for lookup of frequencies for specific strings with the method `toMap`. Read in the quick reference about which extra update methods are offered by `mutable.Map[K, V]`.

```
class FreqMapBuilder:
  private val register = collection.mutable.Map.empty[String, Int]
  def toMap: Map[String, Int] = register.toMap
  def add(s: String): Unit = ???

object FreqMapBuilder:
  def apply(xs: String*): FreqMapBuilder = ???
```

Implement and test `FreqMapBuilder`. *Tip:* You can for example use `mutable.Map`'s methods `addOne` and `getOrElse`.

Task 10. *The method `sliding`.* In this week's lab, you will find the method `sliding` useful, which gives an iterator for special subsequences of a sequence, which can be likened to the "view" in a "sliding window".

a) Run the code below in REPL and describe what happens.

```
scala> val xs = Vector("five", "cucumbers", "is", "more", "than", "four", "toma
scala> xs.sliding(2).toVector
scala> xs.sliding(3).toVector
scala> xs.sliding(10).toVector
```

b) Use `xs.sliding(2)` and convert each element in the result to a pair. Then convert the sequence of pairs into a key-value table. What can the table be used for?

Task 11. *Reading text from file and web servers.* In the lab, you will build tables from data in text format. Then you will find it useful to be able to read text from files and from the web. Test this in REPL:

```
scala> val url = "https://fileadmin.cs.lth.se/pgk/europa.txt"
scala> val xs = io.Source.fromURL(url, "UTF-8").getLines.toVector
scala> val data = xs.map(_.split(';').toVector)
scala> data.head
scala> data.foreach(println)
```

a) Create these tables from the sequence `data`:

```
val populationOf: Map[String, Int]    = ???  // countries' population
val sizeOf:       Map[String, Int]    = ???  // countries' area in km^2
val capitalOf:    Map[String, String] = ???  // countries' capitals
```

Test the tables in REPL.

b) Save the data in a text file `europa.txt`. Read the data from the file with the method
`Source.fromFile(filename, encoding)` in a similar way as with `fromURL` above. If you run in a Linux terminal you can easily download a file as follows (note that the option `-sLO` has a capital letter `O` and not a zero):

```
> curl -sLO https://fileadmin.cs.lth.se/pgk/europa.txt
```

Print all lines in `europa.txt` using `Source.fromFile` in REPL.

9.2 Extra Tasks; additional practice

Task 12. *Create a text game using tables.* Make a simple game to practice various facts about Europe's countries and capitals by reading data from the URL:
https://fileadmin.cs.lth.se/pgk/europa.txt
There is text encoded in UTF-8 with the following content (only the first lines are shown):

```
Land;Invånarantal;Storlek(km^2);Huvudstad
Albanien;3581655;28748;Tirana
Andorra;71201;468;Andorra la Vella
Belgien;10584534;30528;Bryssel
Bosnien-Hercegovina;4590310;51129;Sarajevo
Bulgarien;7385367;110910;Sofia
Cypern;854000;9250;Nicosia
Danmark;5475791;43094;Köpenhamn
Estland;1324333;45226;Tallinn
Finland;5315280;338145;Helsingfors
Frankrike;61538322;551695;Paris
Färöarna;48344;139574;Torshamn
Grekland;10964021;131940;Aten
// ... etcetera for all European countries.
```

Let for example the user answer random questions of the type:

- Does Andorra have more inhabitants than Cyprus?

- What is the capital of Bulgaria?
- Does Denmark have a larger area than Finland?

Use immutable tables with suitable keys and values. You can use a set with countries/-capitals that the user has answered correctly so far to prevent these from recurring again.

9.3 Advanced Tasks; challenges

Task 13. *Registration with `groupBy`.* We will now use a really clever trick to implement registration using just one line of code with the collection methods `groupBy` and `map`.

a) Read about the method `groupBy` in the quick reference. You find it under the heading *"Methods in trait `Iterable[A]`"* since `groupBy` works on all collections. Test `groupBy` as below and describe what happens.

```
scala> val xs = Vector(1, 1, 2, 2, 4, 4, 4).groupBy(x => x > 2)
scala> val ys = Vector(1, 1, 2, 2, 4, 4, 4).groupBy(x => x)
```

b) Create a function `freq` with the function header below that returns a table with the count of occurrences of different integers in `xs`. Test `freq` on a sequence of 1000 random dice throws and explain how the function `freq` works. *Tip:* Do first `groupBy(???)` then `map(???)`.

```
def freq(xs: Vector[Int]): Map[Int, Int] = ???

def toss(n: Int): Vector[Int] =
  Vector.fill(n)(scala.util.Random.nextInt(6) + 1)
```

Task 14. *Iterate at once with `groupMap` and `groupMapReduce`.* A student at LTH has created the following shopping list:

```
enum Department:
  case Vegetables, Fruit, Dairy, Meat, Grain

case class Grocery(product: String, department: Department, price: Int)
val shoppinglist = List(
    Food("tomato", Department.Vegetables, 10),
    Food("ground beef", Department.Meat, 60),
    Food("cucumber", Department.Vegetables, 5),
    Food("banana", Department.Fruit, 20),
    Food("milk", Department.Dairy, 20),
    Food("sausage", Department.Meat, 50),
    Food("apple", Department.Fruit, 30),
    Food("orange", Department.Fruit, 40),
  )
```

To make a rational division of the list and calculate costs, the student studies the methods `groupMap` and `groupMapReduce` carefully here:
https://superruzafa.github.io/visual-scala-reference/

a) Use the function `groupMap` to split the groceries based on the department where they can be found.

b) Use the function `groupMapReduce` to find out what the total costs will be for each department.

Task 15. *Writing to file.* As an aid when you create your own interesting applications or build on the course's labs and exercises with optional extra tasks, you can use the functions in the singleton object `IO` below, which exists in the course's scala library introprog.[39]
The IO module uses `scala.io.Source` to serialize and deserialize strings to and from ordinary text files. The IO module also uses the package `java.io` to offer functions that make it easy to serialize/deserialize arbitrary objects created by serializable classes to/from binary files. Case classes in Scala become automatically serializable.
In the implementation of `IO`, `try ... finally` is used to ensure that files are not left open even if something goes wrong during the read/write process that is handled by the underlying operating system.

a) Compile and run the code below with `introprog` on the classpath, e.g. using `sbt`.

```
import introprog.IO

case class Player(name: String)

@main def run(): Unit =
  println("Test of output/input objects to/from disk:")
  val highscores = Map(Player("Sandra") -> 42, Player("Björn") -> 5)
  IO.saveObject(highscores,"highscores.ser")
  val highscores2 = IO.loadObject[Map[Player, Int]]("highscores.ser")
  val isSameContents = highscores2 == highscores
  val testResult = if (isSameContents) "SUCCESS :)" else "FAILURE :("
  println(testResult)
```

b) Use the `IO` module to save the user's score results in your game about Europe's countries and cities, in the extra task above. The implementation of `introprog.IO` can be found here: `https://github.com/lunduniversity/introprog-scalalib/blob/master/src/main/scala/introprog/IO.scala`

[39]The source code can be found here and also on page **??**:
https://github.com/lunduniversity/introprog-scalalib/blob/master/src/main/scala/introprog/IO.scala

10. Exercise Excercise10

Goals

- ☐ Be able to declare and use an inheritance hierarchy with multiple levels using the keyword **extends**.
- ☐ Be familiar with visibility rules in inheritance and the usefulness of private and protected members and the keywords **private** and **protected**.
- ☐ Be able to declare and use overridden members and the keyword **override**.
- ☐ Be able to declare and use a hierarchy of classes where constructor parameters are passed to superclasses.
- ☐ Be able to declare and use enumerated values with case objects and a common base type.
- ☐ Be familiar with the rules that apply when overriding different kinds of members.
- ☐ Be familiar with the usefulness of final classes and final attributes and the keyword **final**.
- ☐ Knowledge of these concepts: base type, supertype, subtype, runtime type, dynamic binding, polymorphism, trait, mix-in, overridden member, anonymous class, protected member, abstract member, abstract class, reference type,

Preparations

- ☐ Study the concepts in chapter 10

10.1 Basic Tasks; preparation for the lab

Task 1. *Match concepts with descriptions.*

Connect each concept with the (simplified) description that fits best:

base type	1	A	has supertype `AnyRef`, allocated on heap via reference
super type	2	B	can have many forms, e.g., one of several subtypes
subtype	3	C	class without name, extended with extra implementation
runtime type	4	D	a type that is more specific
dynamic binding	5	E	can have parameters, cannot be instantiated, cannot be mixed i
polymorphism	6	F	has no implementation
trait	7	G	has supertype `AnyVal`, stored directly on stack
mix-in	8	H	add properties with **with** and a trait
overridden member	9	I	abstract, can be mixed in, can have parameters
anonymous class	10	J	may be more specific than static type
protected member	11	K	visible only in subtypes
abstract member	12	L	runtime type determines which method executes
abstract class	13	M	subtype member replaces supertype member
sealed type	14	N	most general type in an inheritance hierarchy
reference type	15	O	a type that is more general
value type	16	P	subtyping outside this file is prevented

Task 2. *Common base type.* One often wants to put objects of different types in the same collection.

```
scala> class Cucumber(val weight: Int)
scala> class Tomat(val weight: Int)
scala> val cucumbers = Vector(Cucumber(100), Cucumber(200))
scala> val vegetables = Vector(Cucumber(300), Tomat(42))
```

a) If a collection contains objects of several different types, the compiler tries to infer the most specific common type of the objects. What is the type of the value `vegetables` above?

b) Try to find the sum of the weights as below. What error does the second line give? Why?

```
scala> cucumbers.map(_.weight).sum     // works
scala> vegetables.map(_.weight).sum  // does not work
```

c) You will now make it possible to access the weight of all vegetables by giving cucumbers and tomatoes a common base type that the different concrete vegetable types extend using the keyword **`extends`**. It is said that the subtypes `Cucumber` and `Tomat` **inherit** the properties of the supertype `Vegetable`.
Create a base type `Vegetable` with an abstract attribute `weight`. Then let the concrete vegetables inherit the base type:

```
scala> trait Vegetable { val weight: Int }
scala> class Cucumber(val weight: Int) extends Vegetable
scala> class Tomat(val weight: Int) extends Vegetable
scala> val cucumbers = Vector(Cucumber(100), Cucumber(200))
scala> val vegetables = Vector(Cucumber(300), Tomat(42))
```

When does initialization of the attribute `weight` occur?

d) What is now the type of the variable `vegetables` above?

e) Is it now possible to sum the weights in `vegetables` with the expression below? Why?
`vegetables.map(_.weight).sum`

f) A trait resembles a class, but you cannot instantiate it directly. What error message do you get if you try to create an instance of a trait as below?

```
scala> trait Vegetable { val weight: Int }
scala> new Vegetable
```

g) The trait `Vegetable` has an abstract member `weight`. It is said to be abstract because it lacks implementation – the member has only a name and a type but no value. You can instantiate the abstract trait `Vegetable` if you provide what is ”missing”, namely a value for `weight`. You can supply the missing parts by ”attaching” a block after the type name during instantiation. You then get what is called an **anonymous class**, in this case a rather unusual vegetable that is not a specific kind of vegetable but still has a weight.
What type and string representation does the `anonymousVegetable` below have?

```
scala> val anonymousVegetable = new Vegetable { val weight = 42 }
```

h) What error message do you get if you create an anonymous class `Vegetable` with a body missing the definition of `weight`?

Task 3. *Polymorphism in inheritance, i.e. subtype polymorphism.* Polymorphism means "many shapes". In the context of inheritance this means that several subtypes, for example `Cow` and `Pig`, can be handled jointly as if they were instances of the same supertype, such as `Animal`. Subclasses can implement a method with the same name in different ways. Which method is executed is determined at runtime depending on which subtype is instantiated. In this way, animals can come in many forms.

a) Implement the function `createAnimal` below so that it returns either a new `Cow` or a new `Pig` with equal probability.

```
scala> trait Animal { def makeNoise: Unit }
scala> class Cow   extends Animal { def makeNoise = println("Moo") }
scala> class Pig   extends Animal { def makeNoise = println("Oink oink") }
scala> def createAnimal(): Animal = ???
scala> val farm = Vector.fill(42)(createAnimal())
scala> farm.foreach(_.makeNoise)
```

b) Add an animal of type Horse that makes an appropriate sound and modify `createAnimal` so that it creates cows, pigs, and horses with equal probability.

Task 4. *Various types of integer pairs for the lab* `snake0`. **NOTE! Do this task *before* you look at given code in the lab to avoid spoiling the exercise.**
During this week's lab you will use various types of pairs representing direction and position on a two-dimensional game board, as well as the game board's size. Instead of using a regular 2-tuple for these three different types of pairs, you should create your own specific types all inheriting the base type `Pair[T]`. These types should all reside in the file `pairs.scala` in `package snake`.

```
// this is a sketch of the file pairs.scala
package snake

trait Pair[T]:
  def x: T
  def y: T
  // task a) add the concrete method tuple

// subsequent subtasks implement these subtypes of Pair:
//   case class Dim describes a 2-dimensional surface size
//   case class Pos describes a position on a surface of size Dim
//   enum Dir describes movement towards North, South, East, West
```

The difference between `Pair[T]` and a regular 2-tuple is that the members `x` and `y` are guaranteed to be of the *same* type, whereas a 2-tuple can contain elements of different types.
Figure 2 shows an image of the class hierarchy which you will develop step-by-step in subsequent tasks. The advantage of having different types of pairs is that it is more type-safe: the compiler helps us detect if we accidentally confuse, e.g., a position with a direction.

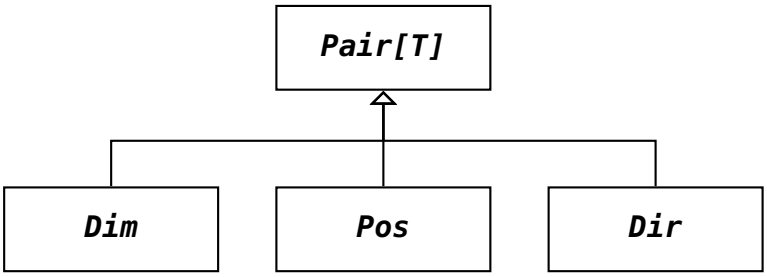


Figure 2: Inheritance hierarchy with `Pair[T]` as base type.

a) Open an editor and write the **`trait`** `Pair[T]` in a file `pairs.scala`. Also add a concrete method `tuple` in `Pair[T]` which returns a 2-tuple with the two elements of the pair, so that it is possible to convert `Pair` instances to 2-tuples if needed. Use REPL to test that this works:

```
scala> val p = new Pair[Int] { override val x = 10; override val y = 20 }
p: Pair[Int]{val x: Int; val y: Int} = $anon$1@784223e9

scala> p.tuple
val res0: (Int, Int) = (10,20)
```

b) Does the above code also work without the keyword **`override`** (test in REPL)? Why? When must **`override`** be used? What are the advantages and disadvantages of using **`override`** even when it is not necessary?

c) Create a case class `Dim` which extends `Pair[Int]`. Instances of this class will be used during this week's lab to represent a game board size by letting `x` specify the number of horizontal positions and `y` the number of vertical positions.
Also add a companion object `Dim` with an `apply` method which can create `Dim` instances given a 2-tuple. Test in REPL as below.

```
scala> Dim(50, 60)
val res1: Dim = Dim(50,60)

scala> Dim((60, 50))
val res2: Dim = Dim(60,50)

scala> res2.tuple
val res3: (Int, Int) = (60,50)
```

d) Add a case class `Pos` which extends `Pair[Int]` representing a position with an `x`-coordinate and a `y`-coordinate, both class parameters. The coordinates must be within a game board size given by the class parameter `dim` of type `Dim`. The coordinate positions are integers and counted from `0` up to (but not including) `dim.x` and `dim.y`, respectively.
Make the primary constructor in the case class `Pos` **private**, by writing the keyword **`private`** after the class name but before the parameter list, so that it is not possible to create instances via the primary constructor outside the class body and the companion object.
Implement methods `+` and `-` in the case class `Pos`. Both methods should take a parameter `p` of type `Pair[Int]` and return a new `Pos`, where `p.x` and `p.y` have been added or subtracted from the current position. Note that you should not write **`new`**

when creating a new instance, since these should always be created via the companion object's `apply` method, which is a "smart" factory method ensuring coordinates stay within the game board.
Add a companion object `Pos` with an `apply` method which creates a new `Pos` instance that ensures coordinates are always within `dim`. The arithmetic should be modulo the size `dim`, i.e. a position should never be outside the game board; instead, it wraps around (see example in REPL below).
Tip: Use `java.lang.Math.floorMod` which handles negative arguments to produce a positive result (unlike the modulo operator %).
Also add the factory method `random` which can create new random positions within `dim`. *Tip:* Use `scala.util.Random.nextInt`.
Test that it works according to below:

```
scala> Pos(-1,20,Dim(10,20))
val res4: Pos = Pos(9,0,Dim(10,20))

scala> new Pos(-1,20,Dim(10,20))  // forbidden with private primary constructor
-- Error:
1 |new Pos(-1,20,Dim(10,20))
  |    ^^^
  |constructor Pos cannot be accessed as a member of Pos

scala> Pos(0,0,Dim(5,5)) + Pos(6,12, Dim(5,5))
val res5: Pos = Pos(1,2,Dim(5,5))

scala> Pos(0,0,Dim(5,5)) - Pos(1,2, Dim(5,5))
val res6: Pos = Pos(4,3,Dim(5,5))

scala> for (_ <- 1 to 3) yield Pos.random(Dim(10,10))
val res7: IndexedSeq[Pos] =
  Vector(Pos(8,8,Dim(10,10)), Pos(2,6,Dim(10,10)), Pos(3,7,Dim(10,10)))
```

e) What happens if you forget to write **new** when calling the private constructor in your `apply` method? Why does this problem not occur in the `apply` method of `Dim`?

f) Add an **enum** `Dir` that extends `Pair[Int]` and has two **val** parameters `x` and `y`. Also add four **case** values all extending `Dir` representing a single-step movement in the four cardinal directions, by giving the parameters `x` and `y` values of 1, −1 or 0. North should be represented with x-coordinate 0 and y-coordinate −1, etc. Verify in REPL that the enumeration works.
Add an **export** that makes it enough to import `snake.*` to get all four directions directly visible (otherwise import of `Dir.*` is needed everywhere direction is used in and outside the `snake` package).

Task 5. *Supertype with parameter.* The Ministry of Education wants to keep track of certain persons with its new data system and therefore creates a class hierarchy as below. Write the code in an editor and test it in REPL with `sbt`.

```
class Person(val name: String)

class Academic(
  name: String,
  val university: String) extends Person(name)
```

```
class Student(
  name: String,
  university: String,
  val program: String) extends Academic(name, university)

class Researcher(
  name: String,
  university: String,
  val title: String) extends Academic(name, university)
```

a) Declare four different `val` variables with suitable names referencing instances of all the different classes above and give the attributes arbitrary initial values through various parameters to the constructors.

b) Write two statements: one that first inserts the instances into a `Vector` and one that then loops through the vector and prints all instances' `toString` and `namn`.

c) The Ministry of Education wants to prevent instantiating objects of types `Person` and `Academic`. This can be achieved by placing the keyword `abstract` before `class`. Update the code accordingly. What is the error message if you try to instantiate an `abstract class`? Is it just as good with a `trait`?

d) The Ministry wants to avoid implementing `toString`. Therefore, convert the types `Student` and `Researcher` to case classes. *Tip:* To avoid a compilation error (which one?), you need to use `override val` in the appropriate places. Create instances of the new case classes `Student` and `Researcher` and print their `toString`.

e) Use abstract attributes instead of parameters for the abstract types, so that you don't need to write `override val` in the class parameters for the concrete case classes. You should also add a case class `NonAcademic` which will be used in various statistical citizen surveys. Also, all persons are given a personal identification number represented as a `Long`. What does the Ministry of Education's code look like now, after all changes? Write a test program that creates some instances and prints their attributes.

10.2 Extra Tasks; additional practice

Task 6. *Base type `Shape` and subtypes `Rectangle` and `Circle`.* You are to create a small library for geometric shapes with immutable objects implemented using case classes. The geometric shapes have a common base type called `Shape`. Start from the code below.

```
case class Point(x: Double, y: Double):
  def move(dx: Double, dy: Double): Point = Point(x + dx, y + dy)

trait Shape:
  def pos: Point
  def move(dx: Double, dy: Double): Shape

case class Rectangle(pos: Point, width: Double, height: Double) extends Shape:
  def move(dx: Double, dy: Double): Rectangle = copy(pos = pos.move(dx, dy))

case class Circle(pos: Point, radius: Double) extends Shape:
  def move(dx: Double, dy: Double): Circle = copy(pos = pos.move(dx, dy))
```

a) Instantiate some circles and rectangles and do some relative moves of your instances by calling `move`.

b) Add a concrete method `moveTo` in `Point` which makes an absolute move to the coordinates `x` and `y`. Add an abstract method `moveTo` to `Shape` which is implemented in the subclasses. Test with REPL on some instances of `Rectangle` and `Circle`.

c) Add the method `distanceTo(that: Point): Double` in the case class `Point` which calculates the distance to another point using `math.hypot`. Paste into REPL and test on some instances of `Point`.

d) Add a concrete method `distanceTo(that: Shape): Double` in the trait `Shape` which calculates the distance to the position of another Shape. Test in REPL on some instances of `Rectangle` and `Circle`.

e) Make it so that `distanceTo` can be called with operator notation.

10.3 Advanced Tasks; challenges

Task 7. *Mixins.* One can extend a class with multiple traits using a comma-separated list. In this way, members can be distributed in different traits and common code parts can be reused by so-called **mixins**, as the following example shows.
An alternative bird taxonomy divides all birds into two specific categories: Crow (`Crow`) and Duck (`Duck`). Crows can fly but not swim, while ducks can swim and often also fly.

```
trait Flock:
  val sound: String
  def makeNoise: Unit = print(sound * 2)
  val isAbleToSwim: Boolean
  val isAbleToFly: Boolean

trait CanSwim          { val isAbleToSwim = true }
trait CannotSwim       { val isAbleToSwim = false }
trait CanFly           { val isAbleToFly = true }
trait MayBeAbleToFly   { val isAbleToFly = math.random() < 0.8 }

class Crow extends Flock, CanFly, CannotSwim:
  val sound = "caw"

class Duck extends Flock, CanSwim, MayBeAbleToFly:
  val sound = "quack"
  override def makeNoise = print(sound * 4)
```

a) A diligent ornithologist finds 42 birds in a perfect forest where all bird species are equally likely, represented by the vector `flock` below. Write in REPL an expression that finds how many of them are flying ducks by using the methods `filter`, `isInstanceOf`, `isAbleToFly`, and `size`.

```
scala> val flock =
         Vector.fill(42)(if math.random() > 0.5 then new Crow else new Duck)
scala> flock.foreach(_.makeNoise)
scala> val duckCount: Int =
```

b) If all the birds the ornithologist found would make sound exactly once each, how many krax and how many kwak would then be heard? Use the methods `filter` and

size, as well as the predicate isAbleToSwim to calculate the numbers of krax and kwak.

```
scala> val numberOfCrows: Int = ???
scala> val numberOfDucks: Int = ???
```

Task 8. *Final classes.* If you want to prevent a class from being extended, you can make it final by placing the keyword **final** before **class**.

a) Since the classification of birds in the previous task into either Duck or Crow is complete and there are no subtypes of either, it is appropriate to make these classes final. Change this in your code.

b) Try nevertheless to create a subclass SwimCrow **extends** Crow. What is the compiler error if you try to extend a final class?

Task 9. *Access rules in inheritance and the keyword **protected**.* If a member in a supertype is private, it cannot be accessed in a subclass. Sometimes you want the subclass to be able to access a member even if it should be inaccessible in other code.

```
trait Super:
  private val myHome = 42
  protected val ourHome = 42

class Sub extends Super:
  def reveal = myHome
  def cryptic = ourHome * math.Pi
```

a) What is the error message when class Sub tries to access myHome?

b) Declare Sub anew, but now without the forbidden method reveal. What is the error message if you try to access ourHome via an instance of class Sub? Try for example with (**new** Sub).ourHome

c) Is it possible to call the method cryptic on instances of class Sub?

Task 10. *Using **protected**.* The diligent ornithologist from task 7 will ring-tag all 42 birds found in the forest. While at it, the ornithologist also decides to try to find out how much noise is created by each bird species. For this purpose the ornithologist equips a Linux computer on all captured birds. You will help write the program.

a) Introduce a **protected var** soundCount in trait Fyle and write code in suitable places to count how many sounds each bird instance has uttered.

b) Add a method soundCount that returns the number of krax or kwak that a bird has uttered since its creation.

c) Why not use **private** instead of **protected**?

d) Why is it good to make the counter variable inaccessible from the ”outside”?

Task 11. *Mixin of properties.* It turns out that the diligent ornithologist from task 7 on page 111 had slept through a zoology lecture when a freshman at Natfak, and therefore completely missed the fyle category Chick. Help our poor ornithologist so that the bird hierarchy now also includes Chick. A Chick can fly like a Crow but is isSmall whereas a Crow isBig. A Chick chirps twice as many times as a Duck quacks. A Chick MaybeCanSwim with swimming ability probability 0.2. Let the ornithologist

again find 42 random birds in the forest and filter out suitable species. Then examine how they make noise, similarly to subtask 7b.

Task 12. *Inheritance hierarchy with mathematical numbers.* Study the deep inheritance hierarchy in the package `numbers` in the code on the following pages. The package `numbers` models various kinds of numbers in mathematics, aiming to offer a so-called DSL[40], i.e. a domain-specific language[41], here: the domain of mathematical numbers.
You can download the `numbers` code here:
github.com/lunduniversity/introprog/blob/master/compendium/examples/numbers.scala
Note especially the method `reduce` which reduces a number to its simplest form. The method `reduce` is overridden as appropriate with relevant reduction.

a) Draw a diagram of the type hierarchy, e.g., as an upside-down tree with the base type `Number` as root.

b) Write code that uses the various concrete classes in **`package`** `numbers`.

```
scala> numbers.  // Press Tab
AbstractComplex   AbstractNatural    AbstractReal   Frac    Nat      Polar
AbstractInteger   AbstractRational   Complex        Integ   Number   Real

scala> numbers.Integ(12)
res0: numbers.Integ = Integ(12)

scala> import numbers.Syntax._
import numbers.Syntax._

scala> 42.j
res1: numbers.Complex = Complex(Real(0),Real(42))

scala> 42.42.j
res2: numbers.Complex = Complex(Real(0),Real(42.42))
```

c) Change the method + in **`trait`** `Number` so that it becomes abstract and implement it in all concrete classes.

d) Implement more arithmetic operations and build on the code as you find interesting.

e) Make the `reduce` method in class `AbstractRational` use the Greatest Common Divisor (GCD) algorithm[42] as described here:
www.artima.com/pins1ed/functional-objects.html#6.8
so that numerator and denominator become as small as possible.

```
package numbers

trait Number:
  def reduce: Number = this
  def isZero: Boolean
  def isOne: Boolean
  def +(that: Number): Number = ???

object Number:
```

[40] https://en.wikipedia.org/wiki/Domain-specific_language
[41] https://stackoverflow.com/questions/49216312/what-is-dsl-in-scala
[42] https://sv.wikipedia.org/wiki/St%C3%B6rsta_gemensamma_delare

```
  def Zero = Nat(0)
  def One  = Nat(1)
  def Im(im: BigDecimal) = Complex(Real(0), Real(im))
  def Im(im: Real)       = Complex(Real(0), im)
  def j                  = Complex(Real(0), Real(1))
  def Re(re: BigDecimal) = Complex(Real(re), Real(0))
  def Re(re: Real)       = Complex(re, Real(0))

trait AbstractComplex extends Number:
  def re: AbstractReal
  def im: AbstractReal
  def abs = Real(math.hypot(re.decimal.toDouble,im.decimal.toDouble))
  def fi =  Real(math.atan2(im.decimal.toDouble, re.decimal.toDouble))
  override def isZero: Boolean = re.decimal == 0 && im.decimal == 0
  override def isOne: Boolean = abs.decimal == 1.0
  override def reduce: AbstractComplex = if im.decimal == 0 then re.reduce else this

final case class Complex(re: Real, im: Real) extends AbstractComplex

object Complex:
  def apply(re: BigDecimal, im: BigDecimal) = new Complex(Real(re), Real(im))

final case class Polar(
    override val abs: Real,
    override val fi: Real
  ) extends AbstractComplex:
  override def re = Real(abs.decimal.toDouble * math.cos(fi.decimal.toDouble))
  override def im = Real(abs.decimal.toDouble * math.sin(fi.decimal.toDouble))

object Polar:
  def apply(abs: BigDecimal, fi: BigDecimal) = new Polar(Real(abs), Real(fi))

trait AbstractReal extends AbstractComplex:
  def decimal: BigDecimal
  override def isZero = decimal == 0
  override def isOne = decimal == 1
  override def re = this
  override def im = Number.Zero
  override def reduce:  AbstractReal =
    if decimal == 0 then Number.Zero else if decimal == 1 then Number.One else this

final case class Real(decimal: BigDecimal) extends AbstractReal

trait AbstractRational extends AbstractReal:
  def numerator: AbstractInteger
  def denominator: AbstractInteger
  override def decimal = BigDecimal(numerator.integ)
  override def isOne = numerator.integ == denominator.integ
  override def reduce: AbstractRational =
    if denominator.isOne then numerator.reduce else this // should use GCD

final case class Frac(numerator: Integ, denominator: Integ) extends AbstractRational:
  require(denominator.integ != 0, "denominator must be non-zero")

object Frac:
  def apply(n: BigInt, d: BigInt) = new Frac(Integ(n), Integ(d))
```

```

trait AbstractInteger extends AbstractRational:
  def integ: BigInt
  override def numerator = this
  override def denominator = Number.One
  override def isZero = integ == 0
  override def isOne = integ == 1
  override def decimal: BigDecimal = BigDecimal(integ)
  override def reduce: AbstractInteger =
    if isZero then Number.Zero else if isOne then Number.One else this

final case class Integ(integ: BigInt) extends AbstractInteger

trait AbstractNatural extends AbstractInteger

final case class Nat(integ: BigInt) extends AbstractNatural:
  require(integ >= 0, "natural numbers must be non-negative")

extension (i: Int)    def j = Number.Im(i)
extension (d: Double) def j = Number.Im(d)
```

11. Exercise `Excercise11`

Goals

- ☐ Be able to explain what a context parameter is.
- ☐ Be able to explain the benefits of context parameters compared to global variables and default arguments when solving the configuration problem.
- ☐ Be able to use simple contextual abstractions with **`given`** and **`using`**.

Preparations

- ☐ Study the concepts in chapter 11

11.1 Basic Tasks; preparation for the lab

Task 1. *Context parameter.* Declare the following functions that take an integer as a context parameter. Also create a **`given`** declaration that provides the given integer value zero:

```
scala> def f(using i: Int) = i + 1

scala> def g(x: Int)(using y: Int) = x + y
```

a) Call the functions `f` and `g` with an explicit given argument that differs from the given integer value using **`using`** in the call. What happens if you omit **`using`**?

b) Call the functions `f` and `g` without specifying a **`using`** argument. Explain what happens.

c) Is it possible to mix regular parameters and context parameters in the same parameter list? If not, what happens?

Task 2. *Multiple given values in local context.* Different values depending on context.

```
case class Delta(value: Int)
object Delta:
  given default: Delta = Delta(1)

def inc(x: Int)(using dx: Delta) = x + dx.value

object Context1:
  val a = inc(1)

object Context2:
  given Delta = Delta(42)
  val a = inc(1)
```

a) What is the value of `Context1.a`?

b) What is the value of `Context2.a`?

c) Explain what happens.

Task 3. *Solution to the configuration problem using given values.* Suppose we want to be able to configure the behavior of a function to make it more flexible. Below are three conceptually different ways to do this for a function `greet` that prints a greeting: 1) a globally accessible variable, 2) default argument, and 3) contextual abstraction with **given** and **using**.

```
object GlobalVar:
  case class GreetConfig(greeting: String, receiver: String)
  object GreetConfig:
    val default = GreetConfig(greeting = "Hello", receiver = "World")
    var config = default

  def greetMsg =
    s"${GreetConfig.config.greeting} ${GreetConfig.config.receiver}!"

object DefaultArgs:
  case class GreetConfig(greeting: String, receiver: String)
  object GreetConfig:
    val default = GreetConfig(greeting = "Hello", receiver = "World")

  def greetMsg(config: GreetConfig = GreetConfig.default) =
    s"${config.greeting} ${config.receiver}!"

object GivenVal:
  case class GreetConfig(greeting: String, receiver: String)
  object GreetConfig:
    given default: GreetConfig = GreetConfig("Hello", "World")

  def greetMsg(using g: GreetConfig) = s"${g.greeting} ${g.receiver}"
```

a) Write code to test the different variants above. Show especially how you can use the default configuration and then provide a configuration that differs from `default`.

b) What are the pros and cons of the different variants above? Discuss especially which solution(s) allow multiple local configurations without affecting each other.

c) Explain what happens when calling `summon[GivenVal.GreetConfig]`.

d) What happens if you try to summon a given value for a type that has none?

e) Must the given value be unique?

11.2 Extra Tasks; additional practice

Task 4. *Context parameter and given value.* Try the following in REPL.

```
1 scala> def add(x: Int)(using y: Int) = x + y
2 scala> add(1)(using 2)
3 scala> add(1)
4 scala> given someName = 42
5 scala> add(1)
```

a) What is the error message on line 3 above?

b) Why does it work on line 5 without error?

c) Define and test a corresponding function `sub` that can subtract a given value.

11.3 Advanced Tasks; challenges

Task 5. *Variance and type bounds.* The code below models pets with the following meaning: Pets can be healthy or sick and are normally born healthy. There can be many cats and dogs, which are all different kinds of pets.

```
trait Pet(var isHealthy: Boolean = true)
class Cat extends Pet()
class Dog extends Pet()
```

a) Modify the code below so that the subsequent REPL statement *does not* give a compilation error?

```
case class Box[A](x: A)
```

```
scala> val b: Box[Any] = Box[Cat](Cat())
```

b) Try below in REPL and explain what happens.

```
scala> val v: Vector[Pet] = Vector[Cat](Cat())

scala> val s: Set[Pet] = Set[Cat](Cat())

scala> :settings -explain

scala> val s: Set[Pet] = Set[Cat](Cat())
```

Hint: In Scala's standard library, `Set[T]` inherits the function type `T => Boolean` which is declared contravariant in its input parameter.

c) There should be veterinarians who can treat pets and make them healthy. Why does the following code not work? Is it a compilation error or runtime error?

```
class Vet[-A]:
  def treat(x: A): Unit = x.isHealthy = true
```

d) Add a type bound to the veterinarian's type parameter to fix the error.

e) Write code to show 1) that the compiler allows cat veterinarians to treat cats but 2) prevents cat veterinarians from treating arbitrary pets, and that 3) a veterinarian who has competence to treat arbitrary pets can treat both cats and dogs. Explain why the compiler allows/prevents these.

Task 6. *Type classes and context parameters.* In Scala there are possibilities for advanced functional programming with so-called **type classes** (i.e. *ad hoc polymorphism*). A type class defines general behaviors that work for arbitrary existing types without needing to change their implementation. We explore here how contextual abstractions can be used to create type classes in Scala, illustrated using given orderings for sorting.

By combining concepts of given values, generic classes, and context parameters, one obtains ad hoc polymorphism, exemplified with the type class `CanCompare` below, which we can get to work for existing types *without* needing to change them. Especially, we

do not have the possibility to add methods to existing types in the standard library since that is not our own code.

a) What happens below? Which lines give error messages? Why?

```
scala> trait CanCompare[T]:
         def compare(a: T, b: T): Int

scala> def sort[T](a: T, b: T)(using cc: CanCompare[T]): (T, T) =
         if cc.compare(a, b) > 0 then (b, a) else (a, b)

scala> sort(42, 41)

scala> given intComparator: CanCompare[Int] with
         override def compare(a: Int, b: Int): Int = a - b

scala> sort(42, 41)

scala> sort(42.0, 41.0)
```

b) Define a given value so that `sort` works for values of type `Double`.

c) Define a given value so that `sort` works for values of type `String`. *Tip:* You can use existing comparison operators on strings, but keep in mind that `compare` still must return an integer even for string comparisons.

Task 7. *Usage of given ordering.* We are now going to create a function `isSorted` that is generally useful by making given orderings available for different types. The function `def isSorted(xs: Vector[Int]): Boolean = ???` works only for collections of type `Vector[Int]`.
If instead we use `def isSorted(xs: Seq[Int]): Boolean = ???` it works for various collections of integers, including `Vector` and `List`.

a) Test the following function in REPL with integer sequences of different types.

```
def isSorted(xs: Seq[Int]): Boolean = xs == xs.sorted
```

b) There is a problem with trying to make `isSorted` generic below. What is the error message? What is missing according to the message?

```
scala> def isSorted[T](xs: Seq[T]): Boolean = xs == xs.sorted
```

c) We want `isSorted` to work for arbitrary types `T` that have an ordering definition. This can be done with the function below where the special type parameter syntax `[T:Ordering]` means that `isSorted` is defined for all collections where `T` has a given ordering `Ordering[T]`. In particular, this applies to all primitive types `Int`, `Double`, `String`, etc., which all have specific implementations of the type class `Ordering`.

```
def isSorted[T:Ordering](xs: Seq[T]): Boolean = xs == xs.sorted
```

Test the method above in REPL as shown below.

```
scala> isSorted(Vector(1,2,3))
scala> isSorted(List(1,2,3,1))
scala> isSorted(Vector("A","B","C"))
scala> isSorted(List("A","B","C","A"))
scala> case class Person(firstName: String, familyName: String)
```

```
scala> val persons = Vector(Person("Kim", "Finkodare"), Person("Voldemort","Fu
scala> isSorted(persons)
```

What error message does the last line give? Why?

d) *Implicit orderings.* A type parameter of the form `[T:Ordering]` is called a context bound and causes the compiler to automatically expand the function header for `isSorted` with a context parameter. Instead of using `[T:Ordering]` we can manually add a context parameter corresponding to the context bound. This also gives us access to a name, here `ord`, for the implicit ordering and can use that name in the function body and call methods that are members of the type class `Ordering`. (The name of the context parameter can also be omitted, but then we must go via the built-in function `summon[T]` to ask the compiler to search for the given instance for the type provided at the call.)

```
def isSorted[T](xs: Seq[T])(using ord: Ordering[T]): Boolean =
  xs.zip(xs.tail).forall(x => ord.lteq(x._1, x._2))
```

Objects of type `Ordering` have comparison methods such as `lteq` (short for *less than or equal*) and `gt` (short for *greater than*).
There are given orderings for all primitive types in the standard library, e.g. `Ordering[Int]`, `Ordering[String]`, etc. Test that the compiler finds the ordering for collections of some primitive types. Also verify below that it still causes problems for user-defined classes, e.g. `Person` (which we will fix in task 8).

```
scala> isSorted(Vector(1,2,3))
scala> isSorted(Array(1,2,3,1))
scala> isSorted(Vector("A","B","C"))
scala> isSorted(List("A","B","C","A"))
scala> class Person(firstName: String, familyName: String)
scala> val persons = Vector(Person("Kim", "Finkodare"), Person("Robin","Fulhack
scala> isSorted(persons)
```

e) *Import implicit ordering operators from an `Ordering`.* If you import members of an `Ordering` object, you get implicit conversions that make comparison operators work. Test the variant of `isSorted` below on different sequence types and verify that <=, >, etc., now work as expected.

```
def isSorted[T](xs: Seq[T])(using ord: Ordering[T]): Boolean =
  import ord.*
  xs.zip(xs.tail).forall(x => x._1 <= x._2)
```

Task 8. *Create your own implicit ordering with `Ordering`.*

a) One way to create a custom, specialized ordering for your own classes is to map your objects to types that already have an implicit ordering. Using the method by in the object `scala.math.Ordering`, you can create orderings by attaching a function `T => S` where `T` is the type of the objects you want to order and `S` is some other type, e.g. `String` or `Int`, where an ordering already exists.

```
scala> case class Team(name: String, rank: Int)
scala> val xs =
         Vector(Team("fnatic", 1499), Team("nip", 1473), Team("lumi", 1601))
scala> xs.sorted  // What is the error message? Why does it error?
scala> val teamNameOrdering: Ordering[Team] = Ordering.by(t => t.name)
```

```
scala> xs.sorted(using teamNameOrdering)   //explicit ordering
scala> given Ordering[Team] = Ordering.by(t => t.rank)
scala> xs.sorted   // Why does it work now?
```

b) To sort in reverse order one can use `Ordering.fromLessThan` which takes a function `(T, T) => Boolean` which should return **`true`** if the first parameter should come before, otherwise **`false`**. To sort by `rank` in reverse order you can do:

```
scala> val highestRankFirst: Ordering[Team] =
         Ordering.fromLessThan((t1, t2) => t1.rank > t2.rank)
scala> xs.sorted(using highestRankFirst)
```

c) If you have a case class with multiple fields and want a predefined implicit sorting order as well as provide an alternative sorting order, you can place a default ordering definition in the companion object; this is one of the places where the compiler searches last for implicit values before giving up.

```
case class Team(name: String, rank: Int)
object Team:
  given highestRankFirst: Ordering[Team] =
    Ordering.fromLessThan((t1, t2) => t1.rank > t2.rank)
  val nameOrdering: Ordering[Team] = Ordering.by(t => t.name)
```

```
scala> val xs =
         Vector(Team("fnatic", 1499), Team("nip", 1473), Team("lumi", 1601))
scala> xs.sorted
scala> xs.sorted(Team.nameOrdering)
```

d) It is also possible with the companion object above to get comparison operators working with your case class by importing members of the appropriate ordering object. Verify that this works as below:

```
scala> Team("fnatic",1499) < Team("gurka", 2)  // What error? Why?
scala> import Team.highestRankFirst.given
scala> Team("fnatic",1499) < Team("gurka", 2)  // No error? Why?
```

Task 9. *Custom ordering by inheriting from `Ordered`* If there is only *one* well-defined, specific order that you want to apply to your case class instances, you can make it ordered by mixing in the trait `Ordered` and implementing the abstract method `compare`. (This illustrates use of subtype polymorphism (i.e. inheritance) instead of ad hoc polymorphism with type classes.)

Background: A trait used this way is called an **interface**, and if you *implement* an interface you fulfill a "contract," which in this case means implementing what is required of ordered objects, namely a concrete `compare` method. You will learn more about interfaces in upcoming courses.

a) Implement the case class `Team` so that it is a subtype of `Ordered` as sketched below. Higher ranked teams should come before lower ranked teams. The method `compare` should return an integer that is negative if `this` comes before `that`, zero if they are equal in ordering, otherwise positive.

```
case class Team(name: String, rank: Int) extends Ordered[Team]:
  override def compare(that: Team): Int = ???
```

Tip: You can call the `compare` method on all primitive types, e.g. `Int`, since they implement the interface `Ordered`. Negating the expression reverses the order.

```
scala> -(2.compare(1))
```

b) Test that your case class now fulfills the requirements to be ordered.

```
scala> Team("fnatic",1499) < Team("gurka", 2)
```

c) Discuss with an instructor or peers the differences and similarities between interfaces and type classes, based on this and the previous exercises.

12. Exercise Excercise12

Goals

- ☐ This week's exercise contains optional deepening.
- ☐ Searching and sorting:
  - Be able to use built-in search methods.
  - Understand when binary search is appropriate and possible.
  - Be able to implement binary search.
  - Be able to implement selection sort, both to a new collection and in place.
- ☐ Threads and concurrent execution:
  - Know what a thread is and be able to explain the concept of concurrent execution.
  - Know what the methods `run` and `start` do in the class `Thread`.
  - Be able to create and start a thread with an overridden `run` method.
  - Be able to create a simple program where two threads compete to update a variable and explain why the behavior can become unpredictable.
  - Be able to use a `Future` to start multiple parallel computations.
  - Be able to register a callback on a `Future` with the method `onComplete`.

12.1 Tasks on searching and sorting

Task 1. *Timing measurement.* In the upcoming tasks, you will find the function `timed` below useful.

```
def timed[T](code: => T): (T, Long) =
  val now = System.nanoTime
  val result = code
  val elapsed = System.nanoTime - now
  println("\ntime: " + (elapsed / 1e6) + " ms")
  (result, elapsed)
```

a) Paste `timed` in the REPL and test that it works by measuring how long the expression below takes to execute.

```
scala> val (v, t1) = timed{ (1 to 1000000).toVector.reverse }
scala> val (s, t2) = timed{ v.toSet }
scala> timed{ v.find(_ == 1) }
scala> timed{ s.find(_ == 1) }
scala> timed{ s.contains(1) }
```

✎ b) Try to explain the differences in execution time between the different ways to search for the number 1 in the collection. Approximately how many times would you need to use `contains` on the integer set `s` for it to be worth creating `s` instead of linear searching in `v` with `find` in the example above?

Task 2. *Searching with built-in search methods.*

a) *Linear search forward with `indexOfSlice`.* Study the documentation for Scala's collection method `indexOfSlice`[43] and write 8 different expressions in the REPL that, using both a string and a vector of integers, test 4 different cases: (1) present at the beginning, (2) present somewhere in the middle, (3) present at the end, and (4) not present.

b) *Linear search backward with `lastIndexOfSlice`.* Study the documentation for Scala's collection method `lastIndexOfSlice`[44] and write 8 different expressions in the REPL that, using both a string and a vector of integers, test 4 different cases: (1) present at the beginning, (2) present somewhere in the middle, (3) present at the end, and (4) not present.

c) *Searching with built-in binary search.* If a collection is sorted, this can be exploited to perform faster search. In **binary search**[45] one begins by looking in the middle and checking which half to continue searching in; then one divides this half in half again to check which quarter to continue in, and so forth.
In the object `scala.collection.Searching`[46] there is a method `search` which, if imported, offers binary search for all sorted sequence collections. If the collection is sorted it returns an object of case class `Found` containing the index where the element was first found; alternatively if the searched-for element does not exist, it returns an object of case class `InsertionPoint` containing the index where the element would have been placed if it existed. Note that if the collection is not sorted, the result is "undefined", i.e., something is returned but it is *not* reliable; so one must first sort the collection or be completely certain that it is sorted.
Investigate how `search` works by explaining what happens below. Which is fastest, `lin` or `bin` below? Use `timed` from task 1.

```
scala> val udda = (1 to 1000000 by 2).toVector
scala> import scala.collection.Searching._
scala> odd.search(odd.last)
scala> odd.search(odd.last + 1)
scala> odd.reverse.search(odd(0))
scala> def lin(x: Int, xs: Seq[Int]) = xs.indexOf(x)
scala> def bin(x: Int, xs: Seq[Int]) = xs.search(x) match
         case Found(i) => i
         case InsertionPoint(i) => -i
scala> timed{ lin(odd.last, odd) }
scala> timed{ bin(odd.last, odd) }
```

d) If a collection contains n elements, how many comparisons are needed in the worst case during binary search? *Tip:* Read about the algorithm on Wikipedia[45].

Task 3. *Search among LTH's courses with linear search.*
TODO!!! Update this exercise so it benefits from the new API on kurser.lth.se, see also `https://github.com/lunduniversity/lth-courses/issues/3`

a) Via this URL, you can download a tab-separated list of all courses offered at LTH this academic year: `https://fileadmin.cs.lth.se/pgk/lthkurser201819.txt`
What data is in the file? You can investigate with e.g.:

[43] docs.scala-lang.org/overviews/collections/seqs.html
[44] docs.scala-lang.org/overviews/collections/seqs.html
[45] `https://en.wikipedia.org/wiki/Binary_search_algorithm`
[46] http://www.scala-lang.org/api/current/scala/collection/Searching$.html

```
scala> import scala.io.Source.fromURL
scala> val url = "https://fileadmin.cs.lth.se/pgk/lthkurser201819.txt"
scala> val data = fromURL(url,"UTF-8").getLines.mkString("\n")
```

b) Paste the object `courses` on page 125 in the REPL.[47] What does the code do? How many courses does `courses.lth` contain?

```
object courses:
  def download(): Vector[Course] =
    val url = "https://fileadmin.cs.lth.se/pgk/lthkurser201819.txt"
    val lines = scala.io.Source.fromURL(url, "UTF-8").getLines.toVector
    lines.drop(1).map(s => Course.fromLine(s, '\t'))

  lazy val lth: Vector[Course] = download()

  case class Course(
    code:    String,
    nameSv:  String,
    nameEn:  String,
    credits: Double,
    level:   String
  )

  object Course:
    def fromLine(line: String, separator: Char): Course =
      import scala.util.Try
      val xs = line.split(separator).toSeq
      def str(i: Int): String = Try(xs(i)).getOrElse("")
      def num(i: Int): Double = Try(xs(i).toDouble).getOrElse(0.0)
      Course(str(0), str(1), str(2), num(3), str(4))
```

Figure 3: Code to download data about all courses at LTH.

c) *Linear search with find.* Student Oddput Clementina wants to attend the first available Computer Science course at G2 level. Help Oddput search for the first such course using linear search with the collection method `find`. CS course codes start with EDA or ETS[48]. *Tip:* Define the predicate `def isCS(s: String): Boolean` which preferably uses the string method `startsWith`.

d) *Implement linear search.* As practice, implement your own linear search function with signature:

```
def linearSearch[T](xs: Seq[T])(p: T => Boolean): Int = ???
```

The function takes a sequence collection `xs` and a predicate `p` which is a function that takes an element and returns a boolean. The type `Seq` is a supertype to all sequence collections, so using it as the parameter type for `xs` makes the function work for

[47] You can download the code from:
github.com/lunduniversity/introprog/tree/master/compendium/examples/lth-courses/courses.scala

[48] This is a simplified view of LTH's course code system. Some courses from the EIT department will also sneak in, but we ignore that here.

`Vector`, `Array`, `List`, etc. By the type parameter `T`, the function is generic and works for any type. The function `p` should return **`true`** if the parameter is the sought element. `linearSearch` should return the index of the first element in `xs` for which `p` holds. If no such element exists, `-1` is returned. First write pseudocode for the function by hand. Use **`while`**.

e) Implement a function **`def`** `rndCode: String` that generates random course codes consisting of 4 letters from A to Z followed by 2 digits from 0 to 9. *Tip:* Use REPL combined with an editor to create and test helper functions gradually solving appropriate subproblems.

f) Use `rndCode` from the previous subtask to fill a vector called `xs` with half a million random course codes. For each random code in `xs`, search with your function `linearSearch` for the index of the code in the vector `courses.lth` from subtask b. Measure the total time for the 500,000 linear searches with the function `timed` from task 1. How many of the random course codes were found among the real course codes at LTH?

g) How can you implement `linearSearch` with the built-in collection method `indexWhere`?

Task 4. *Search among LTH's courses with binary search.* The search algorithm BINSEARCH can be formulated with the pseudocode below:

Input : An ascending sorted sequence xs of length n of integers and a searched integer key
Output : An integer $i \geq 0$ giving the position of x if found, or a negative integer i where $-i$ corresponds to the place where x should be inserted in sorted order if x is not in the collection.

```
set interval (low, high) to (0, n − 1)
found ← false
mid ← −1
while low ≤ high and not found do
    mid ← position midway between low and high
    if xs(mid) == key then
        found ← true
    else
        if xs(mid) < key then
            low ← mid + 1
        else
            high ← mid − 1
        end
    end
end
if found then
    mid
else
    −(low + 1)
end
```

a) Try the algorithm above on paper with a sorted sequence of fewer than 10 integers. Try with even and odd number of elements, a sequence with one integer, and an empty sequence. Try both present and absent elements.

b) Implement binary search in a function with signature

```
def binarySearch(xs: Seq[String], key: String): Int = ???
```

and test in REPL for different cases. What happens if the sequence is not sorted?

c) Use `binarySearch` below to search for LTH courses. Use `rndCode`, `timed`, and `courses` from previous tasks.

```
def binarySearch(xs: Seq[String], key: String): Int = ???

val lthCodesSorted = courses.lth.map(_.code).sorted
val xs = Vector.fill(500000)(rndCode)
val (_, elapsedBin) =
  timed{xs.map(x => binarySearch(lthCodesSorted, x))}
val (_, elapsedLin) =
  timed{xs.map(x => linearSearch(lthCodesSorted)(_ == x))}
println(elapsedLin / elapsedBin)
```

d) How much faster was binary search compared to linear search?[49]

Task 5. *Insertion sort.* Implement sorting of an integer sequence to a sorted sequence with **insertion sort** in a function with this signature:

```
def insertionSort(xs: Seq[Int]): Seq[Int] = ???
```

Idea: Create a new empty sequence which will be our sorted result. For each element in the unsorted sequence: insert it in the correct place in the new sorted sequence.

a) *Pseudocode:* Run the pseudocode below by hand on the sequence 5 1 4 3 2 1. Draw the memory state after each loop iteration. Here we internally use a mutable `ArrayBuffer` which is fast at insertions and finish with `toVector` so that we return an immutable sequence.

```
1 result ← a new, empty ArrayBuffer
2 foreach element e in xs do
3     pos ← find the right position in result
4     insert e at position pos in result
5 end
6 result.toVector
```

b) Implement `insertionSort`. Use a `while` loop to implement line 3 in the pseudocode. Look up the documentation for `insert` on `ArrayBuffer`. Test `insert` on `ArrayBuffer` in REPL and verify it can be used to insert at the end in the "unused" position just after the last index. What happens if you do `insert` at position `size + 2`? Paste your implementation of `insertionSort` in REPL and test it works:

```
scala> insertionSort(Vector())
res0: Seq[Int] = Vector()

scala> insertionSort(Vector(42))
res1: Seq[Int] = Vector(42)

scala> insertionSort(Vector(1,2,3))
res2: Seq[Int] = Vector(1, 2, 3)

scala> insertionSort(Vector(5,1,4,3,2,1))
```

[49] In a run on an i7-4970K at 4.0GHz, `elapsedLin` was about 3000 *ms* and `elapsedBin` about 60 *ms*. Binary search was about 50 times faster in this case.

```
11 res3: Seq[Int] = Vector(1, 1, 2, 3, 4, 5)
```

Task 6. *In-place sorting.* Implement in-place sorting in an `Array[String]` with selection sort.
Idea: For all indices i: find $minIndex$ of the smallest string from position i to the end and swap elements at position i and $minIndex$. See animation here: sv.wikipedia.org/wiki/Urvalssortering
Implement according to the sketch below. *Tip:* You may find a modified version of the solution to task 13 in chapter 2 useful.

```
def selectionSortInPlace(xs: Array[String]): Unit =
  def indexOfMin(startFrom: Int): Int = ???
  def swapIndex(i1: Int, i2: Int): Unit = ???
  for i <- 0 to xs.size - 1 do swapIndex(i, indexOfMin(i))
```

Task 7. *Check if a sequence is sorted.* A simple and readable way to check if a sequence is sorted is shown below.

```
scala> def isSorted(xs: Vector[Int]): Boolean = xs == xs.sorted
```

✎ a) If `xs` has 10^6 elements, how many comparisons are done in the worst case using `isSorted` above? The method `sorted` uses the Timsort algorithm[50]. Look up the number of comparisons in the worst case on Wikipedia.
This solution is relatively slow for large collections. One does not need to sort first to check if sorted (unless you intend to sort anyway). It suffices to check that elements are in ascending order.

b) Implement a more efficient variant of `isSorted` that uses a **`while`** loop checking elements are in ascending order. Your algorithm should stop searching as soon as unsorted elements are found.

✎ c) How many comparisons are done in the worst case with the method in subtask b if you have n elements?

d) One can check if a sequence is sorted by the clever trick of first zipping the sequence with its tail and then checking that all pairs satisfy the sorting condition, i.e. `xs.zip(xs.tail).forall(???)` where `???` is replaced by a suitable predicate. What is the type of the 2-tuple `xs.zip(xs.tail)` if `xs` is of type `Vector[Int]`? Implement `isSorted` with this trick.

Task 8. *In-place insertion sort.* Implement and test in-place sorting in an integer array with the [51] algorithm.
Implement and test the function below in Scala with the following signature:

```
def insertionSort(xs: Array[Int]): Unit
```

Place the method in an appropriate object, and create a main program to test. Compile and run from the terminal. Start by writing the sorting algorithm in pseudocode.

[50] stackoverflow.com/questions/14146990/what-algorithm-is-used-by-the-scala-library-method-vector-sorted
[51] https://en.wikipedia.org/wiki/Insertion_sort

Task 9. *Sorting to new sequence with selection sort.* Implement and test sorting to a new sequence with selection sort [52] in Scala, according to the sketch below. You may find the solution to task 13 in chapter 2 useful.

```
def selectionSort(xs: Seq[String]): Seq[String] =
  def indexOfMin(xs: Seq[String]): Int = ???
  val unsorted = xs.toBuffer
  val result = scala.collection.mutable.ArrayBuffer.empty[String]
  /*
  while unsorted is not empty
    minPos = indexOfMin(unsorted)
    elem   = unsorted.remove(minPos)
    result.append(elem)
  */
  result.toVector
end selectionSort
```

12.2 Tasks on threads and concurrent execution

Task 10. *Threads.* The class `java.lang.Thread` is used to create **threads** with concurrent execution. This way, different parts of code can run simultaneously.
The class `Thread` defines an empty `run` method. To make the thread do something meaningful, one overrides `run` with the desired code.
A thread is started with the method `start`, which automatically calls the `run` method and runs the code in `run` concurrently with other threads. If `run` is called directly, it is *not* concurrent execution.

a) Create a thread that does something that takes a little time and run it with `run` and `start`, respectively.

```
1 def zzz = { print("zzzzzz"); Thread.sleep(5000); println(" AWAKE!")}
2 zzz
3 val t2 = new Thread{ override def run = zzz }
4 t2.run; println("Good morning!")
5 t2.start; println("Good morning!")
6 t2.start
```

b) What happens if you call `start` more than once on the same thread?

c) Create two threads with overridden `run` methods and start them simultaneously as below. In which order are the greetings printed after line 3 and line 4? Explain.

```
1 val g = new Thread{ override def run = for i <- 1 to 100 do print("Cucumber ")
2 val t = new Thread{ override def run = for i <- 1 to 100 do print("Tomato ") }
3 g.run; t.run
4 g.start; t.start
```

d) Use `Thread.sleep` as below. Is the behavior fully predictable (deterministic)? Explain. You can interrupt REPL with CTRL+C or SHIFT+CTRL+C, depending on terminal settings.[53].

[52] https://en.wikipedia.org/wiki/Selection_sort
[53] stackoverflow.com/questions/6248884/can-i-stop-the-execution-of-an-infinite-loop-in-scala-repl

```
def sometimes(block: => Unit) = new Thread {
  override def run = while(true) { block; Thread.sleep(600) }
}.start
sometimes(print("zzz ")); sometimes(print("snark ")); sometimes(println("hi!"))
```

Task 11. *Concurrent variable update.* Write the classes `Bank` and `Kund` in an editor and then paste them in the REPL.

```
class Bank:
  private var _saldo = 0;
  def saldo: Int = _saldo
  def deposit(): Unit = _saldo += 1
  def withdraw(): Unit = _saldo -= 1
end Bank

class Customer(bank: Bank):
  def spendSave(): Unit =
    bank.withdraw()
    Thread.sleep(1)
    bank.deposit()
  end spendSave
end Customer
```

a) Use the function `sometimes` from the previous task and run the lines below in REPL. The concurrent variable update here produces nonsense results that lead to very upset bank customers and owners. Explain what happens.

```
val bank = new Bank
println(bank.saldo)
bank.deposit()
println(bank.saldo)
bank.withdraw()
println(bank.saldo)

val teddybear = new Customer(bank)
val ing = new Customer(bank)

teddybear.spendSave()
ing.spendSave()
println(bank.saldo)

def often(block: => Unit) = new Thread { // every millisecond
  override def run = while true do { block;  Thread.sleep(1)}
}.start

often(teddybear.spendSave()); often(ing.spendSave())

sometimes(println(bank.saldo))
```

Task 12. *Thread-safe `AtomicInteger`.* The JVM supports updates that cannot be interrupted by other threads during memory writing. An operation that cannot be

interrupted is called **atomic**. Study the documentation for `AtomicInteger` [54] and try the code below. Explain what happens.
Use the functions `often` and `sometimes` from earlier tasks.

```
class SafeBank:
  import java.util.concurrent.atomic.AtomicInteger
  private var _saldo = new AtomicInteger
  def saldo: Int = _saldo.get
  def deposit(): Unit = _saldo.incrementAndGet()
  def withdraw(): Unit = _saldo.decrementAndGet()
end SafeBank

class SafeCustomer(bank: SafeBank):
  def spendSave =
    bank.withdraw()
    Thread.sleep(1)
    bank.deposit()
  end spendSave
end SafeCustomer
```

```
val sb = new SafeBank
val grandma = new SafeCustomer(sb)
val wolf = new SafeCustomer(sb)

often(grandma.spendSave); often(wolf.spendSave)

sometimes(println(sb.saldo))
```

Task 13. *Concurrent execution with `scala.concurrent.Future`.* Managing threads can be complicated. Using `scala.concurrent.Future` allows simpler concurrent execution.

Background: With a `Future`, concurrent execution is created "under the hood" using a framework called Akka[55], written in Scala and Java. Akka provides automatic multithreading with thread pools and enables advanced parallel programming at a high abstraction level, so one does not have to create instances of the class `Thread` oneself. Instead one can simply wrap code inside `Future{ "runs concurrently" }` after importing what is needed.

a) To create concurrent execution with `Future`, you must first import as below; this creates an execution context with thread pools ready for use. Restart REPL and observe the error after line 1. Then import as shown. What is the type of `f`?

```
1 scala> concurrent.Future { Thread.sleep(1000); println("One second later!") }
2 scala> import scala.concurrent._
3 scala> import ExecutionContext.Implicits.global
4 scala> val f = Future { Thread.sleep(1000); println("One second later!") }
```

b) Create a procedure `printLater` as below that prints its argument after a random time. Explain what happens below.

```
1 scala> def printLater(a: Any): Unit =
```

[54] docs.oracle.com/javase/8/docs/api/java/util/concurrent/atomic/AtomicInteger.html
[55] http://akka.io/

```
        Future { Thread.sleep((math.random() * 10000).toInt); print(a + " ") }
scala> (1 to 42).foreach(i => printLater(i)); println("all launched!")
```

c) Create as below a `Future` that counts how many digits a very large number has. With `onComplete` one can specify what to do when the heavy computation finishes; this is called registering a callback. What is the return type of `big`? How many digits does the large number have? What is the type of `r`? Adjust the argument to `big` if you do not want to wait too long.

```
scala> BigInt(10).pow(100)
scala> BigInt(10).pow(100).toString.size
scala> def big(n: Int) = Future { BigInt(n).pow(n).toString.size }
scala> big(1234567).onComplete{r => println(r + " digits") }
```

d) The big advantage with `Future` is that you can continue running while waiting, so `Future` calls are **non-blocking**. Sometimes you want to block execution while waiting for the result. You can then use `scala.concurrent.Await` and its method `result` as below. Use `big` from the previous task and do a blocking wait for a result below. What happens? What if you wait too little?

```
scala> import scala.concurrent.duration._
scala> Await.result(big(1234567), 20.seconds)
```

Task 14. *Use `Future` to do multiple things concurrently.* In this task you will download web pages in parallel using `Future`, so that one download can finish while another is delayed.

a) The code for a minimal webpage is below. You can view it here: http://fileadmin.cs.lth.se/pgk/mini.html or write the code below in a file ending with `.html` and open it in your browser.

```
<!DOCTYPE html>
<html>
<body>
HELLO WORLD!
</body>
</html>
```

b) To simulate slow servers, you can download a page via the site
`http://deelay.me/`
Download the above page with 2 seconds delay:
http://deelay.me/2000/http://fileadmin.cs.lth.se/pgk/mini.html

c) You can download webpages with `scala.io.Source`. What happens below? Try to estimate, given how `delay` is computed, how long you have to wait on average, best case, and worst case before you can see the first webpage in vector `loads` below?

```
scala> def load(url: String) = scala.io.Source.fromURL(url).getLines.toVector
scala> def slowload(url: String) =
         val delay = (math.random() * 1000 + 2000).toInt
         val delaySite = s"http://deelay.me/$delay/"
         load(delaySite+url)
       end slowload
scala> load("http://fileadmin.cs.lth.se/pgk/mini.html")
```

```
scala> def seg = slowload("http://fileadmin.cs.lth.se/pgk/mini.html")
scala> val loads = Vector.fill(10)(seg)
scala> loads(0)
```

d) Before we can start a `Future`, as shown in task 13, we must import the underlying execution environment ready for parallelization without creating threads ourselves. What happens below?

```
scala> import scala.concurrent._
scala> import ExecutionContext.Implicits.global
scala> val f = Future(seg)
scala> f   // check if done or try again later
scala> f
```

e) Load input without blocking (non-blocking input). Explain what happens below.

```
scala> val nonBlocking = Future(Vector.fill(10)(seg))
scala> nonBlocking   // check later if not done
scala> nonBlocking
```

f) Load input separately in different parallel threads. Explain what happens below. Run line 3 below repeatedly quickly in REPL using up-arrow + Enter.

```
1 scala> val para = Vector.fill(10)(Future(seg))
2 scala> para
3 scala> para.map(_.isCompleted)
4 scala> para.map(_.isCompleted) // watch how they complete one by one
5 scala> para(0)
```

g) Register a callback with `onComplete`. Explain what happens below.

```
scala> val action = Vector.fill(10)(Future(seg))
scala> action(0).onComplete(xs => println(s"ready:$xs"))
scala> // wait until download at index 0 is done
```

h) Register a callback for error handling in case of exceptions with `onFailure`. Explain what happens below.

```
scala> def happy  = { Thread.sleep(3000); println(":)") }
scala> def unhappy = { Thread.sleep(3000); 42 / 0; happy }
scala> Future(happy ).onFailure{ case e => println(s":( $e") }
scala> Future(unhappy).onFailure{ case e => println(s":( $e") }
```

Task 15. Calculate large primes in parallel using the functions below. Implement `isPrime` according to the pseudocode from the English Wikipedia page on primality test [56] using the "naive algorithm". Compute 10 random primes with 16 digits each. Perform the computations in parallel using `Future`.

```
def isPrime(n: BigInt): Boolean = ???

def nextPrime(start: BigInt): BigInt =
  var i = start
  while !isPrime(i) do i += 1
```

[56] https://en.wikipedia.org/wiki/Primality_test

```
  i
end nextPrime

def randomBigInt(nDigits: Int): BigInt =
   def rndChar = ('0' + (math.random() * 10).toInt).toChar
   val str = Array.fill(nDigits)(rndChar).mkString
   BigInt(str)
randomBigInt
```

✎ **Task 16.** *Answer theory questions.*

a) What is a thread?

b) How do you create a thread with class `Thread`?

c) How do you start a thread?

d) What problems can arise if multiple threads update the same resource?

e) What does it mean that code is *thread-safe*?

f) Name some advantages of using Future compared to using threads directly.

Task 17. *Classes with atomic updates.* Read about and test classes AtomicBoolean, AtomicDouble, and AtomicReference for atomic updates in package `java.util.concurrent.atomic`.
Use some of these together with `scala.concurrent.Future`.

Task 18. *Create your own multithreaded web server.*

a) Type in [57] the code below in an editor and save it as `webserver.scala`, compile and run with `scala run webserver.scala`, then describe what happens when you browse to:
http://localhost:8089/abbasillen

```
package webserver

import java.net.{ServerSocket, Socket}
import java.io.OutputStream
import java.util.Scanner
import scala.util.Try

object html:
  def page(body: String): String = //minimal web page
    s"""<!DOCTYPE html>
       |<html>
       |<head><meta charset="UTF-8"><title>Min Sörver</title></head>
       |<body>
       |$body
       |</body>
       |</html>
       """.stripMargin

  def header(length: Int): String = //standardized header of reply
    s"HTTP/1.0 200 OK\nContent-length: $length\n" +
     "Content-type: text/html\n\n"
```

[57] Or download here: github.com/lunduniversity/introprog/blob/master/compendium/examples/simple-web-server/webserver.scala

```


object start:
  def handleRequest(cmd: String, uri: String, socket: Socket): Unit =
    val os = socket.getOutputStream
    val response = html.page("Backwards: " + uri.reverse)
    os.write(html.header(response.size).getBytes("UTF-8"))
    os.write(response.getBytes("UTF-8"))
    os.close
    socket.close
  end handleRequest

  def serverLoop(server: ServerSocket): Unit =
    println(s"http://localhost:${server.getLocalPort}/hej")
    while true do
      Try {
        var socket = server.accept  // blocks thread until connect
        val scan = new Scanner(socket.getInputStream, "UTF-8")
        val (cmd, uri) = (scan.next, scan.next)
        println(s"Request: $cmd $uri")
        handleRequest(cmd, uri, socket)
      }.recover{ case e: Throwable => s"Connection failed: $e" }

  def main(args: Array[String]) =
    val port = Try{ args(0).toInt }.getOrElse(8089)
    serverLoop(new ServerSocket(port))
```

b) Now create a web server that does something more interesting. It should respond with the 13th Fibonacci number[58] if browsed to `http://localhost:8089/fib/13`. Save your previous webserver as `fibserver.scala` and add the following code to calculate Fibonacci numbers in a deliberately slow exponential-time manner so the server really works, then later implement `compute.fib` with linear time complexity to avoid unnecessary CPU waste.

```
// add below in webserver.scala from
//    https://github.com/lunduniversity/introprog/blob/master/compendium/examples/simple-web-server/web

object compute:
  def fib(n: BigInt): BigInt =
    if n < 0 then 0 else
    if n == 1 || n == 2 then 1
    else fib(n - 1) + fib(n -2)
  end fib
end compute

def fibResponse(num: String) =
  num.toIntOption match
    case Some(n) => html.page(s"fib($n) == " + compute.fib(n))
    case None    => html.page(s"ERROR: enter an integer, not $num")

def errorResponse(uri:String) = html.page(s"Error: $uri </br> use /fib/integer")


// change handleRequest in start in webserver.scala to
  def handleRequest(cmd: String, uri: String, socket: Socket): Unit =
```

[58] https://sv.wikipedia.org/wiki/Fibonaccital

```
    val os = socket.getOutputStream
    val afterSlash = uri.toString.drop(1) // skip initial slash
    println(s"afterSlash:$afterSlash")
    val response: String =
      if afterSlash.startsWith("fib/") then fibResponse(afterSlash.stripPrefix("fib/"))
      else errorResponse(uri)
    os.write(html.header(response.size).getBytes("UTF-8"))
    os.write(response.getBytes("UTF-8"))
    os.close
    socket.close
  end handleRequest
```

Run in terminal with `scala run webserver.scala` and describe what happens in your browser when you navigate to the server.

c) Open several large Fibonacci numbers simultaneously in multiple browser tabs. How does it show that the server is single-threaded?

d) Make your server multithreaded using the new server loop below.

```
import scala.concurrent._
import ExecutionContext.Implicits.global

  def serverLoop(server: ServerSocket): Unit = {
    println(s"http://localhost:${server.getLocalPort}/hej")
    while (true) {
      Try {
        var socket = server.accept  // blocks thread until connect
        val scan = new Scanner(socket.getInputStream, "UTF-8")
        val (cmd, uri) = (scan.next, scan.next)
        println(s"Request: $cmd $uri")
        Future { handleRequest(cmd, uri, socket) }.onFailure {
          case e => println(s"Request failed: $e")
        }
      }.recover{ case e: Throwable => s"Connection failed: $e" }
    }
  }
```

e) Open several large Fibonacci numbers simultaneously in different tabs. How does it show the server is multithreaded?

f) It is unnecessary to compute the same Fibonacci number multiple times. With a cache in the form of a mutable `Map` you can store already computed values. This does not work with a regular `scala.collection.mutable.Map` in our multithreaded web server because it is not **thread-safe**. With thread-safe mutable data structures the same concurrency problems as in task 11 occur.
Instead, use `java.util.concurrent.ConcurrentHashMap`. Look up the documentation for `ConcurrentHashMap` and try to understand the code below. How do the methods `containsKey`, `put`, and `get` work?

```
object compute {
  import java.util.concurrent.ConcurrentHashMap
  val memcache = new ConcurrentHashMap[BigInt, BigInt]

  def fib(n: BigInt): BigInt =
    if (memcache.containsKey(n)) {
      println("CACHE HIT!!! no need to compute: " + n)
```

```
      memcache.get(n)
    } else {
      println("cache miss :( must compute fib:  " + n)
      val f = fastFib(n)
      memcache.put(n, f)
      f
    }

  private def fastFib(n: BigInt): BigInt = {
    if (n < 0) 0 else
    if (n == 1 || n == 2) 1
    else fib(n - 1) + fib(n -2)
  }
}
```

g) Use the above `fib` object in a new version of your server. Save it as `fibserver-memcached.scala`. Examine how fast it computes large Fibonacci numbers with the new variant. How large numbers can you compute? Can the server continue after stack overflow? Explain why.

h) Since we can now get very large Fibonacci numbers, it may be useful to insert line breaks on the webpage. The html tag `</br>` produces a line break.

```
  def insertBreak(s: String, n: Int = 80): String = {
    if (s.size < n) s
    else s.take(n) + "</br>" + insertBreak(s.drop(n),n)
  }
```

Use this recursive function to insert line break tags every n characters in long strings. Test how it looks on the webpage with very large numbers.

i) We will now use the larger heap memory instead of stack, avoiding stack size limitations. Rewrite `fastFib` to use a **`while`** loop instead of recursive calls. This is tricky, but if stuck you may peek at the solutions in the Appendix for inspiration. How large numbers can your server handle now? What happens when memory runs out? How can you protect the server from hanging?

13. Exercise Excercise13

Task 1. *Complete your project.*

Task 2. *Take a final exam.*

Task 3. *Prepare your project presentation.*

Task 4. *Create documentation for your project.* Read more in Appendix E about how to create documentation.

Task 5. *Review exercises and laboratory work.*

Part II

Solutions

Chapter

Solutions to the exercises

1. Solution `Excercise01`

1.1 Basic Tasks; preparation for the lab

Solution Task 1. *Match concepts with description.*

literal	1	⇝	D	represents a specific data value
string	2	⇝	G	a sequence of characters
statement	3	⇝	F	a line of code that does something; can be separated with a semicolon
expression	4	⇝	H	combines values and functions into a new value
function	5	⇝	K	computes a return value when called
procedure	6	⇝	J	performs a side effect when called; return value is empty
runtime error	7	⇝	N	may occur while the program runs
compile error	8	⇝	M	may occur before execution starts
abstract	9	⇝	A	introduce new concepts that simplify coding
compile	10	⇝	C	translate code into executable form
type	11	⇝	I	describes what the data can be used for
for-loop	12	⇝	O	useful when the number of repetitions is known beforehand
while-loop	13	⇝	P	useful when the number of repetitions is unknown beforehand
assignment	14	⇝	L	to change the value of a variable
floating-point	15	⇝	E	decimal number with limited precision
boolean	16	⇝	B	either true or false

Solution Task 2. *Output in Scala REPL.*

a) For example:

```
scala> println("hello lasser")
```

b) If the right parenthesis is missing you can continue typing on the next line. This is indicated by a vertical bar at the start of each new line:

```
scala> println("hello lasser"
     | + "!"
     | )
hello lasser!
```

Solution Task 3. *String concatenation.*

a)

```
scala> "cucumber" + "tomato"
res1: String = cucumbertomato
```

value: `"cucumbertomato"`, type: `String`

b)

```
scala> res1 * 42
```

```
res2: String = cucumbertomatocucumbertomatocucumbertomatocucumbertomatocucumber
```

Solution Task 4. *When is the error detected?*

a) Type: `String`, value: `"hejhejhej"`

b) Runtime error:

```
scala> "hej" * Int.MaxValue
java.lang.OutOfMemoryError: Java heap space
```

c) Compile-time error: (indicated by text `<console> ... error:`)

```
scala> "hej" * true
<console>:12: error: type mismatch;
 found   : Boolean(true)
 required: Int
       "hej" * true
```

A type error means the compiler cannot make the types match in e.g. a function call. In Scala we find out about type errors already at compilation, whereas in other languages (e.g. Javascript) such errors are discovered during execution, possibly leading to hard-to-find bugs that may first manifest much later.

Solution Task 5. *Literals and types.*

a)

`1`	1	$\rightsquigarrow$	E	`Int`
`1L`	2	$\rightsquigarrow$	G	`Long`
`1.0`	3	$\rightsquigarrow$	J	`Double`
`1D`	4	$\rightsquigarrow$	F	`Double`
`1F`	5	$\rightsquigarrow$	H	`Float`
`'1'`	6	$\rightsquigarrow$	I	`Char`
`"1"`	7	$\rightsquigarrow$	A	`String`
`true`	8	$\rightsquigarrow$	C	`Boolean`
`false`	9	$\rightsquigarrow$	B	`Boolean`
`()`	10	$\rightsquigarrow$	D	`Unit`

b) The value goes beyond the limit of what fits in a 32-bit integer and ”wraps around” to the smallest possible integer `Int.MinValue` because that is how binary arithmetic with limited bits works in the CPU.

```
scala> Int.MaxValue + 1
res3: Int = -2147483648

scala> Int.MinValue
res4: Int = -2147483648
```

c) Both are integers but `Long` can represent larger numbers than `Int`.

d) Both are floating-point types but `Double` has double precision and can represent larger numbers with more decimal digits.

Solution Task 6. *Mathematical functions. Using documentation.*

a) Computing $2^{64} - 1$ using `math.pow` as below yields approximately $1.8 \cdot 10^{19}$

```
scala> math.pow(2, 64) - 1
res0: Double = 1.8446744073709552E19
```

b) Yes, the return type annotation `: Double` can be omitted.

- Why can the return type be omitted?
  Because the compiler's type inference can deduce the return type.
- Why might one want to omit it?
  It is shorter to write without it.
- Reasons to specify the return type:
  - With explicit return type you get help from the compiler, already at compile time, to check that the expression to the right of the equals sign is of the expected type.
  - By explicitly specifying the return type, readers who only see the method signature (and not the implementation) can clearly see what is returned.

c) About 500 km.

```
scala> circumference(12750 / 2) / 80
res0: Double = 500.6913291658733
```

Solution Task 7. *Variables and assignment. Mutable and immutable variables.*

a)

After line 1:	a: Int 13		
After line 2:	a: Int 13	b: Int 14	
After line 3:	a: Int 13	b: Int 14	c: Double 54.0
After line 4:	a: Int 13	b: Int 14	c: Double 54.0
After line 5:	a: Int 0	b: Int 14	c: Double 54.0
After line 6:	a: Int 0	b: Int 14	c: Double 55.0

b) Immutable variables are declared with the keyword **val**. It is not possible to assign a new value to an immutable variable; attempting to do so results in a compile-time error that says **error: reassignment to val**. Compile-time errors can be recognized by the text **error:**, as shown below:

```
scala> b = 0
<console>:12: error: reassignment to val
       b = 0
         ^
```

Solution Task 8. *Random numbers with* `math.random()`.

a) From documentation:

```
/** Returns a Double value with a positive sign,
 *  greater than or equal to 0.0 and less than 1.0.
 */
def random(): Double
```

Documentation comments, which start with `/**` and end with `*/`, give a description of how the function works. After the doc comment comes the function header, which here tells us the function is named `random` and always returns a `Double`. (The tool `scaladoc` can use documentation comments to automatically generate websites with special documentation pages and search functions.)

b)

```
scala> def roll: Int = (math.random() * 6 + 1).toInt

scala> roll
res0: Int = 4

scala> roll
res1: Int = 1
```

Solution Task 9. *Repetition with* **for**, `foreach` *and* **while**.

a)

```
for i <- 1 to 100 do print(s"$roll, ")
```

b)

```
(1 to 100).foreach(i => print(s"$roll, "))
```

c)

```
var i = 1
while i <= 100 do { print(s"$roll, "); i = i + 1 }
```

```
var i = 1
while i <= 100 do
    print(s"$roll, ")
    i += 1
```

Solution Task 10. *Alternatives with* **`if`** *statement and* **`if`** *expression.*

a)

```
for i <- 1 to 100 do
  if roll == 6 then print("CONGRATULATIONS! ") else print(":(")
```

or

```
for (i <- 1 to 100) if (roll == 6) print("CONGRATULATIONS! ") else print(":(")
```

b)

```
var i = 1
var n = 0
while i <= 100 do
  if roll == 6 then n = n + 1
  i = i + 1
println("The number of sixes: " + n)
```

c)

```
def isAdult(age: Int) = age >= 18
```

Solution Task 11. *Sequence, statement and block.*

a) The statements create this output:

```
true!hej
true!hej
true!hej
true!hej
```

b)

```
scala> def p = { print("hej"); print("true"); println("!")}
scala> p;p;p;p
```

c)

- Curly braces are used to group multiple statements. Curly braces are needed if you want to define a function consisting of more than one statement. Since Scala 3 you can alternatively use indentation to define a multi-line function with multiple statements.

- Semicolons separate multiple statements. Semicolons are only needed if you want to write multiple statements on the same line.

Solution Task 12. *Integer division.*

`4 / 42`	1	⇝	F	`0: Int`
`42.0 / 2`	2	⇝	C	`21.0: Double`
`42 / 4`	3	⇝	B	`10: Int`
`42 % 4`	4	⇝	G	`2: Int`
`4 % 42`	5	⇝	A	`4: Int`
`40 % 4 == 0`	6	⇝	D	`true : Boolean`
`42 % 4 == 0`	7	⇝	E	`false: Boolean`

Solution Task 13. *Boolean values.*

a) `true`
b) `false`
c) `true`
d) `true`
e) `false`
f) `true`
g) `true`
h) `true`
i) Exception thrown: `java.lang.ArithmeticException: / by zero`
j) `false`

Solution Task 14. *Boolean variables.*

2: Nothing is printed.

4: `watch out!!!`

Solution Task 15. *Turtle graphics with Kojo.*

a) Starting your Kojo program with `erase` causes execution to start in the same initial state: a blank canvas where the turtle points upward, the pen is down, and the pen color is red. That makes reasoning about what the program does from start to finish easier compared to if execution depends on results of previous executions.

b)

```
erase

front; left
front; left
front; left
front; left
```

c)

```
erase

front; left
front; right
```

```
front; left
front; right

front; left
front; right

front; left
```

1.2 Extra Tasks; additional practice

Solution Task 16. *Value and type.*

`1.0 + 18`	1	$\rightsquigarrow$	H	`19.0: Double`
`(41 + 1).toDouble`	2	$\rightsquigarrow$	K	`42.0: Double`
`1.042e42 + 1`	3	$\rightsquigarrow$	A	`1.042E42: Double`
`12E6.toLong`	4	$\rightsquigarrow$	I	`12000000: Long`
`32.toChar.toString`	5	$\rightsquigarrow$	E	`" ": String`
`'A'.toInt`	6	$\rightsquigarrow$	B	`65: Int`
`0.toInt`	7	$\rightsquigarrow$	F	`0: Int`
`'0'.toInt`	8	$\rightsquigarrow$	D	`48: Int`
`'9'.toInt`	9	$\rightsquigarrow$	L	`57: Int`
`'A' + '0'`	10	$\rightsquigarrow$	C	`113: Int`
`('A' + '0').toChar`	11	$\rightsquigarrow$	J	`'q': Char`
`"*!%#".charAt(0)`	12	$\rightsquigarrow$	G	`'*': Char`

Solution Task 17. *Statements and expressions.*

a) An expression can be evaluated and yields a useful value. A statement *does* something (e.g. prints something), but does not yield a useful value.

b) `println()`

c)
`valueMissing` contains Unit
Prints Unit
Prints `"()"`
Prints `"()"`
Prints first 'hej' from the inner call then `()` from the outer call

d) `Unit`

e) `Unit`

Solution Task 18. *Procedure with parameter.*

a)

```
var highscore = 0
```

b)

```
def updateHighscore(points: Int): Unit =
  if points > highscore then
    highscore = points
    println("REKORD!")
  else println("DON'T GIVE UP!")
```

c)

```
def updateHighscore(points: Int): String =
```

```
  if points > highscore then
    highscore = points
    "REKORD!"
  else "DON'T GIVE UP!"
```

Solution Task 19. *Floating point arithmetic.*

a)

```
scala> Double.MinPositiveValue
res0: Double = 4.9E-324
```

b)

```
scala> Double.MaxValue + Double.MinPositiveValue == Double.MaxValue
res2: Boolean = true
```

Solution Task 20. ***`if`*** *statements.*

1. Output: **`false`**
2. Output: **`true`**
3. Nothing is printed, the function is declared but not executed.
4. Output: `1:kop 2:munt 3:kop 4:kop 5:munt` or similar depending on the random values from `math.random()`.

Solution Task 21. ***`if`*** *expressions.* Note the type `Any` on the last two expressions.

```
scala> if vegetable == "tomato" then "good" else "not good"
res0: String = not good

scala> if fruit == "banana" then "good" else "not good"
res1: String = good

scala> if true then vegetable else 42
res2: Any = cucumber

scala> if false then vegetable else 42
res3: Any = 42
```

Solution Task 22. *Modulo operator `%` and Boolean values.*

a)

```
scala> def isEven(n: Int): Boolean = n % 2 == 0

scala> isEven(42)
res0: Boolean = true

scala> isEven(43)
res1: Boolean = false
```

b)

```
scala> def isOdd(n: Int): Boolean = !isEven(n)

scala> isOdd(42)
res2: Boolean = false

scala> isOdd(43)
res3: Boolean = true
```

Solution Task 23. *Differences between* ***var***, ***val***, ***def***.

a)

```
  scala> var x = 30
  x: Int = 30

  scala> x + 1
  res6: Int = 31

  scala> x = x + 1
  x: Int = 31

  scala> x == x + 1
  res7: Boolean = false

  scala> val y = 20
  y: Int = 20

  scala> y = y + 1
  <console>:12: error: reassignment to val
         y = y + 1
           ^

  scala> var z = { println("hej z!"); math.random() }
  hej z!
  z: Double = 0.3381365875903367

  scala> def w = { println("hej w!"); math.random() }
  w: Double

  scala> z
  res8: Double = 0.3381365875903367

  scala> z
  res9: Double = 0.3381365875903367

  scala> z = z + 1
  z: Double = 1.3381365875903368

  scala> w
  hej w!
  res10: Double = 0.06420209879434557

  scala> w
  hej w!
  res11: Double = 0.5777951341051852
```

```

scala> w = w + 1
<console>:12: error: value w_= is not a member of object
       w = w + 1
```

b)

- `var name = expression` is used to declare a mutable variable. The name can be reassigned to new values using assignment statements.
- `val name = expression` is used to declare an immutable variable which cannot be changed after initialization. Attempting to assign it a new value results in a compile-time error.
- `def name = expression` is used to declare a function whose expression is evaluated every time it is called.

Solution Task 24. *Difference between* ***`if`*** *and* ***`while`***.

- Line 3: If lucky (50% chance) you get a single win printed once.
- Line 4: If lucky you get many wins in a row. The probability of n wins in a row is $(\frac{1}{2})^n$.

1.3 Advanced Tasks; challenges

Solution Task 25. *Logic and De Morgan's Laws.*

a) `point > 1000`

b) `point > 100`

c) `point <= highscore`

d) `point <= 0 || point >= highscore`

e) `point >= 0 && point <= highscore`

f) `clear`

g) `!clear`

Solution Task 26. *The string interpolator* `s`.

a)

```
The name 'Kim Finkodare' has 12 letters.
```

b)

```
println(s"$f has ${f.size} letters.")
println(s"$e has ${e.size} letters.")
```

Solution Task 27. *Assignment operators.*

After line1:	a:	Int	40			
After line2:	a:	Int	40	b:	Int	80
After line3:	a:	Int	50	b:	Int	80
After line4:	a:	Int	50	b:	Int	70
After line5:	a:	Int	100	b:	Int	70
After line6:	a:	Int	100	b:	Int	35

Solution Task 28. *Big numbers.*

a) `BigInt` can be used instead of `Int` for very large integers. There is also `Long` which has double the range of `Int`, while `BigInt` can have an arbitrarily large number of digits (until memory runs out) and thus represent enormously large numbers. `BigDecimal` can be used instead of `Double` for very large decimal numbers.

b)

```
scala> BigInt(2).pow(64)
res0: scala.math.BigInt = 18446744073709551616
```

c) Calculations are much slower and they are somewhat trickier to use.

Solution Task 29. *Precedence rules*

a) `77: Int`

b) `13: Int`

c) `-13: Int`

Solution Task 30. *Documentation of packages in Java and Scala.*

a) Scala: `Pi`, Java: `PI`

b) You can search for and filter all occurrences of a particular string.

c) Computes the hypotenuse (Pythagorean theorem) without risking rounding errors in intermediate calculations.

Solution Task 31. *Accuracy and exceptions in arithmetic expressions.*

a) `-2147483648` which corresponds to `Int.MinValue`.

b) An exception is thrown: `java.lang.ArithmeticException: / by zero`

c) `1.0000000000000001E8` (as expected)

d) Rounded to `1E9` (floating-point arithmetic accuracy issue: a big float plus a (too) small float can give the same number; the small number 'disappears').

e) `45.00000000000001` (floating-point arithmetic accuracy issue: mathematically should be exactly 45.)

f) `Infinity` (same as `Double.PositiveInfinity`, representing positive infinity).

g) `2147483647` which is `Int.MaxValue`.

h) `NaN` meaning 'Not a Number'.

i) `NaN` meaning 'Not a Number'.

j) An exception is thrown: `java.lang.Exception: PANG!!!`

Solution Task 32. *Modulo arithmetic with negative numbers.* In Scala the result has the same sign as the dividend.

```
scala> 1 % 2
res0: Int = 1

scala> -1 % 2
res1: Int = -1

scala> -1 % -2
res2: Int = -1

scala> 1 % -2
res3: Int = 1
```

Solution Task 33. *Literal identifiers.*

a) The variable is named 'literal val', backticks allow names otherwise illegal, e.g. containing spaces or keywords.

b) Backticks in Scala allow all kinds of characters in names. Example use: Java has a method `java.lang.Thread.yield` but in Scala `yield` is a keyword; to work around this in Scala you can write `java.lang.Thread.`yield``

Solution Task 34. *`java.lang.Integer`, hexadecimal literals, BigDecimal.*

a)

```
scala> import Integer.{toBinaryString => toBin, toHexString => toHex}

scala> for i <- Seq(33, 42, 64) do println(s"$i \t ${toBin(i)} \t ${toHex(i)}")
33   100001    21
42   101010    2a
64   1000000   40
```

b) The hexadecimal integer $10c$ can be written as literal `0x10c` in Scala, Java and many other languages: [1]

```
scala> 0x10c
res0: Int = 268
```

c) [2]

```
scala> val c = 299792458
c: Int = 299792458

scala> BigDecimal(0x10).pow(c)
res68: scala.math.BigDecimal = 2.1248929632279066130609861108876 72E+360986089
```

Solution Task 35. *String formatting.*

```
val str = f"The giant cucumber is $g%1.3f meter lang"
```

(If you think `$g%1.3f` looks cryptic, take comfort that you now carry on a venerable heritage from the ancient language C and the legendary function `printf` to coming generations of initiated code wizards.)

Solution Task 36. *Multiplication warning.*

a) The second multiplication overflows the maximum allowed value of an `Int`. In the third multiplication an exception `java.lang.ArithmeticException: integer overflow` is thrown instead.

```
scala> Math.multiplyExact(1, 2)
res70: Int = 2

scala> Int.MaxValue * 2
res71: Int = -2

scala> Math.multiplyExact(Int.MaxValue, 2)
java.lang.ArithmeticException: integer overflow
```

[1] https://en.wikipedia.org/wiki/0x10c
[2] https://c418.bandcamp.com/album/0x10c

```
  at java.lang.Math.multiplyExact(Math.java:867)
  ... 42 elided
```

b) Used when you want to be 100

Solution Task 37. *Extra operators for exact multiplication.*

a)

```
scala> Int.MaxValue *! 1
res0: Int = 2147483647

scala> Int.MaxValue *! 2
java.lang.ArithmeticException: integer overflow
  at java.lang.Math.multiplyExact(Math.java:867)
  at IntExtra.$times$bang(<console>:16)
  ... 32 elided
```

b)

```
extension (i: Int)
  def *!(j: Int) = Math.multiplyExact(i,j)
  def +!(j: Int) = Math.addExact(i,j)
  def -!(j: Int) = Math.subtractExact(i,j)
```

c) It quickly becomes very cryptic with names consisting of multiple special characters. If you *really* want such operators it is *very* appropriate to also provide clear-text variants:

```
extension (i: Int)
  def mulExact(j: Int) = Math.multiplyExact(i,j)
  def *!(j: Int) = i mulExact j

  def addExact(j: Int) = Math.addExact(i,j)
  def +!(j: Int) = i addExact j

  def subExact(j: Int) = Math.subtractExact(i,j)
  def -!(j: Int) = i subExact j
```

2. Solution Excercise02

2.1 Basic Tasks

Solution Task 1. *Match concepts with descriptions.*

compile	1	⇝	C	machine code is generated from one or more source files
script	2	⇝	G	single code file, no main program needed
object	3	⇝	M	groups variables and functions
@main	4	⇝	E	where execution of the compiled program starts
program argument	5	⇝	L	can be passed via the args parameter to main
data structure	6	⇝	J	many elements in a whole; element-wise access
collection	7	⇝	K	data structure with elements of the same type
sequence collection	8	⇝	F	data structure with elements in a certain order
Array	9	⇝	D	a mutable, indexable sequence collection
Vector	10	⇝	B	an immutable, indexable sequence collection
Range	11	⇝	N	a collection representing a range of integers
yield	12	⇝	A	used in for-expressions to create a new collection
algorithm	13	⇝	H	step-by-step description of a solution to a problem
implementation	14	⇝	I	a specific realization of an algorithm

Solution Task 2. *Using the terminal.*

a)

```
> mkdir hello
> cd hello
> pwd
```

b)

```
> cd ..
> ls
```

Solution Task 3. *Create and run a Scala script.*

a)

```
Sum of the first 1000 numbers is: 500500
```

b) The compilation error is: `')' expected, but eof found`

c) The file should look like this:

```
val n = args(0).toInt
val summa = (1 to n).sum
println(s"Sum of the first $n numbers is: $summa")
```

The output is:

```
Sum of the first 5001 numbers is: 12507501
```

d) The runtime error is:

```
java.lang.ArrayIndexOutOfBoundsException: Index 0 out of bounds for length 0
```

Since the array `args` is empty when no program arguments are given, the index 0 does not exist.

Solution Task 4. *Scala application with `@main`.*

a) The compiler created 5 files in subdirectories under `.scala-build` named:

```
'hello$package.class'  'hello$package$.class'  'hello$package.tasty'
run.class   run.tasty
```

b) You get the error message if you remove the last curly brace. `eof` in the error message means end-of-file. This error message is common for unmatched parentheses, but the compiler often has difficulty giving a good error message if one of the parentheses is missing and it may point to an incorrect line for where it expects the missing character.

c) `Syntax Error: Expected a toplevel definition`. Without curly braces, indentation determines which parts of the code belong together. If you remove the indentation on the last line with the print statement, the compiler interprets this as being *outside* the main function and you get an error message because it is not allowed to have statements by themselves at top level. (However, it is allowed to have standalone statements in a script with `.sc` at the end of the filename.)

d) The annotation `@main` tells the compiler that the function is a main program and can serve as an entry point for execution.

Under the hood, the compiler creates an object with the same name as your main program. In that object, the compiler generates a method called `main` which takes a string array as a parameter and returns `Unit`. A compiled program must have at least one object with exactly one such main method because the JVM execution environment expects it and calls such a main method with a string array containing any program arguments when execution starts.

An alternative to `@main` is to define a so-called *primitive* main method in a single object. (This was necessary in old Scala 2, before the simpler `@main` annotation arrived in Scala 3.)

```
object Hello:
  def main(args: Array[String]): Unit =
    val message = "Hello world!"
    println(message)
```

Solution Task 5. *Create and use collections.*

`val xs = Vector(2)`	1	⇝	I	new reference to a sequence of length 1
`val ys = Array.fill(9)(0)`	2	⇝	C	new reference to a mutable sequence
`Vector.fill(9)(' ')`	3	⇝	J	new immutable sequence with blank characters
`xs(0)`	4	⇝	E	shortcut for `apply(0)`
`xs.apply(0)`	5	⇝	F	indexing, returns the first element
`xs :+ 0`	6	⇝	A	new collection with a zero appended at the end
`0 +: xs`	7	⇝	H	new collection with a zero prepended at the beginnin
`ys.mkString`	8	⇝	K	new string with all elements concatenated
`ys.mkString(",")`	9	⇝	G	new string with commas between elements
`xs.map(_.toString)`	10	⇝	D	new collection, elements converted to strings
`xs.map(_.toInt)`	11	⇝	B	new collection, elements converted to integers

Solution Task 6. *Compare `Array` and `Vector`.*

a)

Vector	1	⇝	B	immutable
Array	2	⇝	A	mutable

b)

Vector	1	⇝	B	creates variants with more/other elements quickly from existing ones
Array	2	⇝	A	slow when changing size (requires copying everything)

c)

Vector	1	⇝	A	`xs == ys` is **`true`** if all elements are equal
Array	2	⇝	B	unlike other Scala collections, == does not check content equality

Solution Task 7. *Calculate sum, min and max in `args`.*

```
@main def sumMinMax(args: Int*): Unit =
  println(s"${args.sum} ${args.min} ${args.max}")
```

```
> scala run sum-min-max.scala -- hej
Illegal command line: java.lang.NumberFormatException: For input string: "hej"
```

Solution Task 8. *Algorithm: SWAP.*

a) The pseudocode may look like this:

```
Declare integer variable temp.
Copy value from x to temp.
```

```
Copy value from y to x.
Copy value from temp to y.
```

b)
You need to declare a temporary variable where you can save one of the values so that it is not overwritten in the first assignment.

```
val temp = x
x = y
y = temp
```

Solution Task 9. *Indexing and assignment in Array with SWAP.*

```
@main def swapFirstLastArg(args: String*): Unit =
  val xs = args.toArray
  if xs.length > 1 then
    val temp = xs(0)
    xs(0) = xs(xs.length - 1)
    xs(xs.length - 1) = temp
  println(xs.mkString(" "))
```

Solution Task 10. ***for*** *expressions and* `map` *expressions.*

`for x <- xs yield x * 2`	1	⇝ A	`Vector(2, 4, 6)`
`for i <- xs.indices yield i`	2	⇝ E	`Vector(0, 1, 2)`
`xs.map(x => x + 1)`	3	⇝ D	`Vector(2, 3, 4)`
`for i <- 0 to 1 yield xs(i)`	4	⇝ B	`Vector(1, 2)`
`(1 to 3).map(i => i)`	5	⇝ C	`Vector(1, 2, 3)`
`(1 until 3).map(i => xs(i))`	6	⇝ F	`Vector(2, 3)`

Solution Task 11. *Algorithm: SUMBUG*

a) Bug: Since `i` is never incremented, the program gets stuck in an infinite loop. Fix: Add a statement at the end of the while-block that increases the value of i by 1. Bug: Since sum is only increased by 1 each time, the result will be the sum of n ones, not the first n integers. Fix: Change so that the sum increases by `i` each iteration, instead of 1. For -1, the result is 0. Explanation: i starts at 1 and is thus never less than n which is -1. The while block executes zero times, and after sum receives its initial value it never changes.

b) The sum is 39502716.
One possible implementation:

```
@main def sumn(n: Int): Unit =
  var sum = 0
  var i = 1
  while i <= n do
    sum = sum + i
    i = i + 1
```

```
  println(sum)
```

2.2 Extra Tasks; additional practice

Solution Task 12. *Algorithm: MAXBUG*

a) Bug: `i` is never incremented. The program gets stuck in an infinite loop. Fix: Add a statement that increments i at the end of the while loop.

b) Here is one implementation:

```
@main def maxn(args: String*): Unit =
  var max = Int.MinValue
  val n = args.length
  var i = 0
  while i < n do
    val x = args(i).toInt
    if x > max then
      max = x
    i += 1
  println(max)
```

c) Change the line initializing max to `var max = args(0).toInt`

d) To avoid `java.lang.IndexOutOfBoundsException: 0` you need a check that ensures nothing is done if the collection `args` is empty:

```
@main def maxn(args: String*): Unit =
  if args.size > 0 then
    var max = args(0).toInt
    val n = args.size
    var i = 0
    while i < n do
      val x = args(i).toInt
      if x > max then
        max = x
      i += 1
    println(max)
  else
    println("Empty")
```

Solution Task 13. *Algorithm MIN-INDEX.*

a) An unnecessary comparison occurs, but does not affect the result.

b)

```
def indexOfMin(xs: Array[Int]): Int =
  var minPos = 0
  var i = 1
  while i < xs.size do
    if xs(i) < xs(minPos) then
      minPos = i
    i += 1
  if xs.size > 0 then minPos else -1
```

Solution Task 14. *The data structure Range.*

a) value: `Range(1,2,3,4,5,6,7,8,9)`
type: `scala.collection.immutable.Range`

b) value: `Range(1,2,3,4,5,6,7,8,9,10)`
type: `scala.collection.immutable.Range`

c) value: `Range(0,5,10,15,20,25,30,35,40,45)`
type: `scala.collection.immutable.Range`

d) value: `10`, type: `Int`

e) value: `Range(0,5,10,15,20,25,30,35,40,45,50)`
type: `scala.collection.immutable.Range`

f) value: `11`, type: `Int`

g) value: `Range(0,1,2,3,4,5,6,7,8,9)`
type: `scala.collection.immutable.Range`

h) value: `Range(0,1,2,3,4,5,6,7,8,9)`
type: `scala.collection.immutable.Range`

i) value: `Range(0,1,2,3,4,5,6,7,8,9)`
type: `scala.collection.immutable.Range`

j) value: `Range(0,1,2,3,4,5,6,7,8,9,10)`
type: `scala.collection.immutable.Range.Inclusive`

k) value: `Range(0,1,2,3,4,5,6,7,8,9,10)`
type: `scala.collection.immutable.Range.Inclusive`

l) value: `Range(0,5,10,15,20,25,30,35,40,45)`
type: `scala.collection.immutable.Range`

m) value: `Range(0,5,10,15,20,25,30,35,40,45,50)`
type: `scala.collection.immutable.Range`

n) value: `11`, type: `Int`

o) value: `500500`, type: `Int`

2.3 Advanced Tasks; challenges

Solution Task 15. *Rock-Paper-Scissors game.* One (easy-to-understand?) solution that tries all combinations:

```
def winner(user: Int, computer: Int): String =
  if      choices(user) == "Rock" && choices(computer) == "Paper" then "Computer"
  else if choices(user) == "Rock" && choices(computer) == "Scissors"  then "You"
  else if choices(user) == "Paper" && choices(computer) == "Rock" then "You"
  else if choices(user) == "Paper" && choices(computer) == "Scissors"  then "Computer"
  else if choices(user) == "Scissors"  && choices(computer) == "Rock" then "Computer"
  else if choices(user) == "Scissors"  && choices(computer) == "Paper" then "You"
  else "None"
```

A trickier solution (and harder to understand?) using modulo arithmetic:

```
def winner(user: Int, computer: Int): String =
  val result = (user - computer + 3) % 3
  if user == computer then "None"
  else if result == 1 then "You"
  else "Computer"
```

Modulo arithmetic requires the elements in `choices` to be in *loses-to* order, that is Rock, Paper, Scissors. Adding 3 is to avoid negative numbers which behave differently in modulo arithmetic.

Solution Task 16. *Compare execution time for resizing between `Array` and `Vector`.*

a) With a computer that has an `i7-4790K CPU @ 4.00GHz` this was the result:

```
scala> def time(block: => Unit): Double =
     |   val t = System.nanoTime
     |   block
     |   (System.nanoTime - t)/1e6  // gives milliseconds
def time(block: => Unit): Double

scala> val as = Array.fill(1e6.toInt)(0)
val as: Array[Int] = Array(0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
large output truncated, print value to show all

scala> val vs = Vector.fill(1e6.toInt)(0)
val vs: Vector[Int] = Vector(0, 0, 0, 0, 0, ...
large output truncated, print value to show all

scala> val ast = (for i <- 1 to 10 yield time(as :+ 0)).sum / 10.0
val ast: Double = 1.8719819999999998

scala> val vst = (for i <- 1 to 10 yield time(vs :+ 0)).sum / 10.0
val vst: Double = 0.006485099999999999

scala> ast / vst
val res3: Double = 288.6589258453995
```

b) `Vector` is two orders of magnitude faster in this example. The reason is that every size change of an `Array` requires allocation and element-wise copying of a completely

new `Array`, whereas the immutable `Vector` can reuse the entire data structure with already allocated elements when new elements are added.

Solution Task 17. *Memory usage for `Range`.*

a) The variable `interval` refers to objects taking up 12 bytes.

b) The variable `sequence` refers to objects taking up about 4 billion bytes.

Solution Task 18. *Examine the generated byte-code.*

a) The function `plusxy` looks like this:

```
public int plusxy(int, int);
  descriptor: (II)I
  flags: (0x0001) ACC_PUBLIC
  Code:
    stack=2, locals=3, args_size=3
       0: iload_1
       1: iload_2
       2: iadd
       3: ireturn
    LineNumberTable:
      line 2: 0
    LocalVariableTable:
      Start  Length  Slot  Name   Signature
          0       4     0  this   Lplusxy$package$;
          0       4     1     x   I
          0       4     2     y   I
```

The instruction `iadd` performs the actual addition.

b) There is one more parameter in the bytecode. The instruction `iadd` now occurs twice. The `iadd` instruction adds exactly two integers at a time.

```
public int plusxyz(int, int, int);
  descriptor: (III)I
  flags: (0x0001) ACC_PUBLIC
  Code:
    stack=2, locals=4, args_size=4
       0: iload_1
       1: iload_2
       2: iadd
       3: iload_3
       4: iadd
       5: ireturn
    LineNumberTable:
      line 2: 0
    LocalVariableTable:
      Start  Length  Slot  Name   Signature
          0       6     0  this   Lplusxyz$package$;
          0       6     1     x   I
          0       6     2     y   I
          0       6     3     z   I
```

c) The prefix `i` in the instruction name `iadd` stands for "integer" and indicates that integer addition is intended.

3. Solution `Excercise03`

Solution Task 1. *Match concepts with description.*

function header	1	⇝	C	has parameter list and optionally a return type
function body	2	⇝	N	code executed on function call
parameter list	3	⇝	A	describes names and types of parameters
block	4	⇝	O	may have local names; last line gives value
named arguments	5	⇝	H	arguments can be passed in any order
default argument	6	⇝	L	arguments can be omitted
value call	7	⇝	B	argument evaluated before call
name call	8	⇝	D	delayed evaluation of argument
map	9	⇝	F	applies a function to each element of a collection
pure function	10	⇝	I	always returns same result for same arguments
predicate	11	⇝	G	a function returning a boolean value
random seed	12	⇝	M	produces a repeatable pseudo-random sequence
anonymous function	13	⇝	E	function without a name, also called lambda
recursive function	14	⇝	K	a function that calls itself
stack trace	15	⇝	J	list of calls at runtime error

Solution Task 2. *Define and call functions.*

a)

```
def increase(x: Int = 1): Int = x + 1
```

b) `5`

c)

```
def reduce(x: Int = 1): Int = x - 1
```

d) `1`

e)

- *Short, simplified explanation:* The parameter in the function header is a local name for input that can be used in the function body, while the argument is the actual value of the parameter that is passed in the call.

- *Longer, more precise explanation:* A **parameter** is a declaration of an immutable variable in a function header whose name is locally available in the function body. At a call, the parameter name is *bound* to a specific argument. An **argument** is an expression that is applied to a function at call time. Normally the argument is evaluated before the call happens, but if the parameter type is preceded by `=>` the evaluation of the argument is delayed and instead happens *each time* the parameter name occurs in the function body.

Solution Task 3. *Implement function in different ways.*

a)

```
def sumFirst(n: Int): Int = ???
```

b)

```
def sumFirst(n: Int): Int = (1 to n).sum
```

```
scala> sumFirst(-1)
val res0: Int = 0
```

c)

```
def sumFirst(n: Int): Int =
  var result = 0
  var i = 1
  while i <= n do
    result += i
    i += 1
  end while
  result
end sumFirst
```

```
scala> sumFirst(-1)
val res1: Int = 0
```

Solution Task 4. *The text game AliensOnEarth.*

a) `"penguin"` is the best option with probability $\frac{1}{2} + \frac{1}{2} \cdot \frac{1}{3} = \frac{2}{3}$

b)

`options.indices`	1	$\rightsquigarrow$	B	integer sequence with all indices in a sequence
`"1X2".toLowercase`	2	$\rightsquigarrow$	A	convert string to lowercase
`Random.nextInt(n)`	3	$\rightsquigarrow$	C	random number in range `0 until n`
`try { } catch { }`	4	$\rightsquigarrow$	E	catch exceptions to prevent crash
`""" ... """`	5	$\rightsquigarrow$	F	string spanning multiple lines
`s.stripMargin`	6	$\rightsquigarrow$	D	removes margin up to vertical bar
`e.printStackTrace`	7	$\rightsquigarrow$	G	prints exception information

Solution Task 5. *Pure functions.*

- The functions `inc`, `addY` and `isPalindrome` are pure. Note that the variable `y` is initialized to `0` and cannot be changed because it is declared with the keyword `val`.

Solution Task 6. *Apply function to each element in a collection. Function as argument.*

`for i <- 1 to 3 yield increment(i)`	1	⇝ E	`Vector(2, 3, 4)`
`Vector(2, 3, 4).map(i => increment(i))`	2	⇝ A	`xs`
`xs.map(increment)`	3	⇝ B	`Vector(4, 5, 6)`
`xs.map(increment).map(increment)`	4	⇝ D	`Vector(5, 6, 7)`
`xs.foreach(increment)`	5	⇝ C	`()`

Solution Task 7. *Anonymous functions.*

`(0 to 2).map(i => i + 1)`	1	⇝ B	`(2 to 4).map(i => i - 1)`
`(1 to 3).map(_ + 1)`	2	⇝ D	`Vector(2, 3, 4)`
`(2 to 4).map(math.pow(2, _))`	3	⇝ E	`Vector(4.0, 8.0, 16.0)`
`(3 to 5).map(math.pow(_, 2))`	4	⇝ A	`Vector(9.0, 16.0, 25.0)`
`(4 to 6).map(_.toDouble).map(_ / 2)`	5	⇝ C	`Vector(2.0, 2.5, 3.0)`

Solution Task 8. *Create your own control structure using call-by-name parameter.*

a)

```
def repeat(n: Int)(block: => Unit): Unit =
  var i = 0
  while i < n do
    block
    i += 1
  end while
```

b)

```
repeat(100):
  val diceRoll = (math.random() * 6 + 1).toInt
  print(s"\$diceRoll ")
```

c) If the parameter `block` were not declared with call-by-name, the argument would be evaluated once before the call and then the same result used at every iteration. With call-by-name, the block can contain code that e.g. updates a variable which we want to happen every iteration. Call-by-name is similar to the argument code being "pasted" into each place in the function body where the parameter name occurs.

Solution Task 9. *Learn to read a stack trace.*A stack trace contains the following information:

1. an error message
2. names of all functions called at the time of the runtime error, according to all activation records on the call stack
3. current namespace for each function, that is package/singleton object

4. source file name for each function
5. line number in each function
6. the first function shown is the one where the error occurred
7. possibly the error occurs in standard library functions and then your own function is earlier in the call chain

Example of a stack trace:

```
> cat fel.scala
@main def run =
  println("Hej Scala!" + Vector().head)
> scala run fel.scala
Compiling project (Scala 3.3.0, JVM)
Compiled project (Scala 3.3.0, JVM)
Exception in thread "main" java.util.NoSuchElementException: empty.head
  at scala.collection.immutable.Vector.head(Vector.scala:279)
  at fel$package$.run(fel.scala:2)
  at run.main(fel.scala:1)
>
```

3.1 Extra Tasks; additional practice

Solution Task 10. *Function with multiple parameters.*

a)

```scala
def sum(x: Int, y: Int): Int = x + y

def diff(x: Int, y: Int): Int = x - y
```

b) It is `-100` because `0 - 100 == -100`

c) It is 15 because the nested call corresponds to
`diff(100, 42 + 43) == (100 - 85)`

d) It is 185 because the nested call corresponds to
`sum(42 + 43, 100 - 0) == (85 + 100)`

e) It is 256 because `Byte.MaxValue == 127` and `Byte.MinValue == -128` and `sum(127 + 128, 1)`

Solution Task 11. *Average value.*

```scala
def avg(x: Int, y: Int): Double = (x + y) / 2.0
```

Solution Task 12. *Function calls with named arguments.*

a)

```
Namn: Triangelsson, Stina
Namn: Oval, Viktor
```

b)

- The caller can choose the order themselves.

- The code becomes easier to understand if parameter names are self-describing.
- Helps to prevent bugs caused by swapped parameters.

Solution Task 13. *Function as a genuine value.*

a)

`multipleCalls(1, greet)`	1	⇝ D	`f2("Hello!")`
`multipleCalls(3, greet)`	2	⇝ B	`multipleCalls(3, f1)`
`multipleCalls(2, f1)`	3	⇝ A	`f2("Hello!\nHello!")`
`multipleCalls(1, f3)`	4	⇝ C	`f3()`

b) `f1` and `f3` have type `() => Unit` and `f2` has type `String => Unit`.

c) No. `f1` and `f2` have two different function types.

d) Yes, that works fine.

e) No. When the function has no parameter, the compiler needs more information to be sure that you want a function value.

f) Yes! Now with type information in place, the compiler knows what you want to do.

Solution Task 14. *Discarded result values and return type `Unit`.*

a) Procedures return the empty value and `println` is a procedure. When the empty value is printed, you see `()`.

b) Procedures return the empty value. If you explicitly specify return type `Unit`, you have a better chance that the compiler can warn you when calculations are discarded. A warning does not stop execution, but is a way for the compiler to give you tips about things that might need fixing in your code.

c) In Scala variable declaration, just like an assignment statement, is not an expression and has no value.

d) The code becomes easier to read and the compiler gets better possibility to help with warnings if result values risk being discarded.

Solution Task 15. *Call-by-name.*
The block is an expression that has the value `(): Unit`. Evaluation of the block happens where the name `b` appears in the procedure body, which is twice.

```
scala> doThisTwice { println("goededag") }
goededag
goededag
```

3.2 Advanced Tasks; challenges

Solution Task 16. *Mutability of parameters.*

a) No, in Scala the parameter is immutable and it will be a compile error if you try to assign it a new value in the function body.

b) c) Yes, it is perfectly fine in both Java and Python to change the value of the parameter in the function body by assignment, but the code risks to become confusing. https://stackoverflow.com/questions/2970984

Solution Task 17. *Call-by-value and call-by-name.*

a) Each call of `snark` causes a print and a delay before returning 42.
`42 + 42 == 84` which becomes the expression value.

```
scala> snark + snark
snark snark val res1: Int = 84
```

b) The expression `snark` is evaluated directly at the call and the parameter `x` is bound to the value 42 and in the function body $42+42$ is computed. The print happens only once.

```
callByValue(snark)
snark val res2: Int = 84
```

c) The evaluation of the expression `snark` is delayed until each occurrence of parameter `x` in the function body. The print happens twice.

```
callByName(snark)
snark snark val res3: Int = 84
```

d) Evaluation of the expression `zzz` is delayed until each occurrence of parameter `x` in the function body. The print happens once because **`val`** variables are assigned their value once at the delayed initialization.

```
callByName(zzz)
snark val res4: Int = 84
```

Solution Task 18. *Create your own control structure for iteration with loop variable.*

a)

```
def repeat(n: Int)(p: Int => Unit): Unit =
  var i = 0
  while i < n do
    p(i)
    i += 1
  end while
end repeat
```

b)

```
repeat(100){ i =>
  print("i ")
```

```
  println(math.random())
}
```

You can use fewer braces with the help of colon:

```
repeat(100): i =>
  print("i ")
  println(math.random())
```

Solution Task 19. *Split parameter list and curried functions.*

a)

```
1 scala> def add2(a: Int)(b: Int) = a + b
2 def add2(a: Int)(b: Int): Int
3
4 scala> add2(1)(1)
5 val res0: Int = 2
```

b)

- Line 3:

```
doremi doremi doremi
```

- Line 5:

```
lalalalalalala
```

Solution Task 20. *Recursion.*

a) `countdown` prints out `x` and does a recursive call with `x - 1` as argument, but only if the base condition `x > 0` is true. The result is a finite repetition. `finalCountdown` calls itself recursively but lacks a base case that can stop the recursion, which generates an infinite repetition. At -128 there is *overflow* since the bits are not enough for larger negative numbers and the counting starts over at 127. (If you reduce the delay to `Thread.sleep(1)` it becomes quite fast *stack overflow*)

b) Since `1/x` was *after* the recursive call in the previous subtask, we never reached this (potentially disastrous) calculation, but just put many activation frames on the stack at each call. If we place `1/x` *before* the recursive call, we reach this expression immediately and an exception is thrown due to division by zero.

c) The last line leads to many more recursive calls, as the base condition and recursion are constructed. Feel free to insert a `println` statement before the recursive call and examine exactly what happens.

Solution Task 21. *Investigate tail recursion by throwing exceptions.* `countdown` is tail-recursive because the recursive call is *last* and can be optimized to a `while`-loop by the compiler. It is fine to run all the way until it explodes, even with 10000 calls, and the error message only contains one call to `countdown`.
`countdown2` is not tail-recursive because it has an expression `after` the recursive call. In the error printout you can see all recursive calls to `countdown2` before the base case occurs. At `countdown2(10000)` the base case is not fulfilled before a `StackOverflowError` occurs.

Solution Task 22. *`@tailrec` annotation.* The first time `countNoTailrec(100000L)` is called it causes `StackOverflowError`. With annotation `@tailrec` we get a compilation error because the compiler cannot optimize a non-tail-recursive function. If the function is rewritten, the compiler can optimize it so that the recursion is replaced with a **`while`**-loop and we can run as long as we want without stack overflow. And it runs very fast!!

4. Solution `Excercise04`

4.1 Basic Tasks; preparation for the lab

Solution Task 1. *Match concepts with descriptions.*

module	1	⇝	C	code unit with abstractions that can be reused
singleton object	2	⇝	B	module that can have state; exists in a single instance
package	3	⇝	D	module creating namespace; machine code gets own folder
import	4	⇝	F	makes name locally available without full path
export	5	⇝	P	makes name visible outside as a member of this object
lazy initialization	6	⇝	G	allocation occurs only when name is referenced
member	7	⇝	E	belongs to an object; accessed via dot notation if visible
attribute	8	⇝	H	variable representing (part of) an object's state
method	9	⇝	A	function that is a member of an object
private	10	⇝	K	modifies visibility of an object member
overloading	11	⇝	J	methods with same name but different parameter types
name shadowing	12	⇝	L	local name hides same name in enclosing block
namespace	13	⇝	I	environment where all names are unique
uniform access	14	⇝	M	changing between def and val does not affect usage
dot notation	15	⇝	O	used to access non-private parts
type alias	16	⇝	N	alternative name for type, often improves readability

Solution Task 2. *Nested singleton objects, import, visibility and dot notation.*

a)

```
object Underworld:
  var x = 0
  var y = 1

  object Mole:
    var x = Underworld.x + 10
    var y = Underworld.y + 9

  object Worm:
    private var x = Mole.x
    var y = Mole.y + 190
    def isMoleFood: Boolean = x == Mole.x && y == Mole.y
```

b)

```
scala> :load Underworld.scala
scala> import Underworld.*
scala> Worm.isMoleFood
val res0: Boolean = false
scala> Worm.y = Mole.y
scala> Worm.isMoleFood
```

```
val res1: Boolean = true
```

c)

```
scala> import Mole.*
scala> import Worm.*
scala> x = -1
scala> Mole.x
val res2: Int = -1

scala> Worm.x
1 |Worm.x
  |^^^^^^^^
  |variable x cannot be accessed as a member of Underwold.Worm.type from module

scala> Underworld.x
val res3: Int = 0
```

Explanation: When importing all visible members in Worm, those that are not private will shadow other members with the same name. It is Mole's `x` variable that is assigned `-1` because Worm's `x` is private and not visible externally. Underworld's members are shadowed by Worm's `y` and Mole's `x`, but you can still access them using dot notation.

Solution Task 3. *Export.*

a) Similarity: Both **`import`** and **`export`** control visibility. Difference: **`import`** controls local visibility *inside* an object while **`export`** controls visibility *outside* an object.

b) You can conveniently gather members from other objects and make them visible from your own object using **`export`**.

```
object MyObject:
  export java.awt.Color.*  // all colors become members of MyObject
  export math.{atan2, Pi}  // atan2 and Pi become members of MyObject
```

```
scala> object MyObject:
     |   export java.awt.Color.*
     |   export math.{atan2, Pi}
     |

scala> MyObject.RED
val res0: java.awt.Color = java.awt.Color[r=255,g=0,b=0]

scala> MyObject.atan2(3,3) / MyObject.Pi
val res1: Double = 0.25
```

Solution Task 4. *Tuples.*

a) `depth` has type `Double`.

b) `secret` has type `(String, (Int, Int, Double))`.

c)

```
object Underworld3D:
  private val secret = ("the ascent to the uberworld", (3, 4, 0.0))

  object Mole:
```

```
    var pos = (5, 3, math.random() * 10 + 1)

    def depth: Double  = pos._3

  object Worm:
    private var pos = (0, 0, 10.0)

    def isMoleFood: Boolean = pos == Mole.pos

    def isRightUnderTheRise: Boolean =
      pos._1 == secret._2._1 && pos._2 == secret._2._2
```

d) Zero-tuple.

Solution Task 5. *Lazy initialization.*

a) "nu!" is printed only the first time z is used.

```
scala> z
nu!
val res19: Array[Int] = Array(0, 0, 0, 0, 0, 0, 0, 0, 0, 0)

scala> z
val res20: Array[Int] = Array(0, 0, 0, 0, 0, 0, 0, 0, 0, 0)
```

b) The array allocation happens the first time z is used (and not at declaration).

```
scala> lazy val z = { println("nu!"); Array.fill(1e9.toInt)(0)}
val z: Array[Int] = <lazy>

scala> z
nu!
java.lang.OutOfMemoryError: Java heap space
```

c) The print "nu!" occurs only when singleton object zzz is used for the first time. We should move the initialization of b before a or make a a **lazy val**.

d)

```
scala> import test.*
import test.*

scala> zzz.a      // "nu!" is printed when we use zzz
nu!               // this did *not* happen when we imported test
val res0: Int = 42

scala> buggig.a   // a is 0 because b was not initialized
val res1: Int = 0

scala> funkar.a   // with lazy val we avoid the problem
val res2: Int = 42


scala> zzz.a     // second time initialization already done, no "nu!"
val res3: Int = 42
```

e) `lazy val a = expression` means the initialization expression is evaluated *once*, but the evaluation is delayed until the name `a` is actually used. Whereas `def b = expression` means the function body expression is evaluated *each time* the name `b` is (possibly) used.

Solution Task 6. *Extension methods.*

a)

```
scala> extension (i: Int) def inc = i + 1
```

b)

```
scala> extension (i: Int) def dec = i - 1
```

c)

```
extension (i: Int)
  def inc = math.incrementExact(i)
  def dec = math.decrementExact(i)
```

d) Using `math.incrementExact` and `math.decrementExact` throws an exception if we cross the boundary:

```
scala> math.incrementExact(Int.MaxValue)
java.lang.ArithmeticException: integer overflow
  at java.base/java.lang.Math.incrementExact(Math.java:1023)
  at scala.math.incrementExact(package.scala:418)
  ... 34 elided
```

Solution Task 7. *Extension methods.*

a) According to the documentation, the `PixelWindow` class has these parameters:

- `width : Int` specifies the window width, default `800`
- `height: Int` specifies the window height, default `640`
- `title : String` specifies the window title, default `"PixelWindow"`
- `background: Color` specifies background color, default `java.awt.Color.black`
- `foreground: Color` specifies foreground color, default `java.awt.Color.green`

You can create new window instances like this:

```
val w1 = new introprog.PixelWindow()
val w2 = new introprog.PixelWindow(100, 200, "My nice new window")
```

b) You can also download the latest `introprog` like this:

```
curl -o introprog_3-1.4.0.jar -sLO https://fileadmin.cs.lth.se/introprog.jar
```

c)

```
> scala repl --jar introprog_3-1.4.0.jar
scala> val w = new introprog.PixelWindow(400,300,"HEJ")
scala> w.line(100, 100, 200, 100)
scala> w.line(200, 100, 200, 200)
scala> w.line(200, 200, 100, 200)
scala> w.line(100, 200, 100, 100)
```

d)

```
package hello

object Main:
  val w = new introprog.PixelWindow(400, 300, "HEJ")

  var color = java.awt.Color.red

  def square(p: (Int, Int))(side: Int): Unit =
    if side > 0 then
      // side == 1 gives a square that is a single pixel
      val d = side - 1

      w.line(p._1,     p._2,     p._1 + d, p._2,     color)
      w.line(p._1 + d, p._2,     p._1 + d, p._2 + d, color)
      w.line(p._1 + d, p._2 + d, p._1,     p._2 + d, color)
      w.line(p._1,     p._2 + d, p._1,     p._2,     color)

  def main(args: Array[String]): Unit =
    println("Draw square:")
    square(300,100)(50)
```

e)

```
> scala run hello-window.scala --jar introprog_3-1.4.0.jar --main-class hello.M
```

f)

```
> scala run hello-window.scala --dep se.lth.cs::introprog:1.4.0 --main-class he
```

g)

```
//> using scala 3.3
//> using dep se.lth.cs::introprog:1.4.0
```

Solution Task 8. *Color*.

a)

```
object Color:
  import java.awt.{Color as JColor}

  val mole   = new JColor( 51,  51,   0)
  val soil   = new JColor(153, 102,  51)
  val tunnel = new JColor(204, 153, 102)
```

b)

```
package hello

object Color:
  import java.awt.{Color as JColor}
```

```
  val mole   = new JColor( 51, 51,    0)
  val soil   = new JColor(153, 102, 51)
  val tunnel = new JColor(204, 153, 102)


object Main:
  val w = new introprog.PixelWindow(width = 400, height = 300, title = "HEJ")

  type Pt = (Int, Int)

  var color = java.awt.Color.red

  def straight(p:  Pt)(d: Int) = w.line(p._1, p._2, p._1 + d - 1, p._2, color)

  def fill(p: Pt)(s: Int) = for i <- 0 until s do straight((p._1, p._2 + i))(s)

  def square(p: (Int, Int))(side: Int): Unit =
    if (side > 0) then
      val d = side - 1  // side == 1 should give a square that is one pixel big
      w.line(p._1,     p._2,     p._1 + d, p._2,     color)
      w.line(p._1 + d, p._2,     p._1 + d, p._2 + d, color)
      w.line(p._1 + d, p._2 + d, p._1,     p._2 + d, color)
      w.line(p._1,     p._2 + d, p._1,     p._2,     color)

  def main(args: Array[String]): Unit =
    import Color.*
    color = soil
    fill(100,100)(75)
    color = tunnel
    fill(100,100)(50)
    color = mole
    fill(150,150)(25)
```

c) Tuple parentheses can be omitted in calls to `straight` and `fill` when a single tuple argument is the only parameter in its parameter list.

Solution Task 9. *Events.*

a) The immutable integer variable `KeyPressed` in `introprog.PixelWindow.Event` has the value 1.

b) The code line below handles the key press case:

```
case PixelWindow.Event.KeyPressed => println(s"lastKey == \$w.lastKey")
```

c) When the up arrow key on the keyboard is pressed, `w.lastKey` gets the string value `"Up"`. The test program prints the following when the up arrow key is pressed and released:

```
lastEventType: 1 => KeyPressed
lastKey == Up
lastEventType: 2 => KeyReleased
lastKey == Up
```

d) A loop that lets the user draw lines with the mouse:

```
var start = (0,0)
while w.lastEventType != PixelWindow.Event.WindowClosed do
  w.awaitEvent(10)  // wait for next event max 10 milliseconds
  w.lastEventType match {
    case PixelWindow.Event.MousePressed  =>
      start = w.lastMousePos

    case PixelWindow.Event.MouseReleased =>
      w.line(start._1, start._2, w.lastMousePos._1, w.lastMousePos._2)

    case PixelWindow.Event.WindowClosed  =>
       println("Goodbye!");
    case _ =>
  }
  PixelWindow.delay(100) // wait for 0.1 seconds
```

4.2 Extra Tasks; additional practice

Solution Task 10. *Functions are objects with an `apply` method.* Yes, you can write:

```
scala> plus(42, 43)
```

The compiler inserts `.apply` for you.

Solution Task 11. *Create modules using singleton objects.*

a)

```
scala> "pearIceCream".split('I')
val res0: Array[String] = Array(pear,ceCream )
```

b)

```
scala> Test()
--- FREQUENCY ANALYSIS OF:
Five ants are more than four elephants. Eat cucumber.
# letters : 43
# words   : 9
# sentences: 2

--- FREQUENCY ANALYSIS OF:
Galaxies in my breams. Tomato is good. Pear split.
# letters : 39
# words   : 9
# sentences: 3

--- FREQUENCY ANALYSIS OF:
Five ants are more than four elephants. Eat cucumber. Galaxies in my breams. To
# letters : 82
# words   : 18
# sentences: 5
```

c) The object `statistics` has mutable state in the variable `history`. The state changes when calling `printFreq`.

d)

```
object count:
  extension (s: String)
    def nbrOfLetters:Int = s.count(_.isLetter)
    def nbrOfWords:Int = split.words(s).size
    def nbrOfSentences: Int = split.sentences(s).size
```

Solution Task 12. *Tuples as parameters.*

```
def distxy(x1: Int, y1: Int, x2: Int, y2: Int): Double =
  hypot(x1 - x2, y1 - y2)

def distpt(p1: (Int, Int), p2: (Int, Int)): Double =
  hypot(p1._1 - p2._1, p1._2 - p2._2)

def distp(p1: (Int, Int))(p2: (Int, Int)): Double =
```

```
  hypot(p1._1 - p2._1, p1._2 - p2._2)
```

Solution Task 13. *Tuples as function results.*

```
def statistics(xs: Vector[Double]): (Int, Double, (Double, Double)) =
  (xs.size, xs.sum / xs.size, (xs.min, xs.max))
```

```
scala> statistics(Vector(0, 2.5, 5))
val res10: (Int, Double, (Double, Double)) = (3,2.5,(0.0,5.0))
```

Solution Task 14. *Create modules using packages.*

a)

```
> code paket.scala
> scala paket.scala
> find . -type d         # linux command that lists all subdirectories
./.scala-build/project_103be31561-3d0d386400/classes
./.scala-build/project_103be31561-3d0d386400/classes/main
./.scala-build/project_103be31561-3d0d386400/classes/main/gurka
./.scala-build/project_103be31561-3d0d386400/classes/main/gurka/tomat
./.scala-build/project_103be31561-3d0d386400/classes/main/gurka/tomat/banan
./.scala-build/project_103be31561-3d0d386400/classes/main/gurka/tomat/banan/p2
./.scala-build/project_103be31561-3d0d386400/classes/main/gurka/tomat/banan/p2/
./.scala-build/project_103be31561-3d0d386400/classes/main/gurka/tomat/banan/p1
./.scala-build/project_103be31561-3d0d386400/classes/main/gurka/tomat/banan/p1/
./.scala-build/project_103be31561-3d0d386400/classes/main/gurka/tomat/banan/p1/
```

b)

```
> scala run paket.scala --main-class gurka.tomat.banan.Main
Hej paket p1.p11!
Hej paket p1.p12!
Hej paket p2.p21!
```

c) Yes, in Scala 3 packages are allowed to have variables and functions at top level. https://stackoverflow.com/a/56566166

4.3 Advanced Tasks; challenges

Solution Task 15. *How to do without* `do while` *in Scala 3?*

a) It gives a compilation error:

```
> scala repl --scala 3
Welcome to Scala 3.1.3 (17.0.3, Java OpenJDK 64-Bit Server VM).
Type in expressions for evaluation. Or try :help.

scala> var i = 0
var i: Int = 0

scala> do i += 1 while (i < 10)
-- [E103] Syntax Error: ----------------------------------------
1 |do i += 1 while (i < 10)
```

```
  |^^
  |Illegal start of statement
```

b)

```
> scala repl --scala 3
Welcome to Scala 3.1.3 (17.0.3, Java OpenJDK 64-Bit Server VM).
Type in expressions for evaluation. Or try :help.

scala> var i = 0
var i: Int = 0

scala> while
     |   i += 1
     |   i < 10
     | do ()

scala> i
val res0: Int = 10
```

Solution Task 16. *Postfix operators for increment and decrement.*

```scala
extension (i: Int)
  def ++ = i + 1
  def -- = i - 1
```

Solution Task 17. *Use ready package: Color chooser.*

a) The chosen color is returned after the user clicks **OK**

```
scala> introprog.Dialog.selectColor()
val res1: java.awt.Color = java.awt.Color[r=0,g=204,b=0]
```

b) The default color red is returned after the user clicks **Cancel**

c) Color chooser reverts to default color.

Solution Task 18. *Use ready package: user dialogs.*

a)

```
scala> introprog.Dialog.show("Game over!")
```

b) The function `input` returns a string which is the empty string `""` if the user clicks **Cancel**

```
scala> val name = introprog.Dialog.input("What is your name?")
name: String = Oddput Superkodare
```

c) The function `select` returns a string with the text on the button that the user clicked.

```
scala> introprog.Dialog.select("Wat kies jij?",Vector("Steen","Schaar","Papier"
val res4: String = Papier
```

Solution Task 19. *Create your own `jar` file.*

a)

`jar -create -verbose -file <name of created jar file> <name of what to package>`

b)

```
package hello

object Main:
  def main(args: Array[String]): Unit = println("Hello package!")
```

```
scala compile hello.scala --destination .
```

c)

```
> jar -c -v -f my.jar hello
> ls
> scala repl --jar my.jar
scala> hello.Main.main(Array())
Hello package!
```

d)

```
> scala run --jar my.jar  --main-class hello.Main
```

Solution Task 20. *How large is JDK8?* The JDK8 platform includes 4240 precompiled classes, organized in 217 different packages. See Stackoverflow: http://stackoverflow.com/questions/3112882

5. Solution Excercise05

5.1 Basic Tasks; preparation for the lab

Solution Task 1. *Match concepts with description.*

class	1	⇝	H	template for creating multiple instances of the same type
instance	2	⇝	I	copy of an object with its own state memory
constructor	3	⇝	M	creates instance, allocates space for state memory
class parameter	4	⇝	K	binds to arguments given at construction
reference equality	5	⇝	B	instances are considered different even if states are equal
structural equality	6	⇝	J	instances considered equal if they have the same state
case class	7	⇝	F	no need to write new; automatic structural equality
getter	8	⇝	L	indirect access to attribute value
setter	9	⇝	A	indirect assignment to attribute value
companion object	10	⇝	D	sees private members of class with same name
factory method	11	⇝	E	helper function for indirect construction
`null`	12	⇝	G	a value that does not refer to any instance
`new`	13	⇝	C	keyword for direct instance creation of a class

Solution Task 2. *Class and instance.*

a)

`Singelpunkt.x`	1	⇝	B	`1`
`Punkt.x`	2	⇝	G	`value is not a member of object`
`val p  = new Singelpunkt`	3	⇝	C	`Not found: type`
`val p1 = new Punkt`	4	⇝	D	`p1: Punkt = Punkt@27a1a53c`
`val p2 = Punkt()`	5	⇝	F	`p2: Punkt = Punkt@51ab04bd`
`{ p1.x = 1; p2.x }`	6	⇝	E	`3`
`(new Punkt).y`	7	⇝	H	`2`
`{ val p: Punkt = null; p.x }`	8	⇝	A	`java.lang.NullPointerException`

b)

error	*type*	*explanation*
`value is not a member of object`	compile-time error	there is no instance named `Punkt`. *The error message refers to the fact that the class's auto-generated constructor proxy lacks a variable named* `x`.
`Not found: type`	compile-time error	there is no class named `Singelpunkt`
`NullPointerException`	runtime error	you cannot reference attributes in an instance that does not exist

Solution Task 3. *Class parameters.*

a)

`val p1 = Point(1, 2)`	1	$\rightsquigarrow$ C	`p1: Point = Point@30ef773e`
`val p2 = Point()`	2	$\rightsquigarrow$ A	`missing argument for parameter`
`val p2 = Point(3, 4)`	3	$\rightsquigarrow$ E	`p2: Point = Point@218cf600`
`p2.x - p1.x`	4	$\rightsquigarrow$ B	`2`
`Point(0, 1).y`	5	$\rightsquigarrow$ F	`1`
`Point(0, 1, 2)`	6	$\rightsquigarrow$ D	`too many arguments for constructor`

b)

error	*type*	*explanation*
`missing argument for parameter`	compile-time error	you must provide arguments at construction of class `Point`
`too many arguments for constructor`	compile-time error	the number of arguments does not correspond to the number of class parameters

Solution Task 4. *Immutable class with default arguments.*

a)

`val p1 = Point3D()`	1	⇝ C	`p1: Point3D = Point3D@2eb37eee`
`val p2 = Point3D(y = 1)`	2	⇝ F	`p2: Point3D = Point3D@65a9e8d7`
`Point3D(z = 2).z`	3	⇝ E	`value cannot be accessed`
`p2.y = 0`	4	⇝ B	`Reassignment to val`
`p2.y == 0`	5	⇝ A	`false`
`p1.x == Point3D().x`	6	⇝ D	`true`

b) The problem is that as the class `Point3D` is declared it is not possible to read off the `z` coordinate after an instance is constructed. It would be better if the `z` attribute were also a `val`.

Solution Task 5. *Case class, `this`, equality, `toString` and companion object.*

a)

`val p1 = Pt(1, 2)`	1	⇝ E	`Pt(1,2)`
`val p2 = Pt(y = 3)`	2	⇝ C	`Pt(0,3)`
`val p3 = MutablePt(5, 6)`	3	⇝ A	`MPt(5,6)`
`val p4 = Mutable()`	4	⇝ D	`Not found`
`p2.moved(dx = 1) == Pt(1, 3)`	5	⇝ F	`true`
`p3.move(dy = 1) == MutablePt(5, 7)`	6	⇝ B	`false`

b) The compiler infers `MutablePt` because that is the type of the self-reference `this`.

```
scala> :type new MutablePt().move()
MutablePt
```

c) Instantiation with universal apply methods is sugar that makes code easier to read and write. This is possible thanks to the fact that at compilation automatically a constructor proxy is created that instantiates the object with the keyword `new`. A constructor proxy is a companion object with an apply method.
One case where `new` must explicitly be used is when implementing your own apply method in a companion object. If `new` is not used inside the apply method, the same method will be called recursively instead of creating a new instance. See the following example:

```
class Point3D(val x: Int, val y: Int, val z: Int)

object Point3D:
  var secretNumber = 42
  def apply(x: Int, y: Int, z: Int): Point3D =
    if secretNumber == 42 then
      Point3D(x, y, z) // The code will get stuck in an infinite loop.

    else new Point3D(x, y, z) // Works because 'new' is used.
```

d) A method that reads (parts of) an object's (private) state without modifying it is called a *getter*.

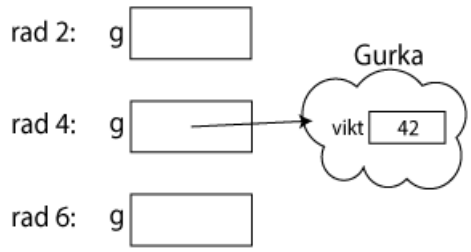
Solution Task 6. *Implement parts of the classes `Pos`, `KeyControl`, `Mole` and `BlockWindow` needed during the lab `blockbattle1`.* This task is lab preparation. Evaluate your solutions through your own tests in REPL.

a) It is not possible to call `Pos.moved(0,1)`. The reason is that `moved` does not exist in the companion object `Pos`, hence the error message "value `moved` is not a member of object `Pos`". To call a method defined inside a class you must call it through a (reference to an) instance of the class.

5.2 Extra Tasks; additional practice

Solution Task 7. *Instantiation with `new` and the value `null`.*

a) Lines 3 and 7 both give the error message `java.lang.NullPointerException`, because of attempts to reference members using a `null` reference, which does not point to any object.



b)

Solution Task 8. *Create a point class that can handle polar coordinates and a class that represents a polygon using these points.*

a)

```
package graphics

case class Point(x: Double, y: Double):
  val r: Double = math.hypot(x, y)
  val theta: Double = math.atan2(y, x)
  def +(p: Point): Point = Point(x + p.x, y + p.y)

object Point:
  def polar(r: Double, theta: Double): Point =
    Point(r * math.cos(theta), r * math.sin(theta))
```

b)

```
package graphics

case class Polygon(points: Vector[Point]):
  val nbrOfCorners: Int = points.length

object Polygon:
  def regular(nbrOfCorners: Int, radius: Double, midPoint: Point): Polygon =
    val points = new Array[Point](nbrOfCorners)
    for i <- 0 until nbrOfCorners do
```

```
      val theta = i * (2 * math.Pi) / nbrOfCorners
      points(i) = Point.polar(radius, theta) + midPoint
    end for
    Polygon(points.toVector)
```

c) A perfect circle cannot be created, but it is possible to get so close that it is impossible to see the corners. Try e.g. with 50 corners as below.

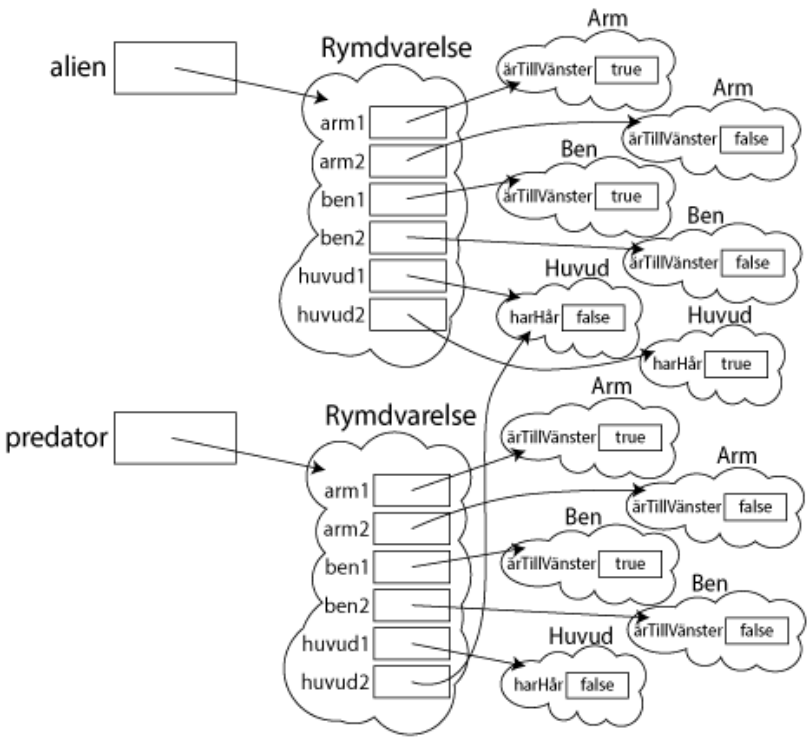
```
scala> import graphics.*
scala> val circle = Polygon.regular(50,70,Point(-25,-25))
scala> val window = PolygonWindow()
scala> window.draw(circle)
```

An irregular polygon can also be created. Use the constructor of `Polygon` directly. For example as below.

```
scala> val irregular =
         Polygon(Vector(Point(1,8), Point(33,14), Point(99,87), Point(42,56)))
scala> window.draw(irregular)
```

Solution Task 9. *Classes, instances and garbage.*

a) We create two aliens, `alien` and `predator`, each with two arms and two legs, and each with two heads (one bald and one hairy). Afterwards, neither `alien` nor `predator` is bald since both have one head with hair. Then the reference to `predator`'s hairy head is made to refer to `alien`'s bald head. Now the predator is totally bald and shares head with the alien.



b) Since there is no longer any reference pointing to that object, the garbage collector will take care of it and eventually the place in memory will be overwritten when the

space is needed. Objects that have no reference to them cannot be accessed.

Solution Task 10. *Case class. Immutable square.*

a)

```
case class Square(val x: Int = 0, val y: Int = 0, val side: Int = 1):
  val area: Int = side * side

  def moved(dx: Int, dy: Int): Square = Square(x + dx, y + dy, side)

  def isEqualSizeAs(that: Square): Boolean = this.side == that.side

  def scale(factor: Double): Square =
    Square(x, y, (side * factor).round.toInt)

object Square:
  val unit: Square = Square()
```

b)

```
scala> val (s1, s2) = (Square(), Square(1, 10, 1))
val s1: Square = Square(0,0,1)
val s2: Square = Square(1,10,1)

scala> val s3 = s1 moved (1,-5)
val s3: Square = Square(1,-5,1)

scala> s1 isEqualSizeAs s3       // same size
val res0: Boolean = true

scala> s2 isEqualSizeAs s1       // same size
val res1: Boolean = true

scala> s1 isEqualSizeAs Square.unit   // s1 has side 1
val res2: Boolean = true

scala> s2.scale(math.Pi) isEqualSizeAs s2  // different size
val res3: Boolean = false

scala> s2.scale(math.Pi) == s2.scale(math.Pi) // same content
val res4: Boolean = true

scala> s2.scale(math.Pi) eq s2.scale(math.Pi)  // different objects
val res5: Boolean = false

scala> Square.unit eq Square.unit   // same object
val res6: Boolean = true
```

5.3 Advanced Tasks; challenges

Solution Task 11. *Content equality between different types.*

```
scala> 42 == "Fourtytwo"
1 |42 == "Fourtytwo"
  |^^^^^^^^^^^^^^^^^
  |Values of types Int and String cannot be compared with == or !=

scala> Cucumber(50) == Car("Sedan")
val res0: Boolean = false
```

The second expression is problematic because it will always result in **false**, since the classes `Cucumber` and `Car` are two incomparable types that should not be compared in terms of content equality. This reduces type safety which increases the risk of subtle bugs where wrong types are compared.
Equality comparisons done between primitive types are type-checked by the compiler and therefore can produce compilation errors if two different types, such as `Int` and `String`, are compared. However, this usually does not apply to user-defined types, which means that an equality comparison between different user-defined types will always result in **false**.
It is possible, however, to achieve the same type checks for user-defined types as for primitive types by importing `scala.language.strictEquality`.

```
import scala.language.strictEquality
class Cucumber(val weight: Int)

class Car(val type: String)
```

```
scala> Cucumber(50) == Car("Sedan")
1 |Cucumber(50) == Car("Sedan")
  |^^^^^^^^^^^^^^^^^^^^^^^^^^^
  |Values of types Cucumber and Car cannot be compared with == or !=
```

Solution Task 12. *Attribute representation. Private constructor. Factory method.*

a) It will be a compilation error, because the constructor is private.

```
scala> class Point private (val x: Int, val y: Int)
     | object Point:
     |   def apply(x: Int = 0, y: Int = 0): Point = new Point(x, y)
     |   val origo = apply()
     |
// defined class Point
// defined object Point

scala> new Point(0, 0)
1 |new Point(0, 0)
  |    ^^^^^
  |constructor Point cannot be accessed as a member of Point from module class
```

b)

- By having a private constructor and only allowing indirect instantiation via the factory method it is easy to change attribute representation in the future without existing code needing to change.
- The access rules for companion objects are such that companions see each other's private parts.

c)

```
class Point private (private val p: (Int, Int)):
  def x: Int = p._1
  def y: Int = p._2

object Point:
  def apply(x: Int = 0, y: Int = 0): Point = new Point(x, y)
  val origo = apply()
```

Solution Task 13. *Visibility of class parameters and constructor,* ***`private`****`[this]`*.

a) `Cucumber5` is broken. Since the weight in `Cucumber5` is private to the instance and not the class, an instance cannot access another instance's weight.

```
11 |  def buddyWeight = buddy.weight
   |                    ^^^^^^^^^^^
   |value weight cannot be accessed as a member of (Cucumber5.this.buddy : Cucu
```

b)

```
scala> new Cucumber1(42).weight
1 |new Cucumber1(42).weight
  |^^^^^^^^^^^^^^^^^^^
  |value weight cannot be accessed as a member of Cucumber1 from module class

scala> new Cucumber2(42).weight
val res0: Int = 42

scala> new Cucumber3(42).weight
1 |new Cucumber3(42).weight
  |^^^^^^^^^^^^^^^^^^^
  |value weight cannot be accessed as a member of Cucumber3 from module class

scala> val noneCucumber: Cucumber4 = null
val noneCucumber: Cucumber4 = null

scala> new Cucumber4(42, noneCucumber).buddyWeight
java.lang.NullPointerException: Cannot invoke "rs$line$1$Cucumber4.weight()" be
  at rs$line$1$Cucumber4.buddyWeight(rs$line$1:8)
  ... 38 elided

scala> new Cucumber4(42, new Cucumber4(84, null)).buddyWeight
val res2: Int = 84

scala> new Cucumber6(42)
1 |new Cucumber6(42)
  |    ^^^^^^
  |constructor Cucumber6 cannot be accessed as a member of Cucumber6 from modul
```

```

scala> new Cucumber7(-42)
1 |new Cucumber7(-42)
  |    ^^^^^^
  |constructor Cucumber7 cannot be accessed as a member of Cucumber7 from modul

scala> Cucumber7(-42)
java.lang.IllegalArgumentException: requirement failed: negative weight: -42

scala> val g = Cucumber7(42)
val g: Cucumber7 = Cucumber7@51fd1c7c

scala> g.weight
val res4: Int = 42

scala> g.weight = -1

scala> g.weight
val res5: Int = -1
```

Solution Task 14.

a)
Line 1:

```
  java.lang.IllegalArgumentException: requirement failed: negative weight: -42
```

`Cucumber8.apply` requires that `weight >= 0` otherwise `require` throws an exception.
Line 5:

```
  java.lang.IllegalArgumentException: requirement failed: negative weight: -1
```

The setter `weight_=` requires `weight >= 0` otherwise `require` throws an exception.
Line 7:

```
  java.lang.IllegalArgumentException: requirement failed: negative weight: -958
```

Since `42 - 1000` is less than zero `require` throws an exception.

b) You can impose your own specific requirements on what values are allowed so you have greater control that nothing unexpected happens.

Solution Task 15. *Object with mutable state.*

a)

```scala
class Frog private (initX: Int = 0, initY: Int = 0):
  private var _x: Int = initX
  private var _y: Int = initY
  private var _distanceJumped: Double = 0

  def x: Int = _x
  def y: Int = _y

  def jump(dx: Int, dy: Int): Unit =
    _x += dx
    _y += dy
```

```
    _distanceJumped += math.hypot(dx, dy)


  def randomJump: Unit =
    def rnd = util.Random.nextInt(10) + 1
    jump(rnd, rnd)

  def distanceToStart: Double = math.hypot(x,y)
  def distanceJumped: Double = _distanceJumped
  def distanceTo(f: Frog): Double = math.hypot(x - f.x, y - f.y)

object Frog:
  def spawn(): Frog = Frog()
```

b) Example test program:

```
object FrogTest:
  def test(): Unit =
    val f1 = Frog.spawn()
    assert(f1.x == 0 && f1.y == 0, "Test of spawn, reqt 1 & 4 failed.")

    f1.jump(4, 3)
    assert(f1.x == 4 && f1.y == 3, "Test of jump, reqt 1 & 4 failed.")

    f1.jump(4, 3)
    assert(f1.distanceJumped == 10, "Test of jump, reqt 2 failed.")

    f1.jump(-4, -3)
    assert(f1.distanceToStart == 5, "Test of jump, reqt 3 failed.")

    for x <- 1 to 10000 do
      val f2 = Frog.spawn()
      f2.randomJump
      assert(f2.x > 0 && f2.x <= 10 && f2.y > 0 && f2.y <= 10,
        "Test of randomJump, reqt 5 failed.")

    println("Test Ok!")
```

c) A method that is an indirect reading of attribute values is called a getter.

d)

```
class Frog private (initX: Int = 0, initY: Int = 0):
  private var _x: Int = initX
  private var _y: Int = initY
  private var _distanceJumped: Double = 0

  def jump(dx: Int, dy: Int): Unit =
    _x += dx
    _y += dy
    _distanceJumped += math.hypot(dx, dy)
```

```
  def x: Int = _x
  def x_=(newX: Int): Unit = // Setter for x
    _distanceJumped += math.abs(x - newX)
    _x = newX

  def y: Int = _y
  def y_=(newY: Int): Unit = // Setter for y
    _distanceJumped += math.abs(y - newY)
    _y = newY


  def randomJump: Unit =
    def rnd = util.Random.nextInt(10) + 1
    jump(rnd, rnd)

  def distanceToStart: Double = math.hypot(x,y)
  def distanceJumped: Double = _distanceJumped
  def distanceTo(f: Frog): Double = math.hypot(x - f.x, y - f.y)

object Frog:
  def spawn(): Frog = Frog()
```

e)

```
object FrogSimulation:
  def isAnyCollision(frogs: Vector[Frog]): Boolean =
    var found = false
    frogs.indices.foreach(i =>  // generate all pairs (i,j)
      for j <- i + 1 until frogs.size do
        if !found then
          found = frogs(i).distanceTo(frogs(j)) <= 0.5
    )
    found

  def jumpUntilCrash(n: Int = 100, initDist: Int = 8): (Int, Double) =
    val frogs = Vector.fill(n)(Frog.spawn())
    (0 until n).foreach(i => frogs(i).x = i * initDist)
    var count = 0
    while !isAnyCollision(frogs) do
      frogs(util.Random.nextInt(n)).randomJump
      count += 1
    (count, frogs.map(_.distanceJumped).sum)


  def run(nbrOfCrashTests: Int = 10) =
    for i <- 1 to nbrOfCrashTests do
      val (n, dist) = jumpUntilCrash()
      println(s"\nAntalet looprundor innan grodkrock: $n")
      println(s"Totalt avstånd hoppat av alla grodor: $dist")
```

Solution Task 16. *Object with mutable state.*

```
class Square private (val initX: Int, val initY: Int, val initSide: Int):
  private var nMoves = 0
  private var sumCost = 0.0

  private var _x = initX
  private var _y = initY

  private var _side = initSide

  private def addCost(): Unit =
    sumCost += math.hypot(x - initX, y - initY) * side

  def x: Int = _x
  def y: Int = _y

  def side = _side

  def scale(factor: Double): Unit = _side = (_side * factor).round.toInt

  def move(dx: Int, dy: Int): Unit =
    _x += dx; _y += dy
    nMoves += 1
    addCost()

  def moveTo(x: Int, y: Int): Unit =
    _x = x; _y = y
    nMoves += 1
    addCost()

  def cost: Double = sumCost

  def pay: Double = {val temp = sumCost; sumCost = 0; temp}

  override def toString: String =
    s"Square[($x, $y), side: $side, #moves: $nMoves times, cost: $sumCost]"

object Square:
  private var created = Vector[Square]()

  def apply(x: Int, y: Int, side: Int): Square =
    require(side >= 0, s"side must be positive: $side")
    val sq = (new Square(x, y, side))
    created :+= sq
    sq

  def apply(): Square = apply(0, 0, 1)

  def totalNumberOfMoves: Int = created.map(_.nMoves).sum

  def totalCost: Double = created.map(_.cost).sum
```

6. Solution Excercise06

6.1 Basic Tasks; preparation for the lab

Solution Task 1. *Match on constant values.*

a) Scala's **match** expression compares stepwise the value with each **case** and then returns a value belonging to the corresponding **case**.

b)

```
scala.MatchError
```

Runtime error, occurs for certain input during execution.

Solution Task 2. *Guards in case branches.*
The guard introduced at **case** `'c'` generates a random number between 0 and 1 and if the number is not greater than 0.5, there will be no match with **case** `'c'` and the program tries further until the default case.
The guard condition must be met in order to match as usual.

Solution Task 3. *Pattern match on attributes in case classes.*
G100true. When changing the order: Gtrue100.
match tests if the companion object `Cucumber` is of type `Cucumber` with two parameter values. The specified parameters are assigned names, `weight` is given the name `w` and `isRotten` the name `rotten` and then printed. If these names are switched, they are printed in the reversed order.

Solution Task 4. *Match on case objects and the usefulness of **sealed**.*

a)

```
Cannot extend sealed trait Color in a different source file
```

The error message occurs because the REPL treats each input individually and therefore does not allow the subtype `Spades` to extend the supertype `Color` since it was sealed. More about this later in the course...

b) -

c) Assuming **import** `Deck._` has been written...

```
def paracolor(f: Color): Color = f match
  case Spades  => Clubs
  case Hearts => Diamonds
  case Diamonds   => Hearts
  case Clubs  => Spades
```

d)

```
<console>:17: warning: match may not be exhaustive.
It would fail on the following input: Diamonds
```

The warning appears already at compilation.

e)

```
scala.MatchError: Diamonds (of class Diamonds)
  at .paracolor(<console>:17)
```

This is a runtime error.

f) If a class is **sealed**, it means that if an element should be matched and is a subtype of this class, Scala gives a warning already at compile time if there is a risk of a `MatchError`, i.e., if the **match** expression is not exhaustive and there are cases not covered by a **case**.
A sealed supertype means that the programmer gets a warning at compile time if a case is not covered, and in that case which of the subtypes is missing, as well as other help from the compiler. This requires, however, that all the subtypes share the same file as the sealed class.

Solution Task 5. *Pattern match enumeration.*a)

```
def paracolor(f: Color): Color = f match
  case Color.Spades => Color.Clubs
  case Color.Hearts => Color.Diamonds
  case Color.Diamonds => Color.Hearts
  case Color.Clubs => Color.Spades
```

Similar to task 4c an **import** statement can also be written here to access members of `Color` without dot notation. However, it is not always advantageous to import members into the global namespace, since name clashes can occur. Suppose an example where we work on a program with a graphical user interface that has a color `Red` defined. Also suppose that we want to import yet another red color for the suits hearts and diamonds, also named `Red`. In this scenario, a name clash would occur since `Red` is already defined, so the import could not proceed.

b) When pattern matching, **sealed trait** used with **case** objects works in practice similarly to using **enum**. We saw in subtask 4d that the REPL warned already at compile time that the match is not exhaustive. This also applies when using **enum**.

Solution Task 6. *Meaning of lowercase and uppercase initial letters when matching.*

a) Both `str` and `whatever` match the input, regardless of what it is, because they start with a lowercase letter.
However, `str` has a guard that the string must start with `c`, which means only **val** `g = "cucumber"` matches this. However, **val** `x = "urka"` is caught by `whatever`, which lacks a guard.

b)

```
<console>:16: warning: patterns after a variable pattern cannot match (SLS 8.1.
```

and

```
<console>:17: warning: unreachable code due to variable pattern 'tomato' on lin
```

Even though a class `tomato` exists, Scala treats the **match** case as a catch-all because it starts with a lowercase letter. This means any object not of type `Cucumber` will print *tomato* and the last case cannot be reached.

c)

```
case `tomato` => println("tomato")
```

Solution Task 7. *Match on the contents of a Vector.*

```
hguod
hguod
42
```

For each element in `xss`, a match is done that results in a string. What happens in each branch is explained below.

1. The first match branch is never activated because `xss` contains no empty vector.
2. The second branch fits `Vector("hej")`, and the variable `a` is bound to `"hej"`.
3. The third branch matches `Vector("on", "dough")` where the first value is ignored (underscore) and the second value is bound to `b`.
4. The fourth branch matches a sequence with three values where the middle value is `"x"`. The last branch is not triggered here but would match anything not caught earlier.

Solution Task 8. *Use `Option` and match on potentially missing values.*

a)

1. `var maybe` becomes an `Option` holding `Int` but with no value, called `None`.
2. Since `var maybe` has no value, its size is 0.
3. `var maybe` is assigned the value 42 which is stored in a `Some` indicating a value exists.
4. Since `var maybe` now contains a value, its size is 1.
5. Since `var maybe` contains a value, it is not empty.
6. Since `var maybe` contains a value, it is defined.
7. `def increaseIfExists` matches an `Option[Int]` with its different cases.
   If there is a value, i.e., `opt: Option[Int]` is a `Some`, it returns a `Some` with the original value plus 1.
   If there is no value, i.e., `opt: Option[Int]` is a `None`, it returns `None`.
8. -
9. -
10. -
11. `def increaseIfExists` is applied to `maybe` and returns a `Some` with the value in `maybe` plus 1, i.e., 43.
12. `def öka` takes an `Int` and returns this value plus 1.
13. `map` applies `def öka` to the sole element in `maybe`, 42. This returns a `Some` with value 43 assigned to `merKanske`.

b)

1. `val` `sentence` becomes a `Some` with value 42.
2. `val` `notSentence` becomes an `Option[Int]` without value, i.e., `None`.
3. `map(_ + 1)` is applied on `sentence` and increases the existing value by 1 to 43.
4. `map(_ + 1)` is applied on `notSentence` but since no value exists, it remains `None`.
5. `map(_ + 1)` is applied again on `notSentence` but now `orElse` method is included. If an `Option` has no value (`None`), the code in `orElse` executes, which here prints out *missing* for the missing value.
6. The same call from the previous line is executed this time on `meningen` and since a value exists, only the first part that increases the value by 1 runs.

This method can be used instead of the `match`-version in the previous example due to its simpler form. An `Option` contains either a value or none so a longer `match` is unnecessary.

c)

1. A vector `xs` is created with every fifth number from 42 to 82.
2. An empty `Int` vector `e` is created.
3. `headOption` takes the first value of vector `xs` and returns it wrapped in an `Option`, `Some(42)`.
4. The first value in vector `xs` is wrapped in an `Option` and then extracted by `get`, 42.
5. Same as previous line but uses `getOrElse`, which returns 0 if the `Option` is `None`. Since `xs` has at least one value, this returns the same as before, 42.
6. Like previous line but instead of returning 0 if missing, returns an `Option[Int]` with 0 as value.
7. `headOption` tries to take the first element of empty vector `e` but since no value exists, returns `None`.
8.
```
java.util.NoSuchElementException: None.get
```

   Like preceding line returns `None`, but calling `get` on `None` causes a runtime error.
9. Like previous returns `None` from empty vector `e`, but `getOrElse` returns 0 instead.
10. Like previous uses `getOrElse` on the `None` and returns an `Option[Int]` holding 0.
11. A vector containing `xs` and three `e` vectors is created.
12. `map` applies `lastOption` on each sub-vector extracting last elements into a new vector. Since `e` is empty, returns `None` for those.
13. Same as previous but uses `flatten` to remove all `None` elements leaving only actual values.

14. `lift` method fetches element at index 0 in `xs` wrapped in an `Option` which is `Some(42)`.
15. `lift` tries to get element at index 1000 in `xs`, which doesn't exist, so returns `None`.
16. Same as previous but applied to vector `e`, and then `getOrElse` returns 0 since `None`.
17. `find` searches `xs` for first element > 50 and returns it in `Some(52)`.
18. `find` searches `xs` for first element < 42 but none exists, so returns `None`.
19. `find` on `e` searching for element > 42 and prints *HITTAT!* if found. Since `e` is empty, `None` is returned and `foreach` does not run.

d) Using `-1` as a return value for missing or error cases can cause runtime errors that are hard to detect. **`null`** can cause crashes if misused during execution. `Option` avoids such problems, and if `getOrElse` or similar is used, the program does not crash. Moreover, functions returning `Option` do not need as much documentation about what can be returned; instead, the code itself shows it explicitly.
Finally, `Option` is more type-safe than **`null`**. When you return an `Option`, the type of the contained value is specified, if any, which makes it easier to understand and limits what can be returned.

Solution Task 9. *Throw exceptions.*

a)

1. An `Exception` is thrown with message *PANG!*.
2. Various types of `Exception` are shown.
3. An exception of type `IllegalArgumentException` is thrown with message *wrong wrong wrong*.
4. An exception with message `stormWind!` is thrown and caught by the **`catch`** expression. A **`match`** inspects the exception and prints the message, and returns -1.

b) For example:
`OutOfMemoryError`, when the program runs out of memory.
`IndexOutOfBoundsException`, when accessing a vector position larger than available.
`NullPointerException`, when calling a method or similar on a null reference.

c) If try and catch branches have the same type, here `Int`, the compiler infers that type for the entire expression. If the catch branch returns a different type such as `String`, then the most precise inferred type for the whole expression is `Matchable`, a direct subtype of the most general type `Any`.

Solution Task 10. *Catch exceptions with `scala.util.Try`.*

a)

1. **`def`** `pang` is defined that throws an `Exception` with message *PANG!*.
2. Scala utilities `Try`, `Success` and `Failure` are imported.

3. `def` `pang` is called in `Try` which catches the exception and wraps it in a `Failure`.
4. The method `recover` matches the exception in the `Failure` from above with a `case` and turns that `Failure` into a `Success`, similar to catch.
5. The string *silent* runs in the previous test but since no exception is thrown it is wrapped in a `Success` and `recover` does nothing. It only handles exceptions.
6. `def` `maybePang` is defined that is equally likely to return the string *silent* or call `pang`.
7. `def` `maybeOk` is defined that tests `def` `maybePang` with `Try`.
8. A vector `xs` is filled with results of 100 runs of `maybeOk` consisting of `Success` and `Failure`.
9. The element at index 13 in vector `xs` is matched to either `Success` (prints ":)") or `Failure` (prints ":(" plus error message).
10. -
11. -
12. Method `isSuccess` tests if element at `xs(13)` is `Success`, returns `true` if so.
13. Method `isFailure` tests if element at `xs(13)` is `Failure`, returns `true` if so.
14. Method `count` counts how many elements in `xs` are `Failure`.
15. Method `find` searches `xs` for a `Failure` element and returns it wrapped in an `Option`.
16. `badOpt` is assigned first `Failure` found in `xs`.
17. `goodOpt` is assigned first `Success` found in `xs`.
18. Printing `badOpt` yields `Option[scala.util.Try[String]] = Some(Failure(java.lang.Exception: PAN`
19. Calling `get` on `badOpt` returns the `Failure` inside the `Option`.
20. Calling `get` again on the `Failure` extracts and throws the exception.
21. Using `getOrElse` on the `Failure` returns the alternative string "bomb disarmed!" since it is a `Failure`.
22. Using `getOrElse` on the `Success` returns the contained string "silent".
23. The same method is mapped over all elements of `xs` and printed.
24. `toOption` is mapped over `xs` turning `Success` into `Some` with the string "silent" and `Failure` into `None`.
25. `flatten` removes `None`s from the vector, leaving just the `Some` elements.
26. The `size` returns the number of `Success` elements counted.

b) `pang` has return type `Nothing`, a special Scala type which is subtype of all types but has no values.

c) `Nothing` is a subtype of every type in Scala's hierarchy, including `String`, which implies `String` includes both strings and `Nothing`, so the inferred type is `String`.

6.2 Advanced Tasks; challenges

Solution Task 11. *Use matching or dynamic binding?*

a)

```
package vegopoly

trait Vegetable:
  def weight: Int
  def isRotten: Boolean
  def isWorthEating: Boolean

case class Cucumber(weight: Int, isRotten: Boolean) extends Vegetable:
  val isWorthEating: Boolean = (!isRotten && weight > 100)

case class Tomato(weight: Int, isRotten: Boolean) extends Vegetable:
  val isWorthEating: Boolean = (!isRotten && weight > 50)

object Main:
  def randomWeight: Int = (math.random()*500 + 100).toInt

  def randomDecay: Boolean = math.random() > 0.8

  def randomCucumber: Cucumber = Cucumber(randomWeight, randomDecay)

  def randomTomato: Tomato = Tomato(randomWeight, randomDecay)

  def randomVegetable: Vegetable =
    if math.random() > 0.2 then randomCucumber else randomTomato

  def main(args: Array[String]): Unit =
    val harvest = Vector.fill(args(0).toInt)(randomVegetable)
    val edible = harvest.filter(_.isWorthEating)
    println("Number of vegetables harvested: " + harvest.size)
    println("Number of edible vegetables: " + edible.size)
```

b) Add the following `case class`:

```
case class Broccoli(weight: Int, isRotten: Boolean) extends Vegetable:
  val isEatable: Boolean = (!isRotten && weight > 80)
```

Also add the following in `object Main` before `def randomVegetable`:

```
def randomBroccoli: Broccoli = Broccoli(randomWeight, randomRotten)
```

Finally, modify `def randomVegetable` to:

```
def randomVegetable: Vegetable =     // choose e.g. this distribution:
  val rnd = math.random()
  if rnd > 0.5 then randomCucumber      // 50% chance cucumber
```

```
else if rnd > 0.2 then randomTomato // 30% chance tomato
else randomBroccoli             // 20% chance broccoli
```

c) The advantages of the **match** version, and pattern matching generally, are that it is easy to change how matching is done. This makes it easy to change the definition of edibility. However, if these change rarely but vegetables change often, the polymorphic alternative may be preferred since it is easier to implement and change when vegetables change.

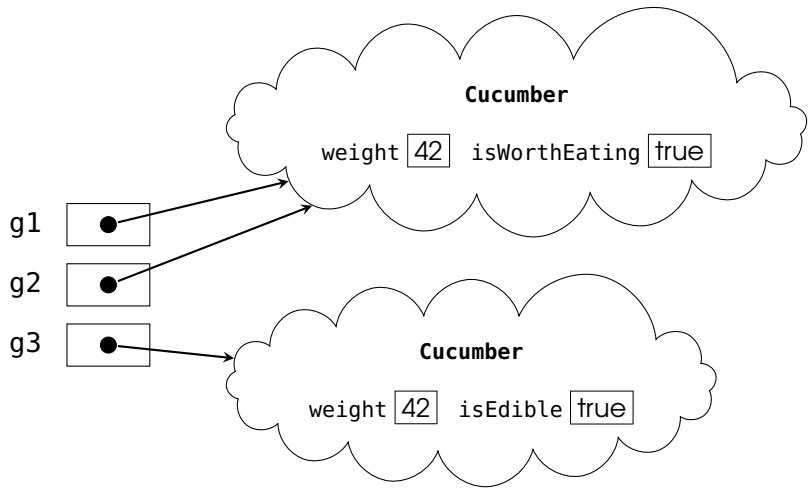
Solution Task 12. *Method* `equals`.

a)

1. A class `Cucumber` is created with parameters `weight` of type `Int` and `isEatable` of type `Boolean`.
2. `g1` is assigned an instance of `Cucumber` with `weight = 42` and `isEatable = true`.
3. `g2` is assigned the same `Cucumber` object as `g1`.
4. `g3` is assigned a new `Cucumber` instance with the same parameter values as `g1`.
5. The == (`equals`) method compares `g1` and `g2` and returns **true**.
6. == (`equals`) method compares `g1` and `g3` and returns **false**.
7. `def equals(x\$1: Any): Boolean`

As seen the parameter of `equals` is of type `Any`. Since the program does not know the class, it uses `Any.equals`. Unlike primitive types that compare content with `equals`, classes default to reference equality unless otherwise specified. `g1` and `g2` refer to the same object but `g3` points to a different one, so `g1 == g3` is false.

b)



c) -

d) In the first three lines the same actions happen as in subtask a. Now when comparing `Cucumber` objects, the `equals` method is overridden by the one defined specifically for `Cucumber`. Since both objects compared are of type `Cucumber`, the case `that: Cucumber` matches. It then compares the weights and edible booleans returning `Boolean` indicating equality, which in both cases is true.

e) In subtask a, `g1 == g3` was false despite content equality. After overriding, this expression returns **`true`**, demonstrating that == now uses content equality.

Solution Task 13. *Polynomial.*

a) TODO!!!

b) TODO!!!

Solution Task 14. *`Option` as a collection.*
Examples of methods that exist both for `Vector` and `Option`: `foreach`, `filter`, `fold` etc.
The method `contains` returns a `Boolean` indicating whether it holds a value or not.

Solution Task 15. *Catch exceptions with **`catch`** in Java and Scala.* TODO!!!

Solution Task 16. *Polynomial, continuation: reduction.*

Solution Task 17. *Type-safe content test with method ===.*

Solution Task 18. *Override `equals` with content equality also for non-final classes.*

Solution Task 19. *Override equals with inheritance.*

Solution Task 20. *Special matchings.* TODO!!!

Solution Task 21. *Extractors.* TODO!!!

Solution Task 22. *Polynomial, continuation: polynomial division.* TODO!!!

7. Solution `Excercise07`

7.1 Basic Tasks; preparation for the lab

Solution Task 1. *Match concepts with description.*

element	1	$\rightsquigarrow$	G	object in a data structure
collection	2	$\rightsquigarrow$	B	data structure with elements of same type
collection library	3	$\rightsquigarrow$	J	many ready-made collections with different properties
sequence(collection)	4	$\rightsquigarrow$	L	zero or more elements of same type in a specific order
sequence algorithm	5	$\rightsquigarrow$	I	solution to problem using a sequence collection
order	6	$\rightsquigarrow$	A	defines how elements of a type are arranged
sorting	7	$\rightsquigarrow$	C	algorithm arranging elements in a specific order
searching	8	$\rightsquigarrow$	D	algorithm finding elements according to a search criterion
linear search	9	$\rightsquigarrow$	F	searching a sequence until element found
counting	10	$\rightsquigarrow$	H	algorithm counting elements with certain properties
time complexity	11	$\rightsquigarrow$	E	how execution time grows with problem size
memory complexity	12	$\rightsquigarrow$	K	how memory usage grows with problem size

Solution Task 2. *Different sequence collections.*

`Vector`	1	$\rightsquigarrow$	B	immutable, quickly returns any modified collection
`List`	2	$\rightsquigarrow$	C	immutable, quickly returns new collection modified at start
`Array`	3	$\rightsquigarrow$	D	primitive, mutable, fast indexing, fixed size
`ArrayBuffer`	4	$\rightsquigarrow$	A	mutable, fast indexing, can change size
`ListBuffer`	5	$\rightsquigarrow$	E	mutable, fast to modify at start

Solution Task 3. *Using sequence collections.*

a)

`x +: xs`	1	$\rightsquigarrow$	G	`Vector(0, 1, 2, 3)`
`xs +: x`	2	$\rightsquigarrow$	L	`error: value +: is not a member of Int`
`xs :+ x`	3	$\rightsquigarrow$	J	`Vector(1, 2, 3, 0)`
`xs ++ xs`	4	$\rightsquigarrow$	M	`Vector(1, 2, 3, 1, 2, 3)`
`xs.indices`	5	$\rightsquigarrow$	E	`(0 until 3)`
`xs apply 0`	6	$\rightsquigarrow$	C	`1`
`xs(3)`	7	$\rightsquigarrow$	I	`java.lang.IndexOutOfBoundsException`
`xs.length`	8	$\rightsquigarrow$	O	`3`
`xs.take(4)`	9	$\rightsquigarrow$	F	`Vector(1, 2, 3)`
`xs.drop(2)`	10	$\rightsquigarrow$	K	`Vector(3)`
`xs.updated(0, 2)`	11	$\rightsquigarrow$	B	`Vector(2, 2, 3)`
`xs.tail.head`	12	$\rightsquigarrow$	N	`2`
`xs.head.tail`	13	$\rightsquigarrow$	D	`error: value tail is not a member of Int`
`xs.isEmpty`	14	$\rightsquigarrow$	H	**`false`**
`xs.nonEmpty`	15	$\rightsquigarrow$	A	**`true`**

b)

error	*type*	*explanation*
`value +: is not a member of Int`	compile-time error	Operators ending with colon are right-associative. The method call `xs +: x` corresponds in dot notation to `x.+:(xs)` and there is no method named `+:` on integers.
`IndexOutOfBoundsException`	runtime error	There are only 3 elements and indexing starts at 0 in sequence collections.
`value tail is not a member of Int`	compile-time error	The method `head` returns the first element and integers do not have the sequence collection method `tail`.

Solution Task 4. *Copying sequences.*

a)

`xs(0)`	`rs$line5$Mutant@66d766b9` new instances get new hex codes
`ys(0).int`	`0` since `ys` contains the same instance as `xs`
`zs(0).int`	`5` since `!(xs(0) eq zs(0))`
`xs(0) eq ys(0)`	`true` because same instance
`xs(0) eq zs(0)`	`false` since different instances
`(ys.toBuffer :+ new Mutant).apply(0).int`	`0` since the non-deep-copied ArrayBuffer copy refers to the same instance at the first position as both `ys` and `xs` and `x(0).int` became zero in an assignment on line 5 in REPL run

Note that copying with `toArray`, `toVector`, `toBuffer`, etc. *is not deep*, i.e., only instance references are copied, not the instances themselves.

b)

```
def deepCopy(xs: Array[Mutant]): Array[Mutant] =
  val result = Array.ofDim[Mutant](xs.length) // filled with null references
  var i = 0
  while i < xs.length do
    result(i) = new Mutant(xs(i).int) // copy with same content at the same place
    i += 1
  result
```

It is also fine to create the result array with `new Array[Mutant](xs.length)`. You can also use `size` instead of `length`.

c)

```
scala> class Mutant(var int: Int = 0)
// defined class Mutant

scala> def deepCopy(xs: Array[Mutant]): Array[Mutant] =
     |   val result = Array.ofDim[Mutant](xs.length)
     |   var i = 0
     |   while i < xs.length do
     |     result(i) = new Mutant(xs(i).int)
     |     i += 1
     |   result

scala> val xs = Array.fill(3)(new Mutant)
xs: Array[Mutant] = Array(rs$line$2$Mutant@46a123e4, rs$line$2$Mutant@44bc2449,
rs$line2$Mutant@3c28e5b6)

scala> val ys = deepCopy(xs)
ys: Array[Mutant] = Array(rs$line$2$Mutant@14b8a751, rs$line2$Mutant@7345f97d,
rs$line$2$Mutant@554566a8)

scala> xs(0).int = 5

scala> ys(0).int
val res0: Int = 0
```

d) No, since elements cannot be mutated one can safely share references between collections. There is no possibility that changes affect multiple collections at the same time. However, deep copies of collections with mutable elements are commonly (though often time-consuming) made to ensure the original collection does not change.

Solution Task 5. *Updating sequences.*

a)

`{ buf(0) = -1; buf(0) }`	1	$\rightsquigarrow$ D	`-1`
`{ xs(0) = -1; xs(0) }`	2	$\rightsquigarrow$ A	`error: value update is not a member`
`buf.update(1, 5)`	3	$\rightsquigarrow$ F	`(): Unit`
`xs.updated(0, 5)`	4	$\rightsquigarrow$ B	`Vector(5, 2, 3, 4)`
`{ buf += 5; buf }`	5	$\rightsquigarrow$ C	`ArrayBuffer(-1, 5, 3, 4, 5)`
`{ xs += 5; xs }`	6	$\rightsquigarrow$ G	`error: value += is not a member`
`xs.patch(1,Vector(-1,5),3)`	7	$\rightsquigarrow$ E	`Vector(1, -1, 5)`
`xs`	8	$\rightsquigarrow$ H	`Vector(1, 2, 3, 4)`

b)

```
def insert(xs: Array[Int], elem: Int, pos: Int): Array[Int] =
  xs.patch(from = pos, other = Array(elem), replaced = 0)
```

c) Pseudocode below is written so that it compiles though it is incomplete.

```
def insert(xs: Array[Int], elem: Int, pos: Int): Array[Int] =
  val result = ??? /* new array with space for one more element than xs */
  var i = 0
  while(???){/* copy elements before pos and increment i */}
  if i < result.length then /* put elem in result at position i */
  while(???){/* copy the rest */}
  result
```

d)

```
def insert(xs: Array[Int], elem: Int, pos: Int): Array[Int] =
  val result = new Array[Int](xs.length + 1)
  var i = 0
  while i < pos && i < xs.length do  { result(i) = xs(i); i += 1}
  if i < result.length then { result(i) = elem; i += 1 }
  while i < result.length do { result(i) = xs(i - 1); i += 1}
  result
```

```
scala> insert(Array(1, 2), 0, pos = -1)
val res2: Array[Int] = Array(0, 1, 2)

scala> insert(Array(1, 2), 0, pos = 0)
val res3: Array[Int] = Array(0, 1, 2)

scala> insert(Array(1, 2), 0, pos = 1)
```

```
val res4: Array[Int] = Array(1, 0, 2)

scala> insert(Array(1, 2), 0, pos = 2)
val res5: Array[Int] = Array(1, 2, 0)

scala> insert(Array(1, 2), 0, pos = 42)
val res7: Array[Int] = Array(1, 2, 0)
```

Solution Task 6. *Comparing strings in Scala.*

a)

```
true
true
true
true
true
false
```

b) *s1* comes first.

Solution Task 7. *Linear search according to different criteria.*

a)

`xs.indexOf(0)`	1	⇝ F	`5`
`xs.indexOf(6)`	2	⇝ B	`-1`
`xs.indexWhere(_ < 2)`	3	⇝ H	`4`
`xs.indexWhere(_ != 5)`	4	⇝ I	`1`
`xs.find(_ == 1)`	5	⇝ D	`Some(1)`
`xs.find(_ == 6)`	6	⇝ J	`None`
`xs.contains(0)`	7	⇝ C	`true`
`xs.filter(_ == 1)`	8	⇝ A	`Vector(1, 1)`
`xs.filterNot(_ > 1)`	9	⇝ E	`Vector(1, 0, 1)`
`xs.zipWithIndex.filter(_._1 == 1).map(_._2)`	10	⇝ G	`Vector(4, 6)`

b) With a boolean variable `found`:

```scala
def indexOf(xs: Vector[String], p: String => Boolean): Int =
  var found = false
  var i = 0
  while i < xs.length && !found do
      found = p(xs(i))
      i += 1
  if found then i - 1 else -1
```

Or without `found`:

```scala
def indexOf(xs: Vector[String], p: String => Boolean): Int =
  var i = 0
  while i < xs.length && !p(xs(i)) do i += 1
```

```
  if i == xs.length then -1 else i
```

Or maybe you want to start searching from the end; the solution below is probably easier to understand (?) and definitely more concise, but it does *not* fulfill the requirement to return the index of the *first* occurrence as stated in the task. However, if the context allows returning *any* index for which the predicate holds, or if you want to search backwards, this is fine:

```
def indexOf(xs: Vector[String], p: String => Boolean): Int =
  var i = xs.length - 1
  while i >= 0 && !p(xs(i)) do i -= 1
  i
```

There are many other ways to solve this. When implementing algorithms, both in programming exams and in professional development, there are often many different solutions with various properties, advantages, and disadvantages. The important thing is that the solution works well according to requirements, the code is understandable, and the implementation is not so inefficient that users get tired waiting for the result...

Solution Task 8. *Lab preparation: Implement integer counting in Array.*

a)

```
def registerDiceRoll(xs: Seq[Int]): Vector[Int] =
  val result = Array.fill(6)(0)
  xs.foreach{ x =>
    require(x >= 1 && x <= 6, "dice roll must be between 1 & 6")
    result(x - 1) += 1
  }
  result.toVector
```

b)

```
scala> registerDiceRoll(toss(1000))
val res0: Vector[Int] = Vector(171, 163, 166, 152, 184, 164)

scala> registerDiceRoll(toss(1000))
val res1: Vector[Int] = Vector(163, 161, 158, 174, 161, 183)
```

Solution Task 9. *Built-in methods for sorting.*

`'a' < 'A'`	1	⇝	E	`false`
`"AÄÖö" < "AÅÖö"`	2	⇝	H	`true`
`xs.sorted.head`	3	⇝	C	`-1`
`xs.sorted.reverse.head`	4	⇝	G	`3`
`ys.sorted.head`	5	⇝	I	`"ak"`
`zs.indexOf('a')`	6	⇝	B	`1`
`ps.sorted.head.firstName.take(2)`	7	⇝	D	`error: ...`
`ps.sortBy(_.firstName).apply(1).firstName.take(2)`	8	⇝	A	`"ka"`
`xs.sortWith((x1,x2) => x1 > x2).indexOf(3)`	9	⇝	F	`0`

It will fail in the expression trying to sort a sequence of `Person` instances directly with `sorted`:

```
scala> ps.sorted
No implicit Ordering defined for Person.
```

It fails because the compiler cannot find any ordering definition for your own classes. Later in the course we will see how to create custom orderings if you want `sorted` to work on sequences of your own classes, but often `sortBy` and `sortWith` suffice.

Solution Task 10. *Built-in shuffle method.*

a) `Random.shuffle` returns a new shuffled sequence collection of the same type. The original collection's order is not affected.

b) Example usage of `random.shuffle`:

```
scala> import scala.util.Random

scala> val xs = Vector("Sten", "Sax", "Påse")
val xs: Vector[String] = Vector(Sten, Sax, Påse)

scala> (1 to 10).foreach(_ => println(Random.shuffle(xs).mkString(" ")))
Sax Påse Sten
Sten Påse Sax
Sten Sax Påse
Sten Sax Påse
Sten Påse Sax
Sten Påse Sax
Sax Sten Påse
Sten Påse Sax
Sax Påse Sten
Sax Påse Sten

scala> (1 to 5).map(_ => Random.shuffle(1 to 6))
val res1: IndexedSeq[IndexedSeq[Int]] =
  Vector(Vector(5, 2, 1, 4, 3, 6), Vector(6, 5, 4, 2, 1, 3),
  Vector(3, 1, 4, 6, 5, 2), Vector(3, 2, 6, 5, 1, 4),
  Vector(5, 3, 4, 6, 1, 2))

scala> (1 to 1000).map(_ => Random.shuffle(1 to 6).head).count(_ == 6)
val res2: Int = 168
```

Solution Task 11. *Repeated parameters.*

a)

```
scala> def stringSizes(xs: String*): Vector[Int] = xs.map(_.size).toVector
def stringSizes(xs: String*): Vector[Int]

scala> stringSizes("hej")
val res0: Vector[Int] = Vector(3)

scala> stringSizes("hej", "on", "dough", "")
val res1: Vector[Int] = Vector(3, 2, 3, 0)

scala> stringSizes()
val res2: Vector[Int] = Vector()
```

Calling with an empty argument list gives an empty integer sequence.

b)

```
scala> val xs = Vector("hej","on","dough", "")
val xs: Vector[String] = Vector(hej, on, dough, "")

scala> stringSizes(xs: _*)
val res0: Vector[Int] = Vector(3, 2, 5, 0)

scala> stringSizes(Vector(): _*)
val res1: Vector[Int] = Vector()
```

Yes, it works fine with an empty sequence.

7.2 Extra Tasks; additional practice

Solution Task 12. *Counting boolean values. Flipping a coin.*

a)

```
def registerCoinFlips(xs: Seq[Boolean]): (Int, Int) =
  val result = Array.fill(2)(0)
  xs.foreach(x => if (x) result(0) += 1 else result(1) += 1)
  (result(0), result(1))
```

b)

Solution Task 13. *Copying and appending at the end.*

```
def copyAppend(xs: Array[Int], x: Int): Array[Int] =
  val ys = new Array[Int](xs.length + 1)
  var i = 0
  while i < xs.length do
    ys(i) = xs(i)
    i += 1
  ys(xs.length) = x
  ys
```

The two bugs in the algorithm are (1) the condition should be strictly less than and (2) the loop variable increment is missing inside the loop.

Solution Task 14.

```
def seqReverseCopy(xs: Array[Int]): Array[Int] =
  val n = xs.length
  val ys = new Array[Int](n)
  var i = 0
  while i < n do
    ys(n - i - 1) = xs(i)
    i += 1
  ys
```

a)

```
def seqReverseCopy(xs: Array[Int]): Array[Int] =
  val n = xs.length
  val ys = new Array[Int](n)
  for i <- (n - 1) to 0 by -1 do
    ys(n - i - 1) = xs(i)
  ys
```

Solution Task 15. *Copy all except one.*

```
def removeCopy(xs: Array[Int], pos: Int): Array[Int] =
  val n = xs.size
  val ys = Array.fill(n - 1)(0)
```

Input : A sequence xs of type `Array[Int]` and pos
Output: A new sequence of type `Array[Int]` which is a copy of xs but with the element at position pos removed

1 $n \leftarrow$ number of elements xs
2 $ys \leftarrow$ a new `Array[Int]` with space for $n-1$ elements
3 **for** $i \leftarrow 0$ **to** $pos-1$ **do**
4 | $ys(i) \leftarrow xs(i)$
5 **end**
6 $ys(pos) \leftarrow x$
7 **for** $i \leftarrow pos+1$ **to** $n-1$ **do**
8 | $ys(i-1) \leftarrow xs(i)$
9 **end**
10 ys

```
  for i <- 0 until pos do
    ys(i) = xs(i)
  for i <- (pos + 1) until n do
    ys(i - 1) = xs(i)
  ys
```

Solution Task 16. *In-place removal in array.*

Input : A sequence xs of type `Array[Int]`, a position pos, and a padding value pad
Output: An updated sequence of xs where the element at position pos is removed and subsequent elements moved one step towards lower index with the last element assigned the value pad

1 $n \leftarrow$ number of elements in xs
2 **for** $i \leftarrow pos+1$ **to** $n-1$ **do**
3 | $xs(i-1) \leftarrow xs(i)$
4 **end**
5 $xs(n-1) \leftarrow pad$

```
def remove(xs: Array[Int], pos: Int, pad: Int = 0): Unit =
  val n = xs.size
  for i <- (pos + 1) until n do
    xs(i - 1) = xs(i)
  xs(n - 1) = pad
```

Solution Task 17. *Copying and inserting.*

a)

```
def insertCopy(xs: Array[Int], x: Int, pos: Int): Array[Int] =
  val n = xs.size
  val ys = Array.ofDim[Int](n + 1)
  for i <- 0 until pos do
    ys(i) = xs(i)
```

```
  ys(pos) = x
  for i <- pos until n do
    ys(i + 1) = xs(i)
  ys
```

b) `pos` must be `0`.

c)

```
java.lang.ArrayIndexOutOfBoundsException: -1
```

d) Element `x` is added to the end of the array, i.e., the returned array is larger than the input array.

e)

```
java.lang.ArrayIndexOutOfBoundsException: 5
```

You get `ArrayIndexOutOfBoundsException` since indexing is out of size bounds.

Solution Task 18. *In-place insertion in array.*

Input : A sequence xs of type `Array[Int]` and integers x and pos
Output : xs updated in-place, where element x is inserted at pos and subsequent elements are shifted one step, the last element lost

$n \leftarrow$ number of elements in xs
$ys \leftarrow$ a clone of xs
$xs(pos) \leftarrow x$
for $i \leftarrow pos + 1$ **to** $n - 1$ **do**
  $xs(i) \leftarrow ys(i - 1)$
end

```
def insertDropLast(xs: Array[Int], x: Int, pos: Int): Unit =
  val n = xs.size
  val ys = xs.clone
  xs(pos) = x
  for i <- pos + 1 until n do
    xs(i) = ys(i - 1)
```

Solution Task 19. *More built-in methods for linear search.*

a)

- `lastIndexOf` is good if you want to search backwards; without it you would otherwise have to use `xs.reverse.indexOf(e)`
- `indexOfSlice(ys)` searches for the index where a whole sequence `ys` begins, unlike `indexOf(e)` which only searches for a single element.
- `segmentLength(p, i)` returns the length of the longest consecutive sequence where all elements satisfy predicate `p`, starting from position `i`
- `xs.maxBy(f)` applies function `f` on all elements in `xs` and returns the element producing the maximum value; similarly `minBy(f)` gives the minimum according to `f`

b) –

7.3 Advanced Tasks; challenges

Solution Task 20. *Fix Swedish sorting order of ÄÅÖ.* TODO!!!

Solution Task 21. *Fibonacci sequence with ListBuffer.*

a)

```
def fib(max: Long): List[Long] =
  val xs = scala.collection.mutable.ListBuffer.empty[Long]
  xs.prependAll(Vector(1, 1))
  while xs.head < max do xs.prepend(xs.take(2).sum)
  xs.reverse.drop(1).toList
```

b)

```
scala> fib(Int.MaxValue).size
val res0: Int = 46
```

c)

```
def fibBig(max: BigInt): List[BigInt] =
  val xs = scala.collection.mutable.ListBuffer.empty[BigInt]
  xs.prependAll(Vector(BigInt(1), BigInt(1)))
  while xs.head < max do xs.prepend(xs.take(2).sum)
  xs.reverse.drop(1).toList
```

```
scala> fibBig(Long.MaxValue).size
val res0: Int = 92

scala> fibBig(BigInt(Long.MaxValue).pow(64)).size
val res1: Int = 5809

scala> fibBig(BigInt(Long.MaxValue).pow(128)).last
val res2: BigInt = 46657280552835544919455361110286315395072000518604554717752

scala> fibBig(BigInt(Long.MaxValue).pow(128)).last.toString.size
val res3: Int = 2428

scala> fibBig(BigInt(Long.MaxValue).pow(256)).last.toString.size
val res4: Int = 4856

scala> fibBig(BigInt(Long.MaxValue).pow(1024)).last.toString.size
java.lang.OutOfMemoryError: Java heap space
```

Solution Task 22. *In-place reverse sequence.*

```
def reverseChars(xs: Array[Char]): Unit =
  val n = xs.length
  for i <- 0 to (n/2 - 1) do
    val temp = xs(i)
    xs(i) = xs(n - i - 1)
    xs(n - i - 1) = temp
```

Solution Task 23. *Palindrome predicate.*

a) Reversing with `reverse` can require one traversal plus memory for the copy. The content comparison requires another traversal. (This is not a big problem since the world's longest palindrome is no longer than 19 letters and is an obscure Finnish word rarely uttered in daily speech. Which one?)

b)

```
def isPalindrome(s: String): Boolean =
  val n = s.length
  var foundDiff = false
  var i = 0
  while i < n/2 && !foundDiff do
    foundDiff = s(i) != s(n - i - 1)
    i += 1
  !foundDiff
```

Solution Task 24. *More useful sequence collection methods.*

```
scala> val xs = Vector.tabulate(10)(i => math.pow(2, i).toInt)
xs: Vector[Int] = Vector(1, 2, 4, 8, 16, 32, 64, 128, 256, 512)

scala> xs.forall(_ < 1024)
val res0: Boolean = true

scala> xs.exists(_ == 3)
val res1: Boolean = false

scala> xs.count(_ > 64)
val res2: Int = 3

scala> xs.zipWithIndex.take(5)
val res3: Vector[(Int, Int)] = Vector((1,0), (2,1), (4,2), (8,3), (16,4))
```

Solution Task 25. *Arrays don't behave, but `ArraySeqs` do!*

a) `xs` provides content equality and is of type `Seq[Int]` with underlying type `ArraySeq[Int]`. You cannot assign elements in an `ArraySeq`, as it is immutable. It does not update when the original array changes.

```
scala> val as1 = Array(1,2,3)
val as1: Array[Int] = Array(1, 2, 3)

scala> val as2 = Array(1,2,3)
val as2: Array[Int] = Array(1, 2, 3)


scala> val (xs1, xs2) = (as1.toSeq, as2.toSeq)
val xs1: Seq[Int] = ArraySeq(1, 2, 3)
val xs2: Seq[Int] = ArraySeq(1, 2, 3)

scala> as1 == as2
val res0: Boolean = false

scala> xs1 == xs2
```

```
val res1: Boolean = true

scala> as1(0) = 42

scala> xs1
val res2: Seq[Int] = ArraySeq(1, 2, 3)

scala> xs1(0) = 42
value update is not a member of Seq[Int]
```

b) With repeated parameters you get an `ArraySeq`.

```
scala> def f(xs: Int*) = xs
def f(xs: Int*): Seq[Int]

scala> println(f(1,2,3))
ArraySeq(1, 2, 3)
```

c) It is not possible to have a generic array as function result without attaching a context bound `ClassTag` to the type parameter so that the compiler can generate code for type conversion required at JVM runtime. See example here:
http://docs.scala-lang.org/overviews/collections/arrays.html

Solution Task 26. The sequence collection `List` is almost twice as fast when processed at the beginning but about 1000 times slower when processed at the end of a sequence with 100000 elements.
Different executions run at different speeds on the JVM, among other things due to optimizations that occur when the JVM warms up and the so-called Just-In-Time compilation does its mighty job. It sometimes suddenly runs significantly slower when the garbage collector is forced to do a time-consuming cleanup of the memory.

Solution Task 27. –

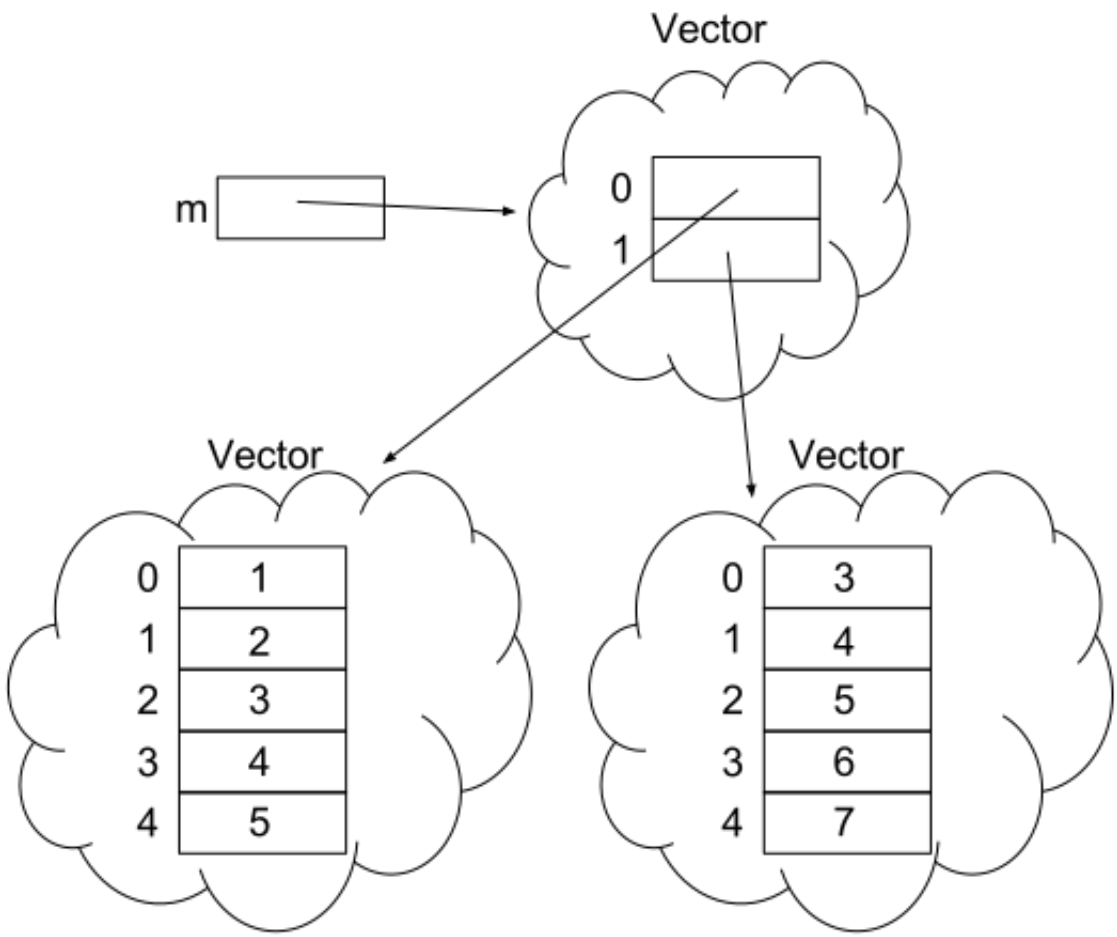
8. Solution `Excercise08`

8.1 Basic Tasks; preparation for the lab

Solution Task 1. *Match concepts with descriptions.*

matrix	1	$\rightsquigarrow$	A	indexable 2D data structure
row vector	2	$\rightsquigarrow$	F	matrix of dimension $1 \times m$ with m horizontal values
column vector	3	$\rightsquigarrow$	G	matrix of dimension $m \times 1$ with m vertical values
column	4	$\rightsquigarrow$	C	another word for column
generic	5	$\rightsquigarrow$	B	has abstract type parameter, type is general
type argument	6	$\rightsquigarrow$	D	concrete type, binds to type parameter at compile time
type inference	7	$\rightsquigarrow$	E	compiler deduces type from context

Solution Task 2. *Create matrices using nested collections.*



a)

Type: `Vector[Vector[Int]]`

Value: `Vector(Vector(1, 2, 3, 4, 5), Vector(3, 4, 5, 6, 7))`

Dimensions: 2×5

In mathematics, indexing is conventionally with 1 as the lowest index. In Scala, the lowest index is 0, called 0-based indexing. This is not the case in all programming languages; you can read more at https://en.wikipedia.org/wiki/Array_data_type#Index_origin.

b)

```
scala> val m = Vector((1 to 5).toVector, (3 to 7).toVector)
m: Vector[Vector[Int]] = Vector(Vector(1, 2, 3, 4, 5), Vector(3, 4, 5, 6, 7))

scala> m.apply(0).apply(1)
res4: Int = 2

scala> m(1)
res5: Vector[Int] = Vector(3, 4, 5, 6, 7)

scala> m(1)(4)
res6: Int = 7
```

c)
m2: `Vector[Vector[Int]]`
m3: `Vector[Vector[Int | Double]]`
m4: `Vector[Vector[Int | Double | String]]`
m5: `Vector[Vector[Int]]`

d) m5, 42×2

Solution Task 3. *Create and iterate over matrices.*

a)

```
def throwDie: Int = (math.random() * 6).toInt + 1
```

Or:

```
def throwDie: Int = scala.util.Random.nextInt(6) + 1
```

b) Matrix dimension in mathematical notation: 1000×5, which corresponds to a matrix with 1000 rows and 5 columns.

c)

```
ds1: IndexedSeq[IndexedSeq[Int]]
ds2: IndexedSeq[IndexedSeq[Int]]
ds3: IndexedSeq[Vector[Int]]
ds4: IndexedSeq[Vector[Int]]
ds5: Vector[Vector[Int]]
ds6: Vector[Vector[Int]]
```

Both `IndexedSeq` and `Vector` above are in the package `scala.collection.immutable`.

d)

```
def roll(n: Int) = Vector.fill(n)(throwDie).sorted
```

e)

```
def isYatzy(xs: Vector[Int]): Boolean = xs.forall(_ == xs(0))
```

f)

```
def diceMatrix(m: Int, n: Int): Vector[Vector[Int]] =
  Vector.fill(m)(roll(n))
```

g)

```
def diceMatrixToString(xss: Vector[Vector[Int]]): String =
  xss.map(_.mkString(" ")).mkString("\n")
```

h)

```
def filterYatzy(xss: Vector[Vector[Int]]): Vector[Vector[Int]] =
  xss.filter(isYatzy)
```

i)

```
def yatzyPips(xss: Vector[Vector[Int]]): Vector[Int] =
  filterYatzy(xss).map(_.head)
```

Solution Task 4. *An immutable, generic matrix class for this week's lab* `life`.

a) The type of `m` becomes `Matrix`.

b) The type of `e` becomes `String`.

c) You need to change 3 places from `String` to `Int`.

d) Generic matrix `Matrix[T]` for elements of any type `T`:

```
case class Matrix[T](data: Vector[Vector[T]]):
  def apply(row: Int, col: Int): T = data(row)(col)

object Matrix:
  def fill[T](dim: (Int, Int))(value: T): Matrix[T] =
    Matrix[T](Vector.fill(dim._1, dim._2)(value))
```

e) Thanks to compiler type inference, `bm` has type `Matrix[Boolean]`.

f) The type of `be` becomes `Boolean`.

g) h) i) j) k) are all implemented in the code below:

```
case class Matrix[T](data: Vector[Vector[T]]):
  require(data.forall(row => row.length == data(0).length))

  val dim: (Int, Int) = (data.length, data(0).length)

  def apply(row: Int, col: Int): T = data(row)(col)

  def updated(row: Int, col: Int)(value: T): Matrix[T] =
    Matrix(data.updated(row, data(row).updated(col, value)))

  def foreachIndex(f: (Int, Int) => Unit): Unit =
    for r <- data.indices; c <- data(r).indices do f(r, c)
```

```
  override def toString =
    s"""Matrix of dim $dim:\n${ data.map(_.mkString(" ")).mkString("\n") }"""

object Matrix:
  def fill[T](dim: (Int, Int))(value: T): Matrix[T] =
    Matrix[T](Vector.fill(dim._1, dim._2)(value))
```

8.2 Extra Tasks; additional practice

Solution Task 5. *Imperative matrix algorithms.*

a)

```
def isYatzy(xs: Vector[Int]): Boolean =
  var foundDiff = false
  var i = 0
  while (i < xs.size && !foundDiff) do
    foundDiff = xs(i) != xs(0)
    i += 1
  end while
  !foundDiff
```

b) The function goes through each matrix row, and for each it iterates over each element in the row and appends to the `StringBuilder` object. If the element is not the last in its row, a space is appended; otherwise, a newline is appended. Exception is the last row, where no newline is appended. Finally, the `StringBuilder` is converted to a `String` and returned.
If `xss` is empty, `xss.indices` is an empty `Range` and the outer for-loop is skipped, returning an empty string. If all rows are empty, the inner for-loops are skipped, with the same result.
Advantage: `StringBuilder` is faster for appending to large strings (though not noticeable here because the string is small).
Disadvantage: The `StringBuilder` code is often perceived as harder to read.

c)

```
def filterYatzy(xss: Vector[Vector[Int]]): Vector[Vector[Int]] =
  var result: Vector[Vector[Int]] = Vector()
  for i <- xss.indices if isYatzy(xss(i)) do result = result :+ xss(i)
  result
```

d) Each loop iteration yields a vector `xss(i)` if the filter condition is satisfied, resulting in a vector of vectors that are yatzy throws.

Solution Task 6. *String table with column headers.*

a)

```
case class Table(
  data: Vector[Vector[String]],
  headings: Vector[String],
  sep: Char
):

  val dim: (Int, Int) = (data.size, headings.size)

  def apply(r: Int, c: Int): String = data(r)(c)

  def row(r: Int): Vector[String]= data(r)

  def col(c: Int): Vector[String] = data.map(r => r(c))
```

```
  lazy val indexOfHeading: Map[String, Int] = headings.zipWithIndex.toMap

  def col(h: String): Vector[String] = col(indexOfHeading(h))

  def values(h: String): Vector[String] = col(h).distinct.sorted

  override def toString: String =
    val s = sep.toString
    headings.mkString(s) + "\n" +data.map(_.mkString(s)).mkString("\n")

object Table:
  def fromFile(fileName: String, sep: Char = ';'): Table =
    val lines = scala.io.Source.fromFile(fileName).getLines.toVector
    val matrix= lines.map(_.split(sep).toVector)
    new Table(matrix.tail, matrix.head, sep)
```

b)

```
@main
def run(fileName: String, separator: String): Unit =
  require(separator.length == 1, "separator ska vara exakt ett tecken")
  val t = Table.fromFile(fileName, separator.head)
  val counts: Vector[Vector[String]] =
    (0 until t.dim._2)
      .map(i => t.values(t.headings(i))
      .map(x => s"$x: ${t.col(i).count(_ == x)}"))
      .toVector
  for (i <- 0 until t.dim._2) do
    println(s"\nColumn: ${i + 1}, ${t.headings(i)}:")
    for (j <- 0 until counts(i).length) do
      println(counts(i)(j))
```

Solution Task 7. *Create a yatzy game for terminal use.*

–

8.3 Advanced Tasks; challenges

Solution Task 8. *Generic functions.*

a)

1. –
2. The string representation of `42` is reversed
3. `"hej"` is reversed — `toString` of a string returns the string itself
4. –
5. The string representation of the `Cucumber` object is reversed
6. The function's type parameter does not match the parameter's type: `42` is not a string
7. Implicit conversion to `Double` happens to match the type parameter, giving a string representation with decimal

b)

1. A function is defined that takes two other functions as arguments, composes them, and applies the composed function to a third argument.
2. A function that increments an integer by 1 is defined.
3. A function that halves a floating point number is defined.
4. `42` is passed to `inc()`, and the result (`43`) is passed to `half()`. Inside `half()`, implicit conversion to `Double` happens since the number is divided by a floating point number (2.0), resulting in `43.0 / 2.0`, that is, `21.5`.
5. The result from `half()` is of type `Double`, but `inc()` expects an `Int`. Since floating point numbers cannot be implicitly converted to integers without loss of information, no implicit conversion occurs, and a compilation error results.

c)

```
def inc(x: Double): Double = x + 1.0
```

Now the compilation error moves to line 4 instead, which can be fixed with the following change:

```
def half(x: Double): Double = x / 2.0
```

Solution Task 9. *Generic classes.*

a) –

b)

```
class Cell[T](var value: T):
  override def toString = "Cell(" + value + ")"
  def concat[U](that: Cell[U]): Cell[String] =
    Cell(s"$value${that.value}")
```

c) Only cells with the same type parameter can now be concatenated. Because `concat()` returns an object of type `Cell[String]`, an odd number of cells of any other type than `String` cannot be concatenated. If the number is even, they can be concatenated pairwise and then concatenate the resulting `Cell[String]` objects, but that is somewhat cumbersome.

Solution Task 10. *Implement more generic methods in `Matrix[T]`.*

```
case class Matrix[T](data: Vector[Vector[T]]):
  require(data.forall(row => row.size == data(0).size))

  val dim: (Int, Int) = (data.length, data(0).length)

  def apply(row: Int, col: Int): T = data(row)(col)

  def updated(row: Int, col: Int)(value: T): Matrix[T] =
    Matrix(data.updated(row, data(row).updated(col, value)))

  def foreach(f: T => Unit): Unit = data.foreach(_.foreach(f))

  def foreachIndex(f: (Int, Int) => Unit): Unit =
    for r <- data.indices; c <- data(r).indices do f(r, c)

  def map[U](f: T => U): Matrix[U] = Matrix(data.map(_.map(f)))

  def mapIndex[U](f: (Int, Int) => U): Matrix[U] =
    var result = Matrix.fill(dim)(f(0,0))
    for
      r <- data.indices
      c <- data(r).indices
    do
      result = result.updated(r, c)(f(r, c))
    end for
    result

  override def toString =
    s"""Matrix of dim $dim:\n${ data.map(_.mkString(" ")).mkString("\n") }"""

object Matrix:
  def fill[T](dim: (Int, Int))(value: T): Matrix[T] =
    Matrix[T](Vector.fill(dim._1, dim._2)(value))
```

9. Solution `Excercise09`

9.1 Basic Tasks; preparation for the lab

Solution Task 1. *Match concepts with description.*

set	1	⇝	H	unordered collection with unique elements
key-value table	2	⇝	F	unordered collection of mappings with unique keys
mapping	3	⇝	G	`key -> value`
key	4	⇝	C	a unique identifier
persistence	5	⇝	D	property of remaining after program ends
serialize	6	⇝	E	encode object into decodable sequence of symbols
deserialize	7	⇝	B	decode symbol sequence and recreate object in memory
linear search	8	⇝	A	search sequence until criterion met

Solution Task 2. *What is a set?* A set is a collection that can quickly answer whether a certain element belongs to the collection or not. The elements in a set are unique. Adding an already existing element is ignored. A set is not a sequence, since traversal with e.g. `map` or `foreach` does not (necessarily) happen in the order in which the elements were given when the set was constructed or updated.

Solution Task 3. *Using sets.*

`Set(1, 2) ++ Set(1, 2)`	1	⇝	I	`Set(1, 2)`
`(1 to 3).toSet`	2	⇝	G	`Set(1) + 2 + 3`
`Vector.fill(3)(1).toSet`	3	⇝	F	`Set(1, 2) - 2`
`Set(1, 2, 3) diff Set(1, 2)`	4	⇝	B	`Set(3)`
`(1 to 7).toSet.apply(8)`	5	⇝	H	`false`
`Set(1, 2, 3).sorted`	6	⇝	D	`error: ...`
`Set(2,4) subsetOf (1 to 7).toSet`	7	⇝	E	`true`
`Set(1, -1, 2, -2).map(_.abs).sum`	8	⇝	A	`3`
`Set(1, 1, 1, 1, 1, 5).sum`	9	⇝	C	`6`

Solution Task 4. *Count unique words using a set.*

a)

```
scala> val hej = "hej hej hemskt mycket hej"
scala> val n = hej.split(' ').toSet.size
n: Int = 3
```

b) The method `distinct` returns a sequence with unique elements and preserved original order.

Solution Task 5. *Creating 2-tuples with the method `->` which can be read as "maps to".*

a) Yes, the factory method returns a completely normal pair:

```
scala> val hereILive = "Skåne" -> "Lund"
val hereILive: (String, String) = (Skåne,Lund)

scala> hereILive._1
val res0: String = Skåne

scala> hereILive._2
val res1: String = Lund
```

b)

```
val capital = Vector(
  "Sverige"  -> "Stockholm",
  "Danmark"  -> "Köpenhamn",
  "Grönland" -> "Nuuk",
  "Skåne"    -> "Lund"
)
```

c)

```
scala> capital(3)._2
val res2: String = Lund
```

Solution Task 6. *Linear search for key in sequence of mappings.*

a)

```
def lookupIndex(xs: Vector[(String, String)])(key: String): Int =
  xs.indexWhere(_._1 == key)
```

b)

```
scala> val i = lookupIndex(huvudstad)("Skåne")
val i: Int = 3

scala> capital(i)._2
val res2: String = Lund
```

Or with functions as reusable partial solutions:

```
scala> val indexOf = lookupIndex(huvudstad) _

scala> def capital(key: String) = huvudstad(indexOf(key))._2

scala> capital("Skåne")
val res3: String = Lund

scala> capital("Sverige")
val res4: String = Stockholm
```

Solution Task 7. *Key-value table.*

```
scala> telnr("Fröken Ur")
```

```
val res0: Long = 464690510

scala> :type telnr
Map[String,Long]

scala> :type telnr.toVector
Vector[(String, Long)]
```

Solution Task 8. *Using key-value table.*

a) No; the key-value pairs are stored in a special order determined by an internal, clever storage principle based on a so-called hash function[3], to achieve fast lookups of values from keys, which normally does not coincide with the order of the sequence from which they were created.

b)

`xs(2) + xs(4)`	1	⇝ A	`8`
`ys(0)`	2	⇝ C	`(10, 11)`
`xs(0)`	3	⇝ I	`NoSuchElementException`
`(xs + (0 -> 1)).apply(0)`	4	⇝ D	`1`
`xs.keySet.apply(2)`	5	⇝ G	`true`
`xs isDefinedAt 0`	6	⇝ H	`false`
`xs.getOrElse(0, 7)`	7	⇝ B	`7`
`xs.maxBy(_._2)`	8	⇝ E	`(16, 17)`
`xs.map(p => p._1 -> -p._2)(8)`	9	⇝ F	`-9`

Solution Task 9. *Registration in mutable key-value table.*

```
class FreqMapBuilder:
  private val register = scala.collection.mutable.Map.empty[String,Int]
  def toMap: Map[String, Int] = register.toMap
  def add(s: String): Unit =
    register.addOne(s -> (register.getOrElse(s, 0) + 1))

object FreqMapBuilder:
  def apply(xs: String*): FreqMapBuilder =
    val result = new FreqMapBuilder
    xs.foreach(result.add)
    result
```

Solution Task 10. *The method `sliding`.*

a)

```
scala> val xs = Vector("five", "cucumbers", "is", "more", "than", "four", "toma
val xs: Vector[String] =
  Vector(five, cucumbers, is, more, thane, four, tomatoes)

```

[3] https://sv.wikipedia.org/wiki/Hashfunktion

```
scala> xs.sliding(2).toVector
val res9: Vector[Vector[String]] =
  Vector(Vector(five, cucumbers), Vector(cucumbers, is), Vector(is, more), Vect

scala> xs.sliding(3).toVector
val res10: Vector[Vector[String]] =
  Vector(Vector(five, cucumbers, is), Vector(cucumbers, is, more), Vector(is, m

scala> xs.sliding(10).toVector
val res11: Vector[Vector[String]] =
  Vector(Vector(five, cucumbers, is, more, than, four, tomatoes))
```

`xs.sliding(n).toVector` creates a sequence that contains sequences of length `n` created by taking each element and its `n - 1` following elements.

b)

```
scala> xs.sliding(2).map(ys => ys(0) -> ys(1)).toMap
val res0: Map[String,String] =
  Map(is -> more,
      than -> four,
      four -> tomatoes,
      cucumbers -> is,
      five -> cucumbers,
      more -> than
  )
```

The table can be used to look up the following word. It works because all words are unique. If there had been multiple identical words with different following words, we would need to create a table with keys that map to a collection registering the following words. This we will do in this week's lab.

Solution Task 11. *Reading text from file and web servers.*

a)

```
val populationOf = data.tail.map(v => v(0) -> v(1).toInt).toMap
val sizeOf       = data.tail.map(v => v(0) -> v(2).toInt).toMap
val capitalOf    = data.tail.map(v => v(0) -> v(3)).toMap
```

```
scala> capitalOf("Sverige")
res2: String = Stockholm

scala> populationOf("Sverige")
res3: Int = 9223766

scala> sizeOf("Sverige")
res4: Int = 449964
```

```
scala> val filename = "europa.txt"
scala> val xs = io.Source.fromFile(filename, "UTF-8").getLines.toVector
scala> val data = xs.map(_.split(';').toVector)
scala> data.map(_.map(_.take(15).padTo(15,' ')).mkString(" ")).foreach(println)
```

9.2 Extra Tasks; additional practice

Solution Task 12. –

9.3 Advanced Tasks; challenges

Solution Task 13. *Registration with `groupBy`.*

a) The method `groupBy` creates a key-value table where the values in the table are a sequence with the elements grouped in a special way. More precisely:
The result of `xs.groupBy(f: K => V)` for a sequence `xs` of type `Vector[K]` is a table of type `Map[V,Vector[K]]` where each element `e` in `xs` is grouped in the same table value if they are equal according to `f(e)`. Each group gets the table key `f(e)`.
Clever trick: If you let the function `f` be the identity function that maps each element onto itself, i.e. `x => x`, then the values are grouped in the same sequence if they are equal.

```
scala> val xs = Vector(1, 1, 2, 2, 4, 4, 4).groupBy(x => x > 2)
val xs: Map[Boolean,Vector[Int]] =
  Map(false -> Vector(1, 1, 2, 2), true -> Vector(4, 4, 4))

scala> val ys = Vector(1, 1, 2, 2, 4, 4, 4).groupBy(x => x)
val ys: Map[Int,Vector[Int]] =
  Map(2 -> Vector(2, 2), 4 -> Vector(4, 4, 4), 1 -> Vector(1, 1))
```

b)

```
def freq(xs: Vector[Int]): Map[Int, Int] =
  xs.groupBy(x => x).map(p => p._1 -> p._2.size)
```

Explanation: the method `groupBy` creates a table with pairs `k, v` where `v` is a sequence with as many `k` as the number of times `k` occurs in `xs`. By converting all values `p._2` to the size `p._2.size` we get a frequency table.

```
scala> freq(toss(1000))
val res0: Map[Int,Int] =
  Map(5 -> 163, 1 -> 174, 6 -> 161, 2 -> 169, 3 -> 167, 4 -> 166)

scala> freq(toss(1000)).toVector.sortBy(_._1).foreach(println)
(1,183)
(2,167)
(3,169)
(4,179)
(5,154)
(6,148)
```

c)

```
shoppinglist.groupMap(_.department)(_.item)
```

d)

```
shoppinglist.groupMapReduce(_.department)(_.price)(_ + _)
```

Solution Task 14. –

10. Solution Excercise10

10.1 Basic Tasks; preparation for the lab

Solution Task 1. *Match concepts with descriptions.*

base type	1	⇝	N	most general type in an inheritance hierarchy
super type	2	⇝	O	a type that is more general
subtype	3	⇝	D	a type that is more specific
runtime type	4	⇝	J	may be more specific than static type
dynamic binding	5	⇝	L	runtime type determines which method executes
polymorphism	6	⇝	B	can have many forms, e.g., one of several subtypes
trait	7	⇝	I	abstract, can be mixed in, can have parameters
mix-in	8	⇝	H	add properties with `with` and a trait
overridden member	9	⇝	M	subtype member replaces supertype member
anonymous class	10	⇝	C	class without name, extended with extra implementation
protected member	11	⇝	K	visible only in subtypes
abstract member	12	⇝	F	has no implementation
abstract class	13	⇝	E	can have parameters, cannot be instantiated, cannot be mixed i
sealed type	14	⇝	P	subtyping outside this file is prevented
reference type	15	⇝	A	has supertype `AnyRef`, allocated on heap via reference
value type	16	⇝	G	has supertype `AnyVal`, stored directly on stack

Solution Task 2. *Common base type.*

a) `Vector[Object]`. The type `Object` in the JVM corresponds to the type `AnyRef` which is the base type for all reference types.

b) Error message:

```
scala> vegetables.map(_.weight).sum
-- Error:
1 |vegetables.map(_.weight).sum
  |               ^^^^^^
  |              value weight is not a member of Object - did you mean wait?
-- Error:
1 |vegetables.map(_.weight).sum
  |                          ^
  |ambiguous implicit arguments: both object DoubleIsFractional in object Nume
```

The first error is because the elements of the vector are of type `Object` and the member `weight` is not defined for this type. The second error is a follow-up error caused by the fact that a sequence with elements of type `Object` cannot be summed because the compiler cannot infer that the element type is numeric.

c) The attribute `weight` is initialized during the construction of `Cucumber` respectively `Tomat`. The value is given by the respective class parameter.

d) `Vector[Vegetable]`.

e) Yes. Since the static type of the elements in the sequence is `Vegetable` (the dynamic type can be any subtype of `Vegetable`) and all instances of this type are

guaranteed to have the attribute `weight` of type `Int`, the compiler can at *compile time* conclude that summing is valid and thus can compile the code to executable machine code.

f)

```
scala> new Vegetable
-- Error:
1 |new Vegetable
  |    ^^^^^^^
  |    Vegetable is a trait; it cannot be instantiated
```

g)

```
scala> val anonymousVegetable = new Vegetable { val weight = 42 }
val anonymousVegetable: Vegetable = anon$1@1edde8b6
scala> anonymousVegetable.toString
val res0: String = anon$1@1edde8b6
```

The type is `Vegetable` and here becomes a so-called *anonymous class*, since we did not use a named class with **`extends`**, but only "attached" a class body within braces directly during construction. When creating anonymous classes you must use the keyword **`new`**.
The compiler invents a unique class name, here anon$1, to keep track of the anonymous class during compilation to machine code. The string representation contains a hexadecimal number that is unique for the instance, here `1edde8b6`.

h)

```
scala> new Vegetable { }
-- Error:
1 |new Vegetable { }
  |^
  |object creation impossible, since val weight: Int in trait Vegetable is not defined
```

Solution Task 3. *Polymorphism in inheritance, i.e. subtype polymorphism.*

a)

```
def createAnimal(): Animal =
  if math.random() > 0.5 then Cow() else Pig()
```

b)

```
class Horse extends Animal:
  def makeNoise = println("Neigh")

def createAnimal(): Animal =
   math.random() match
    case r if r < 0.33 => Cow()
    case r if r < 0.67 => Pig()
    case _             => Horse()
```

Solution Task 4. *Various types of integer pairs for the lab `snake0`.*

a)

```
trait Pair[T]:
  def x: T
  def y: T
  def tuple: (T, T) = (x, y)
```

b)

- Does the above code also work without keyword **override**? Why?

  Yes, it works because **override** is not required when implementing inherited *abstract* members in a subtype. Abstract members lack implementation and there is nothing to override.

- When must **override** be used?

  It is required if you want to give an inherited member a *different* implementation in the subtype, if this member already has an implementation in the supertype. Your new implementation overrides (replaces) the inherited member's implementation.

- What are the advantages and disadvantages of using **override** even when not needed?

  Advantage: the compiler helps check that you are actually implementing an inherited member and not e.g. misspelled the member's name.

  Disadvantage: more to type and thus longer to read.

c)

```
case class Dim(x: Int, y: Int) extends Pair[Int]
object Dim:
  def apply(dim: (Int, Int)): Dim = Dim(dim._1, dim._2)
```

d)

```
case class Pos private (x: Int, y: Int, dim: Dim) extends Pair[Int]:
  def +(p: Pair[Int]): Pos = Pos(x + p.x, y + p.y, dim)
  def -(p: Pair[Int]): Pos = Pos(x - p.x, y - p.y, dim)

object Pos:
  def apply(x: Int, y: Int, dim: Dim): Pos =
    import java.lang.Math.floorMod as mod
    new Pos(mod(x, dim.x), mod(y, dim.y), dim) //NOTE: new is necessary here!

  def random(dim: Dim): Pos =
    import scala.util.Random.nextInt as rni
    Pos(rni(dim.x), rni(dim.y), dim)
```

e) If you forget to write **new** explicitly in the companion object's `apply` method, it becomes a recursive call resulting in an infinite loop at runtime. With **new**, it is guaranteed that the private primary constructor for `Pos` is called.
In `Dim.apply` the parameter types differ between the factory method and the primary constructor, and the compiler chooses the primary constructor since it matches the given two separate integers and not a 2-tuple.

f)

```
enum Dir(val x: Int, val y: Int) extends Pair[Int]:
  case North extends Dir( 0, -1)
  case South extends Dir( 0,  1)
  case East  extends Dir( 1,  0)
  case West  extends Dir(-1,  0)
export Dir.*  // makes North etc visible within package snake
```

Solution Task 5. *Supertype with parameter.*

a)

```
val person = new Person("Person1")
val academic = new Academic("Person2", "LTH")
val student = new Student("Person3", "LTH", "D")
val researcher = new Researcher("Person4", "LTH", "PhD Student")
```

b)

```
val vec = Vector(person, academic, student, researcher)
for(i <- vec){ print(i.toString + i.name) }
```

c) Error message when instantiating `abstract class Academic`: `Academic is abstract; it cannot be instantiated` It does *not* work equally well with a `trait` in the special case of `Academic`, since a trait may not forward parameters to a supertype. Error message: `trait Academic may not call constructor of trait Person`

```
trait Person(val name: String)
abstract class Academic(
name: String,
val university: String) extends Person(name)
class Student(
name: String,
university: String,
program: String) extends Academic(name, university)
class Researcher(
name: String,
university: String,
title: String) extends Academic(name, university)
```

d)

```
scala>
|trait Person(val name: String)
|
| abstract class Academic(
| name: String,
| val university: String) extends Person(name)
|
| case class Student(
| name: String,
| university: String,
| program: String) extends Academic(name, university)
```

```
|
| case class Researcher(
| name: String,
| university: String,
| title: String) extends Academic(name, university)
-- Error:
8 | name: String,
| ^
| error overriding value name in trait Person of type String;
| value name of type String needs override modifier
-- Error:
9 | university: String,
| ^
| error overriding value university in class Academic of type String;
| value university of type String needs override modifier
-- Error:
13 | name: String,
| ^
| error overriding value name in trait Person of type String;
| value name of type String needs override modifier
-- Error:
14 | university: String,
| ^
| error overriding value university in class Academic of type String;
| value university of type String needs override modifier
```

```scala
trait Person(val name: String)
abstract class Academic(
  name: String,
  val university: String) extends Person(name)
case class Student(
  override val name: String,
  override val university: String,
  program: String) extends Academic(name, university)
case class Researcher(
  override val name: String,
  override val university: String,
  title: String) extends Academic(name, university)
```

```
scala> val ps = Vector(Student("Kim", "Lund", "CS"), Researcher("Herz", "Lund", "Dr"))
val ps: Vector[Academic] = Vector(Student(Kim,Lund,CS), Researcher(Herz,Lund,Dr))
scala> ps :+ new Person("Abstract") {}
val res0: Vector[Person] =
  Vector(Student(Kim,Lund,CS), Researcher(Herz,Lund,Dr), anon1@1941bbf3)
```

e)

```scala
trait Person:
  val name: String
  val nbr: Long
trait Academic extends Person:
  val university: String
case class Student(
  name: String,
```

```
  nbr: Long,
  university: String,
  program: String) extends Academic
case class Researcher(
  name: String,
  nbr: Long,
  university: String,
  title: String) extends Academic
case class NonAcademic(
  name: String,
  nbr: Long) extends Person
```

10.2 Extra Tasks; additional practice

Solution Task 6. *Base type* `Shape` *and subtypes* `Rectangle` *and* `Circle`.

a)

```
val c1 = Circle(Point(1, 1), 42)
val r1 = Rectangle(Point(3, 3), 20, 30)
c1.move(2, 3)
r1.move(3, 2)
```

b)

```
case class Point(x: Double, y: Double):
  def move(dx: Double, dy: Double): Point = Point(x + dx, y + dy)
  def moveTo(x: Double, y: Double): Point = Point(x, y)

trait Shape:
  def pos: Point
  def move(dx: Double, dy: Double): Shape
  def moveTo(x: Double, y: Double): Shape

case class Rectangle(pos: Point, width: Double, height: Double) extends Shape:
  def move(dx: Double, dy: Double): Shape = copy(pos = pos.move(dx, dy))
  def moveTo(x: Double, y: Double): Shape = copy(pos.moveTo(x, y))

case class Circle(pos: Point, radius: Double) extends Shape:
  def move(dx: Double, dy: Double): Shape = copy(pos = pos.move(dx, dy))
  def moveTo(x: Double, y: Double): Shape = copy(pos.moveTo(x, y))
```

c) `def distanceTo(that: Point): Double = math.hypot(that.x - x, that.y - y)`

d) `def distanceTo(that: Shape): Double = pos.distanceTo(that.pos)`.

e)

```
case class Point(x: Double, y: Double):
  def move(dx: Double, dy: Double): Point = Point(x + dx, y + dy)
  def moveTo(x: Double, y: Double): Point = Point(x, y)
  infix def distanceTo(that: Point): Double = math.hypot(that.x - x, that.y - y)

trait Shape:
  def pos: Point
  def move(dx: Double, dy: Double): Shape
```

```
  def moveTo(x: Double, y: Double): Shape
  infix def distanceTo(that: Shape): Double = pos.distanceTo(that.pos)

case class Rectangle(pos: Point, width: Double, height: Double) extends Shape:
  def move(dx: Double, dy: Double): Shape = copy(pos = pos.move(dx, dy))
  def moveTo(x: Double, y: Double): Shape = copy(pos.moveTo(x, y))

case class Circle(pos: Point, radius: Double) extends Shape:
  def move(dx: Double, dy: Double): Shape = copy(pos = pos.move(dx, dy))
  def moveTo(x: Double, y: Double): Shape = copy(pos.moveTo(x, y))
```

10.3 Advanced Tasks; challenges

Solution Task 7. *Mixins.*

a) There are many ways, here are some example solutions:

```
val numberOfFlyingDucks: Int =
  flock.count(f => f.isInstanceOf[Duck] && f.isAbleToFly)
```

```
val numberOfFlyingDucks: Int =
  flock.filter(f => f.isInstanceOf[Duck] && f.isAbleToFly).size
```

```
val numberOfFlyingDucks: Int =
  flock.collect{case f: Duck if f.isAbleToFly}.size
```

```
val numberOfFlyingDucks: Int = flock.map(_ match
  case f: Duck if f.isAbleToFly => 1
  case _ => 0
).sum
```

b)

```
val numberOfCrows: Int = flock.filter(f => !f.isAbleToSwim).size * 2
val numberOfDucks: Int = flock.filter(f => f.isAbleToSwim).size * 4
```

Solution Task 8. *Final classes.*

a) Put `final` before `class` in the classes.

b) error: illegal inheritance from final class Crow.

Solution Task 9. *Access rules in inheritance and the keyword* `protected`.

a)

```
2 |  def reveal = myHome
  |                ^^^^^^^^^
  |                Not found: myHome
```

b)

```
scala> class Sub extends Super:
         def cryptic = ourHome * math.Pi
scala> (new Sub).ourHome
-- Error:
1 |(new Sub).ourHome
  |^^^^^^^^^^^^^^^^^^
  |value ourHome in trait Super cannot be accessed as a member of Sub.
  | Access to protected value ourHome not permitted because enclosing object
  | is not a subclass of trait Super where target is defined
```

c) Yes.

Solution Task 10. *Using* ***protected****.*

a) In Bird:

```
protected var soundCount: Int = 0
def makeNoise: Unit = { print(sound * 2); soundCount += 2 }
```

In Duck: `override def makeNoise = { print(sound * 4); soundCount += 4 }`

b) `def numberOfSounds: Int = soundCount`

c) If a class representing a bird that would produce more/fewer sounds than a typical `Bird` needs to change `makeNoise`, that method needs access to `soundCount`, which cannot be **`private`**.

d) The counter variable should not be modifiable from any other part of the program.

Solution Task 11. *Mixin of properties.*

```
trait Bird:
  val sound: String
  def makeNoise: Unit = { print(sound * 2); soundCount += 2 }
  protected var soundCount: Int = 0
  val canSwim: Boolean
  val canFly: Boolean
  val isBig: Boolean
  def numberOfSounds: Int = soundCount

trait CanSwim { val canSwim = true }
trait CannotSwim { val canSwim = false }
trait CanFly { val canFly = true }
trait MaybeCanFly { val canFly = math.random() < 0.8 }
trait MaybeCanSwim { val canSwim = math.random() < 0.2 }
trait IsBig { val isBig = true }
trait IsSmall { val isBig = false }

final class Crow extends Bird, CanFly, CannotSwim, IsBig:
  val sound = "caw"

final class Duck extends Bird, CanSwim, MaybeCanFly, IsBig:
  val sound = "quack"
  override def makeNoise = { print(sound * 4); soundCount += 4 }
```

```
final class Chick extends Bird, CanFly, MaybeCanSwim, IsSmall:
  val sound = "chirp"
  override def makeNoise = { print(sound * 8); soundCount += 8 }
```

In REPL:

```
val birds = Vector.fill(42)(
  if math.random() < 0.33 then Crow()
  else if math.random() < 0.5 then Duck()
  else Chick()
)
fyle.filter(f => f.isInstanceOf[Crow]).size * 2
fyle.filter(f => f.isInstanceOf[Duck]).size * 4
fyle.filter(f => f.isInstanceOf[Chick]).size * 8
```

11. Solution `Excercise11`

11.1 Basic Tasks; preparation for the lab

Solution Task 1. *Context parameter.*

a)

```
scala> given Int = 0
val res0: Int = 83

scala> f(using 41)
val res1: Int = 42

scala> g(41)(using 42)
val res2: Int = 83
```

If you forget **`using`** when giving an explicit context argument, it results in a compilation error. The compiler becomes "confused" and thinks you are trying to provide a "regular" argument to a (in this case) nonexistent regular parameter list.

```
scala> f(41)
-- [E050] Type Error: ----------------------------------------------
1 |f(41)
  |^
  |method f does not take more parameters
  |
  | longer explanation available when compiling with `-explain`
1 error found

scala> :setting -explain

scala> f(41)
-- [E050] Type Error: ----------------------------------------------
1 |f(41)
  |^
  |method f does not take more parameters
  |----------------------------------------------------------------
    | Explanation (enabled by `-explain`)
  |- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
  | You have specified more parameter lists than defined in the
    method definition(s).
   ----------------------------------------------------------------

scala> g(41)(42)
-- [E050] Type Error: ----------------------------------------------
1 |g(41)(42)
  |^^^^^
  |method g does not take more parameters
  |----------------------------------------------------------------
    | Explanation (enabled by `-explain`)
  |- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
  | You have specified more parameter lists than defined in the
    method definition(s).
   ----------------------------------------------------------------
```

It is uncommon to specify **`using`** parameters explicitly; it is more common to let the compiler summon a given value.

b) The given value `0` is bound to the corresponding context parameter, which should be declared in a separate parameter list starting with **`using`**.

```
scala> f
val res3: Int = 1

scala> g(42)
val res4: Int = 42
```

c) No, it results in a compilation error if you try to mix regular and context parameters in the same parameter list:

```
scala> def h(i: Int, using j: Int) = i + j
-- [E040] Syntax Error: ----------------------------------------------
1 |def h(i: Int, using j: Int) = i + j
  |                    ^
  |                    ':' expected, but identifier found
```

It is a deliberate design choice to require separate parameter lists so that there is no confusion about whether a regular parameter or context parameter is meant.

Solution Task 2. *Multiple given values in local context.*

a) 2

b) 43

c) When the compiler tries to summon a given value to automatically use as argument to the **`using`** parameter `dx`, it searches first in the context nearest to the call. If a given value exists in the companion object for the parameter type, the compiler takes this as a last resort if no other given value is found closer to the call.

Solution Task 3. *Solution to the configuration problem using given values.*

Below is a test of the three different solutions given in the task. a)

After each test we discuss associated pros and cons, as requested in the task. b)

```
def testGlobalVar(useDefault: Boolean = true) =
  import GlobalVar.*
  if useDefault then println(greetMsg) else
    GreetConfig.config = GreetConfig("Good morning", "world")
    println(greetMsg)
```

Since `config` here is a mutable variable, a change in one place can affect completely different parts of the program, which sometimes can be an advantage, but often a disadvantage because it can be hard to understand what is happening by reading only one part of the program – a change of `config` can occur anywhere. It is easy to forget to change back to the default value if that is what is expected.

```
scala> testGlobalVar(); testGlobalVar(false); testGlobalVar()
Hello World!
Good morning world!
Good morning world!
```

```
def testDefaultArgs(useDefault: Boolean = true) =
  import DefaultArgs.*
```

```
3   if useDefault then println(greetMsg()) else
4     println(greetMsg(GreetConfig("Godd morning","world")))
```

Here there is no state change and default usage is simple, but it is not easy to deviate from the default that applies in a local context; at *each* individual call you have to explicitly supply all the arguments that should not be default, as shown on line 4 above. Changing the default has only local impact. If all arguments are to follow the default, then you must not forget to call with empty parentheses: `greetMsg()`. (What happens otherwise?)

```
scala> testDefaultArgs(); testDefaultArgs(false); testDefaultArgs()
Hello World!
Good morning world!
Hello World!
```

With context parameters the flexibility is greater; **using** parameters let the user control what applies in different contexts and the call becomes simple regardless of whether the default value or other, locally given values are desired. Changing default has only local impact, but this affects arbitrarily many calls in the local context – arguments that differ can be given once without needing to be repeated at each call. For calls where you want the compiler to summon given values for the context parameter no parentheses should be used, making the calls short and simple.

```
def testGivenVal(using g: GivenVal.GreetConfig) = println(g.greetMsg)
```

```
scala> testGivenVal
Hello World

scala> def localContext =
         import GivenVal.*
         given GreetConfig = GreetConfig("Good morning","world")
         testGivenVal

scala> localContext
Good morning world

scala> testGivenVal
Hello World
```

c) The compiler summons a given value in the local context:

```
scala> summon[GivenVal.GreetConfig]
val res0: GivenVal.GreetConfig = GreetConfig(Hello,World)
```

The compiler follows this priority order when searching for a unique given value:

1. **Explicit** arguments to context parameters marked with **using**
2. **given** and **import given** ... in the current scope
3. **given** values in the **companion objects** for the used type.

If multiple given values can be summoned for types in a common inheritance hierarchy, the compiler chooses the given value of the *most general* type.

d) It results in a compilation error if the compiler does not find a given value for the type in question.

```
scala> summon[Long]
-- Error: ----------------------------------------------------------------
1 |summon[Long]
  |            ^
  |            no given instance of type Long was found for parameter x of
                 method summon in object Predef
1 error found
```

e) Yes! It must *not* be ambiguous which given value should be summoned:

```
scala> def ambiguous =
     |   given a: Int = 42
     |   given b: Int = 43
     |   summon[Int]
     |
-- Error: ----------------------------------------------------------------
4 |  summon[Int]
  |             ^
  |ambiguous given instances: both given instance b and given instance a
  |match type Int of parameter x of method summon in object Predef
1 error found
```

Read more about contextual abstractions here:
https://docs.scala-lang.org/scala3/reference/contextual/

11.2 Extra Tasks; additional practice

Solution Task 4. *Context parameter and given value.*

a)

```
scala> add(1)
-- Error: ----------------------------------------------------------------
1 |add(1)
  |      ^
  |    no given instance of type Int was found for parameter y of method add
1 error found
```

b) Now there is a given value that the compiler can automatically fill in at the call site.

c) `def sub(x: Int)(using y: Int) = x - y`

11.3 Advanced Tasks; challenges

Solution Task 5. *Variance and type bounds.*

a) Make the box flexible in its type parameter with a + before the type parameter as below.

```
case class Box[+A](x: A)
```

The compiler applies the rules for covariance because the type parameter has a plus sign: `Box[Cat]` is a subtype of `Box[Any]` if `Cat` is a subtype of `Any`, which it is since all types are subtypes of `Any`.

b) The explanation for behavior concerns different variance:

- The collection `Vector` is covariant and thus flexible in its type parameter (as are other immutable sequences in Scala's standard library). The compiler thus considers `Vector[Cat]` as a subtype of `Vector[Pet]` since `Cat` is a subtype of `Pet`. Where a `Vector[Pet]` is expected, `Vector[Cat]` conforms and is accepted.

- A set has an apply method from element type to `Boolean` which checks membership. Hence `Set[T]` inherits `Function1[T, Boolean]` which is declared contravariant in `T`, allowing a set to be used where a function `T => Boolean` is expected. Even though it would be practical if `Set[T]` were covariant like `Vector`, `List`, `Seq` etc., a type parameter cannot be both covariant and contravariant at the same time. Therefore `Set` is invariant and sets are not flexible in their type parameter. Thus `Set[Cat]` is *not* a subtype of `Set[Pet]` even though `Cat` is a subtype of `Pet`, which causes the compilation error in the example. See also https://stackoverflow.com/questions/676615/why-is-scalas-immutable-set-not-covariant-in-its-type

- Using `:settings -explain` the compiler outputs a longer explanation clarifying the proof performed during compilation's type checking.

c) It is a compilation error because the method `isHealthy` does not exist for arbitrary types.

d) Add an upper bound to ensure the method `isHealthy` exists for all types bound to the type parameter `A`:

```
class Vet[-A <: Pet]:
  def treat(x: A): Unit = x.isHealthy = true
```

The compiler guarantees that the type parameter `A` is "less than or equal to" `Pet`.

e) The veterinarian `Vet` is flexible in its type parameter and the minus sign indicates contravariance, meaning `Vet[Pet]` is a subtype of `Vet[Cat]` since `Cat` is a subtype of `Pet`. This can be demonstrated with the example below:

```
scala> val pinkPanther = Cat()
val pinkPanther: Cat = Cat@33e7ece5

scala> val somePet: Pet = Cat()
val somePet: Pet = Cat@57f1cb96

scala> val catVet = Vet[Cat]()
val catVet: Vet[Cat] = Vet@1060e784

scala> pinkPanther.isHealthy = false

scala> catVet.treat(pinkPanther)

scala> pinkPanther.isHealthy
val res2: Boolean = true

scala> somePet.isHealthy = false

scala> catVet.treat(somePet)
-- [E007] Type Mismatch Error: --------------
1 |catVet.treat(somePet)
  |             ^^^^^^^
```

```
  |             Found:    (somePet : Pet)
  |             Required: Cat

scala> val powerVet = Vet[Pet]()
val powerVet: Vet[Pet] = Vet@2eb90ae9

scala> pinkPanther.isHealthy = false

scala> powerVet.treat(pinkPanther)

scala> pinkPanther.isHealthy
val res3: Boolean = true

scala> val pluto = Dog()
val pluto: Dog = Dog@6f27db5d

scala> pluto.isHealthy = false

scala> powerVet.treat(pluto)

scala> pluto.isHealthy
val res4: Boolean = true
```

Solution Task 6. *Type classes and context parameters.*

a)

- First we declare a **`trait`**, `CanCompare`, with a generic type parameter `T`. It contains an abstract method `compare` that takes two parameters of type `T` and returns an `Int`.
- Then a method `sort` is defined, also with a generic type parameter `T`. This method takes two parameters, `a` and `b` of type `T`, and a **`using`** parameter `cc` which must be an instance of `CanCompare[T]`. Inside the method, the `compare` method from `CanCompare` is used to determine if `a` and `b` should be swapped.
- When we try to run `sort(42, 41)` we get a compiler error because there is no given instance of `CanCompare[Int]`.
- We fix this by defining **`given`** `intComparator` of type `CanCompare[Int]` overriding the `compare` method. Now the following call `sort(42, 41)` works and returns `(Int, Int) = (41, 42)`.
- When we try `sort` with `Double` arguments, we get a similar error as before because no `CanCompare` for `Double` exists.

b)

```
scala> given doubleComparator: CanCompare[Double] with
         override def compare(a: Double, b: Double): Int = (a - b).toInt
```

c)

```
scala> given stringComparator: CanCompare[String] with
         override def compare(a: String, b: String): Int = a.compareTo(b)
```

Solution Task 7. *Usage of given ordering.*

a) Example tests:

```
scala> isSorted(Vector(1,2,3))
val res0: Boolean = true

scala> isSorted(Vector(1,2,4,3))
val res1: Boolean = false

scala> isSorted(List(1,2,3))
val res2: Boolean = true

scala> isSorted(List(1,2,4,3))
val res3: Boolean = false
```

b)

```
scala> def isSorted[T](xs: Seq[T]): Boolean = xs == xs.sorted
-- [E172] Type Error: -----------------------------------------------------------
1 |def isSorted[T](xs: Seq[T]): Boolean = xs == xs.sorted
  |                                                     ^
  | No implicit Ordering defined for T..
  | I found:
  |
  |     scala.math.Ordering.ordered[T](/* missing */summon[scala.math.Ordering.
  |
  | But no implicit values were found that match type scala.math.Ordering.AsCon
  |
  | The following import might make progress towards fixing the problem:
  |
  |     import scala.math.Ordered.orderingToOrdered
  |
1 error found
```

The error means the compiler cannot derive an ordering for the generic type `T`.

c)

```
scala> isSorted(persons)
-- [E172] Type Error: -----------------------------------------------------------
1 |isSorted(persons)
  |                 ^
  | No implicit Ordering defined for Person..
  | I found:
  |
  |     scala.math.Ordering.ordered[Person](/* missing */summon[scala.math.Orde
  |
  | But no implicit values were found that match type scala.math.Ordering.AsCon
  |
  | The following import might make progress towards fixing the problem:
  |
  |     import scala.math.Ordered.orderingToOrdered
  |
1 error found
```

The error occurs because the compiler does not know any ordering for the `Person` class.

d) -

e) -

Solution Task 8. *Create your own implicit ordering with* `Ordering`.

a)

```
scala> xs.sorted
-- [E172] Type Error: -----------------------------------------------------
1 |xs.sorted
  |         ^
  | No implicit Ordering defined for Team..
  | I found:
  |
  |     scala.math.Ordering.comparatorToOrdering[Team](/* missing */summon[java
  |
  | But no implicit values were found that match type java.util.Comparator[Tean
  |
  | One of the following imports might make progress towards fixing the problem
  |
  |     import scala.math.Ordering.comparatorToOrdering
  |     import scala.math.Ordering.ordered
  |
1 error found
```

The error appears because the compiler doesn't know any ordering for `Team`.
After assigning `given Ordering[Team] = Ordering.by(t => t.rank)` there exists a given ordering for `Team` in the current context. Therefore the following call `xs.sorted` works as intended.

b) -

c) -

d) -

e)

```
scala> Team("fnatic",1499) < Team("gurka", 2)
-- [E008] Not Found Error: ------------------------------------------------
1 |Team("fnatic",1499) < Team("gurka", 2)
  |^^^^^^^^^^^^^^^^^^^^
  |value < is not a member of Team, but could be made available as an extension
  |
  |One of the following imports might fix the problem:
  |
  |  import scala.math.Ordered.orderingToOrdered
  |  import scala.math.Ordering.Implicits.infixOrderingOps
  |
1 error found
```

As the compiler says, < is not a method defined for `Team`. However, after `import Team.highestRankFir` the implicit conversions defined for `highestRankFirst: Ordering[Team]` (because it is an object of type `Ordering`) are imported. Thus, < works. See task 7e for more background.

Solution Task 9. *Custom ordering by inheriting from* `Ordered`

a)

```
case class Team(name: String, rank: Int) extends Ordered[Team]:
```

```
  override def compare(that: Team): Int = -rank.compare(that.rank)
```

b)

```
scala> Team("fnatic",1499) < Team("gurka", 2)
val res1: Boolean = true
```

c) Subtype polymorphism, i.e. using inheritance and interfaces builds on each subtype implementing certain members. The compiler can then guarantee that *something* will happen at calls with a subtype. However, it cannot guarantee *what* will happen, which is decided at runtime by dynamic binding.
Ad hoc polymorphism on the other hand means that the called function behaves differently for different types. This uses some form of type classes. It is the type given at the call that determines what happens, not which behavior is defined in the subtype the type "happens" to have. By compile time the compiler can thus determine what will happen based on the type, i.e. static binding. This makes it easier to reason about how the program behaves.

12. Solution Excercise12

12.1 Tasks on searching and sorting

Solution Task 1. *Timing measurement.*

a) Execute the code and you should find that it takes longer to find the value 1 in our Set s than in the vector v.

b)
A vector has a sequential order that `find` can use, whereas a `Set` is internally organized differently for very fast membership checks. The reason that it takes time for `find` on `Set` is that an iterator must first be created before the set can be traversed from start to end. The method `contains` on `Set` is lightning fast due to the internal structure of objects of type `Set` (which is smartly designed with so-called hash codes, allowing constant time lookup regardless of element position).

Solution Task 2. *Searching with built-in search methods.*

a) Suggested test of `indexOfSlice`:

```
scala> List(1,2,3,35,1,23).indexOfSlice(List(35,1,23))
res73: Int = 3
scala> List(1,2,3,35,1,23).indexOfSlice(List(35,1,3))
res74: Int = -1
```

b) Suggested test of `lastIndexOfSlice`:

```
Vector(1,2,3,4,1,2).lastIndexOfSlice(Vector(1,2))
res2: Int = 4
Vector("apa", "banan", "majs", "banan").lastIndexOfSlice(Vector("banan"))
res3: Int = 3
Vector("apa", "banan", "majs", "banan").lastIndexOfSlice(Vector("banand"))
res4: Int = -1
```

c) Note that the method `search` assumes the collection is sorted in ascending order. When we reverse the order, `search` mostly cannot find what we look for, since it will search the wrong half of the collection.

```
scala> val odd = (1 to 1000000 by 2).toVector
scala> import scala.collection.Searching._
scala> odd.search(odd.last)
res18: collection.Searching.SearchResult = Found(499999)
//Search finds the last element at position 499999 in the collection.

scala> odd.search(odd.last + 1)
res19: collection.Searching.SearchResult = InsertionPoint(500000)
//Search cannot find odd.last + 1 because it does not exist in the collection
//and thus returns an object of type InsertionPoint with value 500000.
//Our element odd.last + 1 would have been at position 500000 if it existed.

scala> odd.reverse.search(odd(0))
res20: collection.Searching.SearchResult = InsertionPoint(0)
//As explained before, search assumes the list is sorted ascending,
//so it cannot find element odd(0) = 1 when the list is reversed.

scala> def lin(x: Int, xs: Seq[Int]) = xs.indexOf(x)
```

```
scala> def bin(x: Int, xs: Seq[Int]) = xs.search(x) match
  case Found(i) => i
  case InsertionPoint(i) => -i

//Defines a method bin using the search method on a sequence.
//With pattern matching it returns only the position i,
//not an object of Found or InsertionPoint.

scala> timed{ lin(odd.last, odd) }
time: 42.294821 ms
res22: (Int, Long) = (499999,42294821)
//Finding odd.last = 499999 with linear search took ~42ms.

scala> timed{ bin(odd.last, odd) }
time: 0.147314 ms
res23: (Int, Long) = (499999,147314)
//Binary search to find 499999 took much less time.
//Because each step halves the search space.
//This is an extreme case since we search for the last element.
//Linear search for the first element would have been as fast as binary search.
```

d) It requires $log_2(n)$ comparisons. This is because at each step, the number of elements left to check is halved. After the first comparison, we have $\frac{n}{2}$ elements remaining. After the second, $\frac{n}{2^2}$, etc. When only one element remains, we have made b comparisons, where:

$$\frac{n}{2^b} = 1$$

Using logarithm laws:

$$\log_2(n) = b$$

Solution Task 3. *Search among LTH's courses with linear search.*

a) The first line contains the column names `Kurskod KursSve KursEng Hskpoang Niva`. Then each row contains a course with data according to the column names.

b) The code downloads data and creates a vector with instances of case class `Course` using the method `fromLine`. Since variable `lth` is declared **`lazy`**, `download()` is not called until `lth` is first referenced. The number of courses is:

```
scala> val n = courses.lth.length
n: Int = 1104
```

c)

```
scala> def isCS(s: String) = s.startsWith("EDA") || s.startsWith("ETS")
scala> val x = courses.lth.find(c => isCS(c.code) && c.level == "G2")
x: Option[courses.Course] = Some(Course(EDAF05,Algoritmer, datastrukturer och
   komplexitet,Algorithms, Data Structures and Complexity,5.0,G2))
```

d)

```
def linearSearch[T](xs: Seq[T])(p: T => Boolean): Int =
   var i = 0
   while(i < xs.length && !p(xs(i))) i += 1
   if (i < xs.length) i else -1
```

e)

```
def rndCode: String =
   //randomizes from 0 to n (inclusive)
   def rnd(n: Int) = (math.random() * (n + 1)).toInt

   def letter = (rnd('Z' - 'A') + 'A').toChar
   def dig = ('0' + rnd(9)).toChar
   Seq(letter, letter, letter, letter, dig, dig).mkString
```

f)

```
val xs = Vector.fill(500000)(rndCode)
val(ixs, elapsedLin) =
  timed { xs.map(x => linearSearch(courses.lth)(_.code == x)) }
val found = ixs.filterNot(_== -1).size
```

g)

```
def linearSearch[T](xs: Seq[T])(p: T => Boolean): Int = xs.indexWhere(p)
```

Solution Task 4. *Search among LTH's courses with binary search.*

a) —

b)

```
def binarySearch(xs: Seq[String], key: String): Int =
  var (low, high) = (0, xs.size - 1)
  var found = false
  var mid = -1

  while (low <= high && !found) do
    mid = (low + high) / 2
    if xs(mid) == key then found = true
    else if xs(mid) < key then low = mid + 1
    else high = mid - 1
  end while
  if found then  mid else -(low + 1)
```

c) On an i7-3770K @ 3.50GHz the searches took:

- Binary search: `time: 142.6 ms`
- Linear search: `time: 3316.5 ms`

On an i7-8700T @ 2.40GHz they took:

- Binary search: `time: 81.5 ms`
- Linear search: `time: 5138.6 ms`

d) Binary search was about 23 times faster on the i7-3770K @ 3.50GHz and about 63 times faster on the i7-8700T CPU @ 2.40GHz.

Solution Task 5. *Insertion sort.*

a) —

b)

```
def insertionSort(xs: Seq[Int]): Seq[Int] =
  val result = scala.collection.mutable.ArrayBuffer.empty[Int]
  for e <- xs do
    var pos = 0
    while pos < result.size && result(pos) < e do pos += 1
    result.insert(pos,e)
  end for
  result.toVector
```

Solution Task 6. *In-place sorting.*

```
def selectionSortInPlace(xs: Array[String]): Unit =
  def indexOfMin(startFrom: Int): Int =
    var minPos = startFrom
    var i = startFrom + 1
    while (i < xs.size) do
      if (xs(i) < xs(minPos)) minPos = i
      i += 1
    end while
    minPos
  end indexOfMin

  def swapIndex(i1: Int, i2: Int): Unit =
    val temp = xs(i1)
    xs(i1) = xs(i2)
    xs(i2) = temp
  end swapIndex

  for i <- 0 to xs.size - 1 do swapIndex(i, indexOfMin(i))
end selectionSortInPlace
```

Solution Task 7. *Check if a sequence is sorted.*

a) The worst case takes $O(n \log n)$ comparisons for Timsort on n elements. Then it requires n comparisons between the sorted and unsorted list. The total comparisons in worst case is at most $n + n \log n$. For 10^6 elements this is about 10^7 comparisons.

```
scala> val n = 1E6
val n: Double = 1000000.0

scala> def worstCase(n: Double) = n + n * math.log(n)
def worstCase(n: Double): Double

scala> println(s"in worst case with n=$n there are ${worstCase(n)} comparisons"
in worst case with n=1000000.0 there are 1.4815510557964273E7 comparisons
```

b) A more efficient version of `isSorted` that stops searching when unsorted elements are detected:

```
def isSorted(xs: Vector[Int]): Boolean =
  if xs.length > 1 then
    var i = 0
    var result = true
    while i < xs.length-1 && result do
      if xs(i) > xs(i+1) then result = false
      i += 1
    end while
    result
  else true
end isSorted
```

c) In worst case one does $n-1$ pairwise comparisons if all elements are sorted except for the last.

d) The 2-tuple is of type `(Int, Int)`.

```
def isSorted(xs: Vector[Int]): Boolean =
  xs.zip(xs.tail).forall(x => x._1 <= x._2)
```

Solution Task 8. *In-place insertion sort.*

```
def insertionSort(xs: Array[Int]): Unit =
  for elem <- 1 until xs.length if xs.length > 0 do
    var pos = elem
    while pos > 0 && xs(pos) < xs(pos - 1) do
      val temp = xs(pos -1)
      xs(pos -1) = xs(pos)
      xs(pos) = temp
      pos -= 1
    end while
  end for
end insertionSort
```

Solution Task 9. *Sorting to new sequence with selection sort.*

```
def selectionSort(xs: Seq[String]): Seq[String] =
  def indexOfMin(xs: Seq[String]): Int = xs.indexOf(xs.min)
  val unsorted = xs.toBuffer
  val result = scala.collection.mutable.ArrayBuffer.empty[String]
  while !unsorted.isEmpty do
    val minPos = indexOfMin(unsorted)
    val elem = unsorted.remove(minPos)
    result.append(elem)
  end while
  result.toVector
end selectionSort
```

12.2 Tasks on threads and concurrent execution

Solution Task 10. *Threads.*

a) -

b) `java.lang.IllegalThreadStateException`. You cannot start a thread more than once. Therefore, you can't restart a thread when it has already executed.

c) When `start` is called, the code in `run` executes in parallel. Therefore, `Cucumber` and `Tomato` are printed interleaved. If `run` is called directly, there is no concurrent execution; `Cucumber` is printed 100 times, then `Tomato` 100 times.

d) `Thread.sleep` does not pause the thread exactly as specified. So it will usually print `zzz snark hi!`, but this is not guaranteed.

Solution Task 11. *Concurrent variable update.*

a) In `spendSave` the balance is fetched, changed, and stored — delayed — repeated. If `teddybear` finishes loading, changing, and storing before `ing` does the same, there is a problem, because they compete in a (eng. *race condition*). The problem is that before a thread stores the changed value, the other thread loads the old value. Only one thread 'wins' the race to store its updated value, causing the other's change to be lost. `ing` and `teddybear` thus become upset because not all their withdrawals and deposits are recorded.

Solution Task 12. *Thread-safe `AtomicInteger`.*
Now the `grandma` thread is guaranteed to be able to load the balance, withdraw/change, and store before the `wolf` thread can overwrite the result. In `spendSave` the thread pauses for a millisecond so the `wolf` thread can withdraw before `grandma` deposits again and the balance becomes negative. However, all withdrawals and deposits are recorded because the operations are atomic and the balance will be restored to zero without lost deposits.

Solution Task 13. *Concurrent execution with `scala.concurrent.Future`.*

a) error: Cannot find an implicit ExecutionContext. Future needs an ExecutionContext to run. `f` has type Future[Unit].

b) The function `printLater` returns a Future, meaning when both `printLater` and `println` are called in the loop, they run concurrently. Since starting a Future takes

longer, the `println` prints "all launched!" first, then numbers 1 to 42 print with irregular intervals as the threads sleep varying amounts.

c) `big` is a Future[Int]. The large number has 7,520,383 digits. `r` is of type Try[Int] (see Future documentation if unsure).

d) Since execution blocks until a result is received, you cannot continue typing in REPL while computation is in progress. If you wait too short, you get a TimeOutException and the computation is aborted.

Solution Task 14. *Use `Future` to do multiple things concurrently.*

a) -

b) -

c) Each page is delayed between 2 and 3 seconds (2000-3000 ms). So on average, each page takes 2.5 seconds to load. The vector must be filled before execution proceeds, so all 10 pages load before the first page is visible. So it takes about 2.5 x 10 = 25 seconds.

d) `f` returns a Vector filled with strings where each element is a line of the webpage. Since `f` runs in background, the program can continue while the contents compute. You can print `f` but the computation may not be done yet.

e) Same as above, but each element is itself a vector of strings.

f) Loads data in parallel, so downloads happen simultaneously but at varying speeds because of method `seg`.

g) Since data loads in parallel threads without blocking, they finish in the order threads complete, not sequentially. Eventually all are done and the result vector shows `true` values.

h) Method `happy` is well-defined and throws no exceptions. It prints `:)`. Method `unhappy` throws `java.lang.ArithmeticException` due to division by zero. This exception is caught in the callback and `:(` plus the exception is printed.

12.3 Extra Tasks; additional practice

Solution Task 15.

```
def isPrime(n: BigInt): Boolean = n match
  case _ if (n <= 1) => false
  case _ if (n <= 3) => true
  case _ if n % 2 == 0 || n % 3 == 0 => false
  case _ =>
    var i = BigInt(5)
    while i * i < n do
      if (n % i == 0 || n % (i + 2) == 0) false
      i += 6
    end while
    true
end isPrime

import scala.concurrent.*
import ExecutionContext.Implicits.global

val primes = Vector.fill(10)(Future{nextPrime(randomBigInt(16))})
```

```
primes.foreach(_.onSuccess{case i => println(i)})
```

Solution Task 16. *Answer theory questions.*

a) Stackoverflow gives the following explanation:
A thread is an independent set of values for the processor registers (for a single core). Since this includes the Instruction Pointer (aka Program Counter), it controls what executes in what order. It also includes the Stack Pointer, which had better point to a unique area of memory for each thread or else they will interfere with each other.

b)

```
val thread = new Thread(new Runnable{
  def run(){println(''This is a thread'')}
})
```

c) `thread.start`

d) Race conditions can occur in which one thread's resource updates overwrite another's, causing data loss.

e) Thread-safe code means multiple threads can run concurrently without incorrect results. For example, mutable variables modified by multiple threads must be handled carefully.

f) For example, you avoid explicitly creating `Thread` instances by placing code inside a Future, which also handles much under the hood for the programmer.

Solution Task 17. –

Solution Task 18. *Create your own multithreaded web server.*

a) `abbasillen` is printed backwards as `nellisabba`.

b)

c)

d)

e)

f)

g)

h)
Solution proposal:

```
package fibserver.threaded.memcache.whileloop

import java.net.{ServerSocket, Socket}
import java.io.OutputStream
import java.util.Scanner
import scala.util.{Try, Success, Failure}
import scala.concurrent._
import ExecutionContext.Implicits.global

object html:
  def page(body: String): String =  //minimal web page
    s"""<!DOCTYPE html>
       |<html><head><meta charset="UTF-8"><title>Min Sörver</title></head>
       |<body>
```

```
      |$body
      |</body>
      |</html>
      """.stripMargin

  def header(length: Int): String = //standardized header of reply to client
    s"HTTP/1.0 200 OK\nContent-length: $length\nContent-type: text/html\n\n"

  def insertBreak(s: String, n: Int = 80): String =
    if s.size < n then s else s.take(n) + "</br>" + insertBreak(s.drop(n),n)

object compute:
  import java.util.concurrent.ConcurrentHashMap
  val memcache = new ConcurrentHashMap[BigInt, BigInt]

  def fib(n: BigInt): BigInt =
    if memcache.containsKey(n) then
      println("CACHE HIT!!! no need to compute: " + n)
      memcache.get(n)
    else
      println("cache miss :( must compute fib:  " + n)
      val f = superFib(n)
      memcache.put(n, f)
      f

  private def superFib(n: BigInt): BigInt =
    if n <= 0 then 0
    else if n == 1 || n == 2 then 1
    else
      var secondLast: BigInt = 1
      var last: BigInt = 1
      var sum: BigInt = secondLast + last
      var i = 3
      while i < n do
        if memcache.containsKey(i) then
          sum = memcache.get(i)
        else
          secondLast = last
          last = sum
          sum = secondLast + last
          memcache.put(i, sum)
        i += 1
      sum


object start:

  def fibResponse(num: String) =
    num.toIntOption match
      case Some(n) => html.page(s"fib($n) == " + compute.fib(n))
      case None    => html.page(s"FEL: skriv ett heltal, inte $num")

  def errorResponse(uri:String) = html.page(s"Error: $uri </br> use /fib/heltal")

  def handleRequest(cmd: String, uri: String, socket: Socket): Unit =
    val os = socket.getOutputStream
    val afterSlash = uri.toString.drop(1) // skip initial slash
```

```
    println(s"afterSlash:$afterSlash")
    val response: String =
      if afterSlash.startsWith("fib/") then fibResponse(afterSlash.stripPrefix("fib/"))
      else errorResponse(uri)
    os.write(html.header(response.size).getBytes("UTF-8"))
    os.write(response.getBytes("UTF-8"))
    os.close
    socket.close
  end handleRequest

  def serverLoop(server: ServerSocket): Unit =
    println(s"http://localhost:${server.getLocalPort}/hej")
    while true do
      Try {
        var socket = server.accept  // blocks thread until connect
        val scan = new Scanner(socket.getInputStream, "UTF-8")
        val (cmd, uri) = (scan.next, scan.next)
        println(s"Request: $cmd $uri")
        Future { handleRequest(cmd, uri, socket) }.recover {
          case e => println(s"Reqest failed: $e")
        }
      }.recover{ case e: Throwable => s"Connection failed: $e" }

  def main(args: Array[String]) =
    val port = Try{ args(0).toInt }.getOrElse(8089)
    serverLoop(new ServerSocket(port))
```

13. Solution Excercise13

Solution Task 1. *Complete your project.* —

Solution Task 2. *Take a final exam.* —

Solution Task 3. *Prepare your project presentation.* –

Solution Task 4. *Create documentation for your project.*–

Solution Task 5. *Review exercises and laboratory work.* –

14. Solution `java`

14.1 Basic Tasks; preparation for the lab

Solution Task 1. *Translate methods from Java to Scala.*

a)

```
import java.net.URL;
import java.util.ArrayList;
import java.util.{Set => JSet};
import java.util.{HashSet => JHashSet};
import java.util.Scanner;

object Hangman {  // This is Java-like, non-idiomatic Scala code!
    private var hangman: Array[String] = Array[String](
        " ======  ",
        " |/   |  ",
        " |    O  ",
        " |   -|- ",
        " |   / \\ ",
        " |       ",
        " |       ",
        " ==========================   RIP  :(");

    private def renderHangman(n: Int): String = {
        var result: StringBuilder = new StringBuilder();
        for (i: Int <- 0 until n){
            result.append(hangman(i));
            if (i < n - 1) {
                result.append("\n");
            }
        }
        return result.toString();
    }

    private def hideSecret(secret: String,
                           found: JSet[Character]): String = {
        var result: String = "";
        for (i: Int <- 0 until secret.length()) {
            if (found.contains(secret.charAt(i))) {
                result += secret.charAt(i);
            } else {
                result += '_';
            }
        }
        return result;
    }

    private def foundAll(secret: String,
                         found: JSet[Character]): Boolean = {
        var foundMissing: Boolean = false;
        var i: Int = 0;
        while (i < secret.length() && !foundMissing) {
            foundMissing = !found.contains(secret.charAt(i));
            i += 1;
        }
```

```
        return !foundMissing;
    }

    private def makeGuess(): Char = {
        var scan: Scanner = new Scanner(System.in);
        var guess: String = "";
        while ({
           System.out.println("Gissa ett tecken: ");
           guess = scan.next();
           guess.length() > 1;
        }) ()
        return Character.toLowerCase(guess.charAt(0));
    }

    def download(address: String, coding: String): String = {
        var result: String = "lackalänga";
        try {
            var url: URL = new URL(address);
            var words: ArrayList[String] = new ArrayList[String]();
            var scan: Scanner = new Scanner(url.openStream(), coding);
            while (scan.hasNext()) {
                words.add(scan.next());
            }
            var rnd: Int = (Math.random() * words.size()).asInstanceOf[Int];
            result = words.get(rnd);
        } catch { case e: Exception =>
            System.out.println("Error: " + e);
        }
        return result;
    }

    def play(secret: String): Unit = {
        var found: JSet[Character] = new JHashSet[Character]();
        var bad: Int = 0;
        var won: Boolean = false;
        while (bad < hangman.length && !won){
            System.out.println(renderHangman(bad));
            System.out.print("Felgissningar: " + bad + "\t");
            System.out.println(hideSecret(secret, found));
            var guess: Char = makeGuess();
            if (secret.indexOf(guess) >= 0) {
                found.add(guess);
            } else {
              bad += 1;
            }
            won = foundAll(secret, found);
        }
        if (won) {
            System.out.println("BRA! :)");
        } else {
            System.out.println("Hängd! :(");
        }
        System.out.println("Rätt svar: " + secret);
        System.out.println("Antal felgissningar: " + bad);
    }

    def main(args: Array[String] ): Unit = {
```

```
        if (args.length == 0) {
            var runeberg: String =
                "http://runeberg.org/words/ord.ortsnamn.posten";
            play(download(runeberg, "ISO-8859-1"));
        } else {
            var rnd: Int = (Math.random() * args.length).asInstanceOf[Int];
            play(args(rnd));
        }
    }
}
```

b)

```
object hangman:
  val hangman = Vector(
    " ======  ",
    " |/   |  ",
    " |    O  ",
    " |   -|- ",
    " |   / \\",
    " |       ",
    " |       ",
    " ========================   RIP  :(")

  def renderHangman(n: Int): String = hangman.take(n).mkString("\n")

  def hideSecret(secret: String, found: Set[Char]): String =
      secret.map(ch => if found(ch) then ch else '_')

  def makeGuess(): Char =
    val guess = scala.io.StdIn.readLine("Gissa ett tecken: ")
    if guess.length == 1 then guess.toLowerCase.charAt(0)
    else makeGuess()

  def download(address: String, coding: String): Option[String] =
    scala.util.Try {
      import scala.io.Source.fromURL
      val words = fromURL(address, coding).getLines.toVector
      val rnd = (math.random() * words.size).toInt
      words(rnd)
    }.toOption

  def play(secret: String): Unit =
    def loop(found: Set[Char], bad: Int): (Int, Boolean) =
      if secret forall found then (bad, true)
      else if bad >= hangman.length then (bad, false)
      else
        println(renderHangman(bad) + s"\nFelgissningar: $bad\t")
        println(hideSecret(secret, found))
        val guess = makeGuess()
        if secret contains guess then loop(found + guess, bad)
        else loop(found, bad + 1)

    val (badGuesses, won) = loop(Set(),0)
```

```
    val msg = if won then "BRA! :)" else "Hängd! :("
    println(s"$msg\nRätt svar: $secret")
    println(s"Antal felgissningar: $badGuesses")

  def main(args: Array[String] ): Unit =
    if args.length == 0 then
      val runeberg = "http://runeberg.org/words/ord.ortsnamn.posten"
      val secret = download(runeberg, "ISO-8859-1").getOrElse("läckalånga")
      play(secret)
    else play(args((math.random() * args.length).toInt))
```

Solution Task 2. *Translate between classes in Scala and classes in Java.*

a)

```
import java.util.List;
import java.util.ArrayList;

public class JPoint {
    private int x, y;

    public JPoint(int x, int y){
      this.x = x;
      this.y = y;
    }

    public JPoint(int x, int y, boolean save){
        this(x, y);
        if (save) {
            saved.add(0, this);
        }
    }

    public JPoint(){
        this(0, 0);
    }

    public int getX(){
        return x;
    }

    public int getY(){
        return y;
    }

    public double distanceTo(JPoint that) {
      return distanceBetween(this, that);
    }

    @Override public String toString() {
        return "JPoint(" + x + ", " + y + ")";
```

```
    }

    private static List<JPoint> saved = new ArrayList<JPoint>();

    public static Double distanceBetween(JPoint p1, JPoint p2) {
        return Math.hypot(p1.x - p2.x, p1.y - p2.y);
    }

    public static void showSaved() {
        System.out.print("Saved: ");
        for (int i = 0; i < saved.size(); i++){
          System.out.print(saved.get(i));
          if (i < saved.size() - 1) {
            System.out.print(", ");
          }
        }
        System.out.println();
    }
}
```

b) -

c)

```
case class Person(name: String, age: Int = 0)
```

d) p.*TAB* - copy, producArity, ProductIterator, productElement, productPrefix
Person.*TAB* - apply, curried, tupled, unapply

```
scala> p.copy
   def copy(name: String,age: Int): Person

scala> p.copy()
res0: Person = Person(Björn,49)

scala> p.copy(age = p.age + 1)
res1: Person = Person(Björn,50)

scala> Person.unapply(p)
res2: Option[(String, Int)] = Some((Björn,49))
```

Solution Task 3. *Immutable Java class.*

```
// Översätt:  class Point3D(val x: Int, val y: Int, val z: Int = 0)

public class JPoint3D {
  private int x;       // Attributen måste vara privata eftersom
  private int y;       //   val-attribut ej finns i Java.
  private int z;       // Typen skrivs före namnet i deklarationer.

  public JPoint3D(int x, int y, int z){// Konstruktor heter som klassen.
    this.x = x;                        // Satser måste sluta med ;
    this.y = y;                        // this behövs p.g.a. skuggning.
```

```
    this.z = z;
  }

  public JPoint3D(int x, int y){  // I stället för default-argument.
    this(x, y, 0);                // Anropa konstruktor i konstruktor.
  }
  public int getX(){    // I Java brukar man ha get i namnet på getter.
    return x;           // Metoder som ej är procedurer måste ha return.
  }

  public int getY(){    // Metoder måste ha parenteser, även getters.
    return y;
  }

  public int getZ(){   // Synlighet public måste anges explicit.
    return z;
  }
}
```

```
> code JPoint3D.java
> javac JPoint3D.java
> ls
JPoint3D.class  JPoint3D.java
> scala

scala> val p = new JPoint3D(1,2)
val p: JPoint3D = JPoint3D@53b1a3f8

scala> p.x
1 |p.x
  |^^^
  |value x is not a member of JPoint3D

scala> p.getX
val res0: Int = 1
```

Solution Task 4. *Mutable Java class.*

```
//Översätt class MutablePoint3D(var x: Int, var y: Int, var z: Int = 0)

public class JMutablePoint3D {
  private int x;  // Attributen brukar vara privata även i föränngsbara
  private int y;  //   klasser eftersom enhetlig access och syntax för
  private int z;  //   setter med tilldelning ej finns i Java.

  public JMutablePoint3D(int x, int y, int z){
    this.x = x;          // Satser måste sluta med ;
    this.y = y;          // this behövs p.g.a. skuggning.
    this.z = z;
  }

  public JMutablePoint3D(int x, int y){  // Motsv. default-argument.
```

```java
    this(x, y, 0);      // Anropa konstruktor i konstruktor.
  }
  public int getX(){    // I Java brukar man ha get i namnet på getter.
    return x;           // Metoder som ej är procedurer måste ha return.
  }

  public int getY(){    // Metoder måste ha parenteser, även getters.
    return y;
  }

  public int getZ(){   // Synlighet public måste anges explicit.
    return z;
  }

  public void setX(int x){ // I Java brukar man ha set i namnet på setter.
    this.x = x;
  }

  public void setY(int y){ // Nyckelordet void liknar Unit i Scala, men
    this.y = y;            //   det finns inget äkta tomt värde som ()
  }

  public void setZ(int zz){ // Om namn ej krockar behövs inte this, men
    z = zz;                 // man brukar ha samma parameternamn som
  }                         // attributnamn i setters trots skuggning så
}                           // att man slipper hitta på nytt namn.
```

```
> code JMutablePoint3D.java
> javac JMutablePoint3D.java
> ls
JMutablePoint3D.class  JMutablePoint3D.java
> scala

scala> val p = new JMutablePoint3D(1,2)
val p: JMutablePoint3D = JMutablePoint3D@625b215b

scala> p.x
1 |p.x
  |^^^
  |value x is not a member of JMutablePoint3D

scala> p.getZ
val res0: Int = 0

scala> p.setZ(3)

scala> p.getZ
val res1: Int = 3
```

Solution Task 5. *Comparing strings in Java.*

a)

```
String = java.lang.String
Boolean = true
Int = 0
```

b) Examples of 3 different expressions to test `compareTo`:

1. Hej comes first since `H < h`.

```
"hej".compareTo("Hej")
res: Int = 32
```

2. These are equivalent, so `compareTo` returns 0.

```
"hej".compareTo("hej")
res: Int = 0
```

3. *h* comes before *ö*.

```
"hej".compareTo("ö")
res: Int = -142
```

c) Examples of 3 different expressions to test `compareToIgnoreCase`:

1.

```
"hej".compareToIgnoreCase("HEj")
res: Int = 0
```

2.

```
"hej".compareToIgnoreCase("Ö")
res: Int = -142
```

3. Same as above, since Ö is converted to ö before comparison.

```
"hej".compareToIgnoreCase("ö") \\ res: Int = -142
```

d)

```
false
true
0
```

Solution Task 6. *Linear search in Java.*

a)

```java
public static boolean isYatzy(int[] dice){
    int col = 1;
    boolean allSimilar = true;
    while(col < dice.length && allSimilar){
        allSimilar = (dice[0] == dice[col]);
        col++; // this line was missing
    }
    return allSimilar;
}
```

b)

```
public static int findFirstYatzyRow(int[][] m){
    int row = 0;
    int result = -1;
    while(row < m.length){
        if(isYatzy(m[row])){
           result = row;
           break;
        }
        row++;
    }
    return result;
}
```

Solution Task 7. *Comparison support in Java.*

a)

b)

```
val teamComparator = new Comparator[Team]{
  def compare(o1: Team, o2: Team) = o2.rank - o1.rank
}
```

c)

d)

e)

```
case class Point(x: Int, y: Int) extends Comparable[Point] {
  def distanceFromOrigin: Double = math.hypot(x, y)
  def compareTo(that: Point): Int =
    (distanceFromOrigin - that.distanceFromOrigin).round.toInt
}
```

Solution Task 8. `java.util.Arrays.binarySearch`

Solution Task 9. *Auto(un)boxing.*

a) -

b) Cell has the type java.lang.Integer. When extracting the value with `c.value` the primitive type `int` is extracted.

c) Using autoboxing 42 is converted to `Integer` and can thereby be compared with another `Integer`.

d) i.compareTo(42) works because of autoboxing. The JVM automatically boxes the primitive type int into an Integer object.

e)

```
0 10 20 30 40 50 60 ... 390 400 410

[0]: 0
[42]: 0
```

```
NOT EQUAL
```

f)

```java
import java.util.ArrayList;

public class Autoboxing2 {
    public static void main(String[] args) {
        ArrayList<Integer> xs = new ArrayList<Integer>();
        for (int i = 0; i < 42; i++) {
            xs.add(i);
        }
        for (int x: xs) {
            int y = x * 10;
            System.out.print(y + " ");
        }
        int pos = xs.size();
        xs.add(pos, 0);
        System.out.println("\n\n[0]: " + xs.get(0));
        System.out.println("[" + pos + "]: " + xs.get(pos));
        if (xs.get(0).equals(xs.get(pos))) {
            System.out.println("EQUAL");
        } else {
            System.out.println("NOT EQUAL");
        }
    }
}
```

g) 42 will be inserted at the front of the list instead of the back, because autounboxing will convert Integer(0) to 0 and vice versa with variable `pos`.

h) If you want to check whether two int variables are equal, use ==, but if variables are of type Integer, you must use `equals`.
The JVM will not warn if you swap `Integer` and `int`, as in `xs.add(0, pos)`.

Solution Task 10. *CollectionConverters.*

a)
Vector[Int] -> java.util.List[Int]
Set[Char] -> java.util.Set[Char]
Map[String, Int] -> java.util.Map[String, Int]

b)
ArrayList[Int] -> scala.collection.mutable.Buffer[Int]
HashSet[Char] -> scala.collection.mutable.Set[Char]
Both become mutable equivalents. This is shown by their membership in `scala.collection.mutable` and as both `ArrayList` and `HashSet` are mutable in Java.

c) `scala.collection.immutable.Set`

d) `sm.asJava.asScala` gives type `scala.collection.mutable.Map[String,Int]`
`sm.asJava.asScala.toMap` gives type `scala.collection.immutable.Map[String,Int]`

e) -

Solution Task 11. *How does a* `switch` *statement in Java (and many other languages) work?*

a) Depending on the first letter of your favorite vegetable you get different responses such as *cucumber is good!* ("cucumber is tasty!") if the first letter is *g*.
Java's `switch` statement tests the first letter of the favorite vegetable by sequentially comparing it with the `case` expressions. If the first letter `firstChar` matches the letter after a `case`, the code after the colon is run until the end of the `switch` statement or until a `break` interrupts the `switch`.
If `firstChar` doesn't match any `case`, there is also a `default`, which is run regardless of the first letter, a general case.

b) If the `case 't'` is run, both *tomato is good!* ("tomato is tasty!") and *broccoli is good!* ("broccoli is tasty!") are printed; this is called "fall through". Without the `break` statement in Java, code in subsequent `case`s runs until a `break` interrupts the execution or the `switch` ends.

Solution Task 12. *Catch exceptions in Java with a* `try-catch` *statement.*

a)

1. Since the first argument is not the string *safe*, an unprotected division of 42 by 42 is done and the result 1 is shown.
2. Since the first argument is not the string *safe*, an unprotected division of 42 by 0 is done which results in an `ArithmeticException` because a number cannot be divided by zero.
3. Since the first argument is the string *safe*, a protected division of 42 by 42 is done and the result 1 is shown.
4. Since the first argument is the string *safe*, a protected division of 42 by 0 is done. This time the `ArithmeticException` is caught by the `try-catch` which replaces the previous division with a safe division by 1 where the result 42 is shown.
5. Since no arguments are given, an `ArrayIndexOutOfBoundsException` is thrown when the program tries to call the `equals` method on a string that does not exist. This could also be checked with a `try-catch` statement.

b)

```
TryCatch.java:16: error: variable input might not have been initialized
```

A compile-time error occurs due to the risk that `input` was not assigned when dividing.

Solution Task 13. *Matrices with arrays in Java.*

a) At initialization, all elements of `xss` are filled with the standard value for the type, `0` in the case of `int`. The outer `for` loop in `showMatrix()` iterates over the rows of `xss`. The inner `for` loop iterates through the elements of the current row and prints row, column, and content. After each row, a newline is printed so that a row in the output corresponds to a row in the matrix.
Examples of differences between usage of matrices in Scala and Java:

- access: `minArray(row)(column)` vs `minArray[row][column]`
- type names: `Array[Array[elementType]]` vs `elementType[][]`

- allocation: `Array.ofDim[typ](xDim,yDim)` vs `new typ[xDim][yDim]`

b)

```
public class JavaArrayTest {

  public static void showMatrix(int[][] m){
    System.out.println("\n--- showMatrix ---");
    for (int row = 0; row < m.length; row++){
      for (int col = 0; col < m[row].length; col++) {
        System.out.print("[" + row + "]");
        System.out.print("[" + col + "] = ");
        System.out.print(m[row][col] + ";");
      } System.out.println();
    }
  }

  public static void fillRnd(int[][] m, int n){
    for (int row = 0; row < m.length; row++){
      for (int col = 0; col < m[row].length; col++) {
        m[row][col] = (int) (Math.random() * n + 1);
      }
    }
  }

  public static void main(String[] args) {
    System.out.println("Hello JavaArrayTest!");
    int[][] xss = new int[10][5];
    fillRnd(xss, 6);
    showMatrix(xss);
  }
}
```

Solution Task 14. *Translate from Java to Scala.*

```
object showInt {
  def show(obj: Any, msg: String = ""): Unit = println(msg + obj)

  def repeatChar(ch: Char, n: Int): String = ch.toString * n

  def showInt(i: Int): Unit = {
    val leading = Integer.numberOfLeadingZeros(i)
    val binaryString = repeatChar('0', leading) + i.toBinaryString
    show(i,                "Heltal : ")
    show(i.asInstanceOf[Char],         "Tecken : ")
    show(binaryString,    "Binärt : ")
    show(i.toHexString,   "Hex    : ")
    show(i.toOctalString, "Oktal  : ")
  }


```

```
  import scala.io.StdIn.readLine
  import scala.util.{Try,Success,Failure}

  def loop: Unit =
    Try { readLine("Heltal annars pang: ").toInt } match {
      case Failure(e) => show(e); show("PANG!")
      case Success(i) => showInt(i); loop
    }

  def main(args: Array[String]): Unit =
    if(args.length > 0) args.foreach(i => showInt(i.toInt))
    else loop
}
```

Solution Task 15. *Content equality and reference equality in Java.*

Solution Task 16. *Implement content equality in Java.*

Solution Task 17. *Arrays and* `for` *loops in Java.*

a) The program simulates 10000 dice rolls (with random seed 42) and prints the occurrence of each die face.

```
Rolling the dice 10000 times with seed 42
Number of 1's: 1654
Number of 2's: 1715
Number of 3's: 1677
Number of 4's: 1629
Number of 5's: 1643
Number of 6's: 1682
```

b) In Java, square brackets are used while Scala has "normal" parentheses. An array in Scala is declared like this:
`val scalaArray = Array.ofDim[Int](6)`
which in Java corresponds to: `int[] javaArray = new int[6];`
`for` loop in Scala is written: `for(i <- 0 to n) {...}` whereas in Java it is written: `for (int i = 0; i < n; i++) { ... }`.
In Java, semicolons must be written after each statement and the type must be explicitly specified at every variable declaration.
In Scala, semicolons are not needed (except to separate statements on the same line) and types can often be inferred by the compiler and do not need to be explicitly written.

c) Add `System.out.println(i);` in the for loops

d)

```
// DiceReg2.java
import java.util.Random;
public class DiceReg2{
  public static int[] diceReg = new int[6];
  private static Random rnd = new Random();

  public static int parseArguments(String[] args){
    int n = 100;
```

```
    if(args.length > 0) {
      n = Integer.parseInt(args[0]);
    }
    if(args.length > 1) {
      int seed = Integer.parseInt(args[1]);
      rnd.setSeed(seed);
    }
    return n;
  }

  public static void registerPips(int n) {
    for(int i = 0; i<n; i++) {
      int pips = rnd.nextInt(6);
      diceReg[pips]++;
    }
  }

  public static void main(String[] args) {
    int n = parseArguments(args);
    registerPips(n);
    printReg();
  }
}
```

e)

```
  // Prints the occurrences of 1000 dice rolls with seed 42.
Number of 1's: 165
Number of 2's: 163
Number of 3's: 178
Number of 4's: 183
Number of 5's: 156
Number of 6's: 155

  // Prints the diceReg attribute
res1: Array[Int] = Array(165, 163, 178, 183, 156, 155)

  // Prints diceReg attribute after 1000 rolls.
res2: Array[Int] = Array(329, 325, 349, 360, 324, 313)

  // Prints diceReg attribute after another 1000 rolls.
res3: Array[Int] = Array(498, 484, 531, 513, 485, 489)

  // It results in a compilation error because the attribute rnd is private
<console>:11: error: value rnd is not a member of object DiceReg2
  DiceReg2.rnd
           ^
```

Solution Task 18. *Read in a sequence of numbers with `Scanner` in Java.*

a)

`hasNextInt()` checks if another number exists and returns **`true`** or **`false`**. `nextInt()` reads the next number.

See https://docs.oracle.com/javase/8/docs/api/java/util/Scanner.html#hasNextInt%28%29 and https://docs.oracle.com/javase/8/docs/api/java/util/Scanner.html#nextInt%28%29.

b)

```
import java.util.Random;
import java.util.Scanner;

public class DiceScanBuggy {
  public static int[] diceReg = new int[6];
  public static Scanner scan = new Scanner(System.in);

  public static void registerPips() {
    System.out.println("Enter pips separated by blanks: ");
    System.out.println("End with -1 and <Enter>.");
    boolean isPips = true;
    while(isPips && scan.hasNextInt()){
      int pips = scan.nextInt();
      if(pips >= 1 && pips <= 6) {
        diceReg[pips-1]++;
      } else {
        isPips = false;
      }
    }
  }

  public static void printReg(){
    for(int i = 1; i<7; i++) {
    System.out.println("Number of " + i + "'s: " + diceReg[i-1]);
    }
  }

  public static void main(String[] args) {
    registerPips();
    printReg();
  }
}
```